

판다스로 배우는 쇼핑물 데이터 분석

목차

1장. 실습 환경과 쇼핑물 데이터셋 준비 [입문]

- 판다스란 무엇이고 이 책은 어떻게 실습하는가
- 설치: pip install pandas numpy pyarrow와 버전 확인
- 쇼핑물 데이터셋 생성: generate_shop_data.py와 시드 고정
- 다섯 파일 소개와 의도적으로 심은 품질 문제
- worked example: 생성부터 첫 적재(read_csv-head)까지
- 단원 미니 프로젝트: 데이터셋 생성과 첫 적재 기록

2장. 판다스 데이터 구조와 불러오기 [입문]

- 시리즈와 데이터프레임: 값·인덱스·자료형
- 데이터 불러오기: read_csv 옵션(dtype·parse_dates·encoding)
- 데이터 저장: to_csv와 index 처리
- 데이터 훑어보기: head·info·describe·value_counts
- worked example: 다섯 테이블 적재와 개요 파악
- 단원 미니 프로젝트: 데이터 품질 점검표 작성

3장. 데이터 선택과 인덱싱 [입문]

- 열 선택과 loc·iloc: 라벨 기반과 위치 기반
- loc + 조건으로 필터와 선택을 한 번에
- 인덱스 다루기: set_index·reset_index
- 정렬: sort_values·sort_index와 날짜 기간 슬라이싱
- worked example: 고객·상품 조회와 정렬
- 단원 미니 프로젝트: 고객·상품 조회기 만들기

4장. 조건으로 걸러내기: 불리언 필터 [기초]

- 불리언 시리즈와 df[불리언] 필터링 원리
- 다중 조건 결합: & | ~와 괄호(and/or 금지)

- 편의 메서드: `isin`·`between`·`str.contains`·`notna`
- query 문자열 필터와 결과 검증
- worked example: 주문 세그먼트 추출과 행수 대조
- 단원 미니 프로젝트: 주문 세그먼트 추출기

5장. 데이터 정제 (1): 결측·중복·이상값·타입 [기초]

- 결측치란: `NaN`·`NaT`와 탐지(`isna`)
- 결측 처리: `dropna`와 `fillna` 전략 고르기
- 중복 제거: `duplicated`·`drop_duplicates(subset=keep)`
- 이상값과 타입 오염: `clip`·`to_numeric`·`astype`
- worked example: 결측·중복·이상값·타입 정제 전후 대조
- 오개념: 결측을 0으로 채우기·이상값 무조건 삭제

6장. 데이터 정제 (2): 문자열 표기 통일과 정제 파이프라인 [기초]

- str 접근자와 문자열 메서드의 벡터화
- 표기 통일: `strip`·`lower`·`replace`로 `category`·`gender` 정리
- 검색·추출: `contains`·`split`·`extract`(정규식 맛보기)
- worked example: 표기 통일 전후 `value_counts` 비교
- 단원 미니 프로젝트: 정제 파이프라인과 clean 테이블 저장

7장. 파생 변수와 데이터 변환 [기초]

- **벡터화 연산**으로 파생 열 만들기(주문 금액)
- `assign`·`map`·`apply`와 조건부 값 `np.where`·`np.select`
- 구간화: `cut`·`qcut`으로 연령대·가격대 만들기
- 범주 매핑: `map`·`replace`로 코드값 라벨링
- worked example: 금액 파생과 구간화 손계산 대조
- 단원 미니 프로젝트: 분석 축 만들기

8장. 그룹화와 집계 [중급]

- `groupby`의 `split`·`apply`·`combine` 원리
- agg 이름 붙은 집계와 `transform`
- **피벗테이블**: `index`·`columns`·`values`·`aggfunc`·`margins`
- 교차표(`crosstab`)와 `normalize` 비율
- worked example: 카테고리·채널 요약표와 총합 대조
- 단원 미니 프로젝트: 핵심 지표 요약표 만들기

9장. 여러 테이블 결합과 재구조화 [중급]

- merge와 조인: on·how(inner·left·outer)
- 네 테이블 순차 병합과 키 무결성 점검
- concat으로 이어 붙이기
- 재구조화: melt·pivot으로 긴/넓은 형식 오가기
- worked example: 통합 테이블 구축과 재구조화
- 단원 미니 프로젝트: 분석용 통합 테이블 구축

10장. 시계열 분석 [중급]

- 날짜·시간 다루기: to_datetime·NaT·dt 접근자
- 기간 계산(Timedelta)과 날짜 인덱스 슬라이싱
- 리샘플링: resample로 월별·주별 집계
- 이동집계: rolling 이동평균과 pct_change 성장률
- worked example: 매출 추세와 대조 검증
- 단원 미니 프로젝트: 매출 추세 분석

11장. 대용량 데이터와 성능 (심화) [실무]

- 메모리 측정: memory_usage(deep=True)
- 최적화: 다운캐스팅과 범주형(category)
- 청크 읽기: read_csv chunksize와 부분 집계
- Parquet과 PyArrow 백엔드: 저장·재적재·비교
- worked example: 최적화 전후 메모리·시간 수치 비교
- 단원 미니 프로젝트: 백만 행 로그 처리

끝

12장. 종합 프로젝트 (1) 분석: 문제 정의와 데이터 품질 점검 [중급]

- 드라이빙 퀘스천·POV와 분석 목표
- 요구사항(FR/NFR)과 답할 질문 정의 PRD, TRD 만드는 연습.
- 데이터 적재와 품질 점검표(단원 개념 매핑)
- 범위 In/Out과 재현성 기준

13장. 종합 프로젝트 (2) 설계: 분석 파이프라인과 통합 스키마 [중급]

- 분석 파이프라인 단계 설계(정제→통합→분석)
- 통합 스키마와 병합 키·조인 결정(ADR)
- 정제·파생·집계 규칙 문서화
- 재현 가능한 스크립트 구조와 실행 순서

14장. 종합 프로젝트 (3) 구현: 정제·통합·분석 수행 [실무]

- M1 세그먼트 추출 · M2 정제 파이프라인
- M3 파생·범주 · M4 핵심 지표 집계
- M5 통합 테이블 · M6 매출 추세 분석
- M7 대용량 로그 처리와 마일스톤 검증

15장. 종합 프로젝트 (4) 발표·평가·성찰: 분석 리포트와 재현성 [실무]

- 분석 리포트: 발견 3개 이상과 개선 제안
 - 재현성 검증: 처음부터 다시 실행
 - 평가 루브릭과 자기 평가
 - 성찰: 개념 적용 회고와 다음 단계
-

이 책에 대하여 (머리말)

이 책의 목적

파이썬 기본 문법을 아는 데이터 분석 입문자가, 실제 쇼핑물 데이터셋(고객·상품·주문·주문상세·웹 행동 로그)을 처음부터 끝까지 판다스로 분석할 수 있게 하는 실습 중심 교재입니다. 결정론적 생성 스크립트로 만든 하나의 데이터셋 위에서 적재·구조 파악·선택·필터·정제·파생·집계·병합·시계열·대용량 처리까지 실제로 수행하며, 매출 추세·핵심 고객·잘 팔리는 상품을 근거와 함께 밝히는 통합 분석 역량을 기릅니다.

이 책을 마치면 할 수 있는 것

- 제공된 생성 스크립트로 쇼핑물 CSV 다섯 개를 만들어 알맞은 dtype·날짜 파싱으로 불러 오고 데이터의 구조·품질 이슈를 파악할 수 있습니다
- 결측·중복·이상값·타입 오염·문자열 불일치를 문서화된 규칙으로 정제하고 정제 전후를 대조 검증할 수 있습니다
- groupby·피벗테이블로 핵심 지표를 집계하고 merge로 흩어진 테이블을 통합하며 시계열 도구로 매출 추세를 읽을 수 있습니다
- 백만 행 규모의 행동 로그를 dtype 최적화·범주형·체크 읽기·Parquet으로 효율적으로 처리해 성능·메모리 개선을 수치로 보일 수 있습니다

대상 독자

파이썬 기본 문법(변수·조건·반복·함수·리스트·딕셔너리)은 아는 데이터 분석 입문자를 대상으로 합니다. 판다스·넴파이는 처음이거나 알게 써 봤고, 엑셀·SQL 경험이 있으면 좋지만 필수는 아닙니다. 통계·머신러닝 지식은 전제하지 않습니다(데이터 분석 도구로서 판다스를 실습으로 익히는 책이지 통계학·모델링 책이 아닙니다).

선수 지식

- 파이썬 기본 문법(변수·자료형·조건·반복·함수·리스트·딕셔너리)
- 터미널에서 python 스크립트 실행과 pip 설치
- 엑셀·SQL로 표 데이터를 다뤄 본 경험이 있으면 도움이 되나 필수 아님

실습 환경

- pip install pandas numpy pyarrow로 판다스 2.2·넘파이·PyArrow 설치
 - ch_01에서 제공된 generate_shop_data.py를 실행해 data 폴더에 CSV 다섯 개 (customers-products-orders-order_items-web_logs) 생성 — 시드 고정으로 누가 실행해도 동일
 - 이후 모든 단원 미니 프로젝트와 캡스톤이 이 하나의 데이터셋 위에 누적
 - 대상 OS: Windows 11(macOS·Linux 동일 동작)
-

1장 실습 환경과 쇼핑물 데이터셋 준비

데이터 분석을 배우겠다고 마음먹으면 흔히 문법부터 외우기 시작합니다. 하지만 문법을 아무리 외워도, 막상 진짜 데이터를 앞에 두면 "그래서 무엇부터 하지?"에서 막힙니다. 이 책은 반대로 갑니다. 실제 쇼핑물 하나를 통째로 분석하는 여정을 처음부터 끝까지 함께 걸으며, 그 길에서 필요한 판다스 도구를 그때그때 배웁니다.

그 여정의 출발점이 이 장입니다. 아직 분석 기법은 배우지 않습니다. 대신 이 책 전체가 닫고 **실습 자리**를 만듭니다. 판다스를 설치하고, 우리만의 쇼핑물 데이터를 직접 만들어 내고, 그 데이터를 판다스로 처음 불러오는 데까지 갑니다. 여기까지 손으로 해내면, 2장부터는 이 데이터 위에서 진짜 분석이 쌓여 갑니다.

앞으로의 여정을 미리 지도로 보면 다음과 같습니다. 모두 이 장에서 만든 하나의 데이터셋 위에서 이어집니다.

단계	배우는 것	답하는 질문
준비(1장)	설치·데이터 생성·첫 적재	무엇으로 분석하나
구조·훑어보기(2장)	시리즈·데이터프레임 ·info·describe	데이터에 무엇이 들었나
선택·필터(3~4장)	loc·iloc·불리언 필터	원하는 부분만 어떻게 뽑나
정제(5~6장)	결측·중복·이상값·문자열 정리	지저분함을 어떻게 고치나
파생·집계·병합(7~9장)	파생 변수·groupby·merge	핵심 지표를 어떻게 계산하나
시계열·대용량(10~11장)	resample·성능 최적화	추세와 대용량을 어떻게 다루나
종합 프로젝트(12~15장)	전 과정 통합 분석	근거 있는 결론을 어떻게 전달하나

지금은 첫 줄, 준비 단계입니다.

이 장의 학습 목표

- pip로 pandas·numpy·pyarrow를 설치하고 import pandas as pd 후 pd.__version__으로 2.2 이상 버전을 확인할 수 있다 [→G1]
- 제공된 generate_shop_data.py를 실행해 data 폴더에 CSV 다섯 개가 생성됐음을 파일 목록과 각 파일 크기로 확인할 수 있다 [→G1]
- pd.read_csv로 customers.csv를 불러와 head().shape로 앞 5행과 행·열 수를 출력해 데이터가 제대로 적재됐는지 확인할 수 있다 [→G1]
- 이 책이 왜 하나의 쇼핑물 데이터셋을 계속 쓰는지, 생성 스크립트가 왜 결측·이상값을 일부러 심는지 설명할 수 있다 [→G1]

선수 확인: 이 장은 파이썬 기본 문법(변수·자료형·조건·반복·함수·리스트·딕셔너리)을 알고, 터미널에서 python 스크립트를 실행하고 pip로 패키지를 설치해 본 경험이 있다고 가정합니다. 판다스·넘파이는 처음이어도 괜찮습니다. 이 장이 그 첫걸음입니다. 실습은 Windows 11을 기준으로 안내하지만 macOS·Linux에서도 명령이 동일합니다.

1.1 판다스란 무엇이고 이 책은 어떻게 실습하는가

도입: 엑셀로는 벅찬 순간

작은 표라면 엑셀이 편합니다. 그런데 주문이 20만 건, 행동 로그가 100만 건이면 어떨까요? 엑셀은 100만 행 근처에서 버거워지고, "지난달과 이번 달의 카테고리별 매출을 매주 똑같은 방식으로 다시 계산해 달라" 같은 반복 작업을 사람이 마우스로 하기 시작하면 실수가 쌓입니다. 데이터 분석에서 진짜 필요한 것은 **한 번 정한 분석 절차를 코드로 적어 두고, 데이터가 바뀌어도 같은 코드로 똑같이 재현하는 능력**입니다. 그 능력을 파이썬으로 주는 도구가 판다스입니다.

정의

이 장에서 배우는 개념은 **판다스 소개와 실습 데이터셋 준비**(introducing pandas and preparing the practice dataset)입니다.

핵심 정의

판다스(pandas)는 표 형태의 데이터를 다루기 위한 파이썬 라이브러리로, 행과 열로 된 데이터를 엑셀·SQL처럼 다루되 **코드로 반복·자동화·대용량 처리**를 할 수 있게 해 줍니다.

- **시리즈(Series)**: 값의 배열에 라벨(인덱스)이 붙은 1차원 자료구조. 표의 한 열에 해당합니다.
- **데이터프레임(DataFrame)**: 같은 인덱스를 공유하는 여러 시리즈(열)를 모은 2차원 표.
- **넘파이(numpy)**: 판다스가 내부적으로 올라타 있는 수치 계산 라이브러리. 덕분에 반복문 없이 열 전체를 한 번에 계산하는 **벡터화 연산**(vectorized operation)이 빠릅니다.

용어를 하나씩 풀면, 판다스로 데이터를 불러오면 그것은 데이터프레임이라는 2차원 표가 되고, 그 표에서 열 하나를 꺼내면 시리즈라는 1차원 값이 됩니다. 엑셀의 "시트"가 데이터프레임, "한 열"이 시리즈라고 생각하면 첫 이미지로 충분합니다. SQL을 써 봤다면, 데이터프레임은 하나의 테이블, 뒤에서 배울 `groupby · merge` 는 SQL의 `GROUP BY · JOIN` 에 대응한다고 보면 됩니다.

왜 넘파이 위의 벡터화 연산이 빠른가

파이썬 리스트로 100만 개 숫자에 각각 1.1을 곱하려면 반복문을 100만 번 돌아야 합니다. 판다스·넘파이는 이 곱셈을 열 전체에 대해 **한 번의 명령**으로 처리합니다. 내부적으로 넘파이가 값을 연속된 메모리에 촘촘히 담아 두고, 최적화된 저수준 코드로 한꺼번에 계산하기 때문입니다. 이것이 벡터화 연산이며, `df['price'] * 1.1` 처럼 반복문 없이 열에 바로 연산을 거는 판다스 특유의 문법이 여기서 나옵니다.

두 방식을 나란히 두면 차이가 분명합니다.

```
# 반복문 방식 (파이썬 리스트) — 한 개씩 처리
result = []
for p in price_list:
    result.append(p * 1.1)

# 벡터화 방식 (판다스 시리즈) — 열 전체를 한 번에
result = df["price"] * 1.1
```

- **반복문 방식** — 값 하나하나를 꺼내 곱하고 다시 담습니다. 100만 개면 100만 번 반복합니다.
- **벡터화 방식** — `df["price"] * 1.1` 한 줄로 열 전체에 곱셈을 겁니다. 코드도 짧고, 내부적으로 훨씬 빠르게 계산됩니다.

지금은 "판다스는 반복문 없이 열 전체를 한 번에 계산한다" 정도만 기억하면 됩니다. 그 위력은 100만 행 로그를 다루는 뒤 단원에서 수치로 체감하게 됩니다.

이 책은 어떻게 실습하는가

이 책은 API를 사전처럼 나열하지 않습니다. 대신 **가상의 쇼핑물 하나를 처음부터 끝까지 분석**합니다. 그 쇼핑물의 데이터는 여러분이 직접 만듭니다. 제공된 생성 스크립트

`generate_shop_data.py` 를 실행하면 고객·상품·주문·주문 상세·웹 행동 로그 다섯 개의 CSV 파일이 만들어집니다. 이 하나의 데이터셋 위에 2장의 훑어보기부터 마지막 종합 프로젝트까지 모든 실습이 차곡차곡 쌓입니다.

한 데이터셋을 끝까지 쓰는 이유는 분명합니다. 매 장 새 예제 데이터로 갈아타면, 도구는 배우지만 "이 도구가 실제 분석의 어느 길목에서 쓰이는지"를 놓칩니다. 반대로 같은 쇼핑물을 계속 파고들면, `loc` 로 뽑은 세그먼트를 나중에 `groupby` 로 집계하고 `merge` 로 다른 테이블과 잇는 식으로 도구들이 하나의 분석으로 이어지는 감각이 생깁니다.

엑셀·SQL·판다스, 무엇이 다른가

이미 엑셀이나 SQL을 써 봤다면, 판다스가 그 사이 어디쯤인지 표로 정리하면 이해가 빠릅니다.

관점	엑셀	SQL	판다스
다루는 단위	시트의 셀	테이블의 행	데이터프레임의 열·행
조작 방식	마우스·수식	질의문(SELECT 등)	파이썬 코드
반복·자동화	어렵다(수동)	질의 재실행	코드로 완전 자동화
대용량(백만 행)	버겁다	강하다	강하다(메모리 한도 내)
재현성	낮다	높다	매우 높다(스크립트)

판다스는 엑셀의 "표를 직접 보고 다루는 직관"과 SQL의 "질의로 집계·결합하는 힘"을 파이썬 코드 위에서 합친 도구라고 보면 됩니다. 그래서 이 책에서 배우는 `groupby` · `merge` 는 SQL의 `GROUP BY` · `JOIN` 과, 필터는 엑셀의 자동 필터와 자연스럽게 대응됩니다. 이미 아는 도구가 있다면 그 경험이 판다스 학습을 돕습니다.

1.2 설치: pip install pandas numpy pyarrow와 버전 확인

동작 원리: 무엇을 왜 설치하나

실습에 필요한 패키지는 세 개입니다. **pandas**는 주인공이고, **numpy**는 판다스가 올라탄 수치 계산 토대이며, **pyarrow**는 뒤 심화 단원에서 쓸 Parquet 파일 형식과 빠른 문자열 처리를 위한

백엔드입니다. 셋을 한 번에 설치합니다.

터미널(Windows라면 명령 프롬프트나 PowerShell)에서 다음을 실행합니다.

```
pip install pandas numpy pyarrow
```

설치가 끝나면 판다스가 제대로 올라왔는지 버전으로 확인합니다. 파이썬을 실행해 다음을 입력합니다.

```
import pandas as pd
print(pd.__version__)
```

- `import pandas as pd` — 판다스를 불러오되 `pd` 라는 짧은 별명으로 씁니다. 이 별명은 전 세계 판다스 코드의 사실상 표준 관례이므로, 이 책도 항상 `pd` 로 씁니다.
- `print(pd.__version__)` — 설치된 판다스의 버전 문자열을 출력합니다. 앞뒤로 밑줄이 두 개씩 붙은 `__version__` 은 라이브러리가 자기 버전을 담아 두는 특수 속성입니다.

실행 결과는 환경에 따라 다음과 비슷하게 나옵니다.

```
2.3.3    3.0.3
```

이 책은 pandas 2.2를 기준으로 쓰였습니다. 출력된 버전이 2.2 이상이면(위 예시는 2.3.3) 실습에 아무 문제가 없습니다. 만약 `ModuleNotFoundError: No module named 'pandas'` 가 나면 설치가 안 된 것이므로 `pip install` 명령을 먼저 다시 실행합니다.

직접 해 보기 — 설치와 버전 확인 (직접 실행·기록)

1. 터미널에서 `pip install pandas numpy pyarrow` 를 실제로 실행하십시오.
2. 파이썬을 열어 `import pandas as pd` 와 `print(pd.__version__)` 을 실행하십시오.
3. 화면에 실제로 출력된 값을 아래에 그대로 옮겨 적으십시오(추측이 아니라 내 화면의 실제 출력).
4. **[내 pandas 버전 출력]:** (예: 2.3.3)
5. **[2.2 이상인가?]:** (예/아니오)
6. **[numpy도 확인]:** `import numpy as np; print(np.__version__)` 의 실제 출력:

자가체크

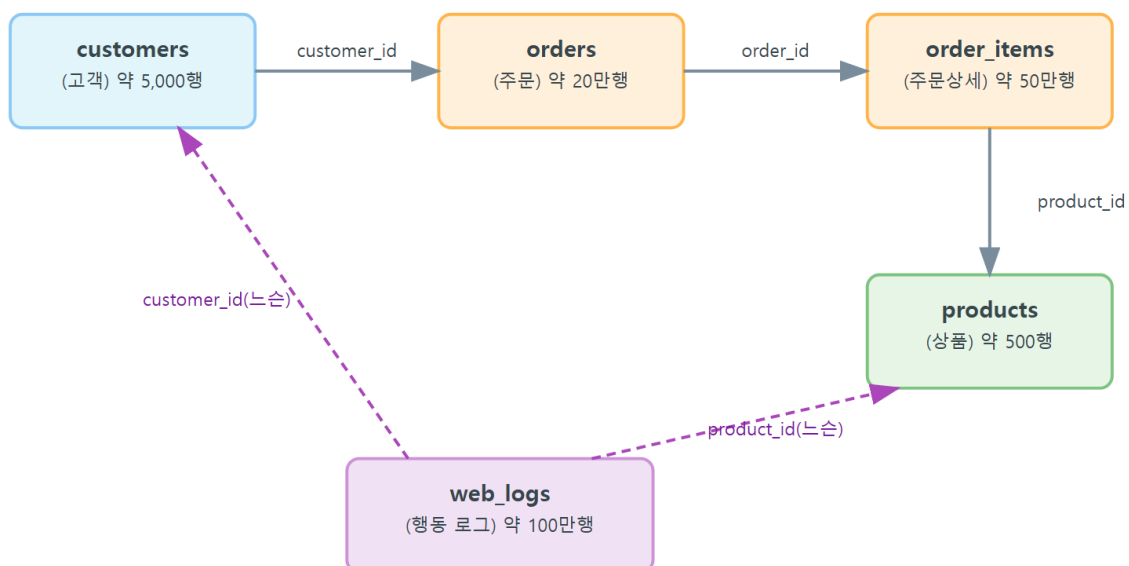
- [] `pip install` 명령을 실제 터미널에서 돌려 설치 로그를 눈으로 봤다.
- [] `pd.__version__` 이 화면에 출력됐고, 그 값이 2.2 이상임을 확인했다.
- [] 오류 없이 `import pandas as pd` 가 통과했다(빨간 에러 메시지가 없었다).

1.3 쇼핑물 데이터셋 생성: generate_shop_data.py와 시드 고정

동작 원리: 스크립트가 다섯 테이블을 만든다

이제 분석할 데이터를 만듭니다. 이 책은 실제 쇼핑물의 개인정보를 쓸 수 없으므로, 현실과 똑 닮은 **가상 쇼핑물 데이터**를 코드로 생성합니다. 제공된 `generate_shop_data.py` 를 실행하면 다섯 개의 테이블이 서로 연결된 채로 만들어집니다. 아래 그림은 다섯 테이블이 어떤 열로 이어지는지, 각 테이블이 대략 몇 행인지를 보여 줍니다.

쇼핑물 데이터셋 다섯 테이블 관계도



다섯 테이블은 다음과 같이 연결됩니다. 한 명의 **고객**(customers)이 여러 **주문**(orders)을 넣고, 한 주문은 여러 **주문 상세**(order_items) 줄을 가지며(장바구니에 담긴 상품 하나가 한 줄), 각 주문 상세는 하나의 **상품**(products)을 가리킵니다. 여기에 고객이 사이트에서 남긴 클릭·검색·장바구니·구매 같은 **웹 행동 로그**(web_logs)가 따로 쌓입니다. 이 구조가 곧 실제 쇼핑물의 데이터 구조이며, 뒤에서 `merge` 로 이 테이블들을 하나로 합쳐 분석하게 됩니다.

시드 고정: 누가 실행해도 같은 데이터

이 스크립트의 중요한 성질은 **결정론**입니다. 스크립트는 난수(무작위 수)를 써서 고객 이름·주문 시각·구매 상품을 정하지만, 그 난수의 출발점인 **시드(seed)**를 42로 고정해 둡니다.

시드 고정이란

난수 생성기는 시드라는 시작값을 주면, 그 시작값에 대해 **항상 똑같은 순서의 난수**를 뱉습니다. 스크립트는 `np.random.default_rng(42)` 와 `random.seed(42)` 로 시드를 42에 고정합니다. 그래서 이 스크립트는 누가, 언제, 어느 컴퓨터에서 실행해도 **바이트 단위로 완전히 동일한 CSV**를 만들어 냅니다. 이것을 **재현성(reproducibility)**이라고 합니다. 여러분의 실습 결과 숫자가 이 책의 출력과 정확히 일치하는 것도 이 시드 고정 덕분입니다.

실행 방법

`generate_shop_data.py` 가 있는 폴더에서 다음을 실행합니다.

```
python generate_shop_data.py
```

스크립트는 다섯 테이블을 차례로 만들며 진행 상황을 출력합니다.

```
[1/5] customers.csv 생성 중...
    완료: 5000행
[2/5] products.csv 생성 중...
    완료: 500행
[3/5] orders.csv 생성 중...
    완료: 200000행
[4/5] order_items.csv 생성 중 ...
    완료: 500000행
[5/5] web_logs.csv 생성 중 (100만행이라 다소 시간이 걸릴 수 있습니다)...
    완료: 1000000행
모든 CSV 생성 완료: ...\data
```

마지막 `web_logs.csv` 는 100만 행이라 몇 초에서 십여 초가 걸릴 수 있습니다. 끝나면 스크립트가 있는 폴더 안에 `data` 라는 하위 폴더가 생기고, 그 안에 CSV 다섯 개가 담깁니다.

직접 해 보기 — 데이터셋 생성 (직접 실행·기록)

1. `generate_shop_data.py` 가 있는 폴더에서 `python generate_shop_data.py` 를 실제로 실행하십시오.

2. 화면에 흐른 [1/5] ... [5/5] 진행 출력의 마지막 줄과, 각 테이블이 몇 행이라고 보고됐는지 옮겨 적으십시오.
3. [마지막 출력 줄]: (예: 모든 CSV 생성 완료: ...)
4. [생성 보고된 행수]: customers ()행 · products ()행 · orders ()행 · order_items ()행 · web_logs ()행
5. [생성에 걸린 대략 시간 느낌]: (예: 십여 초)

자가체크

- [] 스크립트를 실제로 실행해 [1/5] 부터 [5/5] 까지 진행 출력을 눈으로 봤다.
- [] 마지막에 모든 CSV 생성 완료 메시지가 떴다.
- [] 오류(빨간 traceback) 없이 정상 종료됐다.

1.4 다섯 파일 소개와 의도적으로 심은 품질 문제

생성된 다섯 파일이 각각 무엇을 담는지 정리하면 다음과 같습니다.

파일	대략 행 수	담는 내용	핵심 열
customers.csv	5,000	고객 정보	customer_id, name, gender, birth_date, signup_date, city, email
products.csv	500	상품 정보	product_id, product_name, category, price, cost
orders.csv	200,000	주문 헤더(누가·언제·어떤 채널·상태)	order_id, customer_id, order_datetime, channel, status
order_items.csv	500,000	주문 상세(주문 안 상품 줄)	order_item_id, order_id, product_id, quantity, unit_price, discount
web_logs.csv	1,000,000	웹 행동 로그(조회·검색·장바구니·구매)	log_id, customer_id, event_time, event_type, product_id, session_id

데이터는 일부러 지저분합니다

여기서 가장 중요한 점을 짚고 넘어갑니다. 이 데이터는 **깨끗하지 않습니다**. 생성 스크립트가 현실의 지저분한 데이터를 흉내 내려고 결측·중복·이상값·타입 오염·표기 불일치를 **의도적으로** 심어 두었습니다.

오개념: "실습 데이터인데 왜 이렇게 엉망이지? 잘못 만들어진 것 아닌가?"

아닙니다. 지저분함은 실수가 아니라 **교보재**입니다. 실제 현업 데이터는 늘 지저분하고, 데이터 분석 시간의 상당 부분이 이 지저분함을 정리하는 데 쓰입니다. 이 데이터셋에 일부러 심어진 문제의 예:

- **결측**: 고객의 성별(gender)·생일(birth_date)·이메일이 일부 비어 있습니다.
- **표기 불일치**: 도시가 **서울** 과 **서울** (앞 공백)로, 카테고리가 **의류** 와 **의류** (뒤 공백)로, 성별이 **M**·남·**F**·여 로 뒤섞여 있습니다.
- **이상값**: 주문 상세의 수량(quantity)에 음수가, 상품 가격에 0원·음수가 섞여 있습니다.
- **타입 오염**: 숫자여야 할 가격이 **39000원**·**1,200** 처럼 문자열로 들어가 있습니다.
- **미래 날짜·고아 키**: 2025년으로 튜 주문 시각, customers에 없는 고객 번호를 가진 주문도 있습니다.

깨끗한 데이터로만 연습하면 정작 현업에서 막힙니다. 그래서 이 책은 5~6장을 통째로 **정제**에 씁니다. 지금은 "이 데이터에는 고칠 것이 심어져 있고, 그걸 찾아 고치는 것도 분석의 일부"라는 사실만 받아들이면 됩니다.

이 지저분함은 앞으로 우리가 던질 질문 — *누가 얼마나 사는가, 무엇이 잘 팔리는가, 매출은 어디로 가는가* — 에 답하기 전에 반드시 정리해야 할 대상입니다.

핵심 열의 값 미리 알기

앞으로 자주 조건으로 쓰게 될 몇몇 열은 정해진 값 몇 가지만 가집니다. 미리 알아 두면 필터·집계가 수월합니다.

테이블	열	가질 수 있는 값
orders	channel(주문 채널)	web(웹), app(앱), store(매장)
orders	status(주문 상태)	paid(결제), shipped(배송중), delivered(배송완료), canceled(취소), returned(반품)
products	category(카테고리)	의류, 전자, 식품, 가구, 도서, 뷰티
web_logs	event_type(행동 종류)	view(조회), search(검색), cart(장바구니), purchase(구매)

예컨대 뒤에서 "웹으로 들어와 배송 완료된 주문"을 뽑을 때는 `channel` 이 `web` 이고 `status` 가 `delivered` 인 행을 거릅니다. 카테고리별 매출을 볼 때는 `category` 의 여섯 값이 기준 축이 됩니다. 지금은 "이런 값들이 있구나" 정도로 훑어 두면 충분합니다.

1.5 worked example: 생성부터 첫 적재(read_csv·head)까지

이제 처음부터 끝까지 한 번에 해 봅니다. 설치 → 생성 → 첫 적재까지가 이 장의 완성입니다.

먼저 데이터가 실제로 만들어졌는지 파일 목록과 크기로 확인하는 코드입니다. `data` 폴더 안의 파일을 훑어 이름과 바이트 크기를 출력합니다.

```
import os

for name in sorted(os.listdir("data")):
    size = os.path.getsize(os.path.join("data", name))
    print(f"{name}: {size:,} bytes ({size/1024/1024:.1f} MB)")
```

- `os.listdir("data")` — `data` 폴더 안의 파일 이름 목록을 가져옵니다. 판다스가 아니라 파이썬 표준 라이브러리 `os` 로 파일을 확인합니다.
- `os.path.getsize(...)` — 각 파일의 크기를 바이트 단위로 줍니다.
- `f"{size:,}"` — 천 단위마다 쉼표를 찍어 큰 숫자를 읽기 쉽게 만듭니다.

실행 결과는 다음과 같습니다(시드가 고정돼 있어 여러분의 크기도 이와 같습니다).

```
customers.csv: 335,397 bytes (0.3 MB)
order_items.csv: 16,326,730 bytes (15.6 MB)
orders.csv: 9,074,503 bytes (8.7 MB)
products.csv: 24,106 bytes (0.0 MB)
web_logs.csv: 54,464,850 bytes (51.9 MB)
```

파일이 다섯 개 다 있고, 행동 로그(web_logs.csv)가 51.9 MB로 가장 크며 상품(products.csv)이 24 KB로 가장 작습니다. 크기만 봐도 어느 테이블이 대용량인지 짐작됩니다.

이제 첫 파일을 판다스로 불러옵니다. 고객 데이터를 데이터프레임으로 읽어 앞 5행과 표 크기를 확인합니다.

```
import pandas as pd

customers = pd.read_csv("data/customers.csv")

print("shape:", customers.shape)
print(customers.head())
```

- `pd.read_csv("data/customers.csv")` — CSV 파일을 읽어 데이터프레임으로 만듭니다. 이 한 줄이 파일을 판다스 표로 바꾸는 가장 기본 명령이며, 이 책의 거의 모든 실습이 여기서 시작합니다.
- `customers.shape` — 표의 크기를 (행 수, 열 수) 튜플로 돌려줍니다.
- `customers.head()` — 표의 앞 5행만 보여 줍니다. 데이터가 제대로 들어왔는지 눈으로 확인하는 첫 습관입니다.

실행 결과는 다음과 같습니다.

```
shape: (5000, 7)
  customer_id name gender birth_date signup_date city email
0         3293 권준서   여  1954-04-29  2020-04-26  부산 user3293@example.com
1         1862 박준선   F  1980-05-19  2023-09-01  서울 user1862@example.com
2         4955 전영경   여  1980-06-08  2024-04-30  인천 user4955@example.com
3         4653 한재하   M  1971-06-09  2023-08-06  서울 user4653@example.com
4         3331 송준연   여  1971-02-01  2023-12-20  대전 user3331@example.com
```

고객 표가 5,000행 7열로 잘 들어왔습니다. 벌써 지저분함의 단서가 보입니다. `gender` 열에 `여` · `F` · `M` 이 뒤섞여 있죠? 앞서 말한 표기 불일치가 실제로 데이터에 들어 있습니다. 이런 것을 발견하고 정리하는 법을 앞으로 배웁니다.

여기까지 왔다면 실습 자리가 완성된 것입니다. 판다스가 설치됐고, 데이터가 만들어졌고, 첫 파일을 표로 불러왔습니다.

직접 해 보기 — 첫 적재 (직접 실행·기록)

위 두 코드 블록을 여러분의 환경에서 실제로 실행하고 결과를 기록하십시오.

1. 파일 목록·크기 코드를 실행하고, **customers.csv**의 크기를 적으십시오: (예: 335,397 bytes)
2. `pd.read_csv("data/customers.csv")` 로 불러온 뒤 실제 출력을 기록하십시오.
3. **[customers.shape 실제 출력]**: (예: (5000, 7))
4. **[head()의 0번 행 customer_id]**: (내 화면 실제 값)
5. **[gender 열에서 눈에 띈 표기 불일치 예 2개]**: (예: 여, M)

자가체크

- `[] data` 폴더에 CSV 다섯 개가 모두 보였다.
- `[] customers.shape` 가 (5000, 7) 로 출력됐다(내 화면으로 확인).
- `[] head()` 로 앞 5행이 표 형태로 출력됐다.
- `[] gender` 열에서 표기가 뒤섞인 것을 실제로 눈으로 확인했다.

1.6 단원 미니 프로젝트: 데이터셋 생성과 첫 적재 기록

이 단원의 마무리로, 앞에서 배운 것만으로 완성하는 작은 산출물을 만듭니다. 데이터셋 점검 노트입니다. 다섯 개 파일을 각각 불러와 크기와 앞 모습을 한 문서에 기록해, 이후 모든 장에서 참조할 "우리 데이터의 첫인상"을 남깁니다.

모범: 다섯 테이블을 한 번에 점검하는 코드

스스로 해보기

```
import os
import pandas as pd

files = [
    "customers.csv", "products.csv", "orders.csv",
    "order_items.csv", "web_logs.csv",
]

for name in files:
    path = os.path.join("data", name)
    df = pd.read_csv(path)
    size_mb = os.path.getsize(path) / 1024 / 1024
    print(f"[{name}] {df.shape[0]:,}행 x {df.shape[1]}열, {size_mb:.1f} MB")
    print(df.head(3))
    print("-" * 60)
```

- `for name in files:` — 다섯 파일 이름을 하나씩 돌며 같은 점검을 반복합니다. 이것이 바로 판다스를 코드로 다루는 이점입니다. 파일이 늘어도 목록에 이름만 추가하면 됩니다.
- `df.shape[0] / df.shape[1]` — 각각 행 수와 열 수입니다.
- `df.head(3)` — 앞 3행만 미리 봅니다. 표가 너무 넓거나 길 때 앞부분만 확인하는 습관입니다.

실행 결과(일부)는 다음과 같습니다. 각 테이블의 행수가 앞서 소개한 규모와 일치하는지 확인하십시오.

```
[customers.csv] 5,000행 x 7열, 0.3 MB
[products.csv] 500행 x 5열, 0.0 MB
[orders.csv] 200,000행 x 5열, 8.7 MB
[order_items.csv] 500,000행 x 6열, 15.6 MB
[web_logs.csv] 1,000,000행 x 6열, 51.9 MB
```

직접 만들기 (2종 1세트)

미니 프로젝트 — 데이터셋 점검 노트 (직접 실행·기록)

위 모범 코드를 바탕으로, 여러분의 환경에서 다섯 테이블을 모두 불러와 아래 점검표를 **실제 출력값**으로 채워십시오. 값을 추측하지 말고, 코드를 돌려 나온 실제 숫자를 옮겨 적습니다.

파일	내가 확인한 행 수	열 수	크기 (MB)	head(3)에서 눈에 띈 특이 점
customers.csv				(예: gender 표기 뒤섞임)
products.csv				
orders.csv				
order_items.csv				
web_logs.csv				

기록 슬롯:

- [모든 파일이 로드됐는가]: (예/아니오, 안 됐다면 오류 메시지)
- [가장 큰 테이블과 가장 작은 테이블]: () / ()
- [내가 발견한 지저분함 후보 1개]: (어느 테이블 어느 열에서 무엇을 봤는지)

자가체크

- [] 다섯 파일을 모두 `read_csv` 로 불러와 오류 없이 shape가 출력됐다.
- [] 점검표의 행 수가 customers 5,000 / products 500 / orders 200,000 / order_items 500,000 / web_logs 1,000,000과 일치했다.
- [] `head(3)` 로 각 테이블의 앞모습을 실제로 확인하고 특이점을 한 개 이상 적었다.
- [] 이 점검 노트를 다음 장에서 다시 열어 볼 수 있게 저장해 두었다.

자주 묻는 질문

Q. pip install 에서 오류가 납니다. 가장 흔한 원인은 pip 자체가 오래됐거나 파이썬 버전이 낮은 경우입니다. `python -m pip install --upgrade pip` 으로 pip을 올린 뒤 다시 설치해 보십시오. 회사·학교 네트워크라면 방화벽 때문에 설치가 막힐 수 있으니 관리자에게 문의하십시오.

Q. generate_shop_data.py 를 실행했는데 data 폴더가 안 보입니다. 스크립트는 자기 파일이 있는 위치에 `data` 폴더를 만듭니다. 터미널의 현재 위치가 아니라 스크립트가 있는 폴더를 확인하십시오. 또 스크립트 실행 중 오류(빨간 traceback)가 났다면 pandas-numpy 설치가 안 된 것일 수 있으니 설치부터 다시 확인합니다.

Q. 제 실습 결과 숫자가 이 책과 달라도 되나요? 생성 스크립트가 시드를 42로 고정하므로, 정상적으로 실행하면 책과 완전히 같은 숫자가 나와야 합니다. 다르다면 스크립트를 수정했거나 다

른 데이터를 쓰고 있을 가능성이 큼니다. 이 책의 실측값과 대조하는 것이 곧 여러분의 실습이 제대로 됐는지 확인하는 방법입니다.

Q. 왜 굳이 데이터를 직접 만들게 하나요? 그냥 CSV를 주면 안 되나요? 직접 생성해 보면 "데이터가 어떻게 만들어지는지"와 "왜 지저분한지(무엇이 일부러 심겼는지)"를 이해하게 됩니다. 또 시드 고정 덕분에 누구나 같은 데이터를 재현할 수 있어, 실습 결과를 서로 정확히 대조할 수 있습니다.

Q. web_logs.csv가 너무 큼니다. 매번 다 불러와야 하나요? 아닙니다. 100만 행 로그는 대용량 처리 단위(11장)에서 본격적으로 다룹니다. 그전까지의 실습은 대부분 고객·상품·주문·주문 상세 네 테이블로 진행되며, 로그는 필요할 때 `nrows · usecols` 로 일부만 불러오는 법도 배웁니다. 지금은 파일이 만들어졌다는 것만 확인하면 됩니다.

Q. pd, np 같은 별명은 꼭 써야 하나요? 문법상 강제는 아니지만, `import pandas as pd · import numpy as np` 는 전 세계 코드가 따르는 사실상의 표준 관례입니다. 남의 코드를 읽고 도움을 구할 때 혼선을 줄이려면 이 관례를 그대로 따르는 것이 좋습니다. 이 책도 항상 `pd · np` 를 씁니다.

Q. 주피터 노트북에서 해도 되나요? 됩니다. 이 책의 코드는 파이썬 스크립트(.py)로도, 주피터 노트북(.ipynb)으로도 똑같이 동작합니다. 노트북을 쓰면 표가 보기 좋게 출력되고 실행 결과를 셀 단위로 남길 수 있어, 실습 기록을 남기기에 편리합니다. 어느 쪽이든 `data` 폴더의 위치만 코드 기준으로 맞으면 됩니다.

이 점검 노트가 우리 분석의 출발점입니다. 이 장에서 우리는 판다스를 설치하고, 시드가 고정된 생성 스크립트로 다섯 테이블을 직접 만들었으며, `read_csv · head · shape` 로 첫 적재까지 손으로 해냈습니다. 그리고 이 데이터가 일부러 지저분하게 만들어졌다는 것, 그 지저분함을 고치는 것도 분석의 일부라는 것을 확인했습니다.

다음 장에서는 이 데이터의 구조를 더 깊이 들여다보며, `info · describe` 로 각 테이블의 규모·타입·품질을 본격적으로 훑어봅니다.

2장 판다스 데이터 구조와 불러오기

1장에서 쇼핑물 데이터 다섯 개를 만들고 `pd.read_csv` 로 고객 파일을 처음 불러왔습니다. 그런데 "불러왔다"는 것이 정확히 무엇일까요? `customers` 라는 변수 안에 들어간 그 표는 어떤 구조이고, 왜 어떤 열은 숫자로 어떤 열은 문자로 다뤄질까요? 이 장은 판다스의 뼈대를 세우는 장입니다.

데이터 타입

저장소

세 가지를 배웁니다. 먼저 판다스의 두 핵심 자료구조인 **시리즈**와 **데이터프레임**이 무엇인지, 다음으로 파일을 제대로 불러오고 저장하는 법, 끝으로 낯선 데이터를 빠르게 훑어보는 법입니다. 이 세 가지가 손에 익으면, 어떤 데이터를 받아도 "무엇이 들었고 어디가 지저분한지"를 30분 안에 파악할 수 있습니다.

이 장의 학습 목표

행번호 저장할 수 있는 데이터 타입 저장된 값

• `customers` 데이터프레임에서 한 열을 꺼내면 **시리즈**가 됨을 확인하고 그 **시리즈의**

`index-dtype-values`를 출력할 수 있다 [→G2] Series 애기

• `dtypes-shape-columns-index`로 표의 구조와 각 열의 자료형을 파악할 수 있다 [→G2] DataFrame 애기

• `read_csv`에서 `parse_dates-dtype-usecols`를 지정해 `orders` 등을 제대로 불러오고 `to_csv`로 저장할 수 있다 [→G1] `read_csv`

• `info-describe-value_counts-isna().sum()`으로 다섯 테이블의 규모·요약통계·결측을 훑어 품

질 점검표를 작성할 수 있다 [→G2]

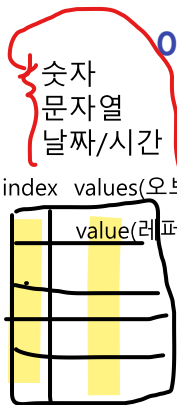
변수: `dtype(values 데이터 타입 저장하고 있는 변수)`

시리즈 오브젝트 : 컬럼 하나 저장

- 변수
- 메소드

변수

메소드



선수 확인: 이 장은 1장에서 `generate_shop_data.py` 로 `data` 폴더에 CSV 다섯 개를 만들고, `pd.read_csv` 로 한 파일을 불러온 상태를 가정합니다. 아직이라면 1장의 미니 프로젝트(데이터셋 점검 노트)부터 끝내고 오시기 바랍니다.

2.1 시리즈와 데이터프레임: 값·인덱스·자료형

도입: 불러온 표의 정체를 알아야 하는 이유

1장에서 `customers = pd.read_csv(...)` 로 얻은 것이 데이터프레임입니다. 그런데 앞으로 우리는 이 표에서 열 하나를 꺼내 계산하고, 특정 행만 골라내고, 열별 타입을 바로잡는 일을 수없이 합니다. 그때마다 "지금 내가 다루는 게 표 전체인가, 열 하나인가, 그 열의 타입은 무엇인가"를 알아야 오류 없이 다룰 수 있습니다. 그래서 구조를 먼저 세웁니다. (선수 개념: 1장의 판다스 소개에서 시리즈·데이터프레임을 이름만 미리 봤습니다. 여기서 제대로 배웁니다.)

정의

이 절의 개념은 **시리즈와 데이터프레임**(Series and DataFrame)입니다.

핵심 정의



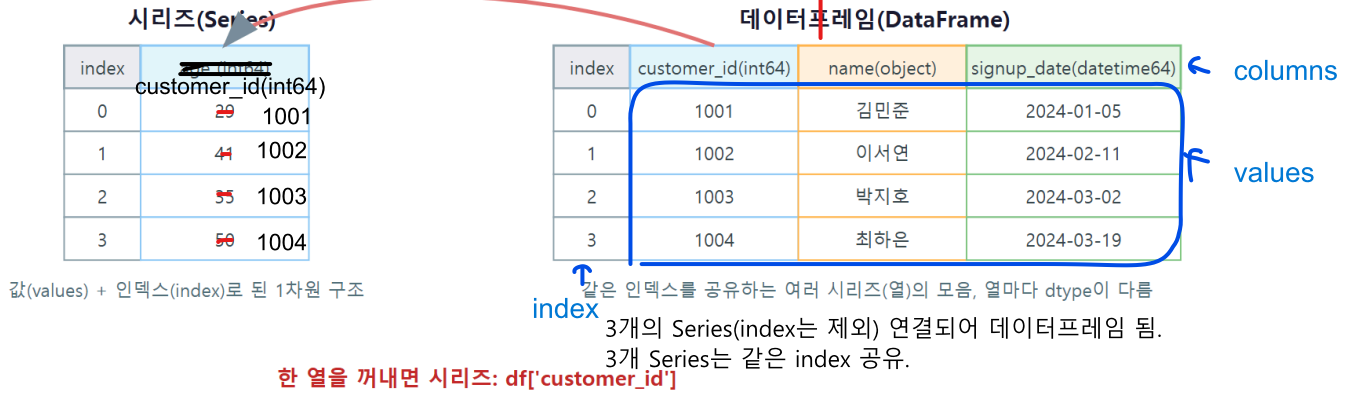
- **시리즈(Series)**: 값의 배열에 **인덱스(index)**라는 라벨이 붙은 **1차원 자료구조**입니다. 데이터프레임에서 열 하나를 꺼내면 시리즈가 됩니다.
- **데이터프레임(DataFrame)**: 같은 인덱스를 공유하는 **여러 시리즈(열)**를 모은 **2차원 표**입니다. 각 열은 하나의 시리즈입니다.
- **인덱스(index)**: 행을 식별하는 **라벨**입니다. 기본은 0부터의 **정수**지만, **고객 번호**나 **날짜** 같은 의미 있는 값으로 바꿀 수 있습니다. int64가 기본(64bit)
- **자료형(dtype)**: 열이 담는 **값의 종류**입니다. **int64 (정수)** · **float64 (실수)** · **object (주로 문자열)** · **bool** · **datetime64 (날짜/시각)** · **category (범주)** 등이 있습니다.

동작 원리

데이터프레임은 "인덱스를 공유하는 시리즈들의 묶음"입니다. 아래 그림은 시리즈 하나의 구조 (인덱스+값)와, 그런 시리즈 여러 개가 같은 인덱스로 정렬돼 데이터프레임이 되는 모습을 보여줍니다. 한 열을 꺼내면 시리즈가 되어 떨어져 나옵니다.

index	name(object)
0	김민준
...	...

시리즈와 데이터프레임 구조도



자료형이 같은 열이 올 수 있음.

핵심은 **열마다 자료형이 다르다**는 점입니다. 고객 표에서 `customer_id`는 정수(`int64`)지만 `name`, `city`는 문자열(`object`)입니다. 판다스는 CSV를 읽을 때 각 열의 값을 보고 자료형을 자동으로 추론합니다. 이 추론이 대체로 맞지만 늘 옳지는 않으며(예: `날짜`를 문자열로 읽음), 그래서 2.2절에서 자료형을 직접 지정하는 법을 배웁니다.

데이터는 다 문자열. `read_csv`가 각각 컬럼을 읽고 데이터 타입을 정함.

worked example: 시리즈와 데이터프레임 구조 확인하기

먼저 작은 데이터프레임을 딕셔너리로 직접 만들어 구조를 봅니다.

```
import pandas as pd

sample = pd.DataFrame({
    "product_name": ["코어 티셔츠", "베이직 노트북", "데일리 원두커피"],
    "category": ["의류", "전자", "식품"],
    "price": [39000, 89000, 15000],
})

print(sample)
print("index:", sample.index.tolist())
print("columns:", sample.columns.tolist())
print(sample.dtypes)
```

- `pd.DataFrame({...})` — 딕셔너리의 키가 열 이름, 값(리스트)이 열의 값이 되어 표가 만들어집니다. 파일 없이 손으로 작은 표를 만들 때 씁니다.
- `sample.index` — 행 라벨입니다. 따로 지정하지 않아 0·1·2 정수 인덱스가 붙습니다.
- `sample.columns` — 열 이름 목록입니다.
- `sample.dtypes` — 열별 자료형입니다. 문자열 열은 `object`, 숫자 열은 `int64`로 추론됩니다.

실행 결과는 다음과 같습니다.

```
product_name category price
0   코어 티셔츠   의류  39000
1   베이직 노트북 전자  89000
2   데일리 원두커피 식품 15000
index: [0, 1, 2]
columns: ['product_name', 'category', 'price']
product_name    object
category         object
price           int64
dtype: object
```

이제 실제 고객 데이터에서 열 하나를 꺼내 시리즈임을 확인합니다.

```
customers = pd.read_csv("data/customers.csv")

city = customers["city"]
print("type:", type(city).__name__)
print("dtype:", city.dtype)
print("앞 5개 값:", city.head().tolist())

print("shape:", customers.shape)
print(customers.dtypes)
```

- `customers["city"]` — 열 하나를 꺼냅니다. 결과는 데이터프레임이 아니라 **시리즈**입니다.
- `type(city).__name__` — 이 값의 실제 타입 이름을 확인합니다. `Series` 가 나옵니다.
- `city.head().tolist()` — 앞 5개 값을 파이썬 리스트로 봅니다.
- `customers.dtypes` — 표 전체의 열별 자료형을 한눈에 봅니다.

실행 결과는 다음과 같습니다.

```
type: Series
dtype: object
앞 5개 값: [' 부산', '서울', '인천', '서울', '대전']
shape: (5000, 7)
customer_id    int64
name           object
gender         object
birth_date     object
signup_date    object
city           object
email          object
dtype: object
```

열 하나를 꺼내니 타입이 `Series` 로 확인됐습니다. 그리고 `city` 의 첫 값이 ' 부산' — 앞에 공백이 붙어 있죠? 1장에서 예고한 표기 불일치가 시리즈 값에서 그대로 드러납니다. 또 `dtypes` 를

보면 `customer_id` 만 정수고 나머지는 전부 `object` 입니다. 특히 `birth_date` · `signup_date` 가 날짜인데 `object` (문자열)로 읽혔습니다. 이것이 다음 절에서 바로잡을 문제입니다.

즉시 연습 (2종 1세트)

직접 해 보기 — 시리즈와 구조 확인 (직접 실행·기록)

`products.csv`를 불러와 아래를 실제로 실행하고 결과를 기록하십시오.

```
products = pd.read_csv("data/products.csv")
# 1) category 열을 꺼내 타입과 dtype 확인
# 2) products.shape, products.columns, products.dtypes 출력
```

- `[products["category"]]`의 type: (예: `Series`)
- `[products.shape]` 실제 출력: `( )`
- `[products.dtypes]`에서 `price` 열의 자료형: (예: `object` 또는 `int64` — 실제로 확인하고, 왜 그런지 한 줄로 추측)

자가체크

- `[ ]` 열 하나를 꺼내니 `Series` 가 나오는 것을 실제 출력으로 확인했다.
- `[ ]` `products.shape` 가 `(500, 5)` 로 나왔다(내 화면으로 확인).
- `[ ]` `dtypes` 에서 각 열의 자료형을 읽고, `price` 가 왜 그 타입인지 한 줄로 적었다.

흔한 오류와 교정

오개념 세 가지

- "열 하나는 리스트다" — 아닙니다. 데이터프레임의 열 하나는 인덱스가 붙은 **시리즈**입니다. 그래서 `.head()` · `.value_counts()` 같은 판다스 메서드를 바로 쓸 수 있습니다.
- "object면 무조건 문자열이다" — 대체로 문자열이지만, 숫자가 문자열로 오염돼 섞이면 숫자 열도 `object` 가 됩니다(`products`의 `price` 가 그렇습니다). `object` 는 "판다스가 순수 숫자로 확신하지 못한 열"이라는 신호입니다.
- "인덱스는 그냥 행 번호다" — 기본은 0부터의 정수지만, 인덱스는 행을 식별하는 **라벨**이며 고객 번호·날짜로 바꿀 수 있습니다(3장에서 배웁니다).

2.2 데이터 불러오기: read_csv 옵션 (dtype·parse_dates·encoding)

도입: 그냥 읽으면 날짜가 문자열로 남는다

날짜 처리 힘들다.

오브젝트면 편함 왜? 메소드로 처리

2.1절에서 봤듯, read_csv 를 옵션 없이 쓰면 order_datetime · birth_date 같은 날짜가 object (문자열)로 읽힙니다. 문자열 날짜로는 "6월 주문만 뽑기"나 "월별 매출 집계" 같은 시간 계산이 안 됩니다. 그래서 불러올 때부터 제대로 지정하는 것이 분석의 첫 품질 관문입니다.

정의

이 절과 다음 절의 개념은 데이터 불러오기와 저장(reading and writing data)입니다.

pd.read_csv 는 CSV를 데이터프레임으로 읽되, 옵션으로 자료형·날짜 파싱·인코딩을 바로잡습니다. 자주 쓰는 옵션은 다음과 같습니다.

옵션	하는 일	예
parse_dates	지정한 열을 datetime으로 파싱 오브젝트 datetime으로 파싱 변환 자동 데이터타입 지정	parse_dates=["order_datetime"]
dtype	열별 자료형을 명시 문자열 DF CSV 자동 데이터타입 지정	dtype={"customer_id": "int64"} key value 변경할 데이터타입
usecols	필요한 열만 읽기	usecols=["order_id", "channel"] 추출할 컬럼 지정
nrows	앞 몇 행만 읽기(미리보기) 데이터가 너무 클 때 다 불러오면 데이터가 너무 많음...	nrows=1000
na_values	누락된 값 결측으로 취급할 값 지정 누락된 값을 어떤 값으로 채울지	na_values=["", "N/A"] 결측값 채울 값
encoding	문자 인코딩 지정 * 처음부터 윈도우를 utf-8로 해두는 것이 필수.(윈도우는 cp949)	encoding="utf-8" 또는 "cp949"

쓰레기데이터 많을 때 수동 지정

동작 원리: 왜 날짜를 datetime으로 파싱하나

날짜 + 시간
오브젝트 -> 날짜와 시간을 다루는 메소드 존재.

"2024-06-15 13:20:00" 은 사람 눈에는 날짜지만, 판다스에는 그냥 글자입니다. 글자로 두면 크기 비교·기간 슬라이싱·월별 집계가 문자열 사전순으로 어긋나거나 아예 불가능합니다.

parse_dates 로 datetime 타입으로 바꾸면 판다스가 이것을 진짜 시각으로 이해해, dt.month 로 월을 뽑고 특정 기간을 슬라이싱할 수 있게 됩니다(자세한 시계열 도구는 뒤 단원에서 배웁니다).

인코딩은 한글이 깨질 때 만나는 문제입니다. 우리 데이터는 UTF-8로 저장돼 있어 별도 지정 없이 잘 읽히지만, 윈도우에서 만든 일부 파일은 cp949 라 한글이 깨지면 encoding="cp949" 를 줘야 합니다. 한글이 깨지면 데이터가 잘못된 게 아니라 인코딩 지정 문제인 경우가 대부분입니다.

worked example: parse_dates·dtype·usecols로 제대로 불러오기

주문 데이터를 옵션 없이 읽었을 때와 parse_dates 로 읽었을 때를 비교합니다.

```
import pandas as pd

# 옵션 없이
orders_raw = pd.read_csv("data/orders.csv")
print("옵션 없이:", orders_raw["order_datetime"].dtype)

# parse_dates 지정
orders = pd.read_csv(
    "data/orders.csv",
    parse_dates=["order_datetime"],
)
print("parse_dates:", orders["order_datetime"].dtype)
print(orders.dtypes)
```

- orders_raw["order_datetime"].dtype — 옵션 없이 읽으면 날짜 열이 object 로 남습니다.
- parse_dates=["order_datetime"] — 이 열을 datetime으로 파싱하라는 지시입니다. 리스트에 여러 열을 넣을 수도 있습니다.
- 파싱 후 dtype이 datetime64[ns] 로 바뀝니다.

실행 결과는 다음과 같습니다.

```
옵션 없이: object
parse_dates: datetime64[ns]
order_id          int64
customer_id       int64
order_datetime    datetime64[ns]
channel           object
status            object
dtype: object
```

같은 파일인데 order_datetime 이 object 에서 datetime64[ns] 로 바뀌었습니다. 이제 이 열로 시간 계산이 가능합니다.

다음은 대용량 파일에서 필요한 열·앞부분만 빠르게 보는 법입니다. 20만 행 주문에서 세 열, 앞 5행만 읽습니다.

```
small = pd.read_csv(
    "data/orders.csv",
    usecols=["order_id", "channel", "status"],
    nrows=5,
)
print(small)
print("shape:", small.shape)
```

실행 결과는 다음과 같습니다.

```
   order_id channel  status
0    25463   store  delivered
1   171088    web  delivered
2    27437    app  delivered
3    98425   store  delivered
4    22325    app  delivered
shape: (5, 3)
```

`usecols` 로 세 열만, `nrows` 로 5행만 읽어 순식간에 표의 모습을 미리 봤습니다. 100만 행 로그를 다룰 때 특히 유용합니다.

즉시 연습 (2종 1세트)

직접 해 보기 — 옵션으로 제대로 불러오기 (직접 실행·기록)

`customers.csv`를 불러오되, `parse_dates` 로 `signup_date` 를 `datetime`으로 파싱해 보십시오.

```
customers = pd.read_csv(
    "data/customers.csv",
    parse_dates=["signup_date"], -> datetime 변환
)
# customers["signup_date"].dtype 를 출력
# usecols로 ["customer_id", "city"] 두 열만 읽어 shape 확인
```

- **[signup_date의 dtype]:** (예: `datetime64[ns]`)
- **[두 열만 읽은 shape]:** `( )`
- **[birth_date도 parse_dates에 넣어 봤을 때 문제가 있었나]:** (결측/이상값 때문에 경고나 `NaT`가 생기는지 실제로 관찰)

자가체크

- `[ ] parse_dates` 전에는 `object`, 후에는 `datetime64[ns]` 임을 실제 출력으로 비교했다.
- `[ ] usecols` 로 두 열만 읽어 `shape` 의 열 수가 2로 줄어든 것을 확인했다.
- `[ ]` 옵션 지정이 왜 "분석의 첫 품질 관문"인지 한 줄로 설명할 수 있다.



2.3 데이터 저장: to_csv와 index 처리

동작 원리: 저장할 때 인덱스를 함께 저장하나

분석 결과나 정제된 표를 파일로 남길 때는 `to_csv` 를 씁니다. 여기서 흔히 놓치는 것이 **인덱스** 입니다. `to_csv` 는 기본으로 인덱스도 한 열로 저장하므로, 의미 없는 0~1~2 정수 인덱스라면 `index=False` 로 빼야 파일이 깔끔합니다.

worked example: 저장하고 다시 불러와 확인하기

앞서 읽은 5행짜리 `small` 을 저장했다가 다시 불러와 같은지 확인합니다.

```
small.to_csv("orders_sample.csv", index=False)

reload = pd.read_csv("orders_sample.csv")
print("재적재 shape:", reload.shape)
print("원본과 동일?:", small.reset_index(drop=True).equals(reload))
```

- `small.to_csv("orders_sample.csv", index=False)` — 표를 CSV로 저장하되 인덱스는 저장하지 않습니다.
- `reset_index(drop=True)` — 비교 전에 인덱스를 0부터로 새로 매겨 두 표의 인덱스를 맞춥니다.
- `.equals(reload)` — 두 데이터프레임이 값·구조 모두 같은지 True/False로 판정합니다.

실행 결과는 다음과 같습니다.

```
재적재 shape: (5, 3)
원본과 동일?: True
```

저장 → 재적재 후 표가 완전히 동일하게 복원됐습니다. 만약 `index=False` 를 빼면 재적재 시 `Unnamed: 0` 라는 정수 인덱스 열이 하나 더 생겨 열 수가 어긋나니, 저장 습관으로 `index=False` 를 기억해 두십시오. Parquet 형식으로 저장하는 `to_parquet` 도 있는데, 이는 대용량 단원에서 다룹니다.

오개념: "read_csv의 자동 추론과 to_csv의 기본값을 믿으면 된다"

- 자동 타입 추론은 틀릴 수 있습니다. 날짜·ID는 명시적으로 `parse_dates · dtype` 으로 지정하는 게 안전합니다.

- `to_csv` 는 기본으로 **인덱스도 저장**합니다. 의미 없는 정수 인덱스는 `index=False` 로 빼아 다음에 읽을 때 이상한 열이 생기지 않습니다.

2.4 데이터 훑어보기: `head·info·describe·value_counts`

도입: **분석 전 정찰이 필요하다**

낮선 데이터를 받으면 곧바로 분석에 뛰어들면 안 됩니다. 규모가 얼마나 되는지, 어떤 타입인지, 어디가 비었는지, 이상한 값은 없는지를 **먼저 정찰**해야 뒤의 분석이 헛발질하지 않습니다. (선수 개념: 2.1의 자료형·구조와 2.2의 제대로 불러오기가 밑바탕입니다.)

정의

이 절의 개념은 **데이터 훑어보기(inspecting data)**입니다. 데이터의 규모·구조·품질을 빠르게 파악하는 도구 모음입니다.

훑어보기 도구

데이터 값 확인

- `head()` / `tail()` — 앞/뒤 몇 행을 눈으로 봅니다.
- `shape` — 행·열 수를 봅니다.
- `info()` — 각 열의 이름·비결측 개수·자료형·메모리를 한 번에 봅니다. **결측 파악의 핵심**입니다.

비누락
사용량
- `describe()` — 숫자 열의 count 개수·mean 평균·std 표준편차·25% 사분위수·50% 최솟값·75% 최댓값·max을 봅니다.
- `value_counts()` — 범주·문자열 열에서 어떤 값이 몇 번 나오는지 셉니다.
- `nunique()` — 고유티 값 개수를 셉니다.
- `isna().sum()` — 열별 결측 수를 셉니다.

컬럼별

동작 원리: `info`와 `describe`가 서로 다른 것을 본다

`info()` 는 모든 열의 구조(비결측 개수·타입)를, `describe()` 는 기본으로 **숫자 열만**의 요약통계를 봅니다. 그래서 둘은 짝입니다. `info()` 로 "어느 열이 비었나"를 보고, `describe()` 로 "숫자 열에 이상한 최댓값·음수 최솟값이 있나"를 봅니다. 문자열·범주 열은 `describe()` 가 잘 요약하지 못하므로 `value_counts()` 로 따로 봅니다.

worked example: 주문·주문 상세·상품 훑어보기

먼저 `info()` 로 주문 표의 결측을 찾습니다.

```
orders = pd.read_csv("data/orders.csv", parse_dates=["order_datetime"])
orders.info()
```

실행 결과는 다음과 같습니다.

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 200000 entries, 0 to 199999
Data columns (total 5 columns):
#   Column          Non-Null Count  Dtype
---  -
0   order_id        200000 non-null  int64
1   customer_id     200000 non-null  int64
2   order_datetime  198067 non-null  datetime64[ns]
3   channel         200000 non-null  object
4   status          200000 non-null  object
dtypes: datetime64[ns](1), int64(2), object(2)
memory usage: 7.6+ MB
```

- **RangeIndex: 200000 entries** — 전체 20만 행입니다.
- **order_datetime 198067 non-null** — 이 열만 비결측이 198,067개입니다. 전체 20만에서 빼면 **1,933개가 결측**입니다. 다른 열은 모두 200,000이라 결측이 없습니다.
- **Dtype 열** — **order_datetime** 이 **datetime64[ns]** 로 제대로 파싱됐음이 보입니다.

`info()` 의 **Non-Null Count** 가 전체 행수보다 작으면 그 차이가 결측 수입니다. 이제 `describe()` 로 주문 상세의 숫자 열을 봅니다.

```
order_items = pd.read_csv("data/order_items.csv")
print(order_items[["quantity", "unit_price", "discount"]].describe())
```

실행 결과는 다음과 같습니다.

	quantity	unit_price	discount
count	500000.000000	484887.000000	500000.000000
mean	2.997008	50925.557707	0.249688
std	1.418193	48908.412198	0.144496
min	-2.000000	1000.000000	0.000000
25%	2.000000	15100.000000	0.120000
50%	3.000000	27000.000000	0.250000
75%	4.000000	73700.000000	0.370000
max	5.000000	291000.000000	0.500000

여기서 두 가지 품질 신호가 잡힙니다. 첫째, `quantity`의 `min`이 -2입니다. 수량이 음수일 수는 없으니 이상값입니다. 둘째, `unit_price`의 `count`가 **484,887**로 전체 50만보다 작습니다. 즉 15,113개가 결측입니다. `describe()` 한 번으로 이상값과 결측을 동시에 잡았습니다.

마지막으로 범주 열의 표기 불일치를 `value_counts()`로 봅니다.

```
products = pd.read_csv("data/products.csv")
print(products["category"].value_counts())
print("고유값 개수:", products["category"].nunique())
```

실행 결과는 다음과 같습니다.

```
category
도서      90
전자      79
식품      75
가구      71
의류      70
뷰티      68
전자      13
의류       9
가구       9
도서       7
뷰티       5
식품       4
Name: count, dtype: int64
고유값 개수: 12
```

카테고리는 6종류여야 하는데 `value_counts`가 12줄을 냈습니다. 전자 (79)와 전자 (13)가 따로 세어지고 있죠? 뒤쪽 값들은 '전자'처럼 공백이 붙은 표기라 판다스가 다른 값으로 봅니다.

`nunique()`도 6이 아니라 **12**로 나옵니다. 이 표기 불일치를 정리하지 않고 카테고리별 매출을 집계하면 같은 카테고리가 둘로 갈라져 잘못된 결과가 나옵니다. 이것이 6장에서 문자열 정제를 배우는 이유입니다.

즉시 연습 (2종 1세트)

직접 해 보기 — 고객 데이터 훑어보기 (직접 실행·기록)

`customers.csv`를 불러와 결측을 집계하십시오.

```
customers = pd.read_csv("data/customers.csv")
print(customers.isna().sum())
```

- **[customers.isna().sum() 실제 출력]:** gender () · birth_date () · email (), 나머지 결측은?
- **[customers.info()에서 결측이 있는 열 목록]:** ()
- **[gender 열 value_counts() 실제 출력]:** (M·F·남·여가 어떻게 갈라져 세어지는지)

자가체크

- [] isna().sum() 으로 gender 304, birth_date 384, email 248 같은 결측 수를 실제로 확인했다.
- [] gender 의 value_counts() 에서 M·남·F·여 가 별개로 세어지는 표기 불일치를 봤다.
- [] info() · describe() · value_counts() 가 각각 무엇을 보는 도구인지 구분해 설명할 수 있다.

흔한 오류와 교정

오개념

- **"describe()면 모든 열을 요약해 준다"** — 기본은 숫자 열만 요약합니다. 문자열·범주 열은 value_counts() · unique() 로 따로 봐야 합니다.
- **"결측 수를 눈으로 세면 된다"** — 20만 행을 눈으로 셀 수는 없습니다. info() 의 Non-Null Count 나 isna().sum() 으로 셉니다.
- **"훑어보기는 시간 낭비다"** — 정반대입니다. 여기서 이상값·결측·표기 불일치를 미리 잡아두면, 뒤의 잘못된 분석과 재작업을 막습니다.

2.5 worked example: 다섯 테이블 적재와 개요 파악

이제 이 장의 도구를 모아, 다섯 테이블을 각각 알맞게 불러와 개요를 한 번에 파악합니다. 각 테이블마다 날짜 열은 parse_dates 로 파싱해 불러옵니다.

```
import pandas as pd

customers = pd.read_csv("data/customers.csv")
products = pd.read_csv("data/products.csv")
orders = pd.read_csv("data/orders.csv", parse_dates=["order_datetime"])
order_items = pd.read_csv("data/order_items.csv")
web_logs = pd.read_csv("data/web_logs.csv", parse_dates=["event_time"])

tables = {
    "customers": customers, "products": products, "orders": orders,
    "order_items": order_items, "web_logs": web_logs,
}

for name, df in tables.items():
    na_total = int(df.isna().sum().sum())
    print(f"[{name}] {df.shape[0]:,}행 x {df.shape[1]}열, 결측 합계 {na_total:,}개")
```

- 다섯 표를 딕셔너리에 담아 하나의 반복문으로 개요를 냅니다.
- `df.isna().sum().sum()` — `isna()` 는 칸별 True/False, 첫 `sum()` 은 열별 결측 수, 둘째 `sum()` 은 표 전체 결측 합계입니다.
- 날짜 열(`order_datetime` · `event_time`)은 `parse_dates` 로 파싱해 처음부터 datetime으로 다룹니다.

실행 결과는 다음과 같습니다.

```
[customers] 5,000행 x 7열, 결측 합계 936개
[products] 500행 x 5열, 결측 합계 0개
[orders] 200,000행 x 5열, 결측 합계 1,933개
[order_items] 500,000행 x 6열, 결측 합계 15,113개
[web_logs] 1,000,000행 x 6열, 결측 합계 550,247개
```

한 번에 다섯 테이블의 규모와 결측 규모가 보입니다. 고객 표에 936개(gender 304 + birth_date 384 + email 248), 주문에 1,933개, 주문 상세에 15,113개, 로그에 55만여 개의 결측이 있습니다. 로그의 결측이 유독 큰 것은 두 가지 때문입니다. 비로그인(익명) 이벤트라 고객 번호(customer_id)가 비어 있고, 조회가 아닌 검색(search) 이벤트에는 상품 번호(product_id)가 없습니다. 즉 이 결측은 실수가 아니라 "익명 방문"·"상품과 무관한 검색"이라는 의미 있는 결측입니다. 이렇게 **결측이 오류인지 의미인지 구분**하는 것이 정제의 첫 판단입니다(5장에서 배웁니다).

2.6 단원 미니 프로젝트: 데이터 품질 점검표 작성

이 단원의 마무리로, 다섯 테이블 전체의 **데이터 품질 점검표**를 작성합니다. 이 점검표는 5~6장 정제 단원과 종합 프로젝트의 분석 단계에서 "무엇을 고쳐야 하는지"의 지도 역할을 합니다.

모범: 품질 이슈를 항목으로 정리

앞 절들에서 발견한 이슈를 표로 정리하면 다음과 같습니다(각자 실제 출력으로 채웁니다).

테이블	열	발견한 이슈	근거(도구·수치)
customers	gender	결측 + 표기 불일치(M/남/F/여)	isna().sum()=304, value_counts 4종
customers	birth_date	결측	isna().sum()=384
customers	city	앞뒤 공백 오염(서울/서울)	head().value_counts
orders	order_datetime	결측 + 미래(2025) 이상 값	info() 198,067 non-null
order_items	quantity	음수 이상값	describe() min=-2
order_items	unit_price	결측	describe() count=484,887
products	category	뒤 공백 표기 불일치	value_counts 12줄, nunique=12
products	price	문자열 오염(39000원) + 0/음수 이상값	dtype=object

직접 만들기 (2종 1세트)

미니 프로젝트 — 데이터 품질 점검표 (직접 실행·기록)

다섯 테이블을 모두 불러와, 각 테이블에 대해 `info() · describe() · value_counts() · isna().sum()`을 실제로 돌리고, 발견한 품질 이슈를 **최소 5개** 아래 표에 채워주세요. 각 이슈에는 반드시 근거(어느 도구가 낸 어떤 수치)를 적습니다. 값은 추측하지 말고 실제 출력을 옮깁니다.

테이블	열	내가 발견한 이슈	근거(도구·실제 수치)

기록 슬롯:

- [결측이 가장 많은 테이블·열]: ()
- [숫자 열에서 발견한 이상값(음수·불가능한 값) 예 1개]: ()
- [표기 불일치로 value_counts가 갈라진 열 1개]: ()
- [web_logs의 결측이 큰 이유(내 해석)]: ()

자가체크

- [] 다섯 테이블 각각에 info() 를 돌려 결측 있는 열을 지목했다.
- [] describe() 로 음수·불가능한 최솟값/최댓값을 최소 1개 짚었다.
- [] value_counts() 로 표기 불일치가 있는 열을 최소 1개 찾았다.
- [] 점검표에 이슈를 5개 이상, 각각 근거 수치와 함께 채웠다.
- [] 이 점검표를 저장해 5장 정제에서 다시 열어 볼 수 있게 했다.

자주 묻는 질문

Q. parse_dates 를 넣었는데 어떤 날짜는 datetime으로 안 바뀝니다. 날짜 형식이 뒤섞였거나 (예: 미래 날짜·잘못된 값), 결측이 있으면 판다스가 그 열을 완전히 파싱하지 못하고 경고를 낼 수 있습니다. 이럴 때는 우선 문자열로 읽은 뒤 pd.to_datetime(..., errors="coerce") 로 변환하며 실패를 결측(NaT)으로 처리하는 방법이 있습니다(자세한 내용은 시계열 단원에서 다룹니다).

Q. info() 와 describe() , 어느 것을 먼저 봐야 하나요? info() 를 먼저 봅니다. 전체 규모·열별 결측·자료형을 한눈에 파악한 뒤, 숫자 열의 이상값이 궁금하면 describe() 로 넘어가는 순서가 자연스럽습니다. 문자열·범주 열은 value_counts() 로 따로 봅니다.

Q. object 타입 열은 전부 문제인가요? 아닙니다. 이름·이메일처럼 원래 문자열인 열은 object 가 정상입니다. 다만 price 처럼 숫자여야 하는데 object 인 열은 오염 신호이므로 정제 대상입니다. 즉 object 자체가 문제가 아니라, "숫자·날짜여야 하는 열이 object 인가"를 봐야 합니다.

Q. 결측을 발견하면 바로 채워야 하나요? 아닙니다. 이 장은 발견·기록까지입니다. 결측을 버릴지·채울지·남길지는 그 열의 의미(무응답인지 계산 오류인지)에 달려 있어, 5장 정제 단원에서 판단합니다. 지금은 품질 점검표에 "어디에 얼마나 있는지"만 정확히 적어 두면 됩니다.

품질 점검표가 완성되면, 우리는 이제 "이 데이터에 무엇이 들었고 어디가 지저분한지"를 손에 쥔 것입니다. 다음 장에서는 이 데이터에서 원하는 부분만 정확히 뽑아내는 선택·인덱싱을 배웁니다.

3장 데이터 선택과 인덱싱

2장에서 다섯 테이블을 불러와 구조와 품질을 파악했습니다. 이제 그 표에서 **원하는 부분만 정확히 꺼내는** 법을 배웁니다. "서울에 사는 고객의 이름과 가입일만", "가격이 비싼 상위 10개 상품만", "1024번 고객의 정보만" — 분석은 대부분 이렇게 전체에서 일부를 골라내는 데서 시작합니다.

이 장은 두 개념을 다룹니다. 열과 행을 콕 집어 꺼내는 **loc-iloc 선택**, 그리고 의미 있는 열을 행 라벨로 삼아 조회를 편하게 만드는 **인덱스 다루기**입니다. 이 선택 기술은 다음 장의 조건 필터, 그 뒤의 집계·병합의 기본 동작이 됩니다.

이 장의 학습 목표

- `df['col']`과 `df[['a','b']]`의 반환 타입(시리즈 vs 데이터프레임) 차이를 확인하며 필요한 열을 선택할 수 있다 [→G3]
- `loc`(라벨)와 `iloc`(위치)로 행·열을 선택하고 슬라이싱의 끝 포함/제외 차이를 설명할 수 있다 [→G3]
- `set_index`·`reset_index`로 `customer_id`를 인덱스로 올려 조회하고 되돌릴 수 있다 [→G3]
- `sort_values`·`sort_index`로 매출·가격·인덱스를 정렬하고 날짜 인덱스로 기간을 슬라이싱할 수 있다 [→G3]

선수 확인: 이 장은 2장의 시리즈·데이터프레임 구조와 자료형, `read_csv` 로 제대로 불러오기를 알고 있다고 가정합니다. 특히 "열 하나를 꺼내면 시리즈"라는 것을 이해했다면 준비가 된 것입니다.

3.1 열 선택과 loc-iloc: 라벨 기반과 위치 기반

도입: 두 가지 인덱서를 왜 구분하나

2장에서 `customers["city"]` 로 열 하나를 꺼내 봤습니다. 그런데 "3번째 행" 같은 위치로 고르고 싶을 때와 "인덱스가 1024인 행" 같은 라벨로 고르고 싶을 때는 방식이 다릅니다. 판다스는 이

둘을 `iloc` (위치)과 `loc` (라벨)로 명확히 나눕니다. 이 구분을 흐릿하게 두면 엉뚱한 행이 나오거나 오류가 나므로, 처음에 확실히 잡고 갑니다.

정의

이 절의 개념은 **열과 행 선택 (loc-iloc)**(selecting columns and rows with loc and iloc)입니다.

핵심 정의

- **열 선택**: `df['열이름']` 은 한 열(시리즈), `df[['열A', '열B']]` 는 여러 열(데이터프레임)을 꺼냅니다.
- **라벨 기반 인덱싱**(loc, label-based): `df.loc[행라벨, 열이름]` 처럼 인덱스 값·열 이름으로 선택합니다. 슬라이싱은 **끝을 포함**합니다.
- **위치 기반 인덱싱**(iloc, integer-location): `df.iloc[행위치, 열위치]` 처럼 0부터의 정수 위치로 선택합니다. 슬라이싱은 **끝을 제외**합니다.

동작 원리

아래 그림은 같은 표에 대해 `df['col']` (열 하나), `df[['a', 'b']]` (여러 열), `loc` (라벨로 행·열), `iloc` (위치로 행·열)가 각각 무엇을 꺼내는지, 그리고 `loc` 슬라이싱은 끝을 포함하고 `iloc` 슬라이싱은 끝을 제외한다는 차이를 함께 보여 줍니다.

 같은 city 열 데이터프레임에서 `loc[10:12]`는 라벨 10~12를 끝 포함으로, `iloc[1:3]`은 위치 1~2를 끝 제외로 선택하는 차이를 나란히 비교하고, `df.loc[조건, 열목록]`으로 필터와 선택을 한 번에 하는 예를 보여주는 비교도.

가장 헷갈리는 지점이 **슬라이싱 끝 처리**입니다. `df.loc[0:5]` 는 라벨 0부터 5까지 **여섯 행** (0,1,2,3,4,5)을, `df.iloc[0:5]` 는 위치 0부터 4까지 **다섯 행** (0,1,2,3,4)을 줍니다. 파이썬 리스트 슬라이싱(`lst[0:5]`)은 끝을 제외하는데, `loc` 만 라벨 기반이라 끝을 포함한다는 점이 예외적이라 실수가 잦습니다.

worked example: 열 선택과 loc-iloc 슬라이싱

먼저 두 열을 데이터프레임으로 선택합니다.

```
import pandas as pd

products = pd.read_csv("data/products.csv")

two_cols = products[["product_name", "cost"]]
print(two_cols.head())
print("여러 열 선택 type:", type(two_cols).__name__)
print("한 열 선택 type:", type(products["cost"]).__name__)
```

- `products[["product_name", "cost"]]` — 대괄호 안에 리스트(`[...]`)를 넣어 두 열을 고릅니다. 결과는 **데이터프레임**입니다.
- `products["cost"]` — 대괄호 안에 문자열 하나면 한 열, 즉 **시리즈**입니다.
- 대괄호가 한 겹이냐 두 겹이냐로 반환 타입이 갈립니다.

실행 결과는 다음과 같습니다.

```
product_name    cost
0      플러스 홍차  3700.0
1   프리미엄 노트북 28900.0
2   프리미엄 원두커피  4700.0
3      베이직 마우스 32600.0
4      베이직 홍차  10500.0
여러 열 선택 type: DataFrame
한 열 선택 type: Series
```

이제 `loc` 와 `iloc` 의 슬라이싱 차이를 직접 봅시다.

```
orders = pd.read_csv("data/orders.csv", parse_dates=["order_datetime"])

print("orders.loc[0:5] 행수:", orders.loc[0:5].shape[0])
print("orders.iloc[0:5] 행수:", orders.iloc[0:5].shape[0])
```

실행 결과는 다음과 같습니다.

```
orders.loc[0:5] 행수: 6
orders.iloc[0:5] 행수: 5
```

같은 `0:5` 인데 `loc` 는 6행, `iloc` 는 5행입니다. `loc` 는 라벨 5를 포함하고, `iloc` 는 위치 5를 제외하기 때문입니다. 이 한 칸 차이가 집계·검증에서 오차를 낳으니 확실히 구분하십시오.

`iloc` 는 행·열을 **위치 번호**로 꼭 집을 때도 씁니다. 행 위치와 열 위치를 함께 주면 원하는 칸을 정확히 꺼냅니다.

```
print("0번 행 1번 열:", customers.iloc[0, 1])
print(customers.iloc[0:3, [1, 5]].to_string(index=False))
```

- `customers.iloc[0, 1]` — 0번째 행, 1번째 열(name)의 값 하나를 꺼냅니다. 위치는 모두 0부터 씁니다.
- `customers.iloc[0:3, [1, 5]]` — 0~2번 행과 1번·5번 열(name·city)을 뽑습니다. 열도 위치 리스트로 고를 수 있습니다.

실행 결과는 다음과 같습니다.

```
0번 행 1번 열: 권준서
name city
권준서 부산
박준선 서울
전영경 인천
```

위치로 특정 칸·구간을 집는 것이 `iloc` 이고, 라벨(이름·번호)로 집는 것이 `loc` 입니다. "몇 번째"로 생각하면 `iloc`, "무엇이라는 라벨"로 생각하면 `loc` 라고 기억하면 헷갈리지 않습니다. 지금까지의 선택 방법을 한 표로 정리하면 다음과 같습니다.

하고 싶은 것	문법	반환
한 열 선택	<code>df["col"]</code>	시리즈
여러 열 선택	<code>df[["a", "b"]]</code>	데이터프레임
라벨로 행·열	<code>df.loc[행라벨, "열"]</code>	값/시리즈/프레임
위치로 행·열	<code>df.iloc[행위치, 열위치]</code>	값/시리즈/프레임
조건+열 한 번에	<code>df.loc[조건, ["a", "b"]]</code>	데이터프레임
라벨 슬라이싱(끝 포함)	<code>df.loc[0:5]</code>	6행
위치 슬라이싱(끝 제외)	<code>df.iloc[0:5]</code>	5행

즉시 연습 (2종 1세트)

직접 해 보기 — 열·행 선택 (직접 실행·기록)

customers.csv로 아래를 실제 실행하고 결과를 기록하십시오.

```
customers = pd.read_csv("data/customers.csv")
# 1) name, city 두 열만 선택해 head()
# 2) customers.loc[0:3] 와 customers.iloc[0:3] 의 행수 비교
```

- [두 열 선택 결과의 type]: (예: DataFrame)
- [customers.loc[0:3] 행수]: () / [customers.iloc[0:3] 행수]: ()
- [두 행수가 다른 이유(내 설명)]: ()

자가체크

- [] 대괄호 두 겹은 데이터프레임, 한 겹은 시리즈임을 실제 출력으로 확인했다.
- [] loc[0:3] 은 4행, iloc[0:3] 은 3행임을 직접 실행해 봤다.
- [] loc는 끝 포함, iloc는 끝 제외라는 규칙을 내 말로 설명할 수 있다.

3.2 loc + 조건으로 필터와 선택을 한 번에

동작 원리: loc 안에 조건과 열 목록을 함께 넣는다

loc의 진짜 힘은 **조건과 열 선택을 한 번에** 하는 데 있습니다. `df.loc[조건, 열목록]` 형태로, "조건을 만족하는 행의, 이 열들만"을 뽑습니다. 조건 자리에는 참/거짓 시리즈가 들어가는데(자세한 원리는 4장), 지금은 `products["cost"] > 100000` 같은 비교가 각 행에 대해 True/False를 만든다고만 알면 됩니다.

worked example: 원가가 높은 상품의 이름·카테고리만

```
products = pd.read_csv("data/products.csv")

expensive = products.loc[
    products["cost"] > 100000,
    ["product_name", "category", "cost"],
]
print(expensive.head())
print("조건 만족 행수:", expensive.shape[0])
```

- `products["cost"] > 100000` — 각 상품의 원가가 10만 원을 넘는지 True/False로 만든 시리즈입니다.
- `products.loc[조건, [열들]]` — True인 행만 남기고, 그중 세 열만 뽑습니다. 필터와 선택이 한 줄로 끝납니다.

- `cost` 는 2장에서 본 대로 오염 없는 깨끗한 실수 열이라 조건 비교가 바로 됩니다(반면 `price` 는 문자열 오염이 있어 5장 정제 후에야 숫자 비교가 가능합니다).

실행 결과는 다음과 같습니다.

```
product_name category    cost
16    베이직 책상    가구  104800.0
34    플러스 스탠드조명    가구  120500.0
49    베이직 노트북    전자  117200.0
58    플러스 스탠드조명    가구  148400.0
77    데일리 의자    가구  113800.0
조건 만족 행수: 20
```

원가 10만 원 초과 상품이 20개이며, 대부분 가구·전자 카테고리입니다. `loc` 하나로 조건 필터와 열 선택을 동시에 끝냈습니다.

즉시 연습 (2종 1세트)

직접 해 보기 — `loc` + 조건 (직접 실행·기록)

`products`에서 아래를 실제 실행하고 결과를 기록하십시오.

```
# 1) cost 가 5만 원 미만인 상품의 product_name, cost 만 뽑기
# 2) 그 결과 행수(.shape[0]) 확인
# 3) customers 에서 city 가 "부산"인 고객의 name 만 loc로 뽑아 행수 확인
```

- `[cost 5만 미만 상품 행수]: ( )`
- `[loc로 뽑은 결과의 열 개수]: (예: 2)`
- `[city == "부산" 고객 행수]: ( )` / `[공백 오염(' 부산')으로 놓친 게 있을지 예상]: ( )`

자가체크

- `[] loc[조건, [열들]]` 한 줄로 필터와 열 선택을 동시에 했다.
- `[]` 결과가 조건을 만족하는 행만 담고 있는지 몇 개 행을 눈으로 확인했다.
- `[]` 값에 공백 오염이 있으면 `==` 조건이 일부를 놓칠 수 있음을 인지했다.

오개념: chained indexing으로 값 넣기

`df[df["cost"] > 100000]["category"] = "고가"` 처럼 대괄호를 연달아(chained) 써서 값을 넣으면, 원본이 아니라 복사본에 쓰여 반영이 안 되고 `SettingWithCopyWarning` 경고가 뜹니다. 값을 넣거나 조건 선택을 할 때는 `loc` 로 한 번에 지정하십시오: `df.loc[df["cost"] > 100000, "category"] = "고가"`. 이 습관은 정제 단원에서 값을 고칠 때 특히 중요합니다.

3.3 인덱스 다루기: set_index·reset_index

도입: 의미 있는 열을 행 라벨로 삼기

지금까지 인덱스는 0~1023 정수였습니다. 그런데 "1024번 고객을 조회"하려면 매번 조건을 걸어야 합니다. `customer_id` 를 아예 **인덱스로 올리면**, `loc[1024]` 로 곧바로 그 고객을 꺼낼 수 있습니다. (선수 개념: 3.1의 `loc`가 라벨 기반이라는 점이 여기서 진가를 발휘합니다.)

정의

이 절의 개념은 **인덱스 다루기**(working with the index)입니다.

핵심 정의

- `set_index('열')` — 지정한 열을 행 인덱스(라벨)로 올립니다. 그 열은 더 이상 일반 열이 아니라 행을 식별하는 라벨이 됩니다.
- `reset_index()` — 인덱스를 다시 기본 정수 인덱스로 되돌리고, 올렸던 열을 일반 열로 복원합니다.

동작 원리

`set_index('customer_id')` 를 하면 `customer_id` 값이 행 라벨이 되어, `loc[1024]` 가 "라벨 1024인 행"을 곧장 찾습니다. 단, 인덱스로 삼은 열에 **중복 값**이 있으면 `loc` 조회가 여러 행을 돌려줄 수 있습니다. 우리 고객 데이터에는 2장에서 본 중복 행이 섞여 있어(50개), 일부 `customer_id` 는 두 번 나타납니다. 그래서 인덱스로 삼을 때는 "이 열이 고유한가"를 의식해야 합니다.

worked example: customer_id로 조회하고 되돌리기

```
import pandas as pd

customers = pd.read_csv("data/customers.csv")
print("customer_id 중복 개수:", customers["customer_id"].duplicated().sum())

by_id = customers.set_index("customer_id")
print(by_id.loc[1000])

restored = by_id.reset_index()
print("되돌린 뒤 columns:", restored.columns.tolist())
```


- `customers["customer_id"].duplicated().sum()` — 중복된 고객 번호가 몇 개인지 셉니다.
- `customers.set_index("customer_id")` — `customer_id` 를 인덱스로 올립니다.
- `by_id.loc[1000]` — 라벨 1000인 고객을 곧바로 조회합니다.
- `by_id.reset_index()` — 인덱스를 정수로 되돌리고 `customer_id` 를 다시 일반 열로 복원합니다.

실행 결과는 다음과 같습니다.

```
customer_id 중복 개수: 50
name                이은진
gender              남
birth_date          2000-02-24
signup_date         2023-06-30
city                고양
email               user1000@example.com
Name: 1000, dtype: object
되돌린 뒤 columns: ['customer_id', 'name', 'gender', 'birth_date', 'signup_date', 'city', 'email']
```

`loc[1000]` 으로 1000번 고객(이은진, 고양)을 한 번에 꺼냈습니다. 조건을 길게 쓰지 않아도 라벨로 바로 조회됩니다. 중복이 50개 있다는 것도 확인했으니, 정제 전에는 조회 결과가 여러 행일 수 있음을 염두에 두어야 합니다.

즉시 연습 (2종 1세트)

직접 해 보기 — `set_index`와 조회 (직접 실행·기록)

```
# 1) customers 를 customer_id 인덱스로 올려 loc로 고객 3명 조회(예: 1000, 2500, 4000)
# 2) reset_index()로 되돌린 뒤 columns 에 customer_id 가 다시 있는지 확인
# 3) products 를 product_id 인덱스로 올려 loc로 상품 1개 조회
```

- [고객 2500의 name·city]: ()
- [reset_index 후 첫 열 이름]: (예: `customer_id`)
- [product_id로 조회한 상품명]: ()

자가체크

- [] `set_index` 로 열을 인덱스로 올려 `loc` 로 라벨 조회를 했다.
- [] `reset_index` 로 되돌리니 올렸던 열이 일반 열로 복원됐다.
- [] 인덱스로 삼은 열이 고유하지 않으면 조회가 여러 행을 낼 수 있음을 이해했다.

오개념: 여러 번 `set_index`하면 열을 잃는다

`set_index` 를 하면 올린 열은 인덱스가 되어 일반 열에서 사라집니다. `reset_index` 없이 다른 열로 또 `set_index` 하면 앞서 올린 열을 잃을 수 있습니다. 인덱스를 바꿀 때는 `reset_index` 로 정리한 뒤 다시 올리는 습관을 들이십시오. 또 `set_index` 는 기본으로 **새 데이터프레임을 반환**하므로, 결과를 변수에 다시 담아야 합니다(원본은 그대로).

3.4 정렬: `sort_values`·`sort_index`와 날짜 기간 슬라이싱

도입: 순서를 매겨 상위/하위를 뽑기

"가장 원가가 높은 상품 5개"나 "가입일 순으로 정렬" 같은 작업은 **정렬**이 필요합니다. 판다스는 열 값 기준 정렬(`sort_values`)과 인덱스 기준 정렬(`sort_index`)을 제공합니다.

정의

이 절도 **인덱스 다루기**(working with the index) 개념의 일부로, 정렬과 날짜 인덱스 슬라이싱을 다룹니다.

핵심 정의

- `sort_values('열')` — 열 값을 기준으로 정렬합니다. `ascending=False` 면 내림차순, `na_position` 으로 결측을 앞/뒤 어디에 둘지 정합니다.
- `sort_index()` — 인덱스(행 라벨)를 기준으로 정렬합니다.
- **날짜 인덱스 슬라이싱** — 날짜를 인덱스로 삼고 정렬해 두면 `df.loc['2024-06']` 처럼 기간으로 잘라 조회할 수 있습니다.

동작 원리

정렬은 두 종류입니다. `sort_values` 는 "이 열의 값 크기대로", `sort_index` 는 "행 라벨 순서대로"입니다. 날짜를 인덱스로 삼은 뒤 `sort_index` 로 시간순 정렬해 두면, 판다스가 날짜 라벨을 이해해 `loc['2024-06']` 처럼 특정 월을 통째로 슬라이싱할 수 있습니다. 이때 인덱스가 정렬돼 있지 않으면 기간 슬라이싱이 예상과 다르게 동작할 수 있으니, 날짜 인덱스는 `sort_index` 로 정렬해 두는 것이 안전합니다.

worked example: 원가 상위 상품과 6월 주문 슬라이싱

먼저 원가 내림차순으로 상위 상품을 뽑습니다.

```
products = pd.read_csv("data/products.csv")

top = products.sort_values("cost", ascending=False).head(5)
print(top[["product_name", "category", "cost"]])
```

- `sort_values("cost", ascending=False)` — 원가 큰 순으로 표 전체를 재정렬합니다.
- `.head(5)` — 그중 상위 5행만 봅니다. 정렬 후 `head` 를 붙이는 것이 "상위 N개" 관용구입니다.
- `sort_values` 는 원본을 바꾸지 않고 정렬된 새 표를 돌려줍니다.

실행 결과는 다음과 같습니다.

	product_name	category	cost
260	베이직 선반	가구	165500.0
58	플러스 스탠드조명	가구	148400.0
321	플러스 소파	가구	143200.0
495	베이직 선반	가구	130700.0
269	스탠다드 소파	가구	126000.0

원가 최상위 상품이 모두 가구 카테고리입니다. 정렬 기준을 **여러 열**로 줄 수도 있습니다. 카테고리 먼저 묶고, 각 카테고리 안에서 원가 높은 순으로 정렬하려면 열 목록과 방향 목록을 함께 줍니다.

```
multi = products.sort_values(
    ["category", "cost"],
    ascending=[True, False],
)
print(multi[["product_name", "category", "cost"]].head(6).to_string(index=False))
```

- `sort_values(["category", "cost"], ascending=[True, False])` — 1차로 `category` 오름차순, 그 안에서 2차로 `cost` 내림차순 정렬합니다.
- `ascending` 에 **리스트** — 얼마다 정렬 방향을 다르게 줄 수 있습니다(카테고리는 오름, 원가는 내림).

실행 결과(앞 6행)는 다음과 같습니다. 카테고리 오름차순 상 첫 그룹이 `가구` 라, 가구 안에서 원가 높은 순으로 나옵니다.

product_name	category	cost
베이직 선반	가구	165500.0
플러스 스탠드조명	가구	148400.0
베이직 선반	가구	130700.0
스탠다드 소파	가구	126000.0
플러스 스탠드조명	가구	125700.0
플러스 스탠드조명	가구	120500.0

이제 날짜를 인덱스로 삼아 특정 월만 슬라이싱합니다.

```
orders = pd.read_csv("data/orders.csv", parse_dates=["order_datetime"])

ord_idx = (
    orders.dropna(subset=["order_datetime"])
    .set_index("order_datetime")
    .sort_index()
)
june = ord_idx.loc["2024-06"]
print("2024-06 주문 건수:", june.shape[0])
print("결측 제외 전체 건수:", ord_idx.shape[0])
```

- `dropna(subset=["order_datetime"])` — 날짜가 결측인 행을 먼저 걸러냅니다(인덱스에는 결측 시각을 넣지 않는 게 안전합니다).
- `set_index("order_datetime").sort_index()` — 주문 시각을 인덱스로 올리고 시간순 정렬합니다.
- `ord_idx.loc["2024-06"]` — 2024년 6월에 해당하는 모든 주문을 한 번에 잘라 옵니다.

실행 결과는 다음과 같습니다.

```
2024-06 주문 건수: 42919
결측 제외 전체 건수: 198067
```

날짜 인덱스 덕분에 "2024-06" 이라는 문자열 하나로 6월 주문 42,919건을 뽑았습니다. 전체 19만여 건 중 6월이 가장 많은데, 이는 생성 데이터가 월이 갈수록 주문이 느는 추세를 담고 있기 때문입니다(뒤 시계열 단원에서 이 추세를 본격 분석합니다).

즉시 연습 (2종 1세트)

직접 해 보기 — 정렬과 기간 슬라이싱 (직접 실행·기록)

아래를 실제 실행해 결과를 기록하십시오.

```
# 1) products를 cost 오름차순으로 정렬해 원가가 가장 낮은 3개 상품
# 2) orders를 날짜 인덱스로 만들고 2024-01 주문 건수 구하기
# 3) customers를 set_index("customer_id") 후 loc[2000] 조회
```

- [원가 최저 3개 상품명]: ()
- [2024-01 주문 건수]: () / [2024-06 건수(앞 예제)와 비교하면?]: ()
- [loc[2000] 고객의 city]: ()

자가체크

- [] sort_values 로 상위/하위를 뽑고, 원본이 바뀌지 않았음을 확인했다.
- [] 날짜 인덱스로 특정 월을 슬라이싱해 건수를 얻었다.
- [] 1월과 6월 주문 건수를 비교해 월별 추세의 방향을 관찰했다.

3.5 worked example: 고객·상품 조회와 정렬

이 장의 도구를 모아, 실제 분석에서 자주 하는 조회 두 가지를 수행합니다. 특정 도시 고객을 뽑고, 카테고리별로 원가 상위 상품을 정렬합니다.

```
import pandas as pd

customers = pd.read_csv("data/customers.csv")
products = pd.read_csv("data/products.csv")

# (1) 특정 고객 번호 범위의 이름·도시만
sel = customers.loc[
    customers["customer_id"].between(5000, 5010),
    ["customer_id", "name", "city"],
]
print(sel)

# (2) 원가 상위 10개 상품을 카테고리·원가만
top10 = products.sort_values("cost", ascending=False).head(10)
print(top10[["product_name", "category", "cost"]].to_string(index=False))
```

- `customers["customer_id"].between(5000, 5010)` — 고객 번호가 5000~5010 범위인지 True/False로 만듭니다(between 은 양끝 포함).
- `customers.loc[조건, [열들]]` — 그 범위 고객의 세 열만 뽑습니다.
- `.to_string(index=False)` — 인덱스 없이 표를 깔끔하게 출력합니다.

실행 결과(일부)는 다음과 같습니다. 고객 번호 5000~5010이 실제로 존재해 11개 행이 나옵니다.

```
customer_id name city
5009 박태재 대전
5002 김하재 서울
5003 장성태 고양
5007 신선수 서울
5008 최예예 수원
...
행수: 11
```

한 가지 눈여겨볼 점은, 5002번 고객의 도시가 출력에서 `서울` 처럼 보여도 실제 값은 `'서울'` (뒤 공백)일 수 있다는 것입니다. 이런 공백 오염 때문에 `city == '서울'` 조건이 일부 서울 고객을 놓치는데, 이는 6장 문자열 정제에서 해결합니다. 지금 핵심은 `loc + 조건 + 열목록`의 한 줄 관용구가 실제 조회에 그대로 쓰인다는 점입니다.

3.6 단원 미니 프로젝트: 고객·상품 조회기 만들기

이 단원의 마무리로, **고객·상품 조회기**를 만듭니다. 주어진 조건으로 고객·상품을 조회하고 정렬해 필요한 열만 보여 주는 작은 스크립트입니다. 이 조회 패턴은 종합 프로젝트에서 세그먼트를 추출할 때 그대로 재사용됩니다.

모범: 조회 함수 두 개

```
import pandas as pd

customers = pd.read_csv("data/customers.csv")
products = pd.read_csv("data/products.csv")
by_id = customers.set_index("customer_id")

def find_customer(cid):
    """고객 번호로 조회. 없으면 안내 메시지."""
    if cid in by_id.index:
        return by_id.loc[cid]
    return f"고객 {cid} 없음"

def top_products_by_cost(category, n=5):
    """카테고리별 원가 상위 n개 상품."""
    subset = products.loc[products["category"] == category]
    return subset.sort_values("cost", ascending=False).head(n)[
        ["product_name", "cost"]
    ]

print(find_customer(1000))
print(top_products_by_cost("전자", 3))
```

- `find_customer` — `set_index` 한 표에서 `loc` 로 고객을 조회하되, 없는 번호는 안내 메시지를 돌려줍니다.
- `top_products_by_cost` — 특정 카테고리를 `loc` + 조건으로 거른 뒤 `sort_values` + `head` 로 상위 상품을 뽑습니다.
- 이 두 함수가 3장의 `loc`-`set_index`-`sort_values`를 모두 씁니다.

직접 만들기 (2종 1세트)

미니 프로젝트 — 고객·상품 조회기 (직접 실행·기록)

위 모범을 바탕으로 여러분의 조회기를 완성해 실제로 돌리고 결과를 기록하십시오. `category` 가 오염된 값(예: '전자')일 수 있으니, 조회가 안 되면 그 이유도 관찰해 적으십시오.

수행 항목:

1. `find_customer` 로 고객 번호 **3개**(예: 1000, 3293, 존재하지 않는 999)를 조회하고 결과를 적습니다.
2. `top_products_by_cost` 로 카테고리 **2개**(예: 가구, 도서)의 원가 상위 3개를 뽑습니다.
3. `customers` 를 `city` 가 특정 값인 고객으로 `loc` 조회할 때, 공백 오염(' 서울') 때문에 일부가 안 잡히는지 관찰합니다.

기록 슬롯:

- [고객 1000 조회 결과의 name-city]: ()
- [존재하지 않는 번호 조회 시 반환값]: ()
- [가구 원가 상위 3개 상품명]: ()
- [city 조회에서 공백 오염으로 놓친 행이 있었나]: (있음/없음 + 관찰 내용)

자가체크

- [] `set_index` + `loc` 로 고객 번호 조회가 동작했다(존재/부재 모두 처리).
- [] 카테고리별 `loc` + 조건, `sort_values` + `head` 로 상위 상품을 뽑았다.
- [] 오염된 값(공백) 때문에 조회가 놓치는 경우를 실제로 관찰했다(6장 정제의 필요성 확인).
- [] 조회기 스크립트를 저장해 다음 장에서 확장할 수 있게 했다.

자주 묻는 질문

Q. `loc` 와 `iloc` 중 무엇을 기본으로 써야 하나요? 분석에서는 대개 `loc` 를 씁니다. 라벨(열 이름·의미 있는 인덱스)로 다루는 것이 코드의 의도를 분명히 드러내기 때문입니다. `iloc` 는 "앞 몇 행만", "n번째 열" 같이 위치가 중요한 특수한 경우에 씁니다.

Q. `set_index` 를 하면 원본 데이터프레임이 바뀌나요? 아닙니다.

`set_index` · `sort_values` · `reset_index` 는 모두 기본으로 새 데이터프레임을 반환하고 원본은 그대로 둡니다. 그래서 결과를 반드시 변수에 다시 담아야 합니다(`by_id = customers.set_index("customer_id").inplace=True` 옵션도 있지만, 실수 추적이 어려워 권장하지 않습니다).

Q. 날짜 인덱스 슬라이싱에서 `loc["2024-06"]` 이 왜 되나요? `order_datetime` 을 `datetime` 으로 파싱해 인덱스로 올리고 `sort_index` 로 정렬해 두면, 판다스가 "2024-06" 을 "2024년 6월 전체 구간"으로 이해합니다. 정렬돼 있지 않으면 기간 슬라이싱이 예상과 다르게 동작할 수 있으니, 날짜 인덱스는 항상 정렬해 두십시오.

Q. 조회 결과가 여러 행 나옵니다. 왜죠? 인덱스로 삼은 열에 중복 값이 있으면 `loc` 가 여러 행을 돌려줍니다. 고객 데이터에는 중복 행 50개가 섞여 있어 일부 `customer_id` 가 두 번 나타납니다. 이는 5장 정제에서 `drop_duplicates` 로 제거합니다.

Q. `df[['col']]` 과 `df['col']` 은 정말 다른가요? 네, 대괄호 겹수로 반환 타입이 갈립니다.

`df['col']` 은 시리즈(1차원), `df[['col']]` 은 열이 하나뿐인 데이터프레임(2차원)입니다.

`.value_counts()` 처럼 시리즈 메서드를 쓰려면 한 겹, 표 모양을 유지하려면 두 겹을 씁니다.

Q. `sort_values` 에서 결측은 어디로 가나요? 기본으로 결측(NaN)은 정렬 순서와 관계없이 맨 뒤로 갑니다. `na_position="first"` 를 주면 맨 앞으로 보낼 수 있습니다. 결측이 많은 열을 정렬할 때는 결측 위치를 의식해야 상위/하위 조회가 어긋나지 않습니다.

Q. `chained indexing` 경고가 뜹니다. 무슨 뜻인가요? `df[조건]["열"] = 값` 처럼 대괄호를 연달아 써서 값을 넣으면, 원본이 아니라 복사본에 쓰여 반영이 안 되고 `SettingWithCopyWarning` 이 뜹니다. 값을 넣거나 조건 선택을 할 때는 `df.loc[조건, "열"] = 값` 처럼 **loc** 로 **한 번에** 지정하십시오. 이 습관은 5장 정제에서 값을 고칠 때 특히 중요합니다.

이 장의 핵심을 다시 새기면 이렇습니다. 열은 대괄호로(한 겹은 시리즈, 두 겹은 데이터프레임), 행·열은 `loc` (라벨)·`iloc` (위치)로 고른다. `loc` 는 슬라이싱 끝을 포함하고 `iloc` 는 제외한다. `set_index` 로 의미 있는 열을 라벨로 올리면 조회가 편해지고, `sort_values` .날짜 인덱스 슬라이싱으로 상위/하위·기간을 뽑는다. 이 선택 기술은 다음 장의 필터와 뒤 단원의 집계·병합에서 계속 재사용됩니다.

이제 라벨·위치로 원하는 부분을 정확히 꺼낼 수 있습니다. 다음 장에서는 이 선택을 한 단계 끌어올려, **조건에 맞는 행을 걸러내는 불리언 필터**를 본격적으로 배웁니다.

4장 조건으로 걸러내기: 불리언 필터

3장에서 라벨과 위치로 원하는 행·열을 꺼냈습니다. 그런데 실제 분석 질문은 대개 위치가 아니라 **조건**으로 던져집니다. "웹으로 들어와 배송 완료된 주문만", "6월에 앱에서 발생한 주문만", "취소·반품된 주문만" — 이런 부분집합을 뽑는 것이 분석의 출발점입니다. 이 장에서 배우는 **불리언 필터**가 바로 그 도구입니다.

3장의 선택이 "어디를 볼까"였다면, 4장의 필터는 "어떤 조건을 만족하는 것만 볼까"입니다. 이 필터는 이후 정제(이상값 걸러내기)·집계(특정 세그먼트 분석)에서 끊임없이 재사용되는, 판다스에서 가장 자주 쓰는 동작 중 하나입니다. 그래서 이 장은 단순히 문법을 넘어 **왜 그렇게 써야 하는지**(and/or를 왜 못 쓰는지, 괄호를 왜 꼭 쳐야 하는지)를 짚습니다.

이 장의 학습 목표

- 비교 연산으로 불리언 시리즈를 만들고 `df[불리언]`으로 조건을 만족하는 행만 추출해 원리를 설명할 수 있다 [→G3]
- `&` | `~`와 괄호로 다중 조건(기간·채널·상태)을 결합해 복합 조건 주문을 정확히 필터링할 수 있다 [→G3]
- `isin`·`between`·`str.contains`·`notna`로 목록·범위·문자열·결측 조건을 표현할 수 있다 [→G3]
- 같은 필터를 `query` 문자열로도 작성하고 결과 행수를 조건과 대조해 검증할 수 있다 [→G3]

선수 확인: 이 장은 3장의 `loc` 선택, 특히 "`loc` 안에 조건과 열 목록을 함께 넣을 수 있다"는 것을 배웠다고 가정합니다. 3.2절에서 잠깐 본 `products["cost"] > 100000` 같은 조건 시리즈가 이 장의 주인공입니다.

4.1 불리언 시리즈와 `df[불리언]` 필터링 원리

도입: 조건이 곧 True/False 시리즈다

3.2절에서 `products["cost"] > 100000` 을 조건 자리에 넣었습니다. 그 비교가 실제로 무엇을 만드는지 이 절에서 파헤칩니다. 결론부터 말하면, 비교 연산은 **각 행마다 조건 만족 여부를 True/False로 담은 시리즈**를 만듭니다. 이 원리를 알면 아무리 복잡한 조건도 조립할 수 있습니다.

정의

이 장의 개념은 **불리언 인덱싱과 조건 필터**(boolean indexing and conditional filtering)입니다.

핵심 정의

불리언 인덱싱(boolean indexing)은 각 행이 조건을 만족하는지를 **True/False로 담은 불리언 시리즈**를 만들어, `df[불리언시리즈]` 로 True인 행만 남기는 필터링 방식입니다.

- `df["channel"] == "web"` → 각 주문이 웹 채널인지 True/False로 만든 시리즈
- `df[그 시리즈]` → True인 행(웹 주문)만 남긴 데이터프레임

동작 원리

아래 그림은 비교 연산이 원본 열의 각 값을 True/False로 바꾸고, 그 불리언 시리즈를 다시 원본에 씌워 True인 행만 걸러내는 과정을 단계로 보여 줍니다.

 price 열에 조건 `price > 1000`을 적용해 True/False 불리언 마스크가 만들어지고, `df[마스크]`로 True인 행만 남기는 3단계(원본 열→불리언 마스크→선택 결과) 흐름과 `&` | `~` 조건 결합 예를 보여주는 개념도.

핵심은 **불리언 시리즈의 길이가 원본 행수와 같다**는 점입니다. 20만 행 주문에 `== "web"` 을 걸면 20만 개의 True/False가 나오고, `df[그것]` 은 True 위치의 행만 골라 옵니다. True가 몇 개인지 세면 곧 조건을 만족하는 행수이므로, `(조건).sum()` 으로 필터 결과 크기를 미리 확인할 수 있습니다(True는 1, False는 0으로 더해집니다).

worked example: 웹 채널 주문 걸러내기

```
import pandas as pd

orders = pd.read_csv("data/orders.csv", parse_dates=["order_datetime"])

mask = orders["channel"] == "web"
print("불리언 시리즈 앞 5개:", mask.head().tolist())
print("True 개수(=web 주문 수):", mask.sum())

web_orders = orders[mask]
print("걸러낸 행수:", web_orders.shape[0])
```

- `orders["channel"] == "web"` — 각 주문의 채널이 "web" 인지 비교해 True/False 시리즈 (`mask`)를 만듭니다.
- `mask.sum()` — True(1)를 모두 더해 웹 주문 수를 셉니다.
- `orders[mask]` — True인 행만 남깁니다. 이것이 필터의 본체입니다.

실행 결과는 다음과 같습니다.

```
불리언 시리즈 앞 5개: [False, True, False, False, False]
True 개수(=web 주문 수): 109958
걸러낸 행수: 109958
```

앞 5개 주문 중 1개만 웹이라 두 번째만 True입니다. 전체로는 웹 주문이 109,958건이고, `orders[mask]` 로 걸러낸 행수가 정확히 그 수와 일치합니다. `sum()` 으로 센 True 개수와 필터 결과 행수가 같다는 것이 필터가 제대로 동작했다는 첫 확인입니다.

즉시 연습 (2종 1세트)

직접 해 보기 — 불리언 시리즈 만들기 (직접 실행·기록)

`orders`에서 아래를 실제 실행하고 기록하십시오.

```
# 1) status == "canceled" 마스크를 만들고 .sum() 으로 취소 주문 수
# 2) orders[그 마스크] 의 행수가 sum()과 같은지 확인
# 3) channel == "store" 마스크의 앞 5개 True/False
```

- [취소 주문 수(`mask.sum()`)]: ()
- [`orders[mask]` 행수]: () / [`sum()`과 일치?]: (예/아니오)
- [store 마스크 앞 5개]: ()

자가체크

- [] 비교 연산이 True/False **시리즈**를 만든다는 것을 실제 출력으로 봤다.

- `[] mask.sum()` (True 개수)과 `orders[mask]` 의 행수가 같음을 확인했다.
- `[]` 불리언 시리즈의 길이가 원본 행수(20만)와 같다는 것을 이해했다.

4.2 다중 조건 결합: & | ~와 괄호(and/or 금지)

도입: 왜 and가 아니라 &인가

"웹 채널이면서 배송 완료"처럼 조건이 둘 이상이면 결합해야 합니다. 여기서 파이썬 입문자가 반드시 걸려 넘어지는 함정이 있습니다. 평소 쓰던 `and` · `or` 를 쓰면 **오류가 납니다**. 판다스는 `&` (그리고) · `|` (또는) · `~` (부정)를 쓰고, 각 조건을 **괄호로 감싸야** 합니다. 이 절은 그 이유를 확실히 이해시키는 데 초점을 둡니다.

정의와 동작 원리: 요소별 결합 vs 스칼라 판단

왜 and/or를 못 쓰나

`and` · `or` 는 파이썬에서 **하나의 참/거짓**을 판단하는 연산자입니다. 그런데 `orders["channel"] == "web"` 은 하나의 값이 아니라 20만 개의 True/False가 든 **시리즈**입니다. 파이썬은 "이 시리즈 전체가 참이냐?"를 판단할 수 없어 오류를 냅니다. 반면 `&` · `|` 는 **요소별(element-wise)**로 두 시리즈의 각 위치를 짝지어 결합하므로, 20만 개 각각에 대해 "둘 다 참인가"를 계산합니다.

- `and / or` → 시리즈 전체를 하나의 참/거짓으로 보려다 실패 → **오류**
- `& / | / ~` → 각 행마다 결합 → **원하는 결과**

또 하나, `&` · `|` 는 비교 연산보다 **연산자 우선순위가 높아**, 괄호를 빼면 엉뚱하게 묶입니다. 예컨대 `orders["channel"] == "web" & orders["status"] == "delivered"` 는 `"web" & orders["status"]` 를 먼저 계산하려 해 깨집니다. 그래서 **각 조건을 괄호로 감싸는 것이 필수**입니다.

worked example: and를 쓰면 나는 오류와 올바른 &

먼저 잘못된 방식이 실제로 어떤 오류를 내는지 봅시다.

```
# 잘못된 방식 — and 사용
try:
    wrong = orders[
        (orders["channel"] == "web") and (orders["status"] == "delivered")
    ]
except ValueError as e:
    print("오류:", str(e)[:60])
```

실행 결과는 다음과 같습니다.

```
오류: The truth value of a Series is ambiguous. Use a.empty, a.boo
```

"시리즈의 진리값이 모호하다"는 이 메시지가 바로 `and` 를 시리즈에 쓸 때 나는 전형적 오류입니다. 올바른 방식은 `&` 와 괄호입니다.

```
seg = orders[
    (orders["channel"] == "web") & (orders["status"] == "delivered")
]
print("web & delivered 행수:", seg.shape[0])
```

- (조건1) & (조건2) — 두 불리언 시리즈를 요소별로 결합해 "둘 다 True"인 행만 True로 만듭니다.
- 각 조건의 괄호 — `&` 의 우선순위가 높아 괄호가 없으면 조건이 엉뚱하게 묶입니다. 괄호는 선택이 아니라 필수입니다.

실행 결과는 다음과 같습니다.

```
web & delivered 행수: 60455
```

웹이면서 배송 완료된 주문이 60,455건입니다. 참고로 웹 주문 전체는 109,958건, 배송 완료 전체는 109,976건인데, 둘 다 만족하는 교집합이 60,455건으로 그보다 작습니다. 교집합이 각 조건보다 작다는 것도 결과가 말이 되는지 가늠하는 감각입니다.

`~` (부정)는 "조건을 만족하지 않는 행"을 뽑습니다. 웹이 아닌 주문을 세어 보겠습니다.

```
not_web = orders[~(orders["channel"] == "web")]
print("웹이 아닌 주문:", not_web.shape[0])
print("웹 + 웹아님 =", 109958 + not_web.shape[0])
```

- `~(orders["channel"] == "web")` — 웹 조건을 뒤집어, 웹이 아닌(앱·매장) 주문을 True로 만듭니다.

- 웹 주문 수와 웹이 아닌 주문 수를 더하면 전체 20만이 되어야 합니다. 이 합으로 부정 필터가 빠짐없이 나뉘는지 검증합니다.

실행 결과는 다음과 같습니다.

```
웹이 아닌 주문: 90042
웹 + 웹아님 = 200000
```

웹(109,958)과 웹이 아닌 주문(90,042)의 합이 정확히 20만입니다. 이렇게 "원조건 + 부정조건 = 전체"가 성립하는지 확인하면 필터가 옳게 나뉘었음을 검증할 수 있습니다.

즉시 연습 (2종 1세트)

직접 해 보기 — 다중 조건 결합 (직접 실행·기록)

아래를 실제 실행하고 결과를 기록하십시오.

```
# 1) (일부러) and로 두 조건을 묶어 어떤 오류가 나는지 관찰
# 2) &로 고쳐 "app 채널 & 취소(canceled)" 주문 행수
# 3) ~(부정)로 "web이 아닌" 주문 행수
```

- [and 사용 시 오류 메시지 앞부분]: (내 화면 실제 메시지)
- [app & canceled 행수]: ()
- [~(channel=="web") 행수]: () / [web 행수(109,958)와 더하면 전체 20만인가?]: ()

자가체크

- [] and 를 시리즈에 써서 실제로 ValueError 가 나는 것을 직접 봤다.
- [] & 와 괄호로 두 조건을 결합해 행수를 얻었다.
- [] ~로 부정 조건을 만들고, 부정+원조건 행수 합이 전체와 같은지 확인했다.

4.3 편의 메서드: isin·between·str.contains·notna

동작 원리: 자주 쓰는 조건을 짧게

==, & 만으로도 조건을 다 표현할 수 있지만, 자주 쓰는 패턴에는 더 짧고 읽기 좋은 메서드가 있습니다.

편의 메서드 네 가지

- `isin([...])` — 값이 목록 안에 있는지. `channel.isin(["web", "app"])` 은 `(channel=="web") | (channel=="app")` 과 같습니다.
- `between(a, b)` — 값이 a 이상 b 이하 범위인지(양끝 포함).
- `str.contains("문자")` — 문자열에 특정 글자가 들어 있는지. 결측이 있으면 `na=False` 를 줍니다.
- `notna()` / `isna()` — 값이 결측이 아닌지 / 결측인지.

worked example: 목록·범위·문자열·결측 조건

```
import pandas as pd

orders = pd.read_csv("data/orders.csv", parse_dates=["order_datetime"])
order_items = pd.read_csv("data/order_items.csv")
products = pd.read_csv("data/products.csv")

# isin: 웹 또는 앱 채널
print("web|app:", orders["channel"].isin(["web", "app"]).sum())

# isin: 취소 또는 반품
print("canceled|returned:", orders["status"].isin(["canceled", "returned"]).sum())

# between: 수량이 정상 범위(1~3)
print("quantity 1~3:", order_items["quantity"].between(1, 3).sum())

# str.contains: 상품명에 '노트북' 포함
mask = products["product_name"].str.contains("노트북", na=False)
print("상품명 '노트북' 포함:", mask.sum())

# notna: 주문 시각이 결측이 아닌 행
print("order_datetime 있는 행:", orders["order_datetime"].notna().sum())
```

- `isin(["web", "app"])` — 두 채널 중 하나면 True. `|` 를 여러 번 쓰는 것보다 간결합니다.
- `between(1, 3)` — 수량 1·2·3을 True로. 음수 이상값이나 4·5는 제외됩니다.
- `str.contains("노트북", na=False)` — 상품명에 "노트북"이 든 행. `na=False` 로 결측은 False 처리합니다.
- `notna()` — 결측(NaT)이 아닌 행만 True로. 시각이 비지 않은 주문을 셉니다.

실행 결과는 다음과 같습니다.

```
web|app: 169852
canceled|returned: 30014
quantity 1~3: 299692
상품명 '노트북' 포함: 18
order_datetime 있는 행: 198067
```

웹·앱 합산 169,852건, 취소·반품 30,014건, 수량 정상(1~3) 299,692건, 상품명에 "노트북"이 든 상품 18개, 시각이 있는 주문 198,067건입니다. `notna()` 의 198,067은 2장에서 본 결측 제외 건수와 정확히 일치하니, 훑어보기 단계의 수치가 필터에서 재확인됩니다.

네 편의 메서드를 언제 쓰는지 한 표로 정리하면 다음과 같습니다.

메서드	대신하는 조건	쓰는 상황
<code>isin([a, b])</code>	<code>(x==a) \ (x==b)</code>	여러 값 중 하나인지
<code>between(a, b)</code>	<code>(x>=a) & (x<=b)</code>	값이 범위 안인지(양끝 포함)
<code>str.contains("s")</code>	문자열 부분 일치	이름·설명에 특정 글자 포함
<code>notna() / isna()</code>	결측 여부	값이 있는/없는 행

이 메서드들은 모두 불리언 시리즈를 돌려주므로, `&` · `|` · `~`로 다른 조건과 자유롭게 결합할 수 있습니다.

즉시 연습 (2종 1세트)

직접 해 보기 — 편의 메서드 (직접 실행·기록)

```
# 1) status.isin(["paid", "shipped"]) 행수
# 2) order_items에서 unit_price.between(10000, 30000) 행수
# 3) products["product_name"].str.contains("소파", na=False) 행수
# 4) order_items["unit_price"].isna() 행수 (결측 상품가)
```

- `[paid|shipped 행수]: ()`
- `[unit_price 10000~30000 행수]: ()`
- `[상품명 '소파' 포함 개수]: ()`
- `[unit_price 결측 행수]: () / [2장 describe의 count와 맞춰 보면?]: ()`

자가체크

- `[]` `isin` 이 여러 `|` 조건을 대체함을 확인했다.

- `[]` `between` 으로 범위 조건을 만들고 양끝 포함임을 이해했다.
- `[]` `str.contains` 에 `na=False` 를 줘 결측 오류를 피했다.
- `[]` `isna / notna` 행수가 2장 훑어보기 수치와 일치하는지 대조했다.

4.4 query 문자열 필터와 결과 검증

동작 원리: 조건을 문자열로 쓰기

조건이 길어지면 `df[(...)& (...)& (...)]` 가 읽기 어려워집니다. `query` 는 조건을 문자열 하나로 표현해 가독성을 높입니다. `orders.query('channel == "web" and status == "delivered"')` 처럼, `query` 문자열 안에서는 오히려 `and` · `or` 를 그대로 쓸 수 있습니다(문자열이 판다스에서 따로 해석되기 때문).

worked example: 같은 필터를 두 방식으로

```
# 대괄호 불리언 방식
seg = orders[
    (orders["channel"] == "web") & (orders["status"] == "delivered")
]

# query 방식 (같은 조건)
segq = orders.query('channel == "web" and status == "delivered"')

print("대괄호 방식 행수:", seg.shape[0])
print("query 방식 행수:", segq.shape[0])
print("두 방식 동일?:", seg.shape[0] == segq.shape[0])
```

- `query('...')` — 조건을 문자열로 씁니다. 열 이름을 따옴표 없이 쓰고, 값 문자열은 `"web"` 처럼 따옴표로 감쌉니다.
- `query` 안에서는 `and` · `or` · `not` 을 그대로 씁니다(대괄호 방식의 `&` · `|` · `~` 에 대응).
- **결과 행수 대조** — 두 방식이 같은 행수를 내는지 확인해 필터가 일관됨을 검증합니다.

실행 결과는 다음과 같습니다.

```
대괄호 방식 행수: 60455
query 방식 행수: 60455
두 방식 동일?: True
```

같은 조건을 두 방식으로 썼더니 행수가 60,455로 정확히 일치합니다. 이렇게 **결과 행수를 조건과 대조**하는 것이 필터 검증의 핵심 습관입니다. 필터를 걸면 "몇 건이 나올 것 같은가"를 먼저 어림한 뒤, 실제 행수와 맞는지 확인하십시오. 어림과 크게 다르면 조건에 오류(괄호 빠짐·잘못된 값)가 있다는 신호입니다.

즉시 연습 (2종 1세트)

직접 해 보기 — 대괄호와 query 두 방식 대조 (직접 실행·기록)

"앱 채널이면서 취소된 주문"을 대괄호 방식과 query 방식 **두 가지**로 뽑아 행수가 같은지 확인하십시오.

```
# 대괄호 방식
seg = orders[(orders["channel"] == "app") & (orders["status"] == "canceled")]
# query 방식 (직접 작성)
segq = orders.query('...') # 여기 직접 작성
print(seg.shape[0], segq.shape[0], seg.shape[0] == segq.shape[0])
```

- [대괄호 방식 행수]: ()
- [내가 쓴 query 문자열]: ()
- [두 방식 일치?]: (예/아니오)

자가체크

- [] 같은 조건을 대괄호·query 두 방식으로 작성했다.
- [] 두 방식의 행수가 일치함을 실제 출력으로 확인했다.
- [] query 문자열 안에서는 and 를 쓰고, 대괄호에서는 & 를 쓴다는 차이를 구분했다.

오개념·흔한 오류 정리

- **다중 조건에 and / or 를 쓴다** → ValueError: truth value of a Series is ambiguous. 대괄호 방식에서는 반드시 & · | · ~ 와 괄호를 씁니다(단, query 문자열 안에서는 and / or 가 정상입니다).
- **괄호를 뺀다** → 연산자 우선순위 때문에 조건이 엉뚱하게 묶여 틀린 결과나 오류가 납니다. 각 조건을 괄호로 감싸십시오.
- **필터가 원본을 바꾼다고 착각** → 필터는 원본을 그대로 두고 **부분집합을 반환**합니다. 결과를 반드시 새 변수(seg = ...)에 담으십시오.
- **필터 결과를 검증하지 않는다** → 행수를 조건과 대조하지 않으면 조용히 틀린 세그먼트로 분석이 이어집니다.

여러 조건을 자유롭게 조합하기

편의 메서드와 `&`, `|`, `~`는 얼마든지 섞어 쓸 수 있습니다. 조건을 세 개 이상 엮을 때는 각 조건을 괄호로 감싸 한 줄씩 세로로 늘어놓으면 읽기 좋습니다.

```
# 웹 + 배송 완료 + 6월, 세 조건 결합
seg3 = orders[
    (orders["channel"] == "web")
    & (orders["status"] == "delivered")
    & (orders["order_datetime"].dt.month == 6)
]
print("web & delivered & 6월:", seg3.shape[0])

# 시각이 있고(notna) 상태가 paid 또는 shipped 인 주문
seg4 = orders[
    orders["order_datetime"].notna()
    & orders["status"].isin(["paid", "shipped"])
]
print("시각있음 & (paid|shipped):", seg4.shape[0])
```

- **세 조건 결합** — `&` 로 이어 붙이되 각 조건을 괄호로 감쌉니다. 조건이 늘어도 규칙은 같습니다.
- **notna() 와 isin() 을 함께** — 결측 조건과 목록 조건을 한 필터에서 섞습니다. 서로 다른 종류의 조건도 자유롭게 결합됩니다.

실행 결과는 다음과 같습니다.

```
web & delivered & 6월: 13077
시각있음 & (paid|shipped): 59425
```

6월의 웹·배송 완료 주문이 13,077건입니다. 앞서 본 웹·배송 완료 전체(60,455건) 중 6월분이 이만큼이라는 뜻으로, 조건을 좁힐수록 행수가 줄어드는 것이 자연스럽습니다. 조건을 하나 추가할 때마다 "행수가 줄었는가"를 확인하면 필터가 의도대로 좁혀지는지 검증할 수 있습니다.

4.5 worked example: 주문 세그먼트 추출과 행수 대조

이 장의 도구를 모아, 실제 분석에서 자주 만드는 **주문 세그먼트** 몇 가지를 추출하고 각각 행수를 대조 검증합니다.

```
import pandas as pd

orders = pd.read_csv("data/orders.csv", parse_dates=["order_datetime"])

# 세그먼트 A: 앱 채널 + 6월 주문
seg_a = orders[
    (orders["channel"] == "app")
    & (orders["order_datetime"].dt.month == 6)
]

# 세그먼트 B: 매장(store) + 취소(canceled)
seg_b = orders[
    (orders["channel"] == "store") & (orders["status"] == "canceled")
]

# 세그먼트 C: 취소도 반품도 아닌(정상) 주문
seg_c = orders[~orders["status"].isin(["canceled", "returned"])]

for label, seg in [("A 앱+6월", seg_a), ("B 매장+취소", seg_b), ("C 정상", seg_c)]:
    print(f"{label}: {seg.shape[0]:,}행")
```

- `orders["order_datetime"].dt.month == 6` — 날짜 열에서 월을 뽑아 6월인지 비교합니다 (dt 로 날짜의 부분을 꺼내는 자세한 방법은 시계열 단원에서 배웁니다).
- `~orders["status"].isin([...])` — ~로 부정해 "취소·반품이 아닌" 정상 주문을 남깁니다.
- 세 세그먼트를 반복문으로 돌려 각각 행수를 출력해 대조합니다.

실행 결과는 다음과 같습니다.

```
A 앱+6월: 12,865행
B 매장+취소: 3,002행
C 정상: 169,986행
```

앱에서 6월에 들어온 주문이 12,865건, 매장에서 취소된 주문이 3,002건, 취소·반품이 아닌 정상 주문이 169,986건입니다. 정상 주문(169,986)에 취소·반품(30,014)을 더하면 정확히 20만이 되니, 부정 필터가 빠짐없이 나뉘었음이 검증됩니다.

이처럼 세그먼트를 만들 때마다 "행수가 조건에 비춰 말이 되는가", "부분들을 더하면 전체가 되는가"를 확인하는 습관이 중요합니다. 이 대조 검증이 없으면, 괄호를 빠뜨리거나 값을 잘못 적은 세그먼트로 분석이 조용히 어긋날 수 있습니다. 종합 프로젝트의 구현 단계에서 세그먼트를 추출할 때도 이 검증을 그대로 적용하게 됩니다.

4.6 단원 미니 프로젝트: 주문 세그먼트 추출기

이 단원의 마무리로 **주문 세그먼트 추출기**를 만듭니다. 조건을 받아 주문 부분집합을 뽑고, 행수를 조건과 대조해 검증 노트까지 남기는 스크립트입니다. 이 추출기는 종합 프로젝트의 구현 단계(M1 세그먼트 추출)에서 그대로 쓰입니다.

모범: 세그먼트 추출 함수와 검증

```
import pandas as pd

orders = pd.read_csv("data/orders.csv", parse_dates=["order_datetime"])

def extract_segment(channel=None, status=None, month=None):
    """채널·상태·월 조건을 조합해 주문 세그먼트를 추출한다."""
    mask = pd.Series(True, index=orders.index)
    if channel is not None:
        mask &= orders["channel"] == channel
    if status is not None:
        mask &= orders["status"] == status
    if month is not None:
        mask &= orders["order_datetime"].dt.month == month
    return orders[mask]

seg = extract_segment(channel="web", status="delivered")
print("web+delivered:", seg.shape[0])

# 대괄호 방식과 query 방식이 같은지 대조
segq = orders.query('channel == "web" and status == "delivered"')
print("query와 동일?:", seg.shape[0] == segq.shape[0])
```

- `mask = pd.Series(True, ...)` — 모두 True인 시리즈에서 출발해, 주어진 조건마다 `&=` 로 좁혀 갑니다. 조건이 없으면 전체가 남습니다.
- `mask &= ...` — 기존 마스크에 새 조건을 `&` 로 누적 결합합니다.
- 마지막에 대괄호 방식과 query 방식 행수를 대조해 필터가 일관됨을 검증합니다.

직접 만들기 (2종 1세트)

미니 프로젝트 — 주문 세그먼트 추출기 (직접 실행·기록)

위 `extract_segment` 를 활용해(또는 직접 조건을 조립해) 아래 세그먼트를 추출하고, 각각 **대괄호 방식**과 **query 방식** 두 가지로 뽑아 행수가 일치하는지 검증하십시오. 값은 실제 출력을 옮깁니다.

추출할 세그먼트:

1. 앱 채널 + 배송 완료(delivered)
2. 웹 채널 + 반품(returned)
3. 5월 + 매장(store)
4. 취소도 반품도 아닌 정상 주문(부정 ~ 사용)

세그먼트	대괄호 방식 행수	query 방식 행수	일치?
앱+배송완료			
웹+반품			
5월+매장			
정상(부정)			

기록 슬롯:

- [가장 큰 세그먼트와 가장 작은 세그먼트]: () / ()
- [두 방식 행수가 어긋난 세그먼트가 있었나]: (없음/있음 + 원인 추적)
- [정상 주문 + 취소·반품 = 전체 20만인지 확인]: (예/아니오)

자가체크

- [] 네 세그먼트를 모두 대괄호·query 두 방식으로 추출했다.
- [] 각 세그먼트의 두 방식 행수가 일치함을 표로 확인했다(불일치 시 원인 추적).
- [] 부정(~) 필터로 정상 주문을 뽑고, 정상+취소·반품 합이 전체와 같은지 검증했다.
- [] 추출기 스크립트를 저장해 종합 프로젝트에서 재사용할 수 있게 했다.

참고: 채널·상태 분포

세그먼트를 만들기 전에 각 조건 열의 전체 분포를 알아 두면, 필터 결과 행수를 어렵하기 좋습니다. `value_counts()` 로 채널·상태의 전체 분포를 확인해 두십시오.

채널(channel)	건수	상태(status)	건수
web	109,958	delivered	109,976
app	59,894	shipped	30,017
store	30,148	paid	29,993
		canceled	20,039
		returned	9,975

이 분포를 알면 "웹+배송 완료"를 걸었을 때 나올 수 있는 최대 행수가 두 조건 중 작은 쪽(약 11만) 이하임을 미리 알 수 있어, 결과가 60,455건으로 나온 것이 타당한지 판단할 수 있습니다.

자주 묻는 질문

Q. `df[mask]` 와 `df.loc[mask]` 는 무엇이 다른가요? 행만 거를 때는 둘이 사실상 같은 결과를 냅니다. 다만 `loc` 는 `df.loc[mask, ["열A", "열B"]]` 처럼 **거르면서 동시에 열까지 고를 수 있다**는 장점이 있어, 조건 필터와 열 선택을 한 번에 할 때는 `loc` 를 씁니다.

Q. `query` 가 대괄호 방식보다 항상 좋은가요? 아닙니다. `query` 는 조건이 길 때 읽기 좋지만, 파이션 변수 값을 넣으려면 `@변수` 문법이 필요하고 열 이름에 공백·특수문자가 있으면 번거롭습니다. 짧은 조건이나 변수를 많이 쓰는 상황에서는 대괄호 방식이 더 편할 수 있습니다. 두 방식 모두 익혀 두고 상황에 맞게 고르십시오.

Q. 필터 결과를 다시 필터해도 되나요? 됩니다. `web = orders[orders["channel"] == "web"]` 로 뽑은 뒤 `web[web["status"] == "delivered"]` 로 또 거를 수 있습니다. 다만 조건을 한 번에 `&` 로 묶는 편이 대개 더 빠르고 명확합니다.

Q. `str.contains` 에서 왜 `na=False` 를 주나요? 문자열 열에 결측(NaN)이 섞여 있으면 `contains` 가 그 칸에서 True/False가 아니라 NaN을 내어, 그대로 필터에 쓰면 오류가 납니다. `na=False` 는 "결측은 포함하지 않은 것으로 본다"는 지정으로, 결측을 안전하게 False 처리합니다.

Q. 필터 결과를 세는 방법이 여러 개인데 무엇이 맞나요? `(조건).sum()` 은 조건을 만족하는 행 수를, `orders[조건].shape[0]` 은 걸러낸 데이터프레임의 행 수를 셉니다. 둘은 같은 값을 냅니다. 미리 개수만 알고 싶으면 `sum()` 이 간편하고, 실제 부분집합이 필요하다면 걸러낸 뒤 `shape[0]` 을 봅니다.

Q. 필터한 부분집합을 다시 인덱싱하면 인덱스가 이상합니다. 필터 결과는 원본의 인덱스를 그대로 물려받습니다. 그래서 걸러낸 표의 인덱스가 0·1·2가 아니라 원본 위치(예: 1, 7, 13...)로 띄엄띄엄 남습니다. 새로 0부터 매기려면 `reset_index(drop=True)` 를 붙입니다. 이는 3장에서 배운 인덱스 되돌리기와 같은 도구입니다.

Q. 조건에 쓴 값의 대소문자·공백이 안 맞으면요? `==` 은 정확히 일치해야 True입니다. `"web"` 과 `"Web"`, `"서울"` 과 `"서울 "` (공백)은 다른 값으로 취급돼 필터가 놓칩니다. `channel · status` 처럼 값이 깔끔한 열은 문제없지만, `city · category` 처럼 오염된 열은 6장 문자열 정제 후에 필터해야 정확합니다.

여기까지 오면 원하는 조건의 부분집합을 정확히·검증 가능하게 뽑을 수 있습니다. 이 장의 핵심 규칙을 다시 새기면 이렇습니다. 첫째, 비교 연산은 True/False 시리즈를 만든다. 둘째, 다중 조건은 `and / or` 가 아니라 `& · | · ~` 와 괄호로 묶는다. 셋째, 필터 결과 행수를 조건과 대조해 검증한다. 이 세 가지가 몸에 배면 어떤 복잡한 조건도 안전하게 조립할 수 있습니다.

이제 우리 손에는 "불러오기 → 구조 파악 → 선택 → 필터"라는 분석의 기본기가 갖춰졌습니다. 다음 단원부터는 이 데이터에 심어진 결측·중복·이상값을 본격적으로 정제해, 신뢰할 수 있는 분석의 토대를 만듭니다.

5장 데이터 정제 (1): 결측·중복·이상값·타입

2장에서 다섯 테이블을 불러와 품질 점검표를 만들 때, `customers`의 `gender`·`birth_date`에 빈칸이 있고 `order_items`의 `quantity`에 음수가 보이며 `products`의 `price`가 숫자가 아니라 문자열로 들어와 있다는 것을 발견했습니다. 4장에서는 불리언 필터로 조건에 맞는 행만 골라내는 법을 배웠습니다. 이제 그 두 가지를 합쳐, 점검표에 적어 둔 품질 문제를 실제로 진단하고 고치는 정제 단계로 들어갑니다.

이 책의 쇼핑물 데이터셋은 완벽하지 않습니다. 생성 스크립트가 정제 연습을 위해 결측·중복·이상값·타입 오염을 일부러 심어 두었기 때문입니다. 이 장에서는 그 문제들을 판다스(pandas)로 직접 찾아내고, 규칙을 정해 고치며, 고치기 전과 후를 대조해 검증합니다. 정제는 데이터를 마음대로 바꾸는 작업이 아니라, 규칙을 정하고 그 규칙대로 바뀔음을 수치로 증명하는 작업입니다.

실무에서 데이터 분석 시간의 절반 이상이 이 정제에 쓰인다고들 합니다. 그만큼 중요하지만 화려하지 않아 건너뛰기 쉬운데, 정제를 소홀히 하면 그 위에 쌓는 모든 집계·시각화·결론이 틀어 집니다. "쓰레기가 들어가면 쓰레기가 나온다"는 말 그대로입니다. 이 장에서 다루는 문제는 모두 이 데이터셋에 실제로 들어 있는 것들입니다 — `customers`의 성별 304건·생년월일 384건 결측, 고객 50건 중복, `order_items`의 음수 수량 500건·단가 결측 15,113건·중복 120건, `products`의 문자열로 오염된 가격 46건. 하나씩 실제 숫자로 확인하며 고쳐 나가겠습니다.

이 장의 학습 목표

- `isna().sum()`·`isna().mean()`으로 열별 결측 수·비율을 집계해 처리 우선순위를 정할 수 있다 [→G4]
- `dropna`·`fillna`를 결측 특성(무응답 vs 계산 오류)에 맞게 골라 적용하고 근거를 설명할 수 있다 [→G4]
- `uplicated`·`drop_duplicates`로 중복을 제거하고 `describe`·분위수로 이상값(outlier)을 진단해 조건 제거·`clip`으로 처리할 수 있다 [→G4]
- 문자열로 오염된 `price`를 `to_numeric(errors='coerce')`·`astype`으로 숫자 자료형(dtype)으로 되돌리고 정제 전후를 대조할 수 있다 [→G4]

선수 확인: 이 장은 2장(데이터 불러오기·훑어보기)과 4장(불리언 인덱싱)을 마쳤다고 가정합니다. `data` 폴더에 CSV 다섯 개가 있고, `pd.read_csv`로 테이블을 불러와

`info()` · `describe()` · `value_counts()` 를 읽을 수 있어야 합니다. 아직이라면 2장의 품질 점검표부터 완성하고 오시기 바랍니다.

5.1 결측치 처리 (개념 c08)

도입: 왜 결측치를 먼저 다루나

분석에서 가장 먼저 만나는 지저분함은 **비어 있는 칸**입니다. `customers` 를 불러와 `birth_date` 로 나이를 계산하려 하면, 값이 없는 384개 행에서 계산이 막히거나 엉뚱한 결과가 나옵니다. 평균을 낼 때 빈칸을 0으로 잘못 취급하면 평균이 실제보다 낮아집니다. 그래서 본격적인 분석에 앞서 **결측이 어디에 얼마나 있는지 파악하고, 각 열의 의미에 맞게 처리하는 것**이 정제의 출발점입니다.

결측을 무시하고 분석하면 조용히 틀린 결과가 나옵니다. 판다스는 대부분의 집계에서 결측을 알아서 빼고 계산하는데(예: `mean()` 은 결측을 제외), 이 "알아서"가 오히려 함정이 될 수 있습니다 — 어떤 열은 결측이 7%나 되는데도 평균이 멀쩡히 나와, 데이터가 온전한 줄 착각하게 만듭니다. 그래서 **결측이 어디에 얼마나 있는지 먼저 눈으로 확인하는 것**이 안전한 분석의 첫걸음입니다.

선수 개념 상기: 결측을 찾을 때는 4장에서 배운 불리언 시리즈를 그대로 씁니다. `df['gender'].isna()` 는 각 행이 결측인지를 True/False로 담은 불리언 시리즈이고, 여기에 `.sum()` 을 붙이면 True(=결측)의 개수가 됩니다.

정의

결측치(missing value) 란 값이 비어 있는 칸으로, 판다스에서는 주로 `NaN` (부동소수 결측)이나 `None` · `NaT` (날짜 결측)로 표현됩니다.

- **NaN** (Not a Number): 숫자·문자열 열의 빈칸에 들어가는 부동소수 결측 표시입니다. `NaN` 은 float 자료형이라, 결측이 하나라도 섞인 정수 열은 float로 바뀝니다.
- **NaT** (Not a Time): 날짜/시간 열의 결측 표시입니다.
- **탐지:** `isna()` (결측이면 True), `notna()` (결측이 아니면 True)로 위치를 찾고, `isna().sum()` 으로 열별 결측 **개수**를, `isna().mean()` 으로 결측 **비율**을 셉니다.

결측을 처리하는 방법은 크게 세 가지입니다 — **버리기**(`dropna`), **채우기**(`fillna`), 그리고 **그대로 두기**(결측 자체가 정보일 때). 무엇을 택할지는 그 열이 왜 비었는지에 달려 있습니다.

동작 원리: 세 가지 처리 전략 고르기

결측을 무조건 버리거나 무조건 채우면 데이터가 왜곡됩니다. 열의 **의미**에 따라 전략을 골라야 합니다.

전략	도구	적합한 경우	이 데이터셋의 예
버리기	<code>dropna(subset=..., how=..., thresh=...)</code>	분석에 꼭 필요한 키가 비었을 때	<code>customer_id</code> 가 빈 행(있을 수 없는 값)
채우기	<code>fillna(값)</code> , <code>ffill</code> / <code>bfill</code> , 그룹 평균	채울 근거가 분명할 때	<code>order_items</code> 의 <code>unit_price</code> 를 상품 기본가로
그대로 두기	(처리 안 함)	결측 자체가 의미일 때	<code>gender</code> 무응답 — 함부로 채우면 성별 분포 왜곡

- `dropna` 는 기본으로 결측이 **하나라도** 있는 행을 통째로 버립니다. 특정 열만 볼 때는 `subset=['gender', 'birth_date']` 로 좁히고, `how='all'` (모든 열이 결측일 때만) · `thresh=3` (비결측이 3개 미만인 행만) 같은 조건으로 조절합니다.
- `fillna` 는 고정값(`fillna(0)`), 앞/뒤 값 복사(`ffill` · `bfill`), 통계값(`fillna(df['price'].median())`)으로 채웁니다.

핵심은 **채우기 전에 "이 결측이 왜 생겼는가"**를 묻는 것입니다. `gender` 가 빈 것은 고객이 응답하지 않은 것일 수 있어, 아무 값으로 채우면 없던 성별을 만들어 내는 셈입니다. 반면 `unit_price` 가 빈 것은 기록 누락이라, 상품의 기본 가격으로 채우는 것이 합리적입니다.

worked example: customers 결측 진단과 처리

`customers.csv` 를 불러와 결측을 진단하고, 열의 의미에 맞게 서로 다른 전략으로 처리해 보겠습니다.

```
import pandas as pd

cust = pd.read_csv("data/customers.csv")

# 1) 열별 결측 개수와 비율 진단
print("결측 개수:")
print(cust.isna().sum())
print("\n결측 비율(%)")
print((cust.isna().mean() * 100).round(2))
```

- `cust.isna()` — 데이터프레임 전체를 같은 크기의 True/False 표로 바꿉니다(값이 비었으면 True).
- `.sum()` — 열마다 True를 세어 결측 **개수**를 냅니다.
- `.mean()` — True를 1, False를 0으로 보고 평균을 내므로 결측 **비율**이 됩니다. `* 100`으로 백분율, `.round(2)`로 소수 둘째 자리까지 반올림합니다.

실행 결과는 다음과 같습니다.

```
결측 개수:
customer_id    0
name           0
gender         304
birth_date     384
signup_date    0
city           0
email          248
dtype: int64

결측 비율(%):
customer_id    0.00
name           0.00
gender         6.08
birth_date     7.68
signup_date    0.00
city           0.00
email          4.96
dtype: float64
```

`gender`가 304개(6.08%), `birth_date`가 384개(7.68%), `email`이 248개(4.96%) 비어 있습니다. `customer_id`·`name`은 결측이 없으므로 키로 믿고 쓸 수 있습니다. 이제 열의 의미에 맞게 처리합니다.

```
# 2) 열 의미에 맞는 서로 다른 처리
# email: 분석에 안 쓰는 부가 정보 -> 그대로 둔다(처리 안 함)
# birth_date: 나이 분석에 필요하지만 지어낼 수 없음 -> 결측 행 표시만
# gender: 무응답일 수 있어 채우지 않고 '미상' 범주로 남긴다
cleaned = cust.copy()
cleaned["gender"] = cleaned["gender"].fillna("미상")

# 나이 분석 전용 테이블은 birth_date 결측 행을 제외
age_ready = cust.dropna(subset=["birth_date"])

print("gender 미상 채운 뒤 결측:", cleaned["gender"].isna().sum())
print("birth_date 기준 dropna 전 행수:", len(cust))
print("birth_date 기준 dropna 후 행수:", len(age_ready))
```

- `cust.copy()` — 원본을 보존하려고 사본에 작업합니다(원본을 직접 바꾸면 되돌리기 어렵습니다).
- `fillna("미상")` — `gender` 결측을 없는 값으로 지어내지 않고 '미상'이라는 별도 범주로 남깁니다. 나중에 성별 분석에서 미상을 따로 셀 수 있습니다.
- `dropna(subset=["birth_date"])` — `birth_date`가 빈 행만 골라 제거합니다. 다른 열이 비었어도 이 열만 기준이라 나이 분석용 테이블에는 적절합니다.

실행 결과는 다음과 같습니다.

```
gender 미상 채운 뒤 결측: 0
birth_date 기준 dropna 전 행수: 5000
birth_date 기준 dropna 후 행수: 4616
```

`gender` 결측 304개가 '미상'으로 바뀌어 결측이 0이 됐고, 나이 분석용 테이블은 5000행에서 `birth_date` 결측 384행을 뺀 4616행이 됐습니다($5000 - 384 = 4616$). 이렇게 **전후 행수를 대조하면 의도대로 처리됐는지 검증**할 수 있습니다.

숫자 열의 결측은 **통계값으로 채우는** 방법도 있습니다. `order_items`의 `unit_price`처럼 채울 근거가 있는 열은 대푯값으로 메웁니다. 다만 어떤 대푯값을 쓰느냐가 결과를 바꿉니다.

```
oi = pd.read_csv("data/order_items.csv")
print("unit_price 결측:", oi["unit_price"].isna().sum())
print("평균:", round(oi["unit_price"].mean(), 2),
      "\n| 중앙값:", oi["unit_price"].median())

# 중앙값으로 채우기(극단값에 덜 흔들림)
oi["unit_price"] = oi["unit_price"].fillna(oi["unit_price"].median())
print("채운 뒤 결측:", oi["unit_price"].isna().sum())
```


- `fillna(oi["unit_price"].median())` — 결측을 중앙값(median)으로 채웁니다. 평균(mean)은 아주 비싼 상품 몇 개에 끌려 올라가지만, 중앙값은 극단값에 덜 흔들려 "가운데 값"을 대표합니다.
- **평균 vs 중앙값** — 이 열은 평균 50,925원, 중앙값 27,000원으로 차이가 큼니다(가격 분포가 한쪽으로 치우침). 어느 값으로 채우느냐에 따라 이후 매출 계산이 달라지므로, **선택 근거를 남기는 것이 중요합니다.**

실행 결과는 다음과 같습니다.

```
unit_price 결측: 15113
평균: 50925.56 | 중앙값: 27000.0
채운 뒤 결측: 0
```

결측 15,113개가 모두 채워졌습니다. 참고로 앞뒤 값을 복사하는 `ffill` (앞 값으로)·`bfill` (뒤 값으로)은 시간 순서가 있는 데이터(예: 날짜순 센서 값)에서 유용하지만, 순서에 의미가 없는 이 주문 상세에는 통계값 채우기가 더 적절합니다.

즉시 연습 (2종 1세트)

직접 해 보기 — `order_items`의 `unit_price` 결측 진단과 채우기

`order_items.csv`에는 `unit_price` (상품 단가)에 결측이 심어져 있습니다. 아래 절차를 **실제로 실행**하고, 화면에 뜬 실제 숫자를 기록하십시오(정답을 추측해 적는 것이 아니라 실행 결과를 옮겨 적습니다).

1단계 — `order_items.csv`를 불러와 `isna().sum()`과 `isna().mean()`을 출력하고, 아래 라벨에 **실제 출력값**을 적습니다.

- **[unit_price 결측 개수]:** (실제 실행값, 예상 근처: 약 1만 5천)
- **[unit_price 결측 비율(%)]:** (실제 실행값)
- **[결측이 있는 다른 열이 있는가]:** (있으면 열 이름과 개수)

2단계 — `unit_price`가 상품 단가라는 점을 생각하며, 이 결측을 **채우기**로 처리하는 게 나은지 **버리기**가 나은지 한 문장으로 판단해 적습니다.

- **[내가 고른 전략과 근거]:** (예: 단가는 0으로 채우면 매출이 0이 되어 왜곡되므로 ...)

3단계 — 고른 전략을 코드로 실행하고(예: `dropna(subset=['unit_price'])` 또는 상품 기본가로 `fillna`), 처리 전후 행수 또는 결측 수를 대조해 기록합니다.

- **[처리 전 행수/결측 수]:** (실제값)

- [처리 후 행수/결측 수]: (실제값)

자가체크

- [] `isna().sum()` 을 실제로 실행해 화면에 뜬 결측 개수를 옮겨 적었다(머리로 추측하지 않았다).
- [] `unit_price` 를 왜 그 전략으로 처리했는지 근거를 한 문장으로 썼다.
- [] 처리 전후 행수 또는 결측 수를 대조해, 처리가 의도대로 됐음을 수치로 확인했다.

흔한 오류와 교정

오개념: "결측은 일단 0으로 채우면 된다"

`fillna(0)` 은 편해 보이지만 위험합니다. `unit_price` 를 0으로 채우면 그 주문의 매출이 0으로 계산되어 **합계·평균이 실제보다 낮아집니다**. `birth_date` 나 `gender` 같은 열은 아예 0이라는 값이 말이 되지 않습니다. 채우기 전에 "이 결측이 왜 생겼고, 무엇으로 채우는 게 맞는가" 를 먼저 물으십시오.

또 하나 흔한 실수는 `dropna()` 를 아무 옵션 없이 부르는 것입니다. 기본 `dropna()` 는 **결측이 하나라도 있는 행을 전부 버립니다**. `customers` 에서 그냥 `dropna()` 를 부르면 `gender·birth_date·email` 중 하나라도 빈 행이 모두 사라져, 5000행이 크게 줄어듭니다. 꼭 필요한 열만 `subset` 으로 지정해 좁히십시오. 마지막으로, `NaN` 은 float이므로 결측이 섞인 정수 열은 float로 바뀐다는 점도 기억하십시오.

5.2 중복·이상값·타입 정제 (개념 c09)

도입: 결측 다음의 세 가지 지저분함

결측을 처리했다고 데이터가 깨끗해진 것은 아닙니다. 실제 데이터에는 세 가지가 더 숨어 있습니다 — **같은 행이 두 번 들어간 중복(duplicate)**, **말이 안 되는 이상값(outlier)**, 그리고 **숫자여야 할 값이 문자열로 들어온 타입 오염**입니다. 2장 점검표에서 `order_items` 의 `quantity` 에 음수가 보이고 `products` 의 `price` 가 문자열이라 `object` 자료형이었던 것을 기억할 것입니다. 이 절에서 그 셋을 차례로 고칩니다.

선수 개념 상기: 이상값을 걸러낼 때도 4장의 불리언 필터를 씁니다.

`oi[oi['quantity'] > 0]` 처럼 조건을 만족하는 행만 남기는 것이 이상값 제거의 기본

입니다.

정의

- **중복(duplicate):** 같은 내용이 두 번 이상 들어간 행입니다. `df.duplicated()` 로 중복 여부(불리언)를 찾고 `df.drop_duplicates(subset=..., keep=...)` 로 제거합니다. `subset` 으로 무엇을 "같은 행"으로 볼지(전체 열 vs 키 열), `keep` 으로 어느 것을 남길지 (`'first' · 'last' · False`) 정합니다.
- **이상값(outlier):** 정상 범위를 크게 벗어난 값입니다. `df.describe()` 와 분위수로 범위를 파악해, **음수 수량·0원 가격·미래 날짜** 같은 불가능한 값을 조건으로 걸러내거나 `df.clip(lower=, upper=)` 으로 상·하한을 씁니다.
- **타입 오염:** 숫자여야 할 값이 `'12300원' · '14,200'` 처럼 문자열로 들어와 열 전체가 `object` 자료형이 된 경우입니다. 문자열을 정리한 뒤 `df.to_numeric(errors='coerce') · df.astype` 으로 숫자로 되돌리고, 변환 실패는 결측으로 만들어 다시 처리합니다.

동작 원리: 진단하고, 규칙을 정하고, 대조한다

세 가지 모두 같은 3단계를 따릅니다 — **진단(무엇이 얼마나 잘못됐나) → 규칙 결정(무엇을 어떻게 고칠까) → 전후 대조(정말 고쳐졌나).**

- **중복:** `df.duplicated().sum()` 으로 완전히 같은 행 수를 세고, `df.duplicated(subset=['customer_id']).sum()` 으로 키 기준 중복을 셉니다. 무엇을 중복으로 볼지 정한 뒤 `df.drop_duplicates` 로 제거하고 행수를 비교합니다.
- **이상값:** `df.describe()` 의 `min · max` 가 불가능한 값(예: `quantity` 의 최솟값 음수)이면 규칙을 세웁니다("수량은 1 이상이어야 한다"). 그 규칙을 불리언 필터나 `df.clip` 으로 적용합니다.
- **타입:** `df.dtypes` 로 잘못된 타입을 찾고, `df.str.replace` 로 `'원' · ','` 를 지운 뒤 `df.to_numeric(errors='coerce')` 으로 숫자화합니다. `errors='coerce'` 는 변환 못 하는 값을 오류 대신 `NaN` 으로 만들어, 이후 결측 처리로 넘길 수 있게 합니다.

이상값을 판단하는 방법은 두 갈래입니다. 하나는 **불가능한 값**을 규칙으로 정의하는 것입니다 — "수량은 1 이상", "가격은 0보다 큼", "주문 날짜는 미래일 수 없음"처럼 도메인 지식으로 명백한 경계를 긋습니다. 이런 값은 데이터 오류이므로 제거·수정이 정당합니다. 다른 하나는 **통계적으로 극단인 값**을 찾는 것입니다 — `df.describe()` 의 사분위수(25%·75%)나 `df.quantile()` 로 분포를 보고, 예컨대 상위 1%처럼 유난히 큰 값을 살핍니다. 다만 통계적 극단값은 **진짜 이상이 아닐 수 있습니다**(프리미엄 고가 상품은 정상적으로 비쌉니다). 그래서 통계적 극단값은 바로 지우지

말고 "왜 이런 값인가"를 확인한 뒤 판단합니다. 이 데이터셋의 정제 대상(음수 수량·0 이하 가격)은 대부분 첫 번째 갈래, 즉 명백히 불가능한 값입니다.

worked example: order_items·products 정제 전후 대조

order_items의 중복·이상값과 products의 타입 오염을 한 번에 진단·정제하고, 전후를 표로 대조하겠습니다.

```
import pandas as pd

oi = pd.read_csv("data/order_items.csv")

# 1) 진단: 중복·이상값
before_rows = len(oi)
dup_cnt = oi.duplicated().sum()
neg_qty = (oi["quantity"] <= 0).sum()
print("정제 전 행수:", before_rows)
print("완전 중복 행수:", dup_cnt)
print("수량 0 이하(이상값) 개수:", neg_qty)
print("quantity 최솟값/최댓값:",
      oi["quantity"].min(), "/", oi["quantity"].max())

# 2) 규칙 적용: 중복 제거 후 수량 1 이상만 남긴다
oi_clean = oi.drop_duplicates()
oi_clean = oi_clean[oi_clean["quantity"] > 0]
print("정제 후 행수:", len(oi_clean))
```

- `oi.duplicated().sum()` — 모든 열이 똑같은 행(완전 중복)의 개수를 셉니다.
- `(oi["quantity"] <= 0).sum()` — 수량이 0 이하인 불가능한 값(이상값)의 개수를 셉니다. 물건을 음수 개나 0개 산다는 것은 있을 수 없으므로 정제 대상입니다.
- `drop_duplicates()` → `[oi_clean["quantity"] > 0]` — 먼저 중복을 지우고, 그다음 수량이 1 이상인 행만 불리언 필터로 남깁니다. 정제 규칙을 코드로 명확히 적은 것입니다.

실행 결과는 다음과 같습니다.

```
정제 전 행수: 500000
완전 중복 행수: 120
수량 0 이하(이상값) 개수: 500
quantity 최솟값/최댓값: -2 / 5
```

정제 전 50만 행에 완전 중복 120건, 수량 0 이하 이상값 500건이 있었습니다. quantity 최솟값이 -2라는 것은 명백히 불가능한 값입니다. 규칙(중복 제거 + 수량 1 이상)을 적용한 뒤 행수는 다음처럼 줄어듭니다.

정제 후 행수: 499380

이제 `products` 의 타입 오염을 고칩니다.

```
prod = pd.read_csv("data/products.csv")
print("price 자료형(정제 전):", prod["price"].dtype)

# 숫자 변환 실패(문자열 오염) 개수 진단
fail_cnt = pd.to_numeric(prod["price"], errors="coerce").isna().sum()
print("바로 숫자 변환 시 실패 개수:", fail_cnt)
print("오염 샘플:", prod.loc[
    pd.to_numeric(prod["price"], errors="coerce").isna(), "price"
].head(4).tolist())

# 규칙: '원'과 ',', ' ' 제거 후 숫자화, 0 이하 가격은 결측 처리
price_str = (prod["price"].astype(str)
             .str.replace("원", "", regex=False)
             .str.replace(",", "", regex=False))
prod["price"] = pd.to_numeric(price_str, errors="coerce")
print("price 자료형(정제 후):", prod["price"].dtype)
print("0 이하 가격 개수:", (prod["price"] <= 0).sum())
```

- `pd.to_numeric(prod["price"], errors="coerce")` — `price` 를 바로 숫자로 바꾸되, '12300원' 처럼 변환 못 하는 값은 오류 대신 `NaN` 으로 둡니다. 그 `NaN` 개수가 곧 문자열 오염 개수입니다.
- `.str.replace("원", "", regex=False)` → `.str.replace(",", "", regex=False)` — 숫자 앞에 붙은 '원' 과 천 단위 심표를 지웁니다. `regex=False` 는 정규식이 아니라 그대로 그 글자를 찾아 바꾸라는 뜻입니다.
- `astype(str)` — `str` 접근자를 쓰려면 열이 문자열이어야 하므로 먼저 문자열로 통일합니다.

실행 결과는 다음과 같습니다.

```
price 자료형(정제 전): object
바로 숫자 변환 시 실패 개수: 46
오염 샘플: ['14,200', '95,500', '3,200', '12300원']
price 자료형(정제 후): float64
0 이하 가격 개수: 8
```

`price` 가 `object` (문자열)에서 `float64` (실수)로 바뀌었고, 문자열 오염 46건이 정리됐습니다. 정리 후에도 0 이하 가격이 8건 남아 있으니, 이것은 이상값으로 별도 규칙(예: 결측 처리 후 상품 기본가로 대체)이 필요하다는 것을 알 수 있습니다. 아래처럼 **정제 전후 대조표**로 정리하면 무엇을 왜 고쳤는지 한눈에 남습니다.

항목	정제 전	규칙	정제 후
order_items 행수	500,000	완전 중복 제거 + 수량 1 이상	499,380
order_items 이상값(수량 $\leq$ 0)	500건	조건 필터 제거	0건
products price 자료형	object	'원','' 제거 후 to_numeric	float64
products price 문자열 오염	46건	coerce로 결측화·정리	0건

이상값을 **삭제 대신 상·하한으로 눌러 두는** 방법도 있습니다. 값을 버리기 아깝지만 극단값의 영향을 줄이고 싶을 때 `clip` 을 씁니다. `order_items` 의 할인율(`discount`)을 예로 봅니다.

```
oi = pd.read_csv("data/order_items.csv")
print("discount 범위(정제 전):",
      oi["discount"].min(), "~", oi["discount"].max())

# 할인율은 0~0.5 범위만 정상이라고 규칙을 정하고 clip으로 상·하한 적용
oi["discount"] = oi["discount"].clip(lower=0, upper=0.5)
print("discount 범위(clip 후):",
      oi["discount"].min(), "~", oi["discount"].max())
```

- `clip(lower=0, upper=0.5)` — 0보다 작은 값은 0으로, 0.5보다 큰 값은 0.5로 눌러 범위 안에 가둡니다. 행을 버리지 않고 값만 상·하한으로 조정합니다.
- **삭제와 clip의 차이** — 명백히 불가능한 값(음수 수량)은 **삭제**가 맞지만, "범위를 벗어난 정도"만 문제일 때는 **clip**으로 행을 살리면서 극단값의 영향을 줄일 수 있습니다. 무엇을 택할지는 규칙으로 정하고 근거를 남깁니다.

실행 결과는 다음과 같습니다(이 데이터의 `discount` 는 이미 0~0.5 범위라 `clip` 후에도 범위가 유지됩니다 — 규칙을 명시적으로 보장한다는 의미입니다).

```
discount 범위(정제 전): 0.0 ~ 0.5
discount 범위(clip 후): 0.0 ~ 0.5
```

즉시 연습 (2종 1세트)

직접 해 보기 — customers 중복 제거와 대조표 작성

`customers.csv` 에는 완전히 같은 행이 중복으로 심어져 있습니다. 아래를 **직접 실행**하고 실제 출력을 기록하십시오.

1단계 — 중복을 진단합니다. 다음을 실행해 실제값을 적습니다.

- [정제 전 행수 `len(cust)`]: (실제값, 예상: 5000)
- [완전 중복 행수 `cust.duplicated().sum()`]: (실제값)
- [customer_id 기준 중복 `cust.duplicated(subset=['customer_id']).sum()`]: (실제값)

2단계 — `drop_duplicates()` 로 중복을 제거하고 제거 후 행수를 기록합니다. 그리고 "정제 전 행수 - 중복 행수 = 정제 후 행수"가 맞는지 손으로 계산해 대조합니다.

- [정제 후 행수]: (실제값)
- [손계산 검증: 정제 전 - 중복 = ?]: (직접 계산해 정제 후 행수와 같은지 확인)

3단계 — 아래 대조표를 자신의 실제 숫자로 채워 완성합니다(빈칸 추측이 아니라 실행 결과 기록).

항목	정제 전	규칙	정제 후
customers 행수	[실제값]	완전 중복 제거	[실제값]

자가체크

- [] `duplicated().sum()` 을 실제로 실행해 중복 개수를 확인했다(추측하지 않았다).
- [] `drop_duplicates()` 후 행수가 "정제 전 - 중복"과 일치함을 손계산으로 대조했다.
- [] 대조표를 실제 실행 숫자로 채웠고, 무엇을 왜 제거했는지 규칙 칸에 적었다.

흔한 오류와 교정

오개념: "중복은 다 지우고, 이상해 보이는 값은 다 삭제하면 된다"

`drop_duplicates` 는 기본으로 모든 열이 같아야 중복으로 봅니다. 그런데 실제로는 "같은 고객 ID면 중복"처럼 키 기준으로 봐야 할 때가 많습니다. 이때 `subset=['customer_id']` 를 주지 않으면, 한 칸이라도 다른 행은 중복으로 잡히지 않아 진짜 중복이 남습니다.

이상값도 무조건 삭제하면 안 됩니다. 수량 -2처럼 명백히 불가능한 값은 제거가 맞지만, "가격이 유난히 높다" 정도는 진짜 프리미엄 상품일 수 있습니다. 불가능한 값(음수 수량·0원 가격·미래 날짜)인지, 단지 극단값인지를 규칙으로 구분하고 근거를 남기십시오. 마지막으로

`to_numeric` 은 `errors='coerce'` 를 주지 않으면 변환 불가 값에서 오류를 내며 멈춥니다. `coerce` 로 결측 처리한 뒤 다시 다루는 흐름을 습관으로 삼으십시오.

자주 묻는 질문

Q. 결측을 채울지 버릴지 매번 헛갈립니다. 판단 기준이 있나요?

세 가지를 물어보십시오. ① **그 값이 분석에 꼭 필요한가?** (안 쓰는 열이면 그냥 뒤도 됩니다) ② **왜 비었는가?** (무응답이면 남기거나 '미상' 범주로, 기록 누락이면 대푯값으로 채웁니다) ③ **채울 근거가 있는가?** (상품 기본가처럼 근거가 있으면 채우고, 없으면 그 행을 분석에서 빼는 게 정직합니다). 핵심은 "편해서"가 아니라 "그 열의 의미에 맞아서" 고르는 것입니다.

Q. dropna 로 행을 지웠더니 인덱스가 중간중간 비어 있습니다. 문제인가요?

dropna 는 행을 지우지만 남은 행의 인덱스 번호는 그대로 둡니다(예: 0,1,3,4). 분석에는 대개 지장이 없지만, 인덱스를 0부터 다시 매기고 싶으면 `reset_index(drop=True)` 를 붙이면 됩니다. `drop=True` 는 기존 인덱스를 열로 남기지 않고 버립니다.

Q. to_numeric(errors='coerce') 로 만든 NaN 은 원래 결측과 어떻게 구분하나요?

구분되지 않습니다 — 둘 다 NaN 이 됩니다. 그래서 변환 **직후에** 몇 개가 새로 결측이 됐는지 (`isna().sum()` 의 전후 차이)를 확인하고, 그 값들이 원래 어떤 문자열이었는지 로그로 남겨 두는 것이 좋습니다. 오염 패턴('원'·',')을 먼저 정리하면 불필요한 결측화를 줄일 수 있습니다.

이 장을 마치며

이 장에서 결측치(개념 c08)와 중복·이상값·타입 오염(개념 c09)을 실제 쇼핑물 데이터로 진단하고 정제했습니다. 핵심은 세 가지입니다. 첫째, **결측은 열의 의미에 따라 버릴지·채울지·남길지를 판단합니다**(무응답 `gender` 는 남기고, 누락된 `unit_price` 는 채우거나 제외). 둘째, **정제는 규칙을 정하는 일입니다** — "수량은 1 이상", "price는 숫자여야 함" 같은 규칙을 코드로 명확히 적습니다. 셋째, **정제 전후를 행수·결측·자료형으로 대조해 검증합니다**.

한 가지 더 기억할 것은, 정제는 **원본을 보존한 채** 사본에 하는 것이 안전하다는 점입니다.

`df.copy()` 로 사본을 만들어 작업하고, 정제 규칙을 함수로 모아 두면, 나중에 규칙이 바뀌어도 원본부터 다시 정제할 수 있습니다. 이 "원본 보존 + 규칙 함수화"가 재현 가능한 분석의 출발점이며, 6장의 정제 파이프라인에서 본격적으로 다룹니다.

다음 6장에서는 마지막 지저분함인 **문자열 표기 불일치**('의류' 와 '의류 ', 'M' 과 '남')를 `str` 접근자로 통일하고, 이 장과 6장의 정제를 하나의 **정제 파이프라인**으로 묶어 깨끗한 `clean` 테이블을 저장합니다.

6장 데이터 정제 (2): 문자열 표기 통일과 정제 파이프라인

5장에서 결측·중복·이상값·타입 오염을 정제했습니다. 그런데 아직 하나가 남았습니다.

`products['category']`에는 '의류'와 ' 의류 ' (뒤에 공백)가, `customers['gender']`에는 'M'·'F'·'남'·'여'가 뒤섞여 있고, `city`에는 '서울'과 ' 서울' (앞 공백)이 따로 있습니다. 눈으로는 같아 보이지만 컴퓨터에게는 다른 값이라, 그대로 `value_counts`로 세면 같은 범주가 둘로 갈라집니다. 이 장에서는 시리즈(Series)의 `str` 접근자로 문자열 표기를 통일하고, 5장과 6장의 정제를 하나의 정제 파이프라인으로 묶어 깨끗한 `clean` 테이블을 저장합니다.

이 장의 학습 목표

- `str.strip`·`str.lower`로 공백·대소문자를 통일해 표기 불일치로 갈라진 범주를 하나로 합칠 수 있다 [→G4]
- `str.replace`·`str.contains`로 패턴을 치환·검색하고 `str.split`·`str.extract`로 부분을 추출할 수 있다 [→G4]
- 정리 전후 `value_counts`를 비교해 표기 통일이 집계 정확도를 어떻게 바꾸는지 확인할 수 있다 [→G4]
- 결측·중복·이상값·타입·문자열 정제를 하나의 규칙 기반 파이프라인으로 묶어 `clean` 테이블을 저장할 수 있다 [→G4]

선수 확인: 이 장은 5장(결측·중복·이상값·타입 정제)을 마쳤다고 가정합니다.

`dropna`·`drop_duplicates`·`to_numeric`을 쓸 수 있고, `value_counts()`로 값 분포를 읽을 수 있어야 합니다.

6.1 문자열 데이터 처리 (개념 c10)

도입: 눈에는 같은데 컴퓨터에는 다른 값

2장에서 `products['category'].value_counts()` 를 봤을 때 '의류' 가 두 줄에 나뉘어 나온 것을 기억할 것입니다. 하나는 '의류', 다른 하나는 '의류' (뒤 공백)입니다. 사람 눈에는 똑같은 "의류"지만, 판다스는 공백 한 칸까지 비교하므로 **다른 범주로 셉니다**. 이 상태로 "카테고리별 매출"을 집계하면 의류 매출이 두 조각으로 갈라져, 실제보다 작아 보입니다. 그래서 집계에 앞서 **표기를 통일하는 문자열 정제**가 필요합니다.

문자열 오염은 데이터를 사람이 손으로 입력하거나 여러 시스템에서 모을 때 거의 반드시 생깁니다. 어떤 담당자는 'M' 으로, 어떤 담당자는 '남' 으로 성별을 적고, 엑셀에서 붙여 넣다 앞뒤에 공백이 딸려 오며, 카테고리 이름을 누군가는 '의류' 로 누군가는 '의류' 로 씁니다. 원본을 만든 사람에게는 다 같은 뜻이지만, 코드는 글자 그대로만 봅니다. 그래서 **분석가가 표기를 통일해 주는 것이 문자열 정제**입니다.

선수 개념 상가: 문자열 정제도 5장의 원칙과 같습니다 — 진단(`value_counts` 로 갈라진 표기 확인) → 규칙(공백 제거·표준값 매핑) → 전후 대조(정리 후 `value_counts` 로 합쳐졌는지 확인).

정의

문자열 처리(working with text data) 는 시리즈의 `str` 접근자로 파이썬 문자열 메서드를 열 전체에 **벡터화 연산(vectorized operation)** 으로 적용하는 것입니다. 반복문 없이

`df['city'].str.strip()` 한 줄로 모든 행의 공백을 한 번에 없앱니다.

- `str.strip()` — 앞뒤 공백 제거(' 서울' → '서울').
- `str.lower()` / `str.upper()` — 대소문자 통일.
- `str.replace(a, b)` — 특정 문자·패턴을 치환.
- `str.contains(패턴)` — 포함 여부를 불리언으로 반환(필터에 사용).
- `str.split(구분자)` / `str.extract(정규식)` — 문자열을 나누거나 일부를 추출.

`str` 메서드는 **원본을 바꾸지 않고 새 시리즈를 반환**하므로, 결과를 다시 열에 할당해야 반영됩니다. 또 결측(`NaN`)이 섞인 열에 `str` 메서드를 쓰면 `NaN` 은 그대로 남습니다.

동작 원리: 표기 통일이 집계를 바꾼다

문자열 정제의 목적은 단순히 "예쁘게"가 아니라 **정확한 집계**입니다. 아래 두 경우를 봅시다.

- **공백 오염:** '의류' 와 '의류' 는 `strip()` 한 번으로 합쳐집니다. `city` 의 ' 서울' · '서울' 도 마찬가지입니다.

- **표기 혼용:** `gender` 의 'M' · '남' 은 같은 성별을 다르게 적은 것입니다. 이때는 `map` 이나 `replace` 로 **표준값 하나에 대응**시킵니다(`{'남':'M', '여':'F', 'M':'M', 'F':'F'}`).

정리 전후로 `value_counts()` 를 비교하면, 갈라졌던 범주 수가 줄고 각 범주의 개수가 커지는 것을 눈으로 확인할 수 있습니다. 이 "전후 대조"가 문자열 정제의 검증입니다.

자주 쓰는 `str` 메서드를 목적별로 정리하면 다음과 같습니다. 모두 `df['열'].str.메서드(...)` 형태로 쓰며, 결과를 다시 열에 할당해야 반영됩니다.

목적	메서드	예	결과
앞뒤 공백 제거	<code>str.strip()</code>	<code>' 서울'.strip()</code>	<code>'서울'</code>
대소 문자 통일	<code>str.lower() · str.upper()</code>	<code>'Web'.lower()</code>	<code>'web'</code>
특정 문자 치환	<code>str.replace(a, b)</code>	<code>'12,000'.replace(',', '')</code>	<code>'12000'</code>
포함 여부 검사	<code>str.contains(패턴)</code>	<code>'프리미엄 니트'.contains('니트')</code>	<code>True</code>
나누 기	<code>str.split(구분자)</code>	<code>'프리미엄 노트북'.split(' ')</code>	<code>['프리미엄', '노트북']</code>
길이	<code>str.len()</code>	<code>'서울'.len()</code>	<code>2</code>
시작· 끝 검 사	<code>str.startswith · str.endswith</code>	<code>'user1@x'.startswith('user')</code>	<code>True</code>

`str.contains` 와 `str.startswith` 는 불리언을 돌려주므로 4장에서 배운 불리언 필터에 바로 넣어 `df[df['product_name'].str.contains('프리미엄')]` 처럼 조건 검색에 씁니다. 즉 문자열 처리는 정제뿐 아니라 필터의 조건으로도 이어집니다.

worked example: category-gender 표기 통일 전후 비교

`products['category']` 의 공백 오염과 `customers['gender']` 의 표기 혼용을 정리하고, `value_counts` 로 전후를 대조하겠습니다.

```
import pandas as pd

prod = pd.read_csv("data/products.csv")

# 1) 정리 전: 공백 때문에 갈라진 category
print("정리 전 category 종류 수:", prod["category"].nunique())
print(prod["category"].value_counts())
```

- `nunique()` — 고윳값(서로 다른 값)의 개수를 셉니다. 공백 오염이 있으면 실제 범주보다 큰 수가 나옵니다.
- `value_counts()` — 값마다 몇 번 나오는지 셉니다. '의류' 와 '의류 ' 가 별도 줄로 나오면 표기가 갈라진 것입니다.

실행 결과는 다음과 같습니다.

```
정리 전 category 종류 수: 12
category
도서      90
전자      79
식품      75
가구      71
의류      70
뷰티      68
전자      13
의류       9
가구       9
도서       7
뷰티       5
식품       4
Name: count, dtype: int64
```

실제 카테고리는 6종인데 공백 때문에 12종으로 갈라졌습니다(전자 79 + 전자 13처럼 같은 범주가 둘씩). 이제 `strip()` 으로 통일합니다.

```
# 2) strip으로 공백 제거 후 다시 세기
prod["category"] = prod["category"].str.strip()
print("정리 후 category 종류 수:", prod["category"].nunique())
print(prod["category"].value_counts())
```

실행 결과는 다음과 같습니다.

```
정리 후 category 종류 수: 6
category
도서    97
전자    92
가구    80
식품    79
의류    79
뷰티    73
Name: count, dtype: int64
```

12종이 6종으로 합쳐졌고, 전자 가 79 + 13 = 92로, 도서 가 90 + 7 = 97로 제대로 합산됐습니다. 이제 gender 표기 혼용을 map 으로 통일합니다.

```
cust = pd.read_csv("data/customers.csv")
print("정리 전 gender:")
print(cust["gender"].value_counts(dropna=False))

# 남/여/M/F를 표준값 M/F로 매핑(결측은 그대로 NaN)
gmap = {"남": "M", "여": "F", "M": "M", "F": "F"}
cust["gender"] = cust["gender"].map(gmap)
print("\n정리 후 gender:")
print(cust["gender"].value_counts(dropna=False))
```

- `map(gmap)` — 각 값을 대응표 `gmap` 의 값으로 바꿉니다. '남' · 'M' 은 모두 'M' 으로, '여' · 'F' 는 'F' 로 통일됩니다.
- **대응표에 없는 값은 NaN 이 됩니다.** 여기서 결측(NaN)은 애초에 대응표에 없으므로 그대로 NaN 으로 남습니다. `dropna=False` 로 세면 그 결측도 함께 보입니다.

실행 결과는 다음과 같습니다.

```
정리 전 gender:
gender
F      1200
M      1191
여      1184
남      1121
NaN      304
Name: count, dtype: int64

정리 후 gender:
gender
F      2384
M      2312
NaN      304
Name: count, dtype: int64
```

네 가지로 흩어졌던 표기가 F (1200 + 1184 = 2384)와 M (1191 + 1121 = 2312) 두 가지로 통일됐고, 결국 304개는 매핑되지 않아 그대로 남았습니다. 이렇게 `value_counts`의 전후를 비교하면 표기 통일이 집계를 얼마나 바꾸는지 수치로 확인됩니다.

문자열 검색·추출도 자주 씁니다. `str.contains`는 특정 단어를 포함한 행을 필터할 때 씁니다.

```
# str.contains로 '프리미엄'이 이름에 든 상품 수 세기
n_premium = prod["product_name"].str.contains("프리미엄").sum()
print("이름에 '프리미엄'이 든 상품 수:", n_premium)
```

실행 결과는 다음과 같습니다.

```
이름에 '프리미엄'이 든 상품 수: 80
```

문자열에서 일부를 떼어내는 작업도 흔합니다. `products`의 `product_name`은 '프리미엄 노트북'처럼 등급 + 상품명 구조라, 공백으로 나누면 등급과 상품을 따로 분석할 수 있습니다. 여기서 앞서 배운 `strip`이 왜 먼저 와야 하는지 도 드러납니다.

```
prod = pd.read_csv("data/products.csv")

# strip을 먼저 하지 않으면 '프리미엄 니트'는 첫 조각이 빈 문자열이 된다
prod["product_name"] = prod["product_name"].str.strip()

# 공백 기준으로 앞 1개만 분리(등급 / 상품명)
parts = prod["product_name"].str.split(" ", n=1, expand=True)
prod["grade"] = parts[0]
prod["item"] = parts[1]

print(prod[["product_name", "grade", "item"]].head(5))
print("\n등급 분포:")
print(prod["grade"].value_counts())
```

- `str.strip()` 먼저 — 원본에는 '프리미엄 니트'처럼 앞 공백이 붙은 이름이 있어, 그대로 `split(" ")`하면 첫 조각이 빈 문자열('')이 됩니다. 공백을 먼저 없애야 등급이 제대로 잡힙니다(공백 정제가 다른 문자열 작업의 전제임을 보여 줍니다).
- `str.split(" ", n=1, expand=True)` — 공백으로 나누되 `n=1`로 앞 한 번만 나눠, '프리미엄 원두커피'가 ['프리미엄', '원두커피']가 되게 합니다. `expand=True`는 결과를 열이 여러인 데이터프레임으로 펼칩니다.
- `parts[0]`, `parts[1]` — 나뉜 두 조각을 각각 등급·상품명 열로 가져옵니다.

실행 결과는 다음과 같습니다.

```

product_name grade item
0      플러스 홍차   플러스   홍차
1   프리미엄 노트북 프리미엄  노트북
2   프리미엄 원두커피 프리미엄  원두커피
3      베이직 마우스 베이직   마우스
4      베이직 홍차   베이직   홍차

```

등급 분포:

grade

베이직 97

플러스 89

스탠다드 81

프리미엄 80

데일리 77

코어 76

Name: count, dtype: int64

`strip`으로 공백을 없앤 덕분에 빈 등급이 하나도 없이 여섯 등급으로 깔끔하게 나뉘었습니다. 만약 `strip`을 건너뛰었다면 앞 공백이 있는 이름들이 빈 등급으로 새어 나왔을 것입니다. 정규식이 필요한 더 복잡한 추출은 `str.extract(정규식)`로 하며, 시계열 코드 파싱에서 다시 만납니다.

팁: 문자열 정제의 순서

문자열 정제는 대개 **공백 제거**(`strip`) → **대소문자 통일**(`lower / upper`) → **표준값 매핑**(`map / replace`) → **분리·추출**(`split / extract`) 순으로 합니다. 앞 단계를 건너뛰면 뒤 단계가 어긋납니다 — 위에서 본 것처럼 공백을 안 없애면 분리 결과에 빈 조각이 생깁니다. 각 단계 뒤에 `value_counts`로 중간 결과를 확인하는 습관이 실수를 줄입니다.

worked example: 표기 통일이 집계를 얼마나 바꾸나

문자열 정제가 왜 중요한지는 **집계 결과의 차이**로 가장 분명하게 드러납니다. `customers`를 도시별로 세어 보면, 공백 오염 때문에 같은 도시가 여러 조각으로 갈라져 실제보다 적게 세어집니다. 정제 전후를 나란히 비교하겠습니다.


```

cust = pd.read_csv("data/customers.csv")

# 정제 전: 공백 때문에 도시가 갈라져 있다
print("정제 전 도시 종류 수:", cust["city"].nunique())
print("정제 전 상위 5개(서울이 조각나 있음):")
print(cust["city"].value_counts().head(5))

# 정제 후: strip으로 공백 통일
cust["city"] = cust["city"].str.strip()
print("\n정제 후 도시 종류 수:", cust["city"].nunique())
print("정제 후 상위 5개:")
print(cust["city"].value_counts().head(5))

```

- 정제 전 `value_counts()` — '서울' · ' 서울' (앞 공백) · '서울 ' (뒤 공백)이 각각 다른 줄로 세어져, 서울 고객 수가 세 조각으로 흩어집니다.
- `str.strip()` 후 다시 세기 — 앞뒤 공백을 없애면 세 조각이 하나로 합쳐집니다.

실행 결과는 다음과 같습니다.

```

정제 전 도시 종류 수: 30
정제 전 상위 5개(서울이 조각나 있음):
city
서울      963
부산      484
수원      424
인천      417
고양      351
Name: count, dtype: int64

정제 후 도시 종류 수: 10
정제 후 상위 5개:
city
서울     1235
부산      627
수원      562
인천      554
고양      452
Name: count, dtype: int64

```

정제 전에는 도시가 30종으로 세어졌지만, 실제로는 10개 도시입니다. 서울만 봐도 정제 전 '서울' 963명으로 보이던 것이, 흩어진 조각('서울' 963 + ' 서울' 141 + '서울 ' 131)을 합치면 실제로는 **1,235명**입니다. 정제하지 않고 "서울 고객이 963명"이라고 보고했다면 272명(약 22%)을 놓친 셈입니다. 이것이 **집계 전에 문자열 표기를 통일해야 하는 실질적 이유**입니다 — 공백 한 칸이 분석 결론을 바꿉니다.

즉시 연습 (2종 1세트)

직접 해 보기 — city 공백 표기 통일하기

`customers['city']`에는 ' 서울' (앞 공백)·'서울 ' (뒤 공백)·'서울' 이 뒤섞여 있습니다. 아래를 **직접 실행**하고 실제 출력을 기록하십시오(추측이 아니라 실행 결과).

1단계 — 정리 전 상태를 진단합니다.

- [정리 전 city 종류 수 `nunique()`]: (실제값, 예상: 10보다 큼)
- [`value_counts().head(12)` 에서 공백으로 갈라진 표기 예 2개]: (예: '서울' 과 ' 서울')

2단계 — `str.strip()` 으로 공백을 제거해 열에 다시 할당하고, 정리 후 종류 수를 셉니다.

- [정리 후 city 종류 수]: (실제값)
- [줄어든 종류 수 = 정리 전 - 정리 후]: (직접 계산)

3단계 — 특정 도시(예: '서울')의 개수가 정리 전(갈라진 조각들의 합)과 정리 후(합쳐진 하나)에서 어떻게 바뀌었는지 대조해 한 문장으로 적습니다.

- [내가 관찰한 변화]: (예: 서울이 세 조각으로 $963 + 141 + 131$ 이던 것이 합쳐져 ...)

자가체크

- [] 정리 전후 `nunique()` 를 실제로 실행해 종류 수가 줄어든 것을 확인했다.
- [] `str.strip()` 결과를 열에 다시 할당했다(할당하지 않으면 원본이 안 바뀔을 이해했다).
- [] 특정 도시의 개수가 정리 후 합쳐진 것을 실제 숫자로 대조해 적었다.

흔한 오류와 교정

오개념: "str 메서드를 부르면 원본이 바뀐다 / 겉보기 같으면 같은 값이다"

`prod["category"].str.strip()` 만 부르고 끝내면 원본은 그대로입니다. `str` 메서드는 새 시리즈를 반환할 뿐이라, `prod["category"] = prod["category"].str.strip()` 처럼 결과를 다시 할당해야 반영됩니다.

또 '의류' 와 '의류 ' 처럼 겉보기 같은 값도 공백·대소문자가 다르면 판다스는 다른 값으로 셉니다. 그래서 집계 전에 반드시 표기를 통일해야 합니다. 마지막으로, 결측(`NaN`)이 섞인 문자열 열에 `str` 메서드를 쓰면 `NaN` 은 처리되지 않고 그대로 남습니다. `map` 으로 표준값을 만들 때 대응표에 없는 값(오타·결측)이 `NaN` 이 되므로, 매핑 후 `value_counts(dropna=False)` 로 누락된 값이 없는지 꼭 확인하십시오.

6.2 단위 미니 프로젝트: 정제 파이프라인과 clean 테이블 저장

5장과 6장에서 배운 정제 기술(결측·중복·이상값·타입·문자열)을 **하나의 함수로 묶어** 재사용 가능한 정제 파이프라인을 만듭니다. 파이프라인은 "규칙을 정하고, 순서대로 적용하고, 전후를 대조하는" 정제의 원칙을 코드로 굳힌 것입니다.

정제에는 **순서**가 있습니다. 대체로 다음 흐름을 따릅니다.

1. **타입 교정** — 문자열로 오염된 숫자 열을 먼저 숫자로 되돌립니다(`to_numeric`). 이후 이상값을 숫자로 비교할 수 있어야 하기 때문입니다.
2. **문자열 표기 통일** — `strip`·`map`으로 범주 표기를 통일합니다. 이후 그룹화·중복 판단이 정확해집니다.
3. **이상값 처리** — 불가능한 값(음수 수량·0 이하 가격)을 결측화하거나 제거합니다.
4. **중복 제거** — 표기·타입이 통일된 상태에서 중복을 판단해야 "진짜 같은 행"이 제대로 잡힙니다.
5. **결측 처리** — 마지막으로 남은 결측을 열의 의미에 맞게 채우거나 남깁니다.

순서가 어긋나면 문제가 생깁니다 — 예를 들어 표기를 통일하기 전에 중복을 제거하면, '서울'과 '서울'이 다른 값이라 같은 고객의 중복이 안 잡힐 수 있습니다.

이 미니 프로젝트가 쓰는 개념: 결측치 처리(c08)·중복·이상값·타입(c09)·문자열 처리(c10). 5장과 6장 전체의 종합 적용입니다.

모범: products 정제 파이프라인

먼저 `products`를 정제하는 함수 하나를 완성한 모습을 봅니다. 각 단계에 5·6장에서 배운 규칙이 그대로 들어 있습니다.

```
import pandas as pd

def clean_products(path: str) -> pd.DataFrame:
    """products.csv를 규칙대로 정제해 clean 데이터프레임을 반환한다."""
    df = pd.read_csv(path)
    before = len(df)

    # 규칙 1) category 공백 통일(문자열 정제 - c10)
    df["category"] = df["category"].str.strip()

    # 규칙 2) price 타입 오염 정리 후 숫자화(타입 정제 - c09)
    price_str = (df["price"].astype(str)
                 .str.replace("원", "", regex=False)
                 .str.replace(",", "", regex=False))
    df["price"] = pd.to_numeric(price_str, errors="coerce")

    # 규칙 3) 0 이하 가격은 이상값 -> 결측 처리(이상값 - c09)
    df.loc[df["price"] <= 0, "price"] = pd.NA

    # 규칙 4) 완전 중복 제거(중복 - c09)
    df = df.drop_duplicates()

    after = len(df)
    print(f"[products] 정제 전 {before}행 -> 정제 후 {after}행")
    print(" category 종류 수:", df["category"].nunique())
    print(" price 자료형:", df["price"].dtype,
          "\t| 가격 결측:", df["price"].isna().sum())
    return df

clean_prod = clean_products("data/products.csv")
```

- **함수로 묶은 이유** — 정제 규칙을 한곳에 모아 순서대로 적으면, 나중에 같은 파일을 다시 정제하거나 규칙을 바꿀 때 이 함수만 고치면 됩니다(재현성).
- `df.loc[df["price"] <= 0, "price"] = pd.NA` — 5장의 loc + 조건을 써서 0 이하 가격 칸만 결측으로 바꿉니다. 이상값을 "삭제"가 아니라 "결측화"해, 이후 상품 기본가로 채우는 등의 후속 처리 여지를 남깁니다.
- **각 규칙에 주석으로 어느 개념(c08~c10)인지 표시** — 정제 규칙을 문서화하는 습관입니다.

실행 결과는 다음과 같습니다.

```
[products] 정제 전 500행 -> 정제 후 500행
category 종류 수: 6
price 자료형: object
가격 결측: 8
```

`category` 가 6종으로 통일되고 0 이하 가격 8건이 결측으로 처리됐습니다(자료형은 `pd.NA` 가 섞여 `object` 로 표시될 수 있으며, 이후 `astype('float64')` 로 확정할 수 있습니다).

모범: 정제 결과 검증하기

정제 함수를 만들었다면, "정말 규칙대로 정제됐는가"를 **코드로 확인**하는 것이 좋습니다. 사람이 눈으로 보는 대신 `assert` 로 조건을 걸어 두면, 나중에 데이터가 바뀌거나 규칙을 잘못 고쳤을 때 바로 오류로 드러납니다.

```
def check_products(df: pd.DataFrame) -> None:
    """정제된 products가 규칙을 만족하는지 검증한다."""
    # 1) category는 6종이어야 한다(공백 통일 확인)
    n_cat = df["category"].nunique()
    assert n_cat == 6, f"category 종류가 6이 아님: {n_cat}"

    # 2) product_id에 중복이 없어야 한다
    dup = df.duplicated(subset=["product_id"]).sum()
    assert dup == 0, f"product_id 중복 {dup}건 남음"

    # 3) 남은 가격은 0보다 커야 한다(0 이하는 결측 처리됐어야 함)
    bad_price = (df["price"] <= 0).sum()
    assert bad_price == 0, f"0 이하 가격 {bad_price}건 남음"

    print("검증 통과: category 6종, product_id 중복 0, 0 이하 가격 0")

check_products(clean_prod)
```

- **assert 조건, 메시지** — 조건이 참이 아니면 오류를 내며 메시지를 보여 줍니다. 정제가 어긋나면 조용히 넘어가지 않고 즉시 멈춥니다.
- **세 가지 규칙을 검증** — 표기 통일(category 6종)·중복 제거(product_id 유일)·이상값 처리(0 이하 가격 없음). 정제 함수가 의도대로 동작했는지 확인하는 계약(contract) 역할을 합니다.
- **정제와 검증의 분리** — 정제하는 함수와 검증하는 함수를 나누면, 규칙이 바뀔 때 어느 쪽을 고쳐야 하는지 분명해집니다.

실행 결과는 다음과 같습니다(모든 조건을 만족하면 통과 메시지만 출력됩니다).

```
검증 통과: category 6종, product_id 중복 0, 0 이하 가격 0
```

만약 정제 규칙 하나를 빠뜨렸다면(예: `strip` 을 안 했다면) 첫 `assert` 에서 `category` 종류가 6이 아님: 12 같은 오류가 나서 곧바로 알아챌 수 있습니다. 이렇게 **정제 뒤에 검증을 붙이는 습관**이 신뢰할 수 있는 분석의 바탕입니다.

직접 만들기 (2종 1세트)

직접 작성 — customers 정제 파이프라인 완성과 clean 저장

위 `clean_products` 를 본떠, `clean_customers` 함수를 **직접 완성**하고 실행하십시오. 아래 스타터의 `#` 여기 직접 작성 자리에 5.6장 규칙을 채워 넣습니다(빈칸 토큰이 아니라 실제 코드를 작성하고 실행해 통과 확인).

```
def clean_customers(path: str) -> pd.DataFrame:
    df = pd.read_csv(path)
    before = len(df)

    # 규칙 1) gender 표기 통일: 남/여/M/F -> M/F (map)
    # 여기 직접 작성

    # 규칙 2) city 앞뒤 공백 제거 (str.strip)
    # 여기 직접 작성

    # 규칙 3) 완전 중복 행 제거 (drop_duplicates)
    # 여기 직접 작성

    after = len(df)
    print(f"[customers] 정제 전 {before}행 -> 정제 후 {after}행")
    print(" gender 종류:", df["gender"].value_counts(dropna=False).to_dict())
    print(" city 종류 수:", df["city"].nunique())
    return df

clean_cust = clean_customers("data/customers.csv")
# 정제 결과를 파일로 저장 (index=False로 의미 없는 정수 인덱스 제외)
clean_cust.to_csv("data/customers_clean.csv", index=False)
```

실행한 뒤 화면에 뜬 **실제 출력**을 아래에 기록하십시오.

- [정제 전 행수 → 정제 후 행수]: (실제 출력, 중복 제거로 줄었는지)
- [정리 후 gender 종류]: (M·F·NaN 각 몇 개인지 실제값)
- [정리 후 city 종류 수]: (실제값, 10 근처인지)
- [저장 확인]: `data/customers_clean.csv` 파일이 생겼는지, 다시 `read_csv` 로 불러와 행수가 같은지

추가 단계 — 위 `check_products` 를 본떠 `check_customers` 검증 함수를 **직접 작성**하십시오. 아래 세 규칙을 `assert` 로 확인하고 실행해, 통과하는지 봅시다(빈칸이 아니라 실제 코드 작성).

- 규칙 1) `customer_id` 에 중복이 없다(`duplicated(subset=['customer_id']).sum() == 0`)
- 규칙 2) `city` 종류 수가 10 이하다

- 규칙 3) `gender` 의 값이 `{'M','F'}` 와 결측만으로 이뤄진다(다른 값이 없다)
- [검증 통과 여부와, 만약 실패했다면 어느 규칙에서 걸렸는지]: (실제 실행 결과)

자가체크

- [] 세 규칙(`gender map city strip drop_duplicates`)을 스타터에 직접 작성해 실행했다.
- [] 정제 전후 행수·`gender` 종류·`city` 종류 수를 실제 출력으로 확인했다(추측하지 않았다).
- [] `to_csv(index=False)` 로 `clean` 파일을 저장하고, 다시 불러와 행수가 같은지 대조했다.
- [] 각 규칙이 어느 개념(`c08~c10`)에서 배운 것인지 주석으로 남겼다.

자주 묻는 질문

Q. `str.strip()` 을 했는데 `city` 종류 수가 여전히 많습니다. 왜인가요?

`strip()` 은 앞뒤 공백만 없앱니다. '서 울' 처럼 글자 사이에 공백이 있으면 남습니다. 이럴 때는 `str.replace(" ", "", regex=False)` 로 모든 공백을 없애거나, `value_counts()` 로 어떤 표기가 남았는지 확인한 뒤 개별 `replace` 로 다듬습니다. 늘 "정리 후 `value_counts` 로 눈으로 확인"하는 것이 답을 줍니다.

Q. `map` 과 `replace` 는 어떻게 다른가요?

`map` 은 시리즈의 모든 값을 대응표로 바꾸며, 대응표에 없는 값은 `NaN` 이 됩니다. `replace` 는 지정한 값만 바꾸고 나머지는 그대로 둡니다. 그래서 "몇 개 코드만 라벨로 바꾸고 나머지는 유지"할 때는 `replace` 가, "정해진 몇 값을 표준값으로 통일하고 나머지는 결측 처리"할 때는 `map` 이 편합니다. `gender` 처럼 값이 네 종류로 고정된 열은 `map` 이 누락을 드러내 줘 안전합니다.

Q. 정제 파이프라인은 왜 함수로 만드나요? 그냥 순서대로 실행하면 안 되나요?

함수로 묶으면 세 가지 이점이 있습니다. 첫째, **재현성** — 같은 파일을 다시 정제하거나 다른 사람이 실행해도 같은 규칙이 같은 순서로 적용됩니다. 둘째, **수정 용이성** — 규칙이 바뀌면 함수 한 곳만 고치면 됩니다. 셋째, **문서화** — 함수 안의 주석이 "이 데이터에 어떤 정제 규칙을 적용했는가"의 기록이 됩니다. 종합 프로젝트(12~15장)에서는 이런 정제 함수들이 재현 가능한 분석 스크립트의 핵심이 됩니다.

이 장을 마치며

이 장에서 문자열 처리(개념 c10)로 표기 불일치를 통일하고, 5·6장의 정제를 하나의 **정제 파이프라인 함수**로 묶었습니다. 핵심은 세 가지입니다. 첫째, **겉보기 같아도 공백·대소문자가 다르면 다른 값**이므로, 집계 전에 `str.strip.map` 으로 표기를 통일합니다. 둘째, `str` 메서드는 새 시리즈를 반환하므로 **결과를 다시 할당**해야 합니다. 셋째, 정제 규칙을 **함수로 문서화**하면 재현 가능하고 수정하기 쉬운 파이프라인이 됩니다.

5장과 6장을 통틀어, 우리는 지저분한 원본 데이터를 **믿을 수 있는 상태로** 바꿨습니다. 결측을 열의 의미에 맞게 처리하고, 중복·이상값을 규칙으로 걸러내며, 타입 오염을 바로잡고, 문자열 표기를 통일했습니다. 그리고 그 모든 규칙을 함수로 문서화하고 검증까지 붙였습니다. 이제 이 깨끗한 데이터는 어떤 집계·병합·시계열 분석에도 흔들림 없는 토대가 됩니다.

이제 데이터가 깨끗해졌으니, 다음 7장에서는 이 정제된 데이터 위에 **분석 축을 더합니다** — 단가·수량·할인으로 주문 금액을 계산하고, 나이·가격을 연령대·가격대로 구간화(binning)해 집계의 재료를 만듭니다. 정제가 "데이터를 믿을 수 있게" 만드는 일이었다면, 파생은 "데이터에서 답을 끌어낼 준비"를 하는 일입니다. 지저분한 데이터를 신뢰할 수 있는 분석 결과로 바꾸는 이 여정에서, 정제는 가장 눈에 안 띄지만 가장 중요한 발판이었습니다.

7장 파생 변수와 데이터 변환

5~6장에서 지저분한 데이터를 정제해 믿을 수 있는 상태로 만들었습니다. 그런데 정작 분석에 필요한 값은 원본에 없을 때가 많습니다. `order_items`에는 단가(`unit_price`)·수량(`quantity`)·할인(`discount`)만 있고 **주문 금액**은 없습니다. `customers`에는 생년월일은 있지만 **연령대**는 없습니다. 이런 값은 기존 열로 **계산해 새로 만들어야** 합니다. 이 장에서는 벡터화 연산(vectorized operation)으로 파생 변수(derived column)를 만들고, 연속값을 구간화(binning)해 연령대·가격대 같은 분석 축을 만드는 법을 배웁니다. 여기서 만든 열들이 8장의 그룹화·집계에서 그룹을 나누는 기준이 됩니다.

이 장의 학습 목표

- 벡터 연산으로 `unit_price · quantity · discount`에서 주문 금액(`amount`) 열을 만들고 표본 행을 손계산과 대조해 검증할 수 있다 [→G5]
- `assign · map · apply · np.where`로 파생 열과 조건부 값을 만들 수 있다 [→G5]
- `cut · qcut`으로 나이·가격·금액을 연령대·가격대·등급으로 구간화하고 `labels`로 이름을 붙일 수 있다 [→G5]
- `map · replace`로 코드값(`status`)을 읽기 쉬운 라벨로 매핑하고 각 범주 분포를 확인할 수 있다 [→G5]

선수 확인: 이 장은 5~6장(정제)을 마쳤다고 가정합니다. `order_items`의 `unit_price` 결측을 제거하고 `quantity > 0`으로 걸러 수 있으며, `customers`의 `birth_date`를 다룰 수 있어야 합니다. 파생은 정제된 데이터 위에서 해야 의미가 있으므로(오염된 값으로 계산하면 결과도 오염됨), 정제를 먼저 끝낸 상태를 전제합니다.

7.1 파생 열 만들기와 변환 (개념 c11)

도입: 반복문 없이 열 전체를 한 번에 계산한다

주문 금액을 구하려면 각 행에서 `단가 × 수량 × (1 - 할인율)` 을 계산해야 합니다. 파이썬만 안다면 `for` 문으로 한 행씩 돌 생각이 들 텐데, 50만 행에서는 느리고 코드도 길습니다. 판다스는 **열 전체를 한 번에 계산하는 벡터화 연산**을 지원하므로 `df['unit_price'] * df['quantity']` 한 줄이면 모든 행이 동시에 계산됩니다. 이것이 판다스가 빠른 이유이자, 파생 열을 만드는 기본 방식입니다.

파생 변수는 데이터 분석에서 흔히 "피처(feature)"라고도 부르며, 좋은 파생 변수를 잘 만드는 것이 분석·모델링의 성패를 가른다고 할 만큼 중요합니다. 원본 그대로는 답이 안 나오던 질문도, `단가 × 수량` 으로 매출을 만들고 나이를 연령대로 묶는 순간 답할 수 있게 됩니다. 이 장의 도구는 단순하지만, 어떤 축을 만들지 정하는 것은 분석가의 판단입니다.

선수 개념 상기: 파생 열도 5장의 `loc` + 조건과 이어집니다. 조건에 따라 다른 값을 넣을 때는 `np.where` 를 쓰고, 특정 조건 행만 골라 값을 넣을 때는 `df.loc[조건, '열'] = 값` 을 씁니다.

정의

파생 변수(derived column) 는 기존 열을 계산해 새로 만든 열입니다(주문 금액·마진 등). 판다스에서 만드는 방법은 여러 가지입니다.

- **벡터 연산:** `df['amount'] = df['unit_price'] * df['quantity']` — 열끼리 사칙연산하면 행 별로 자동 계산됩니다(반복문 불필요).
- **assign:** `df.assign(amount=...)` — 원본을 바꾸지 않고 새 열을 더한 **사본**을 반환해, 메서드를 이어 붙이는 체이닝에 좋습니다.
- **map:** 시리즈의 각 값을 딕셔너리·함수로 바꿉니다(요소별 변환).
- **apply:** 행/열 단위로 사용자 함수를 적용합니다(유연하지만 큰 데이터에서 느릴 수 있음).
- **np.where(조건, 참값, 거짓값):** 조건에 따라 두 값 중 하나를 넣습니다. 여러 조건은 `np.select` 를 씁니다.

파생 결과는 **표본 몇 행을 손으로 계산해 대조**하는 검증이 중요합니다.

동작 원리: 벡터화가 우선, apply는 마지막 수단

같은 파생을 여러 방법으로 만들 수 있지만 성능·가독성이 다릅니다.

방법	언제 쓰나	성능
벡터 연산(<code>a * b</code>)	사칙연산·비교로 표현되는 대부분	가장 빠름
<code>np.where</code> / <code>np.select</code>	조건에 따라 다른 값	빠름
<code>map</code> (딕셔너리)	값 하나하나를 대응표로 치환	빠름
<code>apply</code> (함수)	벡터화로 표현 안 되는 복잡한 행 계산	느릴 수 있음

원칙은 "벡터 연산으로 되면 벡터 연산, 조건 분기는 `np.where`, 정말 안 되면 `apply`" 입니다. `apply` 는 편하지만 내부적으로 행마다 파이썬 함수를 부르므로 50만 행에서는 눈에 띄게 느립니다.

원본을 보존하며 여러 파생 열을 한 번에 더할 때는 `assign` 이 편합니다. `products` 에 마진(판매가 - 원가)과 마진율을 함께 더해 봅니다.

```
import pandas as pd

prod = pd.read_csv("data/products.csv")
prod["price"] = pd.to_numeric(
    prod["price"].astype(str).str.replace("원", "", regex=False)
    .str.replace(",", "", regex=False), errors="coerce")
prod = prod[prod["price"] > 0]

# assign으로 두 파생 열을 한 번에(원본은 그대로, 사본 반환)
prod2 = prod.assign(
    margin=prod["price"] - prod["cost"],
    margin_rate=(prod["price"] - prod["cost"]) / prod["price"],
)
print(prod2[["product_name", "price", "cost",
             "margin", "margin_rate"]].head(4).round(3))
```

- `prod.assign(margin=..., margin_rate=...)` — 두 파생 열을 한 번에 더한 사본을 반환합니다. 원본 `prod` 는 바뀌지 않으므로, 결과를 `prod2` 에 받습니다.
- `margin_rate` — 마진을 판매가로 나눈 비율입니다. 파생 열을 다른 파생의 재료로 바로 쓸 수 있습니다.

실행 결과는 다음과 같습니다.

	product_name	price	cost	margin	margin_rate
1	프리미엄 노트북	68600.0	28900.0	39700.0	0.579
2	프리미엄 원두커피	10500.0	4700.0	5800.0	0.552
3	베이직 마우스	52100.0	32600.0	19500.0	0.374
4	베이직 홍차	16000.0	10500.0	5500.0	0.344

첫 행 노트북의 마진을 손으로 확인하면 $68600 - 28900 = 39700$ 으로 표와 일치하고, 마진율은 $39700 / 68600 \approx 0.579$ 로 맞습니다.

worked example: 주문 금액 파생과 손계산 대조

정제한 `order_items` 에 주문 금액(`amount`) 열을 벡터 연산으로 만들고, 앞 몇 행을 손으로 계산해 대조하겠습니다.

```
import pandas as pd
import numpy as np

oi = pd.read_csv("data/order_items.csv")
# 5장 정제: 단가 결측 제거 + 수량 1 이상
oi = oi.dropna(subset=["unit_price"])
oi = oi[oi["quantity"] > 0].copy()

# 벡터 연산으로 주문 금액 파생: 단가 x 수량 x (1 - 할인율)
oi["amount"] = oi["unit_price"] * oi["quantity"] * (1 - oi["discount"])

print(oi[["unit_price", "quantity", "discount", "amount"]].head(5))
print("\namount 요약:")
print("건수:", oi["amount"].count(),
      "| 최소:", round(oi["amount"].min(), 2),
      "| 최대:", round(oi["amount"].max(), 2))
```

- `oi["unit_price"] * oi["quantity"] * (1 - oi["discount"])` — 세 열이 행별로 곱해집니다. `discount` 가 0.05면 $(1 - 0.05) = 0.95$ 를 곱해 5% 할인이 반영됩니다. 반복문 없이 모든 행이 한 번에 계산됩니다.
- `.copy()` — 필터링한 결과에 새 열을 더할 때 경고 없이 안전하게 쓰려고 사본으로 확정합니다.
- `.count()` — 결측이 아닌 값의 개수로, 파생이 몇 행에 적용됐는지 봅니다.

실행 결과는 다음과 같습니다.

```

    unit_price  quantity  discount  amount
0    20300.0         1      0.05  19285.0
1    90000.0         2      0.45  99000.0
2    20300.0         3      0.23  46893.0
3     8600.0         1      0.38   5332.0
4    42600.0         1      0.31  29394.0

amount 요약:
건수: 484400 | 최소: 500.0 | 최대: 1440450.0

```

첫 행을 손으로 계산해 대조해 봅니다. $20300 \times 1 \times (1 - 0.05) = 20300 \times 0.95 = 19285$. 표의 amount 와 정확히 일치합니다. 둘째 행도 $90000 \times 2 \times 0.55 = 99000$ 으로 일치합니다. 이렇게 표 본 몇 행을 손계산과 대조하면 파생 공식이 맞는지 검증됩니다. 50만 행을 다 계산할 수는 없지만, 서로 다른 몇 행(할인율이 큰 행·수량이 여럿인 행)을 골라 맞으면 공식 전체를 신뢰할 근거가 됩니다.

이제 금액에 따라 "고액"."일반"을 표시하는 조건부 파생 열을 np.where 로 만듭니다.

```

# 금액 10만 이상이면 '고액', 아니면 '일반'
oi["grade"] = np.where(oi["amount"] >= 100000, "고액", "일반")
print(oi["grade"].value_counts())

```

실행 결과는 다음과 같습니다.

```

grade
일반    320442
고액     163958
Name: count, dtype: int64

```

50만에 가까운 행이 반복문 한 번 없이 두 등급으로 나뉘었습니다. 이 grade 열은 8장에서 "등급별 매출"을 집계하는 축이 됩니다.

두 가지가 아니라 세 단계 이상으로 나눌 때는 np.where 를 겹치기보다 np.select 가 깔끔합니다. 조건 목록과 값 목록을 짝지어 줍니다.

```

# np.select로 세 등급: 20만 이상 '최고액', 5만 이상 '고액', 나머지 '일반'
conditions = [oi["amount"] >= 200000, oi["amount"] >= 50000]
choices = ["최고액", "고액"]
oi["tier"] = np.select(conditions, choices, default="일반")
print(oi["tier"].value_counts())

```

- np.select(conditions, choices, default=...) — 조건을 위에서부터 검사해, 처음 참이 되는 조건의 값을 넣습니다. 어느 조건에도 안 맞으면 default 를 씁니다.

- **조건 순서가 중요** — 금액 25만 원인 행은 첫 조건(20만 이상)에서 이미 '최고액' 이 됩니다. 큰 경계를 먼저 두어야 의도대로 나뉩니다.

실행 결과는 다음과 같습니다.

```
tier
일반      214435
고액      182057
최고액     87908
Name: count, dtype: int64
```

세 등급으로 나뉘었고, 세 개수의 합($214,435 + 182,057 + 87,908 = 484,400$)이 전체 행수와 같아 누락 없이 분류됐음이 확인됩니다. 이렇게 분류 결과의 개수 합을 전체와 대조하는 것도 파생의 검증 방법입니다 — `default` 를 빠뜨리면 어느 조건에도 안 맞는 행이 생겨 합이 안 맞습니다.

`apply` 는 벡터 연산·`np.where` 로 표현하기 어려운 **복잡한 행 계산**에 씁니다. 다만 느낄 수 있어 마지막 수단입니다.

```
# apply 예: 금액과 할인율을 함께 보는 사용자 정의 라벨(느릴 수 있음)
def label(row):
    if row["amount"] >= 100000 and row["discount"] >= 0.3:
        return "고액-대폭할인"
    return "기타"

# 앞 5행에만 적용해 결과를 확인(전체 적용은 벡터화로 대체 권장)
print(oi.head(5).apply(label, axis=1).tolist())
```

- `apply(label, axis=1)` — `axis=1` 은 **행 단위**로 함수를 적용한다는 뜻입니다. 함수 안에서 `row["amount"]` 처럼 여러 열을 함께 봅니다.
- **성능 주의** — `apply` 는 행마다 파이썬 함수를 부르므로 50만 행에서는 느립니다. 위 조건은 사실 `np.where((oi["amount"]>=100000) & (oi["discount"]>=0.3), ...)` 로 벡터화할 수 있습니다. `apply` 는 정말 벡터화가 안 될 때만 쓰십시오.

실행 결과는 다음과 같습니다(앞 5행 기준).

```
['기타', '기타', '기타', '기타', '기타']
```

즉시 연습 (2종 1세트)

직접 해 보기 — 마진 파생과 손계산 대조

`products`에는 판매가(`price`)와 원가(`cost`)가 있습니다. **마진 = price - cost**를 파생 열로 만들고 손계산으로 검증하십시오. 아래를 **직접 실행**하고 실제 출력을 기록합니다.

1단계 — `products.csv`를 불러와 6장 규칙으로 `price`를 숫자로 정리한 뒤, `assign`으로 `margin = price - cost` 열을 더합니다. 앞 3행의 `price · cost · margin`을 출력해 기록합니다.

- [앞 3행 `price / cost / margin`]: (실제 출력값)

2단계 — 첫 행에서 `price - cost`를 손으로 계산해 파생 결과와 같은지 대조합니다.

- [내 손계산값]: (직접 계산)
- [`margin` 열의 실제값과 일치하는가]: (예/아니오)

3단계 — `np.where`로 마진이 원가보다 큰 상품(`margin > cost`)을 '고마진', 아니면 '일반'으로 표시한 열을 만들고 `value_counts`를 기록합니다.

- [고마진 / 일반 각 개수]: (실제 출력값)

자가체크

- [] `assign`으로 `margin` 열을 만들고 앞 3행을 실제로 출력했다.
- [] 첫 행의 `price - cost`를 손으로 계산해 파생 결과와 일치함을 확인했다(추측하지 않았다).
- [] `np.where`로 조건부 열을 만들고 `value_counts`로 분포를 확인했다.

흔한 오류와 교정

오개념: "새 열은 반복문으로 만든다 / `assign`이 원본을 바꾼다"

파생 열을 만들 때 `for i, row in df.iterrows(): ...`로 한 행씩 도는 것은 **느립니다**. 50만 행이면 벡터 연산의 수십 배 시간이 걸립니다. 사칙연산·비교로 표현되면 먼저 **벡터 연산**을, 조건 분기는 `np.where`를 쓰십시오. `apply`도 편하지만 큰 데이터에서 느릴 수 있으니 벡터화로 대체 가능한지 먼저 보십시오.

또 `df.assign(amount=...)`는 **원본을 바꾸지 않고 새 사본을 반환**합니다. `df.assign(...)`만 부르고 결과를 안 받으면 원본에는 아무 열도 안 생깁니다. `df = df.assign(...)`로 다시 받거나, 원본을 직접 바꾸려면 `df['amount'] = ...` 대입을 쓰십시오.

7.2 구간화와 범주 매핑 (개념 c12)

도입: 연속값은 구간으로 나눠야 패턴이 보인다

나이가 27, 34, 41처럼 제각각인 수백 개 값을 그대로 두면 "나이별 매출"이 의미 없이 잘게 쪼개집니다. 대신 **20대·30대·40대** 같은 구간으로 묶으면 패턴이 드러납니다. 가격도 마찬가지로 저가·중가·고가로 나누면 가격대별 판매 경향을 볼 수 있습니다. 이렇게 연속값을 몇 개의 구간으로 나누는 것이 구간화(binning)이며, 이미 코드로 된 값(`status · gender`)을 읽기 쉬운 라벨로 바꾸는 것이 범주 매핑입니다.

선수 개념 상기: 여기서 만든 연령대·가격대 열은 7.1의 파생 열과 함께 8장 그룹화의 그룹 축이 됩니다. 구간화는 곧 "집계의 재료 만들기"입니다.

정의

구간화(binning) 는 연속값을 여러 구간으로 나눠 범주로 만드는 것입니다.

- `cut(x, bins, labels=...)` — **지정한 경계**로 값을 구간에 배정합니다. 예: 나이를 `[0, 19, 29, 39, ...]` 경계로 10년 단위 연령대로. 경계를 직접 정하므로 "20대", "30대"처럼 **의미 있는 구간**을 만들 때 씁니다.
- `qcut(x, q, labels=...)` — **분위수 기준**으로 각 구간에 **같은 개수**가 들어가게 나눕니다. 예: 지출을 4등분해 하위 25%·상위 25% 등급으로. 개수를 균등하게 나눌 때 씁니다.
- `labels` — 구간에 '20대' · '고가' 같은 읽기 쉬운 이름을 붙입니다.
- **범주 매핑**(`map · replace`) — 이미 코드로 된 값('paid')을 사람이 읽는 라벨('결제완료')이나 표준값으로 바꿉니다.

동작 원리: cut과 qcut의 차이

나이를 `cut(bins=[0,20,30,40,100])`으로 나누면 구간 폭이 제각각이고, `qcut(q=3)`으로 나누면 개수는 같지만 값 폭이 다른 하·중·상 구간이 되는 모습을 두 수직선으로 비교한 구간화 개념도.

`cut` 과 `qcut` 은 목적이 다릅니다.

- **cut** 은 **경계값 기준**입니다. `[0, 29, 49, 200]` 으로 나누면 값이 어느 경계 사이에 있느냐로 구간이 정해지므로, 각 구간의 개수는 데이터 분포에 따라 다릅니다. "나이 30세 미만/30~50세/50세 초과"처럼 **의미가 정해진 경계**가 있을 때 적합합니다.
- **qcut** 은 **개수 기준**입니다. `q=4` 면 값을 크기순으로 정렬해 네 덩어리에 **같은 개수**씩 넣습니다. "지출 상위 25%"처럼 **상대적 등급**을 매길 때 적합합니다.

주의할 점으로 `cut` 의 구간은 기본이 **오른쪽 포함**(`right=True`)입니다. `[0, 29, 49]` 면 29는 첫 구간(0 초과 29 이하)에 들어갑니다. 경계값이 어디 들어갈지 헷갈리면 결과를 `value_counts` 로 확인하십시오.

구간 경계와 라벨을 **어떻게 정하느냐가 분석 결과의 해석을 좌우**한다는 점도 중요합니다. 연령대를 10년 단위로 나누느냐 20년 단위로 나누느냐에 따라 "어느 연령대가 큰 손님인가"의 답이 달라집니다. 정답은 없지만, 왜 그 경계로 나눴는지(업계 관례·데이터 분포·분석 목적) 근거를 남기면 결과를 설득력 있게 설명할 수 있습니다.

worked example: 연령대·가격대 구간화와 status 라벨링

`customers` 의 나이를 연령대로 `cut` 하고, `products` 의 가격을 `qcut` 으로 등급화하며, `orders` 의 `status` 를 읽기 쉬운 라벨로 매핑하겠습니다.

```
import pandas as pd

cust = pd.read_csv("data/customers.csv")
cust = cust.dropna(subset=["birth_date"]).copy()
cust["birth_date"] = pd.to_datetime(cust["birth_date"])
cust["age"] = 2024 - cust["birth_date"].dt.year # 2024년 기준 나이

# cut: 지정 경계로 연령대 구간화(오른쪽 포함)
bins = [0, 19, 29, 39, 49, 59, 200]
labels = ["10대이하", "20대", "30대", "40대", "50대", "60대이상"]
cust["age_group"] = pd.cut(cust["age"], bins=bins, labels=labels)
print(cust["age_group"].value_counts().sort_index())
```

- `2024 - cust["birth_date"].dt.year` — 생년월일에서 연도만 뽑아(`.dt.year`) 기준 연도에 서 빼 나이를 계산합니다(자세한 날짜 다루기는 10장).
- `pd.cut(cust["age"], bins=bins, labels=labels)` — 나이를 여섯 경계로 나눠 각 구간에 라벨을 붙입니다. `bins` 가 7개면 구간은 6개(경계 사이)입니다.
- `.sort_index()` — 결과를 라벨 순서대로 정렬해 봅니다.

실행 결과는 다음과 같습니다.

```
age_group
10대이하    168
20대        817
30대        893
40대        897
50대        903
60대이상    938
Name: count, dtype: int64
```

나이가 여섯 연령대로 나뉘었고, 각 구간의 개수가 다릅니다(분포에 따라 다름). 이제 가격을 `qcut` 으로 등급화합니다.

```
import pandas as pd

prod = pd.read_csv("data/products.csv")
# 6장 규칙으로 price 숫자화
price = (prod["price"].astype(str)
         .str.replace("원", "", regex=False)
         .str.replace(",", "", regex=False))
prod["price"] = pd.to_numeric(price, errors="coerce")
prod = prod[prod["price"] > 0].copy()

# qcut: 분위수 기준 4등급(같은 개수씩)
prod["price_tier"] = pd.qcut(prod["price"], q=4,
                             labels=["저가", "중가", "고가", "최고가"])
print(prod["price_tier"].value_counts().sort_index())

# 범주 매핑: status 코드값을 읽기 쉬운 라벨로
orders = pd.read_csv("data/orders.csv")
smap = {"paid": "결제완료", "delivered": "배송완료", "shipped": "배송중",
        "canceled": "취소", "returned": "반품"}
orders["status_ko"] = orders["status"].map(smap)
print("\nstatus 라벨링 결과:")
print(orders["status_ko"].value_counts())
```

- `pd.qcut(prod["price"], q=4, ...)` — 가격을 크기순으로 4등분해 각 등급에 거의 같은 개수를 넣습니다. `cut` 과 달리 경계를 데이터가 정합니다.
- `orders["status"].map(smap)` — 각 코드값을 대응표의 한국어 라벨로 바꿉니다. 대응표의 다섯 키를 모두 채웠으므로 누락 없이 매핑됩니다.

실행 결과는 다음과 같습니다.

```
price_tier
저가      124
중가      123
고가      122
최고가    123
Name: count, dtype: int64

status 라벨링 결과:
status_ko
배송완료    109976
배송중      30017
결제완료    29993
취소        20039
반품        9975
Name: count, dtype: int64
```

qcut 이라 네 등급에 거의 같은 개수(124·123·122·123)가 들어갔고, status 코드가 모두 한국어 라벨로 바뀌었습니다. 만약 대응표에 'shipped' · 'returned' 를 빠뜨렸다면 그 값들은 NaN 이 됐을 것입니다(아래 오개념 참고).

cut 의 **경계값이 어느 구간에 들어가는지**는 실수가 잦은 지점이라 작은 예로 확인해 봅니다. 정확히 29·30·49·50 같은 경계 근처 값이 핵심입니다.

```
import pandas as pd

ages = pd.Series([19, 20, 29, 30, 49, 50])
# 기본 right=True(오른쪽 포함): (0,29], (29,49], (49,200]
g_right = pd.cut(ages, bins=[0, 29, 49, 200],
                 labels=["20대이하", "30-40대", "50대이상"])
print("right=True :", g_right.tolist())
```

- **right=True (기본)** — 구간이 (0, 29] 처럼 **오른쪽을 포함**합니다. 그래서 29세는 첫 구간에, 30세는 둘째 구간에 들어갑니다.
- 경계값(29·49)이 **아래 구간에 붙는**다는 점을 기억하면 됩니다. 반대로 하려면 right=False 를 줍니다.

실행 결과는 다음과 같습니다.

```
right=True : ['20대이하', '20대이하', '20대이하', '30-40대', '30-40대', '50대이상']
```

29세까지가 '20대이하', 30~49세가 '30-40대', 50세부터가 '50대이상' 으로 나뉩니다. 이렇게 경계 근처 값 몇 개로 확인하면 구간이 의도대로 잡혔는지 검증됩니다.

즉시 연습 (2종 1세트)

직접 해 보기 — 주문 금액 등급화(qcut)와 채널 라벨링(map)

7.1에서 만든 주문 금액(amount)을 qcut 으로 등급화하고, channel 코드를 한국어로 매핑하십시오. **직접 실행**하고 실제 출력을 기록합니다.

1단계 — order_items 에 amount 를 만든 뒤(7.1 방식), qcut(amount, q=3, labels=['소액', '중간', '고액']) 으로 세 등급을 만들고 value_counts 를 기록합니다.

- **[소액 / 중간 / 고액 각 개수]**: (실제 출력값 — 세 등급이 거의 같은 개수인지 확인)

2단계 — orders 의 channel (web · app · store)을 map 으로 한국어 ({'web': '웹', 'app': '앱', 'store': '매장'})로 바꾸고 value_counts 를 기록합니다.

- [웹 / 앱 / 매장 각 개수]: (실제 출력값)
- [매핑 후 NaN이 있는가]: (`value_counts(dropna=False)` 로 확인 — 없어야 정상)

3단계 — `qcut(q=3)` 의 세 등급 개수가 왜 거의 같은지, `cut` 으로 만들면 어떻게 다를지 한 문장으로 적습니다.

- [qcut과 cut의 차이 관찰]: (직접 작성)

자가체크

- [] `qcut(q=3)` 으로 금액을 등급화해 세 등급 개수가 거의 같음을 실제 출력으로 확인했다.
- [] `channel` 을 `map` 으로 라벨링하고 `dropna=False` 로 누락(NaN)이 없는지 확인했다.
- [] `qcut` (개수 기준)과 `cut` (경계 기준)의 차이를 내 말로 한 문장 적었다.

흔한 오류와 교정

오개념: "cut과 qcut은 같다 / map은 모든 값을 바꿔 준다"

`cut` 은 **경계값 기준**, `qcut` 은 **개수 기준**입니다. "20대·30대"처럼 경계가 정해진 구간은 `cut`, "상위 25%"처럼 균등 등급은 `qcut` 을 쓰십시오. 둘을 혼동해 `cut` 을 쓰면 개수가 한쪽에 몰릴 수 있습니다. 또 `cut` 의 구간은 기본 **오른쪽 포함**(`right=True`)이라, 경계값(예: 29세)이 어느 구간에 들어갈지 확인해야 합니다.

`map` 은 **대응표에 없는 값을 NaN으로 만듭니다**. `status` 를 매핑할 때 'shipped' 를 대응표에 안 넣으면, 배송중 주문 3만여 건이 조용히 NaN 이 되어 집계에서 사라집니다. 매핑 후에는 반드시 `value_counts(dropna=False)` 로 **누락된 값이 없는지** 확인하십시오. 누락이 있으면 대응표에 키를 추가합니다.

7.3 단원 미니 프로젝트: 분석 축 만들기

정제된 데이터에 이 장에서 배운 파생·구간화·매핑을 모두 적용해, 8장 집계에 바로 쓸 **분석 축**이 갖춰진 테이블을 만듭니다.

이 미니 프로젝트가 쓰는 개념: 파생 열 만들기(c11)·구간화와 범주 매핑(c12). 7장 전체의 종합 적용입니다.

직접 만들기 (2종 1세트)

직접 작성 — 세 테이블에 분석 축 더하기

아래 세 가지 파생·범주 열을 **직접 만들어** 각 테이블에 더하고, 결과를 저장하십시오(빈칸 추측이 아니라 실제 코드 작성·실행).

1단계 — `order_items` (정제본)에 `amount` (주문 금액, 벡터 연산)와 `grade` (`np.where` 로 10만 이상 '고액')를 더합니다.

- **[amount 앞 3행과 grade value_counts]:** (실제 출력값)

2단계 — `customers` (정제본)에 `age_group` (`cut` 으로 연령대)을 더합니다. 경계·라벨은 본문의 6 구간을 그대로 써도 되고 직접 정해도 됩니다.

- **[age_group value_counts]:** (실제 출력값)
- **[내가 정한 경계와 라벨]:** (직접 작성 — 왜 그 경계로 나뉘는지 근거 한 줄)

3단계 — `orders` 에 `status_ko` (`map` 으로 한국어 라벨)를 더하고, 매핑 후 `NaN` 이 없는지 확인합니다.

- **[status_ko value_counts(dropna=False)]:** (실제 출력값 — `NaN` 0 확인)

4단계 — 세 테이블을 각각 `to_csv(index=False)` 로 저장합니다(예: `order_items_axis.csv`).

- **[저장한 파일 3개 이름과 각 행수]:** (실제값)

자가체크

- [] `amount` 를 벡터 연산으로 만들고 첫 행을 손계산과 대조했다.
- [] `age_group` 경계를 스스로 정하고 왜 그렇게 나뉘는지 근거를 적었다.
- [] `status_ko` 매핑 후 `dropna=False` 로 누락(`NaN`)이 없음을 확인했다.
- [] 세 테이블을 파일로 저장하고 각 행수를 기록했다.

자주 묻는 질문

Q. `cut` 의 경계값은 구간에 포함되나요, 안 되나요?

기본은 **오른쪽 포함** (`right=True`)입니다. `cut(age, bins=[0, 29, 49])` 이면 29세는 첫 구간 `(0, 29]` (0 초과 29 이하)에 들어가고, 30세는 둘째 구간에 들어갑니다. 왼쪽을 포함하고 싶으면 `right=False` 를 줘 `[0, 29)` 형태로 바꿉니다. 경계값(정확히 29·49 같은 값)이 어느 쪽에 들어가

야 맞는지 생각하고, 헛갈리면 그 경계 근처 값 몇 개를 만들어 `cut` 결과를 눈으로 확인하십시오.

Q. `qcut` 을 했더니 `ValueError: Bin edges must be unique` 가 납니다. 왜인가요?

같은 값이 너무 많아 분위수 경계가 겹칠 때 납니다(예: 절반이 0이면 하위 50% 경계가 모두 0). `qcut(x, q, duplicates='drop')` 으로 겹치는 경계를 버리게 하거나, 구간 수(`q`)를 줄이거나, 애초에 값의 분포가 `qcut` 에 맞는지(연속적이고 고르게 퍼졌는지) 확인하십시오.

Q. 파생 열을 만들 때 원본이 바뀌는 게 무섭습니다. 안전하게 하려면?

`df.assign(새열=...)` 을 쓰면 **원본을 그대로 두고** 새 열이 더해진 사본을 돌려줍니다. 여러 파생 열을 이어서 만들 때 `df.assign(a=...).assign(b=...)` 처럼 체이닝하면 각 단계가 원본을 건드리지 않아 안전합니다. 원본을 정말 바꾸려는 의도가 분명할 때만 `df['새열'] = ...` 대입을 쓰십시오.

이 장을 마치며

이 장에서 파생 열 만들기(개념 c11)와 구간화·범주 매핑(개념 c12)으로 데이터에 **분석 축**을 더했습니다. 핵심은 세 가지입니다. 첫째, 파생은 **벡터 연산이 먼저**이고 조건 분기는 `np.where`, `apply` 는 마지막 수단입니다. 둘째, 연속값은 `cut` (**경계 기준**)·`qcut` (**개수 기준**) 으로 목적에 맞게 구간화합니다. 셋째, 코드값은 `map` 으로 라벨링하되 **누락(NaN)이 없는지 반드시 확인**합니다. 파생 결과는 항상 표본 손계산으로 검증합니다.

파생과 구간화가 왜 분석의 핵심 준비인지 다시 정리하면 이렇습니다. 원본에는 `unit_price · quantity` 같은 **날것의 값**만 있습니다. 여기서 "얼마나 팔렸나"(주문 금액), "어떤 고객층인가"(연령대), "고가인가 저가인가"(가격대) 같은 **분석이 실제로 묻는 질문의 축**은 우리가 계산해 만들어야 합니다. 이 축이 없으면 8장의 "연령대별 매출", "가격대별 판매량" 같은 집계 자체가 불가능합니다. 즉 파생·구간화는 집계로 가는 다리입니다.

다음 8장에서는 이렇게 만든 축(`category · channel · age_group · grade`)으로 데이터를 **그룹으로 나눠 집계**합니다 — "카테고리별 매출", "채널별 주문 건수"처럼 분석의 결론에 가장 가까운 요약표를 만듭니다.

8장 그룹화와 집계

지금까지 데이터를 불러오고(2장), 원하는 부분을 선택·필터하고(3~4장), 정제하고(5~6장), 분석 축을 더했습니다(7장). 이제 드디어 **분석의 결론에 가장 가까운 요약**을 만들 차례입니다. "어느 카테고리가 가장 잘 팔리는가?", "채널별 주문 건수는?", "카테고리별로 채널 비중이 다른가?" 같은 질문은 모두 **기준별로 데이터를 나눠 계산하는 그룹화(groupby)**로 답합니다. 이 장에서는 그룹화·집계(aggregation)의 핵심 원리인 '쪼개서-계산해-합치기'와, 두 축을 한 표로 요약하는 피벗테이블(pivot table)·교차표(crosstab)를 배웁니다.

50만 행을 그대로 놓고 보면 아무것도 안 보입니다. 하지만 카테고리별로 묶어 매출을 더하면 여섯 줄로 줄어들고, "전자와 가구가 매출을 이끈다"는 결론이 한눈에 드러납니다. 집계란 이렇게 **많은 행을 의미 있는 요약으로 압축하는 일**이며, 데이터 분석에서 "그래서 결론이 뭔데?"에 답하는 가장 직접적인 도구입니다. 엑셀에서 피벗테이블을 만들어 본 적이 있다면, 이 장은 그 작업을 코드로 — 반복 가능하고 재현 가능하게 — 하는 법을 배우는 것입니다.

이 장의 학습 목표

- `groupby` 로 카테고리별·채널별 매출 합계와 건수를 집계하고 정렬해 상위 그룹을 뽑을 수 있다 [→G6]
- `agg` 의 이름 붙은 집계로 한 그룹에 여러 지표를 계산하고 `transform` 으로 그룹 통계를 원래 행에 되돌릴 수 있다 [→G6]
- `pivot_table` 로 두 축(카테고리×채널) 매출 요약표를 만들고 `margins` · `fill_value` 로 정리할 수 있다 [→G6]
- `crosstab` 으로 빈도·비율표를 만들고 집계 총합을 원본과 대조해 검증할 수 있다 [→G6]

선수 확인: 이 장은 5~7장을 마쳤다고 가정합니다. 정제한 `order_items` · `orders` · `products` 를 병합해 만든 통합 테이블(또는 `category` · `channel` · `amount` 가 있는 테이블)을 다룰 수 있어야 합니다. 병합(merge)은 9장에서 자세히 배우지만, 이 장의 예제는 세 테이블을 간단히 합친 통합 테이블 위에서 진행합니다. 집계의 그룹 축으로 쓰는 `category` · `amount` · `grade` 같은 열이 앞 장(5~7장)에서 정제·파생된 결과임을 기억하면, 각 장이 어떻게 이어지는지 보입니다.

8.1 그룹화와 집계 (개념 c13)

도입: '쪼개서-계산해-합치기'가 분석의 뼈대다

"카테고리별 매출"을 구한다는 것은 무슨 일일까요? 데이터를 카테고리(전자·가구·의류...)별로 **쪼개고**, 각 덩어리에서 매출을 **더하고**, 그 결과를 하나의 표로 **합치**는 것입니다. 이 세 단계 — 쪼개기(split) → 계산(apply) → 합치기(combine) — 가 그룹화의 본질이며, 판다스에서는 `groupby` 한 줄로 표현됩니다. 이 패턴을 이해하면 "채널별 건수", "연령대별 평균 구매액"처럼 모든 "○○별 △△"를 같은 방식으로 풀 수 있습니다.

선수 개념 상기: 그룹을 나누는 기준 열은 대부분 7장에서 만든 범주 열 (`category` · `grade` · `age_group`)이나 원본의 범주 열(`channel`)입니다. 구간화가 곧 "그룹 축 만들기"였음을 여기서 확인합니다.

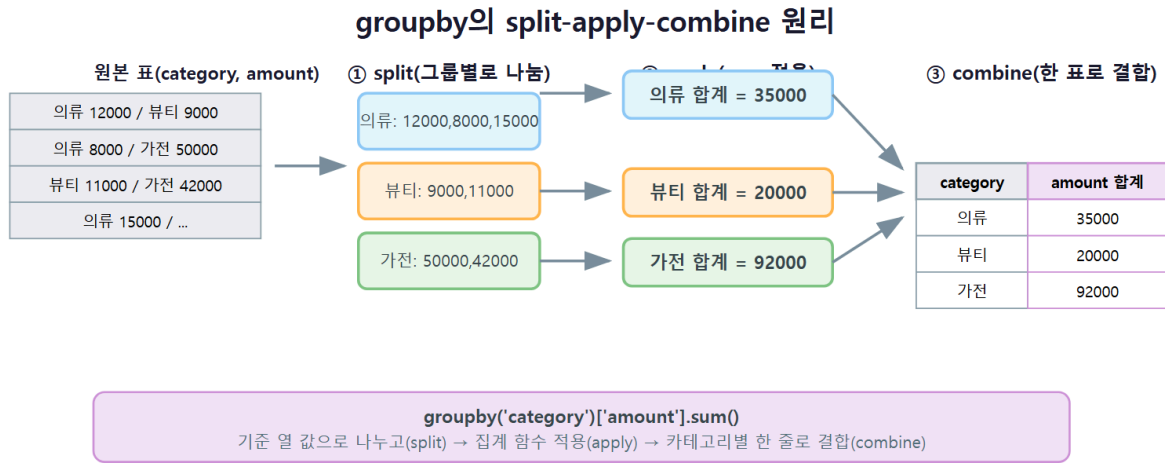
정의

그룹화(groupby) 는 기준 열의 값으로 데이터를 그룹으로 나눈 뒤, 각 그룹에 집계 함수를 적용하고, 결과를 하나로 합치는 분석 패턴입니다.

- `df.groupby('category')['amount'].sum()` — 카테고리별 매출 집계.
- **집계 함수:** `sum` (합계) · `mean` (평균) · `count` (비결측 개수) · `size` (결측 포함 행수) · `nunique` (고유 개수) 등.
- **agg 이름 붙은 집계(named aggregation):** `groupby('category').agg(매출=('amount', 'sum'), 건수=('order_id', 'count'))` — 한 그룹에 **여러 지표를 한 번에**, 결과 열 이름까지 지정.
- **transform:** 집계값을 **원래 행 개수 그대로** 되돌립니다. "각 주문 금액 ÷ 그 카테고리 전체 매출" 같은 그룹 상대 계산이나 그룹별 결측 채우기에 씁니다.

집계(aggregation) 란 여러 행을 합계·평균·건수 등 하나의 값으로 요약하는 계산입니다. 그룹화 없이 `df['amount'].sum()` 을 하면 전체 하나의 합이지만, 그룹화와 결합하면 그룹마다 하나씩 값이 나옵니다.

동작 원리: split-apply-combine



`df.groupby('category')['amount'].sum()` 이 실행되는 과정을 단계로 보면 이렇습니다.

1. **split(쪼개기)**: `category` 값이 같은 행끼리 그룹으로 묶습니다(전자 그룹, 가구 그룹, ...).
2. **apply(계산)**: 각 그룹의 `amount` 열에 `sum` 을 적용해 그룹마다 하나의 합계를 냅니다.
3. **combine(합치기)**: 그룹별 결과를 모아 `category` 를 인덱스로 하는 시리즈로 돌려줍니다.

여기서 주의할 두 가지가 있습니다. 첫째, `groupby` 결과는 그룹 키가 인덱스가 됩니다. 열로 쓰려면 `reset_index()` 를 붙입니다. 둘째, `count` 는 결측을 빼고 세고 `size` 는 결측을 포함한 행수를 셉니다. 결측이 있는 열의 개수를 셀 때 둘의 값이 달라질 수 있습니다.

worked example: 카테고리·채널별 매출 집계와 총합 대조

정제한 세 테이블을 합친 통합 테이블을 만들어, 카테고리별 매출을 집계하고 여러 지표를 `agg` 로 한 번에 구하겠습니다.

```
import pandas as pd

# 통합 테이블 준비(9장 merge 미리보기 - 여기선 결과만 사용)
prod = pd.read_csv("data/products.csv").drop_duplicates("product_id")
prod["category"] = prod["category"].str.strip()
orders = pd.read_csv("data/orders.csv")
oi = pd.read_csv("data/order_items.csv").drop_duplicates("order_item_id")
oi = oi.dropna(subset=["unit_price"])
oi = oi[oi["quantity"] > 0].copy()
oi["amount"] = oi["unit_price"] * oi["quantity"] * (1 - oi["discount"])

m = oi.merge(orders[["order_id", "channel", "status"]], on="order_id")
m = m.merge(prod[["product_id", "category"]], on="product_id")

# 카테고리별 매출 합계, 큰 순으로 정렬
cat_sales = m.groupby("category")["amount"].sum().sort_values(
    ascending=False)
print(cat_sales.round(0).astype("int64"))
```

- `m.groupby("category")["amount"]` — `category` 로 그룹을 나눈 뒤 각 그룹의 `amount` 열만 봅니다.
- `.sum()` — 그룹마다 매출을 더합니다(split-apply-combine의 apply 단계).
- `.sort_values(ascending=False)` — 매출 큰 카테고리가 위로 오게 정렬해 "상위 그룹"을 바로 봅니다.

실행 결과는 다음과 같습니다.

```
category
전자      23706632357
가구      16883450865
의류       4773475968
도서       4550701174
뷰티       3627218386
식품       1953853982
Name: amount, dtype: int64
```

전자(약 237억)와 가구(약 169억)가 매출을 이끄는 두 카테고리임이 한눈에 드러납니다. 50만 행이 여섯 줄로 압축되면서, "무엇이 잘 팔리는가"라는 질문의 답이 곧바로 보입니다 — 이것이 그룹화·집계가 분석에서 하는 일입니다. 이제 `agg` 로 매출·건수·평균단가를 한 번에 구합니다.

```
# agg 이름 붙은 집계: 한 그룹에 여러 지표
report = m.groupby("category").agg(
    sales=("amount", "sum"),
    orders=("order_item_id", "count"),
    avg_price=("unit_price", "mean"),
).sort_values("sales", ascending=False)
print(report.round(1))

# 총합 대조 검증: 그룹별 매출의 합 == 전체 매출
print("\n그룹 매출 합:", int(report["sales"].sum()))
print("원본 전체 매출:", int(m["amount"].sum()))
```

- `agg(sales=("amount", "sum"), ...)` — 결과열이름=("원본열", "집계함수") 형식으로 여러 지표를 한 번에 계산합니다. 매출(합계)·건수(개수)·평균단가(평균)를 각각 다른 열에서 구합니다.
- **총합 대조** — 그룹별 매출을 다 더하면 원본 전체 매출과 같아야 합니다. 다르면 집계에서 행이 누락·중복된 것이므로, 이 대조가 집계의 검증입니다.

실행 결과는 다음과 같습니다.

	sales	orders	avg_price
category			
전자	2.370663e+10	108736	96798.2
가구	1.688345e+10	59089	127242.2
의류	4.773476e+09	58637	36082.2
도서	4.550701e+09	133445	15150.2
뷰티	3.627218e+09	62009	26023.3
식품	1.953854e+09	62371	13916.0

그룹 매출 합: 55495332732
원본 전체 매출: 55495332732

그룹별 매출의 합(554억여 원)이 원본 전체 매출과 **정확히 일치**하므로, 집계에 누락·중복이 없었음이 검증됐습니다. 흥미롭게도 도서는 건수(133,445)가 가장 많지만 평균단가(15,150원)가 낮아 매출은 중위권입니다 — 이런 해석이 그룹 집계로 드러나는 통찰입니다.

`transform` 은 집계값을 원래 행에 되돌릴 때 씁니다.

```
# transform: 카테고리 전체 매출을 각 행에 붙여 비중 계산
m["cat_total"] = m.groupby("category")["amount"].transform("sum")
m["share"] = m["amount"] / m["cat_total"]
print(m[["category", "amount", "cat_total", "share"]].head(3))
```

실행 결과는 다음과 같습니다.

	category	amount	cat_total	share
0	도서	19285.0	4.550701e+09	0.000004
1	가구	99000.0	1.688345e+10	0.000006
2	도서	46893.0	4.550701e+09	0.000010

`sum` 은 그룹당 하나의 값을 주지만, `transform('sum')` 은 각 행에 그 행이 속한 그룹의 합계를 붙여 행 개수를 그대로 유지합니다. 그래서 `amount / cat_total` 로 각 주문이 카테고리 매출에서 차지하는 비중을 계산할 수 있습니다.

그룹 기준은 여러 열이 될 수 있습니다. `groupby(['channel', 'status'])` 처럼 두 열을 주면 두 값의 조합마다 그룹이 만들어집니다. `size` 로 조합별 주문 건수를 세어 봅니다.

```
orders = pd.read_csv("data/orders.csv")
# 채널 x 상태 조합별 주문 건수(size는 결측 포함 행수)
combo = orders.groupby(["channel", "status"]).size()
print(combo)
```

- `groupby(["channel", "status"])` — 두 열의 조합(web-delivered, web-paid, ...)마다 그룹을 만듭니다. 결과는 두 단계로 된 인덱스(멀티인덱스)가 됩니다.
- `.size()` — 각 조합의 행수를 셉니다. `count` 와 달리 결측을 포함한 순수 행 개수입니다.

실행 결과는 다음과 같습니다(일부).

channel	status	
app	canceled	5865
	delivered	32845
	paid	9169
	returned	2975
	shipped	9040
store	canceled	3002
	delivered	16676
	paid	4460
	returned	1570
	shipped	4440
web	canceled	11172
	delivered	60455
	paid	16364
	returned	5430
	shipped	16537

dtype: int64

채널마다 다섯 상태의 건수가 나옵니다. 모든 채널에서 `delivered` (배송완료)가 가장 많고, 웹이 전반적으로 건수가 큼니다. 이처럼 두 열로 묶으면 한 열로는 안 보이던 "채널마다 취소·반품 비중이 다른가" 같은 질문에도 답할 수 있습니다. 이렇게 다중 키 그룹화는 "채널별로, 상태별로"

같은 교차 분석의 기초이며, 8.2의 피벗테이블은 이 결과를 보기 좋은 2차원 표로 펼친 것입니다.

즉시 연습 (2종 1세트)

직접 해 보기 — 채널별 매출·건수 집계와 총합 대조

위 통합 테이블 `m`에서 **채널별**(`channel`) 매출과 건수를 집계하고 총합으로 검증하십시오. **직접 실행**하고 실제 출력을 기록합니다.

1단계 — `groupby('channel')`로 매출 합계를 구하고 큰 순으로 정렬해 기록합니다.

- **[채널별 매출: web / app / store]:** (실제 출력값 — 어느 채널이 1위인지)

2단계 — `agg`로 채널별 `sales`(합계)·`orders`(건수)·`avg_price`(평균단가)를 한 번에 구해 기록합니다.

- **[채널별 3개 지표 표]:** (실제 출력값)

3단계 — 채널별 매출의 합이 원본 전체 매출(`m['amount'].sum()`)과 **일치하는지** 대조합니다.

- **[채널 매출 합 / 원본 전체 매출]:** (두 값을 적고 같은지 확인)

자가체크

- `[]` `groupby('channel')`로 매출을 집계하고 정렬해 1위 채널을 확인했다.
- `[]` `agg`로 세 지표를 한 번에 구했다(따로 세 번 계산하지 않았다).
- `[]` 채널별 매출의 합이 원본 전체 매출과 일치함을 수치로 대조했다(추측하지 않았다).

흔한 오류와 교정

오개념: "groupby 결과는 바로 열로 쓸 수 있다 / count와 size는 같다"

`groupby` 결과는 **그룹 키가 인덱스**가 됩니다. `groupby('category')['amount'].sum()`의 `category`는 열이 아니라 인덱스라, 이후 `merge`나 열 참조가 필요하면 `reset_index()`로 열로 되돌려야 합니다.

또 `count`와 `size`는 다릅니다. `count`는 **결측을 빼고** 세고, `size`는 **결측을 포함한** 행수를 셉니다. 결측이 있는 열(예: `unit_price`)에서 개수를 세면 두 값이 달라지므로, "몇 건인가"를 셀 때 무엇을 세는지 의식해야 합니다. 마지막으로, 집계 후에는 **총합 대조**(그룹 합계의 합 = 원본 전체)를 습관으로 삼으십시오 — 병합·필터 실수로 행이 새거나 사라지면 이 대조에서 드러납니다.

8.2 피벗테이블과 교차표 (개념 c14)

도입: 한 축이 아니라 두 축으로 보고 싶을 때

8.1에서 "카테고리별 매출"을 봤습니다. 그런데 "카테고리별로, 그리고 채널별로 매출이 어떻게 다른가?"처럼 **두 축을 동시에** 보고 싶을 때가 있습니다. 카테고리를 행에, 채널을 열에 놓고 매출을 채우면 표 하나로 교차 분석이 됩니다. 엑셀을 써 봤다면 이것이 바로 **피벗테이블**입니다. 판다스의 `pivot_table` 은 같은 일을 코드로, 마진(합계)·빈 칸 처리까지 한 줄로 해 줍니다.

선수 개념 상기: 피벗테이블은 8.1의 `groupby` 를 두 축으로 확장한 것입니다. 실제로 같은 결과를 `groupby(['category', 'channel']).sum().unstack()` 으로도 만들 수 있습니다.

정의

피벗테이블(pivot table) 은 한 열을 행 축, 다른 열을 열 축으로 놓고 값 열을 집계 함수로 요약해 2차원 표로 펼치는 도구입니다.

- `pivot_table(data, index='category', columns='channel', values='amount', aggfunc='sum')`
— 카테고리(행)×채널(열)별 매출 합계표.
- `margins=True` — 행·열 끝에 합계(All)를 붙입니다.
- `fill_value=0` — 빈 조합(NaN)을 0으로 채웁니다.

교차표(crosstab) 는 두 범주형 열의 빈도표(개수)를 만드는 특화 함수입니다.

- `crosstab(행범주, 열범주)` — 기본은 개수를 셉니다.
- `normalize='index'` (행 기준)·`'columns'` (열 기준)·`True` (전체 기준) — 개수를 비율로 바꿉니다.

동작 원리: `pivot_table` = 두 축 `groupby`

피벗테이블: 긴 형식 → 카테고리×채널 요약표

긴 형식 원본(category, channel, amount)

category	channel	amount
의류	web	12000
의류	app	8000
뷰티	web	9000
가전	offline	50000
...	...	...

→ pivot_table

2차원 요약표(index=category, columns=channel)

category	web	app	offline	합계
의류	12000	8000	0	20000
뷰티	9000	0	2000	11000
가전	0	0	50000	50000
합계	21000	8000	52000	81000

values='amount', aggfunc='sum' · margins=True로 행·열 합계 추가 · fill_value=0으로 빈 칸 채움

```
pivot_table(index='category', columns='channel', values='amount', aggfunc='sum', margins=True, fill_value=0)
```

`pivot_table(index='category', columns='channel', values='amount', aggfunc='sum')` 은 내부적으로 `category` 와 `channel` 두 열로 그룹을 나눠 `amount` 를 합친 뒤, `channel` 을 열 방향으로 펼친 것입니다. 그래서 `groupby(['category', 'channel'])['amount'].sum().unstack('channel')` 과 같은 결과를 냅니다. 차이는 **표현의 편의**입니다 — `pivot_table` 은 두 축 요약을 한 줄로 쓰고 `margins · fill_value` 를 바로 붙일 수 있어 리포트 표를 만들기에 좋습니다.

`crosstab` 은 값을 집계하지 않고 **빈도(개수)** 를 세는 데 특화된 피벗테이블입니다. "연령대별로 성별 분포가 어떤가"처럼 두 범주의 조합이 몇 건인지 볼 때 씁니다.

worked example: 카테고리×채널 매출 피벗테이블과 연령대×성별 교차표

두 축 매출 요약표를 `pivot_table` 로 만들고 마진으로 총합을 확인한 뒤, `crosstab` 으로 빈도·비율표를 만들겠습니다.

```
import pandas as pd

# (앞 8.1의 통합 테이블 m을 그대로 사용한다고 가정)
pt = pd.pivot_table(m, index="category", columns="channel",
                    values="amount", aggfunc="sum",
                    margins=True, fill_value=0)
print(pt.round(0).astype("int64"))
```

- `index="category", columns="channel"` — 카테고리를 행, 채널을 열로 놓습니다.
- `values="amount", aggfunc="sum"` — 각 칸을 그 카테고리·채널 조합의 매출 합계로 채웁니다.
- `margins=True` — 맨 끝에 행·열 합계(All)를 붙여, 카테고리 총매출·채널 총매출·전체 총합을 한 표에서 봅니다.

- `fill_value=0` — 매출이 없는 조합을 `NaN` 대신 0으로 채웁니다.

실행 결과는 다음과 같습니다.

channel	app	store	web	All
category				
가구	5011512370	2625032543	9246905952	16883450865
도서	1355128274	682645063	2512927837	4550701174
뷰티	1097429571	535683843	1994104972	3627218386
식품	590618118	292373904	1070861960	1953853982
의류	1444389650	712135574	2616950744	4773475968
전자	7077176119	3588209569	13041246669	23706632357
All	16576254102	8436080496	30482998134	55495332732

All 행·열 덕분에 세 가지를 한 표에서 읽습니다 — 채널별 총매출(웹 약 305억이 가장 큼), 카테고리별 총매출(오른쪽 All 열, 8.1과 일치), 그리고 전체 총합 55,495,332,732원(우하단). 이 전체 총합은 8.1의 그룹 매출 합과 **정확히 같아**, 피벗테이블이 누락 없이 집계했음이 검증됩니다. 모든 카테고리에서 웹이 매장보다 매출이 크다는 패턴도 한눈에 보입니다.

`pivot_table` 은 여러 집계 함수를 한 번에 적용할 수도 있습니다. `aggfunc` 에 목록을 주면 각 함수의 결과가 나란히 붙습니다.

```
# 카테고리별 매출 집계·평균·건수를 한 표에
multi = pd.pivot_table(m, index="category", values="amount",
                        aggfunc=["sum", "mean", "count"])
print(multi.round(1))
```

- `aggfunc=["sum", "mean", "count"]` — 매출의 집계·평균·건수를 한 번에 계산해 세 열로 나란히 보여 줍니다. `agg` 의 이름 붙은 집계와 비슷하지만, 피벗테이블은 여기에 `columns` 축을 더 붙일 수 있습니다.

실행 결과는 다음과 같습니다.

	sum	mean	count
category	amount	amount	amount
가구	1.688345e+10	285729.2	59089
도서	4.550701e+09	34101.7	133445
뷰티	3.627218e+09	58495.0	62009
식품	1.953854e+09	31326.3	62371
의류	4.773476e+09	81407.2	58637
전자	2.370663e+10	218020.1	108736

가구는 건수(59,089)가 적어도 평균 주문액(285,729원)이 가장 높아 매출 2위이고, 도서는 건수(133,445)가 가장 많지만 평균(34,102원)이 낮습니다. 집계만 보면 놓칠 이 대비가 세 지표를 나

란히 놓으니 드러납니다.

이제 `crosstab` 으로 연령대×성별 분포를 봅니다.

```
# 고객의 연령대 x 성별 빈도표
cust = pd.read_csv("data/customers.csv").drop_duplicates()
cust = cust.dropna(subset=["birth_date", "gender"]).copy()
cust["birth_date"] = pd.to_datetime(cust["birth_date"])
cust["age"] = 2024 - cust["birth_date"].dt.year
cust["age_group"] = pd.cut(cust["age"], bins=[0, 29, 49, 200],
                           labels=["20대이하", "30-40대", "50대이상"])
cust["gender_ko"] = cust["gender"].map(
    {"남": "남", "M": "남", "여": "여", "F": "여"})

freq = pd.crosstab(cust["age_group"], cust["gender_ko"])
print("빈도표:")
print(freq)
print("\n행 기준 비율:")
print(pd.crosstab(cust["age_group"], cust["gender_ko"],
                  normalize="index").round(3))
```

- `crosstab(cust["age_group"], cust["gender_ko"])` — 연령대(행)×성별(열) 조합별 고객 수를 셉니다.
- `normalize="index"` — 각 **행의 합을 1**로 보고 비율을 냅니다. 연령대 안에서 남/여 비중을 보는 것입니다("columns" 면 열 기준, True 면 전체 기준).

실행 결과는 다음과 같습니다.

```
빈도표:
gender_ko   남   여
age_group
20대이하    432  482
30-40대     799  871
50대이상    876  826

행 기준 비율:
gender_ko   남      여
age_group
20대이하    0.473  0.527
30-40대     0.478  0.522
50대이상    0.515  0.485
```

빈도표는 절대 수를, 비율표는 각 연령대 안의 남/여 구성을 보여 줍니다. 20대이하·30-40대는 여성 비중이 약간 높고, 50대이상은 남성이 조금 더 많습니다 — `normalize` 로 비율을 보면 그룹 크기가 달라도 **구성을 공정하게 비교**할 수 있습니다.

즉시 연습 (2종 1세트)

직접 해 보기 — 카테고리×상태 피벗테이블과 groupby 재현

통합 테이블 `m`에서 **카테고리×상태(status)** 매출 피벗테이블을 만들고, 같은 결과를 `groupby`로도 만들어 **일치하는지** 확인하십시오. **직접 실행**하고 실제 출력을 기록합니다.

1단계 — `pivot_table(index='category', columns='status', values='amount', aggfunc='sum', margins=True, fill_value=0)` 을 만들어 기록합니다.

- **[카테고리×상태 피벗표의 All 행·열 값]**: (실제 출력값 — 전체 총합이 55,495,332,732인지)

2단계 — 같은 요약을 `m.groupby(['category', 'status'])['amount'].sum().unstack('status')` 로 만듭니다.

- **[groupby로 만든 표의 전자·delivered 칸 값]**: (실제값)
- **[피벗테이블의 같은 칸 값과 일치하는가]**: (예/아니오)

3단계 — `crosstab` 으로 **채널×상태** 빈도표를 만들고 `normalize='index'` 로 채널별 상태 비율을 기록합니다.

- **[채널별 배송완료(delivered) 비율]**: (실제 출력값)

자가체크

- [] `pivot_table` 의 `margins` 로 전체 총합이 8.1의 매출 총합과 같음을 확인했다.
- [] 같은 요약을 `groupby(...).unstack()` 으로도 만들어 특정 칸 값이 일치함을 대조했다.
- [] `crosstab` 에 `normalize='index'` 를 줘 채널 안에서의 상태 비율을 확인했다(개수와 비율의 차이를 이해했다).

흔한 오류와 교정

오개념: "pivot과 pivot_table은 같다 / 빈 칸은 알아서 0이 된다"

`pivot_table` 은 **집계(aggfunc)** 를 하지만, `pivot` 은 **집계 없이 모양만** 바꿉니다. 그래서 같은 인덱스·열 조합에 값이 여러 개면 `pivot` 은 **오류**를 냅니다. 집계가 필요하면(대부분 그렇습니다) `pivot_table` 을 쓰십시오.

또 매출이 없는 조합은 기본으로 `NaN` 이 됩니다. `fill_value=0` 을 주지 않으면 합계 행이 `NaN` 에 오염되거나 표가 지저분해집니다. 값의 성격상 "없음 = 0"이 맞으면 `fill_value=0` 으로 채우십시오. 마지막으로 `crosstab` 의 `normalize` 는 '`index`' (**행**)·'`columns`' (**열**)·`True` (**전체**) 중 무엇을

기준으로 하느냐에 따라 비율의 의미가 완전히 달라집니다 — "연령대 안의 성별 비율"을 원하면 'index', "성별 안의 연령대 비율"을 원하면 'columns' 입니다.

8.3 단원 미니 프로젝트: 핵심 지표 요약표 만들기

이 장에서 배운 그룹화·집계·피벗테이블·교차표를 모두 써서, 쇼핑물의 **핵심 지표 요약표 세트**를 만듭니다. 이것이 종합 프로젝트(12~15장) 분석 리포트의 뼈대가 됩니다.

이 미니 프로젝트가 쓰는 개념: 그룹화와 집계(c13)·피벗테이블과 교차표(c14). 8장 전체의 종합 적용입니다.

직접 만들기 (2종 1세트)

직접 작성 — 핵심 지표 요약표 노트

통합 테이블 `m`으로 아래 요약표 세 개를 **직접 만들어** 저장하고, 총합으로 검증하십시오(빈칸 추측이 아니라 실제 코드 작성·실행).

1단계 — 카테고리별 요약: `agg`로 `sales` (매출)·`orders` (건수)·`avg_price` (평균단가)를 구하고 매출 내림차순 정렬해 저장합니다.

- **[매출 1위·2위 카테고리**와 **각 매출]**: (실제 출력값)
- **[건수는 많지만 매출은 중위권인 카테고리]**: (관찰해 적기 — 평균단가가 낮은 곳)

2단계 — 카테고리×채널 매출 피벗테이블을 `margins=True`로 만들어 저장하고, 전체 총합이 1단계 매출의 합과 같은지 대조합니다.

- **[피벗표 전체 총합 / 카테고리 매출의 합]**: (두 값을 적고 일치 확인)

3단계 — 연령대×성별(또는 채널×상태) 교차표를 빈도·비율(`normalize='index'`) 두 가지로 만들어 저장합니다.

- **[내가 고른 두 범주와 관찰한 패턴]**: (직접 작성 — 어느 그룹에서 어떤 편향이 보이는지)

4단계 — 세 요약표를 각각 `to_csv`로 저장합니다.

- **[저장한 파일 3개 이름]**: (실제값)

자가체크

- [] 카테고리별 `agg` 요약을 만들고 매출 순 정렬로 상위 그룹을 확인했다.
- [] 피벗테이블 전체 총합이 카테고리 매출의 합과 일치함을 대조했다(집계 검증).
- [] 교차표를 빈도와 비율 두 가지로 만들어 `normalize`의 차이를 이해했다.
- [] 세 요약표를 파일로 저장했고, 관찰한 패턴을 한 문장 이상 적었다.

자주 묻는 질문

Q. `groupby` 결과를 다시 데이터프레임으로 다루려면 어떻게 하나요?

`groupby(...).sum()`의 결과는 그룹 키가 **인덱스**인 시리즈·데이터프레임입니다. `reset_index()`를 붙이면 그룹 키가 다시 **열**이 되어, 이후 `merge`나 필터에 쓰기 편해집니다. 예:

`m.groupby('category')['amount'].sum().reset_index()`는 `category`·`amount` 두 열을 가진 데이터프레임을 돌려줍니다.

Q. `pivot_table`과 `groupby` 중 무엇을 써야 하나요?

결과는 같게 만들 수 있습니다 — `pivot_table(index=A, columns=B)`는 `groupby([A, B]).agg(...).unstack(B)`와 같습니다. 두 축을 **행×열 표**로 보고 싶고 마진(합계)을 붙이고 싶으면 `pivot_table`이 간결합니다. 반대로 결과를 이어서 다른 계산에 쓰거나 한 축만 필요하면 `groupby`가 유연합니다. 리포트 표는 `pivot_table`, 중간 계산은 `groupby`로 생각하면 대체로 맞습니다.

Q. 집계 결과가 원본 총합과 안 맞습니다. 무엇을 봐야 하나요?

가장 흔한 원인은 ① 그룹 기준 열에 결측이 있어 그 행이 집계에서 빠졌거나(`dropna=False`로 확인), ② 앞선 병합·필터에서 행이 늘거나 줄었거나(병합 전후 행수 확인), ③ `count`와 `size`를 혼동한 경우입니다. 집계 뒤 "그룹 합계의 합 == 원본 전체"를 늘 대조하면 이런 문제가 바로 드러납니다.

이 장을 마치며

이 장에서 그룹화·집계(개념 c13)와 피벗테이블·교차표(개념 c14)로 분석의 결론에 가장 가까운 **요약표**를 만들었습니다. 핵심은 세 가지입니다. 첫째, 그룹화는 **쪼개서-계산해-합치기**(`split-apply-combine`)이며 `agg`로 여러 지표를 한 번에 구합니다. 둘째, 두 축 요약은 `pivot_table`로,

두 범주의 빈도·비율은 `crosstab` 으로 만들되 `margins · normalize` 를 목적에 맞게 씁니다. 셋째, 집계는 항상 **총합 대조**(그룹 합계의 합 = 원본 전체)로 검증합니다.

그룹화·집계는 이 책에서 배우는 도구 중 **분석 결론에 가장 가까운** 것입니다. 정제·파생이 재료를 다듬는 일이었다면, 집계는 그 재료로 "누가 얼마나 사는가", "무엇이 잘 팔리는가"라는 질문에 실제로 답하는 일입니다. 그래서 종합 프로젝트(12~15장)의 핵심 발견도 대부분 이 장의 도구로 만들어집니다 — 카테고리별 매출 순위, 채널별 비중, 고객 세그먼트별 구매액 같은 요약이 모두 `groupby · pivot_table` 의 결과입니다.

지금까지는 세 테이블을 간단히 합친 통합 테이블을 가정해 썼습니다. 다음 9장에서는 그 **병합(merge)** 자체를 제대로 배웁니다 — 어떤 키로, 어떤 조인 방식으로 네 테이블을 합쳐야 행이 새거나 사라지지 않는지, 병합 전후 행수로 무결성을 점검하는 법을 다룹니다.

9장 여러 테이블 결합과 재구조화

8장까지 우리는 표 하나를 정제하고, 파생 열을 만들고, groupby와 피벗테이블로 요약했습니다. 그런데 지금까지 다룬 질문에는 한계가 있었습니다. "어느 **카테고리**가 잘 팔리는가"를 물으려면 주문 상세(order_items)에 있는 판매 수량과 상품(products)에 있는 카테고리를 **한 표에서** 봐야 하는데, 이 둘은 서로 다른 파일에 흩어져 있습니다. 실무 데이터는 거의 항상 이렇게 여러 테이블로 나뉘어 있습니다. 이 장에서는 흩어진 테이블을 공통 키로 **병합(merge)**해 분석용 통합 테이블을 만들고, 같은 구조의 조각을 **연결(concat)**하며, 표의 모양 자체를 바꾸는 **재구조화(reshape)**로 긴 형식과 넓은 형식을 오가는 법을 배웁니다.

이 장은 문법을 나열하지 않습니다. 여러분이 실제 쇼핑물 데이터셋 다섯 개 중 네 개 (order_items-orders-customers-products)를 손으로 병합해 보고, 병합하면서 **행수가 왜 늘어나는지, 어떤 키가 짝을 잃는지**를 직접 눈으로 확인하게 합니다. 이 데이터에는 이후 정제 단원에서 다룰 문제(중복 고객, 짝 없는 주문)가 그대로 들어 있어, 병합이 그 문제를 어떻게 드러내는지도 함께 봅니다.

이 장의 학습 목표

- on-left_on-right_on으로 키를 지정해 order_items-orders-customers-products를 순차 병합해 통합 테이블을 만들 수 있다 [→G7]
- how(inner-left-outer)의 차이를 이해하고 병합 전후 행수로 키 중복·고아 키를 진단할 수 있다 [→G7]
- concat으로 같은 구조 데이터를 이어 붙이고 melt로 넓은 형식을 긴 형식으로 녹일 수 있다 [→G7]
- pivot-unstack으로 긴 형식을 넓은 형식으로 펼쳐 각 형식의 쓰임(집계·시각화·리포트)을 설명할 수 있다 [→G7]

선수 확인: 이 장은 3장(개념 c05 열·행 선택, loc-iloc)과 4장(개념 c07 불리언 필터)을 마쳤다고 가정합니다. 병합은 결국 "키가 맞는 행을 골라 옆에 붙이는" 선택 연산의 확장이고, 병합 결과의 무결성 점검은 불리언 필터로 결측·고아 키를 세는 일이기 때문입니다. 또한 이 장의 통합 테이블은 5~6장에서 배운 정제(결측·중복·타입·문자열)를 전제로 만듭니다. 아직 그 장을 안 보셨다면, 병합 자체는 따라올 수 있으나 왜 중간에 정제가 끼는지가 낯설 수 있습니다.

9.1 개념 c15 — 데이터프레임 병합 (merge)

도입: 왜 병합이 필요한가

쇼핑물 데이터는 정규화되어 있습니다. 즉 같은 정보를 여러 곳에 중복 저장하지 않으려고, 주문의 뼈대(누가·언제·어떤 채널)는 orders에, 그 주문에 담긴 상품 한 줄 한 줄(무슨 상품·몇 개·단가)은 order_items에, 고객 신상은 customers에, 상품 정보는 products에 따로 둡니다. 이 구조는 저장에는 효율적이지만, "전자 카테고리를 서울 고객이 앱에서 얼마나 샀나" 같은 질문에는 답할 수 없습니다. 필요한 값이 네 표에 흩어져 있기 때문입니다. **데이터프레임 병합(merge)**은 이 흩어진 표들을 공통 키로 연결해, 한 행에서 주문·상품·고객을 모두 볼 수 있는 넓은 통합 테이블을 만드는 도구입니다.

정의

병합(merge)이란 두 데이터프레임을 **공통 키 열**로 연결해 옆으로(열 방향) 합치는 연산으로, SQL의 조인(join)에 해당합니다. 왼쪽.merge(오른쪽, on='키', how='방식') 형태로 쓰며, 키가 같은 행끼리 짝지어 두 표의 열을 나란히 붙입니다. 여기서 두 가지가 병합의 성격을 결정합니다. 첫째는 **키(key)** — 어느 열로 짝을 맞추는지이고(on 또는 양쪽 이름이 다르면 left_on·right_on), 둘째는 **조인 방식(how)** — 짝이 없는 행을 어떻게 처리할지입니다.

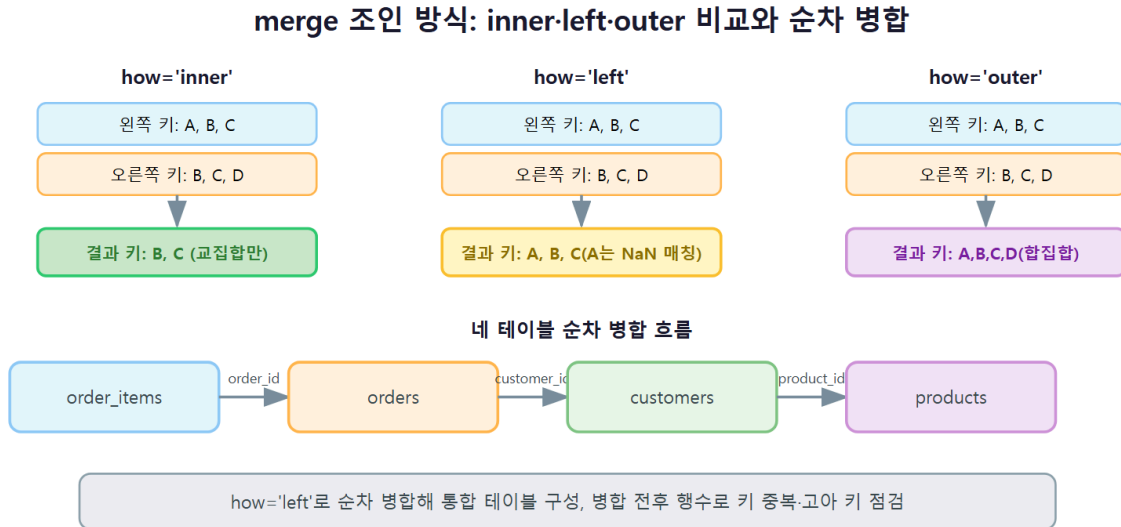
핵심 용어

- **키(key)**: 두 표를 짝짓는 기준 열입니다. 예를 들어 order_items와 orders는 둘 다 가진 order_id 로 짝을 맞춥니다.
- **조인 방식(how)**: 짝 없는 행의 운명을 정합니다. inner 는 **양쪽에 다 있는 키만** 남기고, left 는 **왼쪽 전체**를 남기되 오른쪽에 짝이 없으면 그 열을 결측(NaN)으로 채우며, right·outer 는 각각 오른쪽 전체·양쪽 합집합입니다.
- **고아 키(orphan key)**: 한쪽에는 있으나 다른 쪽에는 짝이 없는 키입니다. 예를 들어 orders에는 있는데 customers에는 없는 customer_id입니다.
- **1:다 관계(one-to-many)**: 오른쪽 한 행이 왼쪽 여러 행과 짝지어지는 관계입니다. 주문 하나(orders)에 상세 여러 줄(order_items)이 달리는 것이 대표적입니다.

동작 원리

병합의 핵심은 **"키가 몇 대 몇으로 만나는가"**입니다. 왼쪽 표의 각 행을 들고, 오른쪽 표에서 같은 키를 가진 행을 **모두** 찾아 짝을 만듭니다. 그래서 오른쪽에 같은 키가 여러 개면(중복 키) 왼

쪽 한 행이 여러 행으로 불어나고, 오른쪽에 짝이 없으면(고아 키) 조인 방식에 따라 그 행이 사라지거나(inner) 결국으로 남습니다(left). 아래 그림은 inner와 left가 짝 없는 행을 어떻게 다르게 처리하는지, 그리고 중복 키가 어떻게 행을 늘리는지를 보여 줍니다.



이 때문에 병합에서 가장 중요한 습관은 **병합 전후 행수를 대조하는** 것입니다. 행수가 예상과 다르면 둘 중 하나입니다. 늘었다면 키 중복(1:다 또는 다:다)이 있는 것이고, 줄었다면 inner 조인이 고아 키를 버린 것입니다. `indicator=True` 는 각 행이 양쪽에 다 있었는지(both), 한쪽에만 있었는지(left_only)를 표시해 주고, `validate='m:1'` 은 "왼쪽 다 대 오른쪽 하나" 관계가 맞는지 검사해 어긋나면 오류로 멈춰 줍니다. 이 두 옵션이 무결성 점검의 자동화 도구입니다.

worked example: 주문 상세에 주문·고객을 붙이기

order_items를 기준으로 orders와 customers를 차례로 붙여 보겠습니다. 여기서 **행수가 어떻게 변하는지**에 주목하십시오. 데이터는 3장까지 배운 대로 `data/` 폴더에서 불러옵니다.

```
import pandas as pd

customers = pd.read_csv("data/customers.csv")
orders = pd.read_csv("data/orders.csv", parse_dates=["order_datetime"])
items = pd.read_csv("data/order_items.csv")

# 1) 주문 상세 + 주문: order_id로 짝짓기
oi = items.merge(orders, on="order_id", how="left")
print("order_items:", items.shape, "-> 병합 후:", oi.shape)
```

실행 결과는 다음과 같습니다.

order_items: (500000, 6) -> 병합 후: (500000, 10)

- `items.merge(orders, on="order_id", how="left")` — order_items의 각 행에 대해, 같은 order_id 를 가진 orders 행을 찾아 그 열(order_datetime·channel·status)을 옆에 붙입니다.
- 행수가 500,000으로 그대로입니다. orders의 order_id 는 유일(중복 없음)하고 order_items의 모든 order_id가 orders에 존재하므로, 각 상세 행이 정확히 하나의 주문과 짝지어져 행이 늘지도 줄지도 않았습니다. 열은 6개에서 10개로 늘었습니다(orders의 4개 열이 붙음).

이제 여기에 고객 정보를 붙여 보겠습니다. 그런데 이번에는 행수가 예상 밖으로 변합니다.

```
# 2) + 고객: customer_id로 짝짓기 (원본 그대로)
m = oi.merge(customers, on="customer_id", how="left")
print("병합 전:", oi.shape[0], "행, 병합 후:", m.shape[0], "행")
print("customers의 customer_id 중복 개수:",
      customers["customer_id"].duplicated().sum())
```

병합 전: 500000 행, 병합 후: 503678 행
customers의 customer_id 중복 개수: 50

- 행수가 500,000에서 503,678로 늘었습니다. how="left" 인데도 왼쪽 행수가 보존되지 않은 이유는, 오른쪽 customers에 customer_id가 중복된 행이 50개 있기 때문입니다(5장에서 다룰 중복 데이터). 중복 키를 가진 고객과 짝지어진 주문 상세는 그 중복 수만큼 복제되어 행이 늘어납니다.
- 이것이 병합의 가장 위험한 함정입니다. 집계 전에 이 중복을 못 잡으면 매출·건수가 부풀려집니다. 그래서 병합 후 반드시 행수를 확인해야 합니다.

행이 왜 늘었는지, 그리고 짝 없는 고객(고아 키)은 얼마나 되는지 indicator 로 진단합니다.

```
# 3) indicator로 매칭 상태 진단
chk = oi.merge(customers, on="customer_id", how="left", indicator=True)
print(chk["_merge"].value_counts())
orphan_rate = (chk["_merge"] == "left_only").mean() * 100
print("고아 키 비율: %.2f%%" % orphan_rate)
```

```
_merge
both      501199
left_only   2479
right_only    0
고아 키 비율: 0.49%
```

- **both 501,199행**은 양쪽에 다 있는 정상 매칭입니다. **left_only 2,479행**은 order/order_items 쪽에는 있으나 customers에 없는 **고아 customer_id**로, 전체의 약 0.49%입니다. 이 데이터셋은 고객 테이블에서 일부 고객이 빠진 상황(약 0.5%)을 일부러 심어 두었습니다.
- **how="left"** 였으므로 이 2,479행은 사라지지 않고, 고객 열(name-city 등)이 NaN인 채로 남습니다. 만약 **how="inner"** 였다면 이 행들은 통째로 버려졌을 것입니다.

이번에는 관계 자체가 올바른지 **validate** 로 강제 검사해 보겠습니다.

```
# 4) validate로 "왼쪽 다 : 오른쪽 하나" 관계 검사
try:
    oi.merge(customers, on="customer_id", how="left", validate="m:1")
except pd.errors.MergeError as e:
    print("MergeError:", str(e)[:48])
```

MergeError: Merge keys are not unique in right dataset

- **validate="m:1"** 은 "왼쪽은 키가 여러 번 나와도 되지만(many), 오른쪽은 키가 유일해야 한다(one)"는 관계를 검사합니다. customers의 customer_id가 중복되므로 이 검사는 실패하고 MergeError 로 즉시 멈춥니다.
- 이렇게 오류로 멈추는 것이 **좋은 일**입니다. 조용히 행이 불어나 잘못된 집계로 이어지느니, 병합 시점에 "관계가 깨졌다"고 알려 주는 편이 낫습니다.

문제를 확인했으니 중복을 제거하고(6장에서 배운 **drop_duplicates**) 다시 병합하면, 행수가 보존되고 검사도 통과합니다.

```
# 5) 정제 후 재병합 (중복 고객 제거)
cust = customers.drop_duplicates(subset="customer_id")
m_ok = oi.merge(cust, on="customer_id", how="left", validate="m:1")
print("정제 customers:", cust.shape[0], "행, 병합 후:", m_ok.shape[0], "행")
print("inner 병합 행수:", oi.merge(cust, on="customer_id", how="inner").shape[0])
```

정제 customers: 4950 행, 병합 후: 500000 행
inner 병합 행수: 497521

- 중복 50개를 제거하니 customers가 5,000행에서 4,950행이 되고, **how="left"** 병합은 **행수 500,000을 정확히 보존**하며 **validate="m:1"** 도 통과합니다.
- 마지막 줄은 같은 병합을 **how="inner"** 로 한 결과입니다. **497,521행**으로, left (500,000)보다 2,479행 적습니다. 그 차이가 바로 앞에서 본 고아 키의 수입니다. 즉 inner는 고아 키

를 버리고, left는 남깁니다. "고객 없는 주문도 매출에는 포함해야 한다"면 left를, "고객 정보가 확실한 것만 본다"면 inner를 고릅니다.

즉시 연습 (2종 1세트)

직접 해 보기 — 상품 병합의 행수 변화 관찰하기

위 worked example의 oi (order_items + orders, 500,000행)에 이번에는 **products**를 붙여 보십시오. products에는 product_id가 3개 중복되어 있고, order_items의 모든 product_id는 products에 존재합니다. 아래 순서로 **직접 실행하고 실제 출력을 기록하십시오**(정답을 추측해 적는 것이 아니라, 여러분의 화면에 뜬 숫자를 옮겨 적습니다).

```
products = pd.read_csv("data/products.csv")

# (1) 원본 products로 left 병합 후 행수 기록
mp = oi.merge(products, on="product_id", how="left")
print("병합 후 행수:", mp.shape[0])

# (2) product_id 중복 수와 indicator 매칭 상태 기록
print("product_id 중복:", products["product_id"].duplicated().sum())
chk = oi.merge(products, on="product_id", how="left", indicator=True)
print(chk["_merge"].value_counts())

# (3) 중복 제거 후 다시 병합해 행수가 보존되는지 확인
prod = products.drop_duplicates(subset="product_id")
print("정제 후 병합 행수:", oi.merge(prod, on="product_id", how="left").shape[0])
```

관찰한 실제 값을 아래 라벨에 채워 넣으십시오.

- [원본 병합 후 행수]: (500,000보다 큰가? 얼마나 늘었나)
- [product_id 중복 수]: (몇 개인가)
- [left_only 고아 키 행수]: (products에 없는 상품을 참조하는 주문 상세가 있는가)
- [중복 제거 후 병합 행수]: (500,000으로 보존됐는가)

자가체크

- [] 원본 병합에서 행수가 늘어난 것을 **실제 출력으로** 확인했다(머리로 추측하지 않았다).
- [] 늘어난 행수와 product_id 중복 수의 관계를 설명할 수 있다.
- [] how="left" 인데도 행이 늘 수 있는 이유(오른쪽 키 중복)를 한 문장으로 적었다.
- [] 중복 제거 후 행수가 500,000으로 보존됨을 확인했다.

흔한 오류와 교정

오개념: "how='left'면 왼쪽 행수는 무조건 그대로다"

가장 자주 하는 착각입니다. `how` 는 **짜깁 없는 행**의 처리 방식일 뿐, **키 중복으로 인한 행 증가**는 막지 못합니다. 오른쪽 키가 중복이면 `left` 병합도 행이 붙어납니다. 그래서 병합 직후에는 반드시 `print(before, "->", after)` 로 행수를 대조하고, 관계가 1:다여야 하는 병합에는 `validate="m:1"` 을 붙여 오른쪽 키의 유일성을 강제하십시오.

또 하나 흔한 실수는 **양쪽에 같은 이름의 비(非)키 열이 있을 때** 생기는 `_x · _y` 접미사입니다. 예를 들어 `orders`와 `order_items`에 둘 다 `id` 같은 열이 있으면 `id_x · id_y` 로 갈라져 헷갈립니다. `suffixes=("_주문", "_상세")` 로 의미 있는 접미사를 직접 지정하거나, 병합 전에 필요 없는 열을 정리하십시오.

9.2 개념 c16 — 연결과 재구조화 (concat·melt·pivot)

도입: 붙이는 것과 모양을 바꾸는 것

`merge`가 서로 다른 정보를 키로 **짜지어 옆에 붙이는** 일이라면, 이 절의 도구들은 성격이 다릅니다. **연결(concat)**은 이미 **같은 구조인** 데이터 조각을 단순히 위아래(또는 좌우)로 이어 붙입니다. 예를 들어 월별로 나뉘어 저장된 로그 파일을 하나로 합칠 때 씁니다. **재구조화(reshape)**는 값은 그대로 두고 **표의 모양만** 바꿉니다. 8장에서 만든 "카테고리×월 매출 표"처럼 사람이 읽기 좋은 넓은 표와, 집계·시각화 도구가 다루기 좋은 긴 표 사이를 오가는 일입니다.

정의

연결(concat)은 `pd.concat([df1, df2, ...])` 로 여러 데이터프레임을 이어 붙이는 함수로, 키 매칭 없이 단순히 쌓습니다. `axis=0` (기본)이면 위아래(행 방향), `axis=1` 이면 좌우(열 방향)이고, `ignore_index=True` 로 이어 붙인 뒤 인덱스를 0부터 다시 매깁니다.

재구조화(reshape)는 표의 형식을 바꾸는 것으로, 두 방향이 있습니다. **melt**는 여러 지표 열이 옆으로 펼쳐진 **넓은 형식(wide format)**을 "변수 이름 열 + 값 열" 두 개로 세워 **긴 형식(long format)**으로 **녹입니다**. 반대로 **pivot**은 긴 형식을 다시 넓은 형식으로 **펼칩니다**. `stack · unstack` 은 열과 인덱스 사이를 오가는 같은 계열의 도구입니다.

핵심 용어

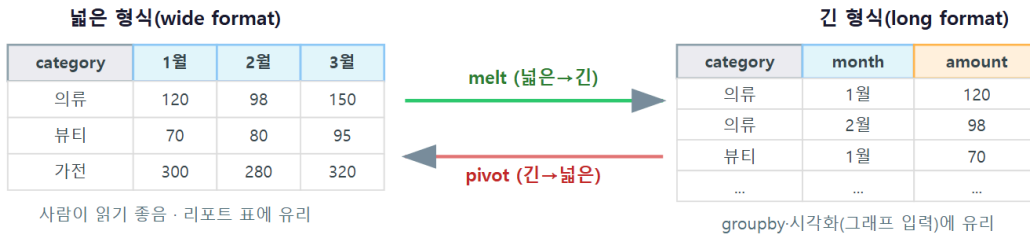
- **긴 형식(long format)**: 한 행이 "하나의 관측값"인 형태입니다. 예: (카테고리, 월, 매출) 이 한 행. `groupby`·시각화 라이브러리가 다루기 좋습니다.

- **넓은 형식(wide format):** 지표가 여러 열로 펼쳐진 형태입니다. 예: 행은 카테고리, 열은 1월~6월, 칸은 매출. 사람이 표로 읽기 좋습니다.
- **melt(녹이기):** 넓은 → 긴. `id_vars` (그대로 둘 열)와 나머지 지표 열을 `variable`·`value` 두 열로 세웁니다.
- **pivot(펼치기):** 긴 → 넓은. `index`·`columns`·`values` 로 어느 열을 행·열·칸에 놓을지 정합니다.

동작 원리

긴 형식과 넓은 형식은 **우열이 아니라 용도**의 문제입니다. 아래 그림은 같은 "카테고리×월 매출" 데이터가 두 형식으로 어떻게 표현되는지, 그리고 melt와 pivot이 그 사이를 어떻게 오가는지를 보여 줍니다.

재구조화: melt(넓은→긴)와 pivot(긴→넓은)



목적에 맞게 두 형식을 오가며 사용: 리포트=넓은 형식, 집계:그래프=긴 형식

핵심 규칙 하나를 기억하십시오. **pivot은 집계를 하지 않습니다.** `index`와 `columns`의 조합이 유일해야 하며, 같은 조합이 여러 번 나오면(중복) pivot은 "어느 값을 넣어야 할지 모른다"며 오류를 냅니다. 이 경우 값을 합쳐야 하므로 8장의 `pivot_table` (집계 기능이 있음)을 씁니다. 즉 **긴 데이터가 이미 유일한 조합으로 집계돼 있으면 pivot, 아직 원자료라 합쳐야 하면 pivot_table**입니다.

worked example: concat으로 잇고 pivot·melt로 모양 바꾸기

먼저 concat으로 월별 주문 조각을 이어 붙여 보겠습니다.


```
# 주문 일시를 datetime으로 두고 월 열 추가 (10장에서 자세히)
o = orders.dropna(subset="order_datetime").copy()
o["month"] = o["order_datetime"].dt.month

jan = o[o["month"] == 1]
feb = o[o["month"] == 2]
cc = pd.concat([jan, feb], ignore_index=True)
print("1월", jan.shape[0], "+ 2월", feb.shape[0], "= concat", cc.shape[0])
```

1월 23583 + 2월 25761 = concat 49344

- `pd.concat([jan, feb], ignore_index=True)` — 1월 조각(23,583행)과 2월 조각(25,761행)을 위아래로 이어 붙여 정확히 49,344행이 됩니다. `concat`은 키를 맞추지 않고 단순히 쌓기 때문에, 두 조각의 열 구조가 같아야 자연스럽게 붙습니다.
- 참고로 월별 주문 건수가 1월 23,583건에서 늘어나는 것이 보입니다. 이 데이터셋은 상반기에 걸쳐 주문이 증가하는 추세를 담고 있어, 이후 10장 시계열 분석에서 이 추세를 정면으로 다룹니다.

이제 재구조화입니다. 카테고리별·월별 매출을 집계한 긴 형식을 pivot으로 넓은 표로 펼칩니다. 매출(amount)은 정제된 통합 테이블에서 계산합니다(정제 과정은 5~6장, 통합 방법은 9.3절 참고).

```
# full: 정제·병합이 끝난 통합 테이블 (9.3절에서 구축, amount 열 포함)
f = full.dropna(subset="order_datetime").copy()
f["month"] = f["order_datetime"].dt.month

# 긴 형식: (카테고리, 월, 매출) 한 행씩
long = f.groupby(["category", "month"])["amount"].sum().reset_index()

# pivot으로 넓은 표: 행=카테고리, 열=월, 칸=매출
wide = long.pivot(index="category", columns="month", values="amount")
pd.options.display.float_format = "{:,.0f}".format
print(wide)
```

month	1	2	3	4	5	6
category						
가구	2,092,437,367	2,220,471,099	2,772,762,273	2,984,746,284	3,425,288,120	3,702,632,829
도서	548,088,948	596,980,465	755,908,058	792,341,059	909,583,849	990,922,354
뷰티	436,000,735	494,956,566	599,888,135	648,971,531	724,970,039	796,555,060
식품	241,996,428	259,822,954	324,128,329	346,196,519	389,416,815	429,751,784
의류	561,991,327	642,391,079	803,708,285	845,420,235	953,851,647	1,050,134,797
전자	2,857,536,408	3,145,439,576	3,913,578,929	4,240,967,749	4,757,509,122	5,238,382,550

- `groupby(["category", "month"])["amount"].sum().reset_index()` — 카테고리 와 월의 조합마다 매출 합계를 낸 긴 형식입니다. 한 행이 "특정 카테고리의 특정 달 매출"입니다.
- `long.pivot(index="category", columns="month", values="amount")` — 그 긴 형식을 넓게 펼칩니다. 카테고리가 행, 월(1~6)이 열, 매출이 칸이 됩니다. 조합이 이미 유일하므로 `pivot`으로 충분합니다. 사람이 "어느 카테고리가 어느 달에 얼마 팔렸나"를 한눈에 읽기 좋은 리포트 표입니다. 전자·가구가 매출을 이끌고, 모든 카테고리가 달을 거듭하며 커지는 추세가 보입니다.

반대로, 이 넓은 표를 시각화 라이브러리에 넣으려면 다시 **긴 형식**이 편합니다. `melt`로 녹입니다.

```
# melt로 넓은 표를 다시 긴 형식으로
back = wide.reset_index().melt(
    id_vars="category", var_name="month", value_name="amount")
print("melt 결과 shape:", back.shape)
print(back.head(4))
```

```
melt 결과 shape: (36, 3)
category month amount
0   가구      1  2,092,437,367
1   도서      1   548,088,948
2   뷰티      1   436,000,735
3   식품      1   241,996,428
```

- `melt(id_vars="category", var_name="month", value_name="amount")` — `category`는 그대로 두고(`id_vars`), 나머지 월 열(1~6)을 `month` (변수 이름)와 `amount` (값) 두 열로 세웁니다. 6개 카테고리 × 6개 월 = **36행**의 긴 형식이 됩니다.
- 이렇게 `melt`한 긴 형식은 `seaborn·plotly` 같은 도구에 그대로 넣어 카테고리별 선그래프를 그리기 좋습니다. **넓은 표는 사람용, 긴 표는 도구용**이라는 원칙을 기억하면 언제 무엇으로 바꿀지 판단할 수 있습니다.

즉시 연습 (2종 1세트)

직접 해 보기 — 채널×월 표를 넓게·길게 오가기

이번에는 카테고리 대신 **채널(channel: web·app·store)**을 축으로 삼아 채널별·월별 주문 건수 표를 만들고, 넓은/긴 형식을 오가 보십시오. 아래를 **직접 실행하고 실제 출력을 기록**하십시오.

```
# (1) 채널x월 주문 건수 긴 형식
long_ch = (o.groupby(["channel", "month"])["order_id"]
           .count().reset_index())
print(long_ch.head())

# (2) pivot으로 넓은 표: 행=채널, 열=월, 칸=건수
wide_ch = long_ch.pivot(index="channel", columns="month",
                        values="order_id")
print(wide_ch)

# (3) melt로 다시 긴 형식으로 되돌리고 행수 확인
back_ch = wide_ch.reset_index().melt(
    id_vars="channel", var_name="month", value_name="건수")
print("되돌린 긴 형식 행수:", back_ch.shape[0])
```

관찰한 실제 값을 기록하십시오.

- [넓은 표의 모양]: (몇 행 × 몇 열인가 — 채널 수와 월 수)
- [web 채널 6월 건수]: (넓은 표의 해당 칸 값)
- [melt 후 긴 형식 행수]: (채널 수 × 월 수와 일치하는가)
- [어느 형식이 채널 3개를 겹친 선그래프에 편한가]: (긴 형식인 이유를 한 줄로)

자가체크

- [] pivot으로 만든 넓은 표의 행·열이 각각 채널·월임을 실제 출력으로 확인했다.
- [] melt 후 긴 형식의 행수가 (채널 수 × 월 수)와 같은지 계산해 대조했다.
- [] concat과 merge의 차이(단순 쌓기 vs 키 매칭)를 한 문장으로 적었다.
- [] pivot과 pivot_table을 언제 각각 쓰는지(중복 조합 여부) 설명할 수 있다.

흔한 오류와 교정

오개념: "concat도 merge처럼 알아서 짝을 맞춰 준다"

concat은 키 매칭을 하지 않습니다. 단순히 쌓기만 하므로, 이어 붙이는 조각들의 열 이름이 정확히 같아야 합니다. 한 조각에만 있는 열이 있으면 다른 조각의 그 자리는 NaN으로 채워집니다. 또 ignore_index=True를 빼먹으면 두 조각의 원래 인덱스가 그대로 남아 0,1,2,...,0,1,2,... 처럼 중복 인덱스가 생겨 이후 조치가 꼬입니다.

또 하나, pivot에 중복 조합을 넣으면 오류가 납니다. (카테고리, 월) 이 두 번 이상 나오는 원자료를 그대로 pivot 하면 "인덱스가 유일하지 않다"는 오류를 만납니다. 이때는 값을 합쳐야 하므로 pivot 이 아니라 8장의 pivot_table(..., aggfunc="sum") 을 써야 합니다. 먼저 groupby로 집계 조합을 유일하게 만든 뒤 pivot하는 것도 같은 해법입니다.

9.3 종합 worked example: 분석용 통합 테이블 구축

이제 이 장의 두 개념(merge-재구조화)과 5~6장의 정제를 합쳐, 이 책의 나머지 분석이 계속 쓸 **통합 테이블 하나**를 완성하겠습니다. 순서는 "정제 → 순차 병합 → 파생 → 무결성 점검"입니다.

```
import pandas as pd

customers = pd.read_csv("data/customers.csv")
products = pd.read_csv("data/products.csv")
orders = pd.read_csv("data/orders.csv", parse_dates=["order_datetime"])
items = pd.read_csv("data/order_items.csv")

# (1) 정제 (5~6장 규칙): 문자열로 오염된 가격 숫자화, 카테고리 공백 통일,
# 중복 키 제거
products["price"] = pd.to_numeric(products["price"], errors="coerce")
products["category"] = products["category"].str.strip()
prod = products.drop_duplicates(subset="product_id")
cust = customers.drop_duplicates(subset="customer_id")
items["unit_price"] = pd.to_numeric(items["unit_price"], errors="coerce")

# (2) 결측 단가는 상품 기본가로 채움(5장 fillna 전략)
items = items.merge(prod[["product_id", "price"]], on="product_id", how="left")
items["unit_price"] = items["unit_price"].fillna(items["price"])
items = items.drop(columns="price")

# (3) 순차 병합: 상세 -> 주문 -> 고객 -> 상품
full = (items
        .merge(orders, on="order_id", how="left", validate="m:1")
        .merge(cust, on="customer_id", how="left", validate="m:1")
        .merge(prod, on="product_id", how="left", validate="m:1"))

# (4) 파생: 주문 금액 (7장 벡터 연산)
full["amount"] = full["unit_price"] * full["quantity"] * (1 - full["discount"])

print("통합 테이블 shape:", full.shape)
print("행수 보존 확인:", full.shape[0] == items.shape[0])
print("주문 금액 결측:", full["amount"].isna().sum())
```

```
통합 테이블 shape: (500000, 21)
행수 보존 확인: True
주문 금액 결측: 2324
```

- (1)~(2) 정제를 병합보다 먼저 합니다. 중복 키를 남겨 두면 9.1절에서 봤듯 행이 불어 나고, 오염된 문자열 가격을 두면 금액 계산이 깨집니다.
- (3) 세 번의 병합을 메서드 체이닝으로 이었고, 각 병합에 `validate="m:1"` 을 붙여 "주문 상세 여러 개 대 주문/고객/상품 하나"라는 관계가 실제로 성립하는지 강제 검사했습니다. 정제를 마쳤으므로 이 검사를 통과합니다.

- (4) 벡터 연산으로 주문 금액을 만들었습니다. 최종 통합 테이블은 500,000행 21열이고, 행수가 원본 order_items와 같아(무결성 확인) 매출·건수가 부풀러지지 않았음을 보장합니다.
- 다만 **주문 금액 결측이 2,324건 남았습니다**. 상품 46개는 가격 문자열이 숫자로 복원되지 않는 진짜 결측이라, 그 상품을 참조한 주문 상세의 단가를 기본가로도 채우지 못했기 때문입니다. 이렇게 정제로도 못 살리는 잔여 결측이 있음을 아는 것이 중요합니다. 실무에서는 이런 행을 분석에서 제외하거나(플래그 표시) 별도 확인 대상으로 남깁니다. 병합·파생 뒤에는 이처럼 결측을 다시 한번 세어 보는 습관이 필요합니다.

즉시 연습 (2종 1세트)

직접 해 보기 — 통합 테이블의 무결성을 스스로 점검하기

위 full 통합 테이블을 만든 뒤, 병합이 데이터를 왜곡하지 않았는지 **직접 검증**하십시오. 아래를 실행해 실제 값을 기록합니다.

```
# (1) 총 주문 상세 건수가 원본과 같은지
print("full 행수:", full.shape[0], "== 원본:", items.shape[0])

# (2) 고객 정보가 불지 않은(고아 키) 행이 몇 개인지
print("고객명 결측 행수:", full["name"].isna().sum())

# (3) full의 총 매출과, 병합 전 order_items에서 직접 계산한 총 매출 대조
oi_amount = (items["unit_price"] * items["quantity"]
              * (1 - items["discount"])).sum()
print("full 총매출:", round(full["amount"].sum()))
print("원본 총매출:", round(oi_amount))
```

기록:

- [full 행수 == 원본 행수]: (True인가)
- [고객명 결측(고아 키) 행수]: (약 0.5%인가)
- [full 총매출 vs 원본 총매출]: (두 값이 일치하는가 — 병합이 금액을 바꾸지 않았는가)

자가체크

- [] 통합 테이블 행수가 원본 order_items와 같음을 확인했다(중복으로 부풀지 않음).
- [] 고아 키(고객 없는 주문) 행이 left 병합으로 살아남았음을 확인했다.
- [] 병합 전후 총매출이 일치함을 실제 출력으로 대조했다.
- [] 정제를 병합보다 먼저 해야 하는 이유를 한 문장으로 적었다.

9.4 단원 미니 프로젝트: 분석용 통합 테이블 구축

이 단원(8단원: 여러 테이블 결합과 재구조화)의 개념 c15-c16만으로 완성하는 작은 산출물을 만듭니다. 네 테이블을 병합해 통합 테이블을 만들고, 무결성을 점검하고, 카테고리×월 매출을 재구조화하는 스크립트 `build_master.py` 입니다. 정답 코드를 그대로 베끼지 말고, 아래 마일스톤을 직접 실행하며 실제 출력을 기록하십시오.

미니 프로젝트 — `build_master.py` 만들기 (마일스톤 3단계)

마일스톤 1: 정제 후 순차 병합 9.3절을 참고해 정제(가격 숫자화·카테고리 공백 통일·중복 키 제거·결측 단가 채우기) 후 네 테이블을 순차 병합해 `full` 을 만드십시오. 각 병합에 `validate="m:1"` 을 붙입니다. 직접 실행해 기록하십시오.

- **[통합 테이블 shape]:** (실제 출력)
- **[행수가 원본 `order_items`와 같은가]:** (True/False)

마일스톤 2: 무결성 점검표 만들기 병합이 데이터를 왜곡하지 않았는지 아래 세 지표를 계산해 표로 정리하십시오.

- **[고아 `customer_id` 행수와 비율]:** (name이 NaN인 행)
- **[병합 전후 총매출 일치 여부]:** (원본에서 계산한 매출과 대조)
- **[카테고리별 결측(상품 정보 없는 상세) 행수]:** (category가 NaN인 행)

마일스톤 3: 카테고리×월 매출 재구조화 `full` 에서 주문 일시의 월을 뽑아 카테고리×월 매출을 `groupby`로 집계(긴 형식)한 뒤, `pivot` 으로 넓은 리포트 표를 만들고, 다시 `melt` 로 긴 형식으로 되돌려 두 형식의 행수를 기록하십시오.

- **[넓은 표 모양]:** (몇 행 × 몇 열)
- **[가장 매출이 큰 카테고리 그 달]:** (넓은 표에서 읽기)
- **[`melt` 후 긴 형식 행수]:** (카테고리 수 × 월 수와 일치하는가)

마지막으로, 이 통합 테이블을 `to_csv("master.csv", index=False)` 로 저장해 다음 장(시계열 분석)에서 다시 불러 쓸 수 있게 하십시오.

자가체크

- [] `build_master.py`를 실행하면 처음부터 끝까지 오류 없이 통합 테이블이 만들어진다(직접 실행해 확인).
- [] 통합 테이블 행수가 원본 `order_items`와 같아 매출이 누락되지 않았음을 검증했다.
- [] 무결성 점검표에 고아 키·총매출 대조·상품 결측 세 지표를 실제 값으로 채웠다.

- [] 카테고리×월 매출을 pivot(넓은)·melt(긴) 두 형식으로 오가며 각 형식의 쓰임을 적었다.
- [] master.csv로 저장해 다음 장에서 재사용할 준비를 마쳤다.

이 장에서 우리는 흩어진 네 테이블을 하나로 통합했습니다. **merge**는 키로 짝지어 옆에 붙이되 행수 변화(중복·고아 키)를 반드시 점검해야 하고, **concat**은 같은 구조를 단순히 쌓으며, **melt·pivot**은 값을 그대로 두고 긴/넓은 형식을 오갑니다. 이 통합 테이블(master.csv)은 이제 이 책의 나머지 분석이 달고 설 토대입니다. 다음 10장에서는 이 테이블의 주문 일시를 시간 축으로 삼아, 상반기 내내 커지는 매출 추세와 주말에 몰리는 주문 패턴을 시계열 도구로 정면으로 읽어 보겠습니다.

10장 시계열 분석

9장에서 흩어진 네 테이블을 하나로 병합해 통합 테이블(master.csv)을 만들었습니다. 그 통합 테이블에는 지금까지 **거의 쓰지 않은 열**이 하나 있습니다. 바로 주문이 일어난 시각, `order_datetime` 입니다. 8장까지의 집계는 "누가·무엇을·어디서"를 물었지만, "**언제**"는 다루지 않았습니다. 그런데 쇼핑물 데이터에서 가장 중요한 질문 중 하나가 "매출이 늘고 있는가, 줄고 있는가"입니다. 이 질문은 시간 축 위에서만 답할 수 있습니다.

이 장에서는 문자열로 저장된 시각을 **날짜·시간(datetime) 타입**으로 바꿔 연·월·요일을 뽑고, 날짜를 인덱스로 삼아 원하는 기간을 조회하며, **리샘플링(resampling)**으로 월별·주별 매출을 집계하고 **이동평균(rolling mean)**으로 추세를 매끄럽게 보는 법을 배웁니다. 실제 데이터로 작업하면서 이 데이터셋에 숨어 있는 두 패턴 — 상반기 내내 커지는 매출과 주말에 몰리는 주문 — 을 직접 수치로 확인하게 됩니다.

이 장의 학습 목표

- `to_datetime`으로 시각을 `datetime`으로 변환하고 `dt` 접근자로 연·월·요일·시간대를 추출할 수 있다 [→G8]
- 두 시각의 차(`Timedelta`)로 기간을 계산하고 날짜 인덱스로 기간을 슬라이싱할 수 있다 [→G8]
- `resample`로 월별·주별 매출·건수를 집계하고 `rolling`으로 이동평균을 구할 수 있다 [→G8]
- `shift·pct_change`로 전월 대비 성장률을 계산하고 특정 달 합계를 부분집합 합과 대조해 검증할 수 있다 [→G8]

선수 확인: 이 장은 6장(개념 c06 인덱스 다루기, `set_index`-기간 슬라이싱)과 8장(개념 c13 그룹화·집계)을 마쳤다고 가정합니다. 리샘플링은 본질적으로 "시간 단위로 나눈 `groupby`"이고, 날짜 인덱스 슬라이싱은 `set_index`의 특수한 경우이기 때문입니다. 또한 이 장의 매출 예제는 9장에서 만든 통합 테이블(주문 금액 `amount` 열 포함)을 씁니다. 아직 그 표를 안 만드셨다면 9.3절을 먼저 완성하고 오시기 바랍니다.

10.1 개념 c17 — 날짜와 시간 다루기

도입: 문자열 시각으로는 계산이 안 된다

CSV에서 갓 읽어 온 `order_datetime` 은 "2024-06-25 00:17:41" 같은 **문자열**입니다. 문자열로는 "6월 주문만 뽑기", "가입 후 며칠 만에 첫 구매했나", "요일별로 나누기" 같은 시간 계산을 할 수 없습니다. 문자열끼리는 뺄셈이 안 되고, "6월"을 뽑으려면 문자열을 잘라 비교하는 번거로운 일을 해야 하며, 그마저 "2024-6-5" 처럼 표기가 흔들리면 정렬이 어긋납니다. 시간 데이터는 **datetime** 타입으로 바뀌야 비로소 계산·비교·집계가 가능해집니다.

정의

날짜와 시간 다루기란 문자열로 된 시각을 판다스의 **datetime64** 타입으로 변환한 뒤, 그 타입이 제공하는 시간 연산(연·월·요일 추출, 기간 계산, 기간 조회)을 활용하는 것입니다. 변환은 `pd.to_datetime(시리즈, errors="coerce")` 로 하며, 변환에 실패한 값(형식이 깨졌거나 비어 있는 값)은 **NaT(Not a Time, 날짜 결측)**가 됩니다. 변환된 datetime 시리즈에서는 `.dt` 접근자로 연·월·요일 등을 열로 뽑습니다.

핵심 용어

- **datetime64**: 판다스의 날짜·시간 자료형입니다. 이 타입이어야 시각 뺄셈·기간 슬라이싱·리샘플링이 됩니다.
- **NaT (Not a Time)**: 날짜의 결측값입니다. 숫자 열의 NaN에 해당하며, 변환 실패나 빈 값이 여기로 모입니다.
- **dt 접근자**: datetime 시리즈에 붙는 특수 접근자로, `dt.year · dt.month · dt.dayofweek · dt.hour · dt.day_name()` 등으로 구성 요소를 꺼냅니다. (문자열 열의 `str` 접근자와 같은 계열입니다.)
- **Timedelta**: 두 시각의 차로 얻는 "기간" 값입니다. `(끝 - 시작).dt.days` 로 일수를 얻습니다.

동작 원리

`dt.dayofweek` 는 **월요일을 0, 일요일을 6**으로 매기는 정수를 돌려줍니다. 그래서 `dt.dayofweek >= 5` 가 주말(토·일)을 뜻합니다. 요일 이름이 필요하면 `dt.day_name()` 이 "Monday" 같은 문자열을 줍니다. 이렇게 뽑은 연·월·요일·시간대 열은 그 자체로 **분석 축**이 되어, `groupby`의 기준 열로 바로 쓸 수 있습니다. "요일별 주문 건수"는 `dt.day_name()` 으로 요일 열을 만든 뒤 그 열로 `groupby`하면 나옵니다.

한 가지 주의할 점은 `dt` 접근자가 **datetime** 타입 **시리즈에만** 동작한다는 것입니다. 문자열 시리즈에 `dt.month` 를 쓰면 오류가 납니다. 그래서 순서가 중요합니다. **먼저 `to_datetime`으로 변**

환하고, 그다음 dt로 추출합니다.

worked example: 시각을 변환하고 요일 편향을 확인하기

주문 데이터를 `parse_dates` 없이 일부러 문자열로 불러온 뒤, 직접 변환해 보겠습니다.

```
import pandas as pd

orders = pd.read_csv("data/orders.csv") # 일부러 parse 안 함

print("변환 전 dtype:", orders["order_datetime"].dtype)
orders["order_datetime"] = pd.to_datetime(
    orders["order_datetime"], errors="coerce")
print("변환 후 dtype:", orders["order_datetime"].dtype)
print("NaT(변환 실패·결측) 개수:", orders["order_datetime"].isna().sum())
```

```
변환 전 dtype: object
변환 후 dtype: datetime64[ns]
NaT(변환 실패·결측) 개수: 1933
```

- `pd.to_datetime(..., errors="coerce")` — 문자열 object 를 `datetime64[ns]` 로 바꿉니다. `errors="coerce"` 는 변환할 수 없는 값을 오류로 멈추는 대신 **NaT로 처리**하라는 뜻입니다.
- 변환 결과 **1,933개의 NaT**가 나왔습니다. 이 데이터셋은 주문 일시 일부를 비워 두었는데 (결측), 그 빈 값들이 NaT로 모였습니다. 만약 `errors="coerce"` 없이 변환했다면 이 지점에서 오류로 멈췄을 것입니다.

변환이 왜 중요한지는 **비교·필터**에서 바로 드러납니다. `datetime`이 되면 날짜를 숫자처럼 크기 비교할 수 있어, "6월 이후 주문"을 문자열 자르기 없이 한 줄로 거를 수 있습니다(4장의 불리언 필터가 시각에도 그대로 적용됩니다).

```
june_on = orders[orders["order_datetime"] >= "2024-06-01"]
print("6월 1일 이후 주문:", june_on.shape[0], "건")
print("가장 이른 주문:", orders["order_datetime"].min())
print("가장 늦은 주문:", orders["order_datetime"].max())
```

```
6월 1일 이후 주문: 42959 건
가장 이른 주문: 2024-01-01 00:02:25
가장 늦은 주문: 2025-06-30 03:01:14
```

- `orders["order_datetime"] >= "2024-06-01"` — `datetime` 열은 문자열 날짜와 직접 비교됩니다(판다스가 오른쪽 문자열을 자동으로 시각으로 해석). 문자열 열이었다면 사전순 비교라 엉뚱한 결과가 나왔을 것입니다.

- `min()` · `max()` 로 데이터의 시간 범위를 보니, 대부분 2024년 상반기인데 **가장 늦은 주문이 2025년 6월**입니다. 이는 이후 정제에서 다룰 미래 날짜 이상값이 섞여 있음을 시사합니다. `datetime`으로 바꿨기 때문에 이런 범위 점검이 한 줄로 가능합니다.

이제 `dt` 접근자로 시간 축 열을 뽑고, 요일별 주문을 세어 보겠습니다.

```
o = orders.dropna(subset="order_datetime").copy()
o["month"] = o["order_datetime"].dt.month
o["dow"] = o["order_datetime"].dt.dayofweek      # 0=월 ... 6=일
o["dow_name"] = o["order_datetime"].dt.day_name()

# 요일별 주문 건수 (월~일 순서로)
order_names = ["Monday", "Tuesday", "Wednesday", "Thursday",
               "Friday", "Saturday", "Sunday"]
print(o.groupby("dow_name")["order_id"].count().reindex(order_names))
```

```
dow_name
Monday      25286
Tuesday     25134
Wednesday   24977
Thursday    25568
Friday      25478
Saturday    36037
Sunday      35587
```

- `dt.month` · `dt.dayofweek` · `dt.day_name()` 로 세 개의 분석 축 열을 만들었습니다. `dayofweek` 는 정수(월=0)라 계산에 편하고, `day_name()`은 사람이 읽기 좋습니다.
- 요일별 건수를 보면 **주말(토 36,037건·일 35,587건)이 평일(약 25,000건)보다 뚜렷하게 많습니다**. 이 데이터셋은 주말에 주문이 몰리는 편향을 담고 있고, `dt` 접근자 덕분에 그 패턴이 한눈에 드러났습니다.

일평균으로 환산하면 편향이 더 분명해집니다.

```
# 날짜별 주문 수를 세고, 그 날이 주말인지로 나눠 일평균 비교
days = o["order_datetime"].dt.normalize()      # 시각을 자정으로 잘라 날짜만
day_counts = o.groupby(days).size()
is_weekend = day_counts.index.dayofweek >= 5
print("평일 일평균 주문: %.0f건" % day_counts[~is_weekend].mean())
print("주말 일평균 주문: %.0f건" % day_counts[is_weekend].mean())
```

```
평일 일평균 주문: 800건
주말 일평균 주문: 1119건
```

- `dt.normalize()` 는 시각의 시·분·초를 0으로 만들어 "같은 날"끼리 묶을 수 있게 합니다. 그 날짜로 groupby해 하루당 주문 수를 셉습니다.
- 일평균으로 보면 **주말 1,119건 대 평일 800건**으로, 주말이 하루 평균 약 40% 더 많습니다. 단순 합계는 평일이 5일이라 커 보일 수 있지만, 일평균으로 환산하니 주말 편향이 정확히 드러납니다.

즉시 연습 (2종 1세트)

직접 해 보기 — 시간대별 주문 분포와 첫 구매까지 걸린 일수

datetime의 다른 두 활용을 직접 실행하고 실제 출력을 기록하십시오.

```
# (1) 시간대(hour)별 주문 건수 — 언제 가장 많이 주문하나
o["hour"] = o["order_datetime"].dt.hour
by_hour = o.groupby("hour")["order_id"].count()
print("가장 주문 많은 시간대:", by_hour.idxmax(), "시,", by_hour.max(), "건")

# (2) Timedelta: 가입일부터 첫 주문까지 걸린 일수
customers = pd.read_csv("data/customers.csv")
customers["signup_date"] = pd.to_datetime(
    customers["signup_date"], errors="coerce")
first_order = o.groupby("customer_id")["order_datetime"].min()
cust = customers.drop_duplicates("customer_id").set_index("customer_id")
gap = (first_order - cust["signup_date"]).dt.days.dropna()
print("첫 구매까지 평균 %.1f일, 중앙값 %.0f일" % (gap.mean(), gap.median()))
```

기록:

- [가장 주문이 많은 시간대와 건수]: (실제 출력)
- [첫 구매까지 평균·중앙값 일수]: (실제 출력 — 두 시각의 뺄셈이 Timedelta가 되고 .dt.days로 일수가 됨을 확인)
- [gap에 결측(dropna로 제거된 행)이 있었는가]: (가입일이나 주문 기록이 없는 고객)

자가체크

- [] to_datetime 변환 전후의 dtype이 object → datetime64로 바뀌는 것을 실제로 확인했다.
- [] dt.hour로 만든 시간대 열로 groupby해 실제 건수를 기록했다.
- [] 두 datetime의 뺄셈이 Timedelta가 되고 .dt.days 로 일수가 나옴을 확인했다.
- [] dt 접근자는 datetime 타입에만 쓸 수 있음을 이해했다(문자열엔 안 됨).

흔한 오류와 교정

오개념: "변환 실패(NaT)는 무시해도 된다"

`errors="coerce"` 로 변환하면 실패한 값이 조용히 NaT가 됩니다. 이것을 확인하지 않고 넘어가면, 이후 집계에서 그 행들이 슬그머니 빠져 **합계가 실제보다 작게** 나옵니다. 변환 직후에는 반드시 `isna().sum()` 으로 NaT 개수를 세고, 그 규모가 분석에 영향을 줄지 판단하십시오. 이 데이터에서는 1,933건(전체의 약 1%)이 NaT였습니다.

또 흔한 실수는 **문자열 날짜를 변환하지 않고 그냥 정렬·비교하는 것**입니다. "2024-6-5" 와 "2024-12-1" 을 문자열로 비교하면 사전순이라 "2024-12-1" 이 더 작다고 나올 수 있습니다. `datetime`으로 변환하면 이런 착오가 사라집니다. **시간을 다루기 전에 먼저 `datetime`으로 바꾼다** — 이것이 첫 단계입니다.

10.2 개념 c18 — 리샘플링과 이동집계

도입: 월별·주별로 다시 묶어 추세를 본다

앞 절에서 요일·시간대 축을 만들었습니다. 이제 진짜 질문에 답할 차례입니다. "매출이 달마다 어떻게 변하나?" 이 질문은 주문을 **월 단위로 다시 묶어** 각 달의 매출을 합해야 답할 수 있습니다. `dt.month` 로 월 열을 만들어 `groupby`할 수도 있지만, 연도가 섞이거나 주·분기 단위로 바꾸려면 번거롭습니다. 판다스는 시간 축에 특화된 **리샘플링(resampling)**이라는 전용 도구로 이 일을 훨씬 간결하게 처리합니다.

정의

리샘플링(resample)은 날짜 인덱스를 가진 데이터를 원하는 **시간 단위(일·주·월·분기)**로 다시 묶어 집계하는 시계열 전용 `groupby`입니다. `df.resample("ME")["amount"].sum()` 은 월별(ME=월말) 매출 합계를, `resample("W")` 는 주별 집계를 냅니다. **이동집계(rolling)**는 연속한 구간(창, window)에 대해 계산해 값을 매끄럽게 보는 도구로, `rolling(window=7).mean()` 은 7일 **이동평균(rolling mean)**을 냅니다.

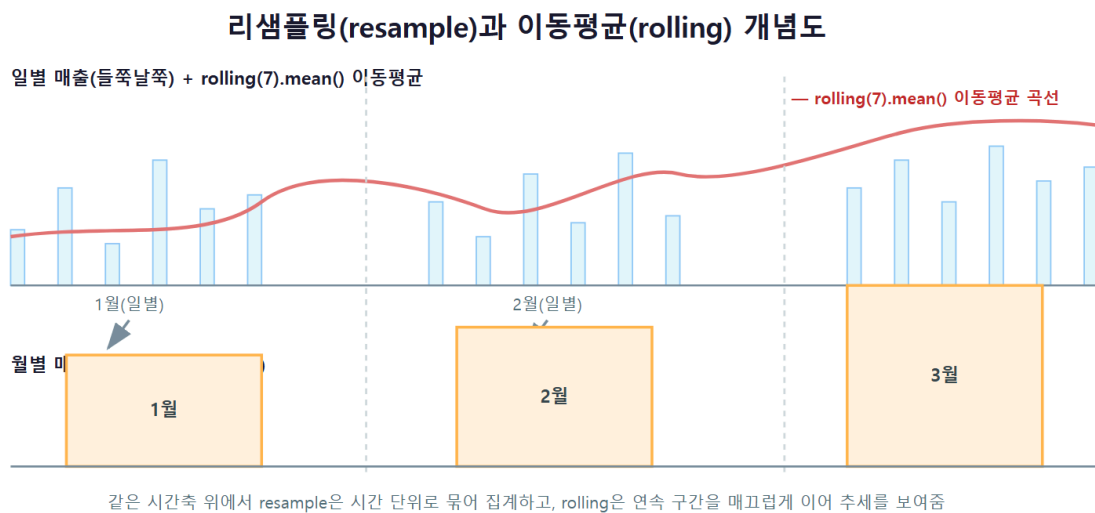
핵심 용어

- **규칙 문자열(rule)**: 리샘플링의 시간 단위입니다. D (일)·W (주)·ME (월말)·MS (월초)·QE (분기말) 등. 판다스 2.2에서는 월말이 ME, 월초가 MS 입니다.
- **rolling window(창)**: 이동집계가 한 번에 보는 연속 구간의 크기입니다. `window=7` 이면 "직전 7개"입니다. 창이 클수록 더 매끄럽지만 반응이 느립니다.

- **shift**: 값을 앞뒤로 한 칸 미는 연산입니다. "전월 값"을 옆에 두어 증감을 계산할 때 씁니다.
- **pct_change**: 직전 값 대비 변화율(성장률)을 바로 계산합니다. 내부적으로 shift를 씁니다.

동작 원리

리샘플링이 동작하려면 데이터가 **날짜 인덱스**를 가져야 합니다(또는 `on="날짜열"`로 지정). 그래서 순서는 "주문 일시를 인덱스로 올리고 → `resample`"입니다. 아래 그림은 개별 주문(원자료)이 월 단위 구간으로 묶여 각 달의 합계가 되는 과정과, 그 위에 이동평균 창이 미끄러지며 매끄러운 추세선을 만드는 과정을 함께 보여 줍니다.



이동평균의 특징 하나를 기억하십시오. `rolling(window=7)`의 **앞 6개 값은 NaN**입니다. 7개가 모여야 첫 평균을 낼 수 있기 때문입니다. 이것은 오류가 아니라 정상 동작이며, 필요하면 `min_periods`로 "몇 개만 모여도 계산"하도록 조절할 수 있습니다.

worked example: 월별 매출 추세와 이동평균

9장에서 만든 통합 테이블(`full`, `amount` 열 포함)을 날짜 인덱스로 두고 월별 매출을 집계합니다.

```
# full: 9.3절에서 만든 통합 테이블 (amount 열 포함, order_datetime은 datetime)
full = full.dropna(subset="order_datetime")
ts = full.set_index("order_datetime").sort_index()
ts = ts.loc["2024-01":"2024-06"] # 분석 대상 상반기로 한정

pd.options.display.float_format = "{:,.0f}".format
monthly = ts.resample("ME").agg(매출=("amount", "sum"),
                               건수=("order_item_id", "count"))

print(monthly)
```

	매출	건수
order_datetime		
2024-01-31	6,736,803,392	58994
2024-02-29	7,358,969,754	64458
2024-03-31	9,168,134,804	80810
2024-04-30	9,856,419,707	85984
2024-05-31	11,157,759,981	97424
2024-06-30	12,205,608,216	107444

- `set_index("order_datetime").sort_index()` 로 주문 일시를 인덱스로 올리고 시간순으로 정렬했습니다. `resample`은 정렬된 날짜 인덱스에서 가장 잘 동작합니다.
- `resample("ME").agg(...)` 는 월말(ME) 단위로 묶어 매출 합계와 건수를 한 번에 냈습니다. 8장의 named aggregation(이름 붙은 집계)을 `resample`에도 똑같이 씁니다.
- 결과가 명확한 **상승 추세**를 보입니다. 1월 약 67억에서 6월 약 122억으로, 반년 만에 매출이 거의 두 배가 됐습니다. 이 데이터셋은 상반기 성장 추세를 담고 있고, 리샘플링이 그 추세를 숫자로 뽑아냈습니다.

집계가 옳은지 반드시 **대조 검증**합니다. 리샘플링 결과의 3월 매출을, 3월만 직접 뽑아 합한 값과 비교합니다.

```
mar_resample = float(monthly.loc["2024-03-31", "매출"])
mar_subset = float(ts.loc["2024-03", "amount"].sum()) # 날짜 인덱스 슬라이싱
print("resample 3월:", format(mar_resample, ",.0f"))
print("부분집합 3월:", format(mar_subset, ",.0f"))
print("일치?", round(mar_resample, 2) == round(mar_subset, 2))
```

```
resample 3월: 9,168,134,804
부분집합 3월: 9,168,134,804
일치? True
```

- `ts.loc["2024-03"]` 는 날짜 인덱스의 **기간 슬라이싱**입니다. "2024-03" 이라는 부분 문자열만으로 3월 전체 행을 뽑습니다(6장의 인덱스 다루기가 시계열에서 이렇게 쓰입니다).

- 리샘플링이 낸 3월 매출과, 3월만 직접 뽑아 합한 값이 **정확히 일치**합니다. 이 대조로 `resample`이 누락·중복 없이 집계했음을 확인했습니다. 집계 결과는 늘 이렇게 다른 경로로 계산하는 습관을 들이십시오.

이제 일별 매출에 7일 이동평균을 얹어 추세를 매끄럽게 봅니다.

```
daily = ts["amount"].resample("D").sum()
roll7 = daily.rolling(window=7).mean()
tbl = pd.DataFrame({"일매출": daily, "이동평균7": roll7})
print(tbl.head(9))
```

	일매출	이동평균7
order_datetime		
2024-01-01	179,280,476	NaN
2024-01-02	194,048,713	NaN
2024-01-03	200,231,805	NaN
2024-01-04	180,888,007	NaN
2024-01-05	199,688,053	NaN
2024-01-06	260,134,445	NaN
2024-01-07	296,094,295	215,766,542
2024-01-08	182,510,646	216,227,995
2024-01-09	207,501,531	218,149,826

- `resample("D").sum()` 으로 일별 매출을 만든 뒤 `rolling(window=7).mean()` 으로 7일 이동평균을 얹었습니다.
- 예상대로 **앞 6일의 이동평균은 NaN**입니다. 7일이 모이는 1월 7일부터 첫 평균(약 2.16억)이 나옵니다. 일별 매출은 주말·평일에 따라 크게 출렁이지만(1월 6일 2.6억 vs 1월 8일 1.8억), 이동평균은 그 출렁임을 눌러 완만한 추세만 남깁니다.

마지막으로 전월 대비 성장률을 `pct_change` 로 계산합니다. 퍼센트는 소수 첫째 자리까지 보려고 별도 시리즈로 출력합니다.

```
growth = (monthly["매출"].pct_change() * 100).round(1)
print(growth)
```

order_datetime	
2024-01-31	NaN
2024-02-29	9.2
2024-03-31	24.6
2024-04-30	7.5
2024-05-31	13.2
2024-06-30	9.4
Freq: ME	

- `pct_change()` 는 각 달의 매출을 직전 달과 비교해 변화율을 냅니다. 첫 달은 비교할 이전 값이 없어 NaN입니다.
- 매달 7~25% 성장하며, 특히 **3월에 24.6%로 크게 뛩니다**. 이런 월별 성장률은 "추세가 얼마나 가파른가"를 한눈에 보여 주는 지표로, 캡스톤 리포트의 핵심 발견이 됩니다.

`pct_change` 가 어떻게 동작하는지 이해하려면 그 밑바탕인 `shift` 를 직접 봐야 합니다.

`shift(1)` 은 값을 한 칸 아래로 밀어 "직전 달 값"을 같은 행에 놓아 줍니다. 그러면 이번 달과 전월을 나란히 두고 증감을 계산할 수 있습니다.

```
m = monthly[["매출"]].copy()
m["전월매출"] = m["매출"].shift(1)      # 값을 한 칸 아래로
m["증감액"] = m["매출"] - m["전월매출"]  # 이번 달 - 전월
print(m)
```

	매출	전월매출	증감액
order_datetime			
2024-01-31	6,736,803,392	NaN	NaN
2024-02-29	7,358,969,754	6,736,803,392	622,166,362
2024-03-31	9,168,134,804	7,358,969,754	1,809,165,050
2024-04-30	9,856,419,707	9,168,134,804	688,284,903
2024-05-31	11,157,759,981	9,856,419,707	1,301,340,274
2024-06-30	12,205,608,216	11,157,759,981	1,047,848,235

- `shift(1)` 은 각 값을 한 행 아래로 밀어, 2월 행의 "전월매출" 칸에 1월 매출이 오게 합니다. 첫 행은 밀려올 이전 값이 없어 NaN입니다.
- 이렇게 이번 달과 전월을 한 행에 두면 증감액(뽀셈)이나 성장률(나눗셈)을 자유롭게 계산할 수 있습니다. `pct_change()` 는 결국 $(\text{현재} - \text{shift}(1)) / \text{shift}(1)$ 을 한 번에 해 주는 편의 함수입니다. 3월 증감액이 약 18억으로 가장 크고, 이는 앞서 본 24.6% 성장과 맞아떨어집니다.

누적 추세가 필요하면 `expanding` 을 씁니다. `rolling`이 고정 크기 창(직전 7개 등)을 미끄러뜨리는 것과 달리, `expanding`은 **처음부터 현재까지 전부**를 창으로 삼아 누적합·누적평균을 냅니다.

```
print(monthly["매출"].expanding().sum())      # 처음부터 현재까지 누적 매출
```

```
order_datetime
2024-01-31    6,736,803,392
2024-02-29    14,095,773,146
2024-03-31    23,263,907,950
2024-04-30    33,120,327,657
2024-05-31    44,278,087,638
2024-06-30    56,483,695,854
Freq: ME
```

- `expanding().sum()` 은 1월부터 각 달까지의 누적 매출을 냅니다. 6월 값(약 565억)이 상반기 전체 매출 합계입니다.
- `rolling`은 "최근 흐름", `expanding`은 "지금까지 누적", `pct_change/shift`는 "직전 대비 변화"를 봅니다. 세 도구를 목적에 맞게 골라 쓰면 같은 시계열에서 서로 다른 이야기를 읽어 낼 수 있습니다.

즉시 연습 (2종 1세트)

직접 해 보기 — 주별 매출과 채널별 월 추세

`resample`의 다른 단위와 조합을 직접 실행하고 실제 출력을 기록하십시오.

```
# (1) 주별(W) 매출 추세 — 앞 8주
weekly = ts["amount"].resample("W").sum()
print(weekly.head(8))

# (2) 채널별 월별 매출: resample과 groupby를 함께
by_ch = (ts.groupby("channel")["amount"]
         .resample("ME").sum().unstack(level=0))
print(by_ch)

# (3) 대조 검증: 6월 전체 매출 = 채널별 6월 매출의 합
total_jun = float(ts.loc["2024-06", "amount"].sum())
by_ch_jun = float(by_ch.loc["2024-06-30"].sum())
print("6월 전체:", round(total_jun), " 채널합:", round(by_ch_jun))
```

기록:

- [1주차 매출]: (`weekly`의 첫 값)
- [채널별 월 매출 표의 모양]: (몇 행 × 몇 열 — 월 수와 채널 수)
- [6월 전체 매출 == 채널별 6월 매출 합]: (두 값이 일치하는가 — 집계 검증)

자가체크

- [] `resample("W")`로 주별 매출을 실제로 뽑아 기록했다.
- [] 채널별 월 매출 표를 만들고 `unstack`으로 채널을 열로 펼쳤다.

- [] 6월 전체 매출과 채널별 6월 매출의 합이 일치함을 대조 검증했다.
- [] 리샘플링이 날짜 인덱스(또는 on=날짜열)를 필요로 함을 이해했다.

흔한 오류와 교정

오개념: "resample은 날짜 열만 있으면 된다"

resample은 데이터가 **날짜 인덱스**를 가져야 동작합니다. 날짜가 그냥 한 열로 있고 인덱스가 정수라면 resample은 오류를 냅니다. `set_index("order_datetime")`으로 인덱스로 올리거나, `resample("ME", on="order_datetime")`처럼 on으로 날짜 열을 지정해야 합니다.

또 하나, **월 단위 규칙 문자열**을 헛갈리기 쉽습니다. 판다스 2.2에서 ME는 각 달의 **말일**을 라벨로 씁니다(1월은 2024-01-31). 월초 기준이 필요하다면 MS를 씁니다. 그리고 원자료에 없는 기간(예: 주문이 하나도 없는 달)도 resample은 **0 또는 NaN으로 채워** 연속된 시간 축을 만듭니다. 만약 원본 날짜에 잘못된 미래 날짜(2025년 등)가 섞여 있으면 resample 결과의 꼬리에 그 달들이 드러나므로, 이는 오히려 이상값을 발견하는 계기가 됩니다. 분석 전 `ts.loc["2024-01":"2024-06"]`처럼 대상 기간을 명시하면 이런 잡음을 걸러낼 수 있습니다.

10.3 단원 미니 프로젝트: 매출 추세 분석

이 단원(9단원: 시계열 분석)의 개념 c17~c18만으로 완성하는 산출물을 만듭니다. 9장의 통합 테이블(master.csv)을 불러 **월별·주별 매출 추세와 이동평균·성장률**을 뽑고, **특정 달 합계를 대조 검증**하는 스크립트 `trend_report.py`입니다. 아래 마일스톤을 **직접 실행하며 실제 출력을 기록**하십시오.

미니 프로젝트 — trend_report.py 만들기 (마일스톤 3단계)

마일스톤 1: 시간 축 준비 master.csv를 `parse_dates=["order_datetime"]`로 불러(또는 to_datetime 변환) NaT 개수를 확인하고, 상반기(2024-01~2024-06)로 한정된 뒤 날짜 인덱스로 올리십시오. 직접 실행해 기록하십시오.

- **[NaT(주문 일시 결측) 개수]:** (실제 출력)
- **[상반기 한정 후 행수]:** (실제 출력)

마일스톤 2: 월별·주별 추세와 성장률 resample로 월별 매출·건수를 집계하고, pct_change로 전월 대비 성장률을, resample("W")로 주별 매출을 뽑으십시오.

- **[월별 매출 6줄]:** (1~6월 실제 값)

- **[성장률이 가장 높은 달과 그 값]:** (pct_change 결과에서 읽기)
- **[월별 매출 + 7일 이동평균에서 첫 이동평균이 나오는 날짜]:** (앞 6일이 NaN인지 확인)

마일스톤 3: 대조 검증과 발견 정리 아무 달 하나를 골라 resample 결과와 그 달 부분집합 합이 일치하는지 검증하고, 이 추세 분석에서 읽어 낸 **발견 2가지**를 한 문장씩 적으십시오(예: "상반기 매출이 X% 성장", "주말 매출이 평일보다 큼").

- **[검증한 달과 두 값의 일치 여부]:** (True/False)
- **[발견 1-발견 2]:** (내가 데이터에서 읽은 근거 있는 문장)

마지막으로 월별 매출 표를 `to_csv("monthly_trend.csv")` 로 저장해 캡스톤 리포트에서 재사용하십시오.

자가체크

- [] trend_report.py가 처음부터 끝까지 오류 없이 실행된다(직접 실행해 확인).
- [] 월별 매출이 상승 추세를 실제 숫자로 확인하고 성장률을 계산했다.
- [] 7일 이동평균의 앞 6일이 NaN인 것을 확인하고 그 이유를 적었다.
- [] 특정 달 합계를 부분집합 합과 대조해 집계 정확성을 검증했다.
- [] 데이터에서 읽은 발견 2가지를 근거(숫자)와 함께 문장으로 적었다.

자주 묻는 질문 (FAQ)

Q. resample 과 groupby(df['order_datetime'].dt.month) 는 무엇이 다른가요? 둘 다 시간 단위로 묶어 집계하지만, `groupby(dt.month)` 는 **월 숫자(1~12)만** 보므로 연도가 다른 같은 달(2024년 3월과 2025년 3월)이 한데 섞입니다. 반면 `resample("ME")` 는 **실제 연-월**을 구분하고, 주문이 없는 달도 0으로 채워 연속된 시간 축을 만듭니다. 여러 해에 걸친 데이터라면 resample이 안전하고, 요일-시간대처럼 순환하는 축을 볼 때는 dt 추출 후 groupby가 적합합니다.

Q. rolling 의 window를 며칠로 잡아야 하나요? 목적에 달렸습니다. 창이 클수록(예: 30일) 더 매끄럽지만 최근 변화에 둔감해지고, 작을수록(예: 3일) 민감하지만 출렁임이 남습니다. 주간 리듬(주말 편향)을 없애고 추세만 보려면 7일이 흔한 선택입니다. 정답은 없으니 여러 window를 그려 비교하십시오.

Q. 날짜 인덱스 슬라이싱 df.loc["2024-03"] 이 예상과 다르게 나옵니다. 두 가지를 확인하십시오. 첫째, 인덱스가 **정렬돼 있어야** 기간 슬라이싱이 올바르게 동작합니다(`sort_index()`). 둘째, 인덱스가 정말 datetime 타입인지 확인하십시오. 문자열 인덱스라면 부분 문자열 매칭이 사전순으로 엉뚱하게 동작합니다.

Q. 월 합계가 원본보다 작게 나옵니다. NaT(날짜 결측) 행이 resample에서 빠졌을 가능성이 큼니다. `to_datetime(errors="coerce")` 직후 `isna().sum()` 으로 NaT 규모를 확인하고, 그 행들을 어떻게 처리할지(제외할지, 원인을 찾을지) 먼저 정하십시오.

이 장에서 우리는 시간 축을 다뤘습니다. 문자열 시각을 **datetime**으로 바꿔야 계산이 시작되고, **dt** 접근자로 연·월·요일 축을 뽑으며, **resample**로 시간 단위를 다시 묶고 **rolling**으로 추세를 매끄럽게 봤습니다. 그 결과 이 데이터셋의 두 실제 패턴 — 상반기 내내 매달 7~25%씩 커지는 매출과 평일 800건 대 주말 1,119건의 요일 편향 — 을 숫자로 밝혀냈습니다. 다음 11장에서는 지금까지 다룬 20만 행 규모를 넘어, 백만 행짜리 웹 행동 로그를 메모리 안에서 효율적으로 처리하는 성능 기법을 배웁니다.

11장 대용량 데이터와 성능 (심화)

지금까지 다룬 표들은 가장 큰 order_items도 50만 행이었고, 노트북 메모리에 통째로 올려 놓고 작업해도 문제가 없었습니다. 그런데 이 데이터셋에는 아직 열어 보지 않은 파일이 하나 남아 있습니다. 웹 행동 로그 `web_logs.csv` — **백만 행**짜리입니다. 사용자가 상품을 보고(view), 검색하고(search), 장바구니에 담고(cart), 구매하는(purchase) 모든 행동이 기록된 이 로그는, 그냥 `read_csv` 로 불러오는 것만으로도 메모리를 상당히 잡아먹습니다. 데이터가 이 정도로 커지면 "어떻게 계산하나"보다 "**메모리에 들어가긴 하나, 얼마나 빠른가**"가 문제가 됩니다.

이 장에서는 실제로 백만 행 로그를 다루며 성능을 배웁니다. 먼저 무엇이 메모리를 잡아먹는지 **측정**하고, **다운캐스팅**과 **범주형(category)**으로 메모리를 줄이며, 파일이 메모리보다 클 때 쓰는 **체크 읽기(chunked reading)**로 나눠 처리하고, CSV보다 작고 빠른 **Parquet** 형식으로 저장·재적재해 그 효과를 **직접 측정한 수치**로 확인합니다. 이 장의 모든 숫자는 실제로 실행해 얻은 값이며, 여러분의 컴퓨터에서도 직접 측정해 대조하게 합니다.

이 장의 학습 목표

- `memory_usage(deep=True)`로 `web_logs` 열별 메모리를 측정해 최적화 대상을 지목할 수 있다 [→G9]
- 다운캐스팅과 `category` 변환으로 메모리를 줄이고 `category`가 유리/불리한 경우를 설명할 수 있다 [→G9]
- `read_csv chunksize`로 체크 단위 집계를 수행해 전체를 메모리에 다 올리지 않고 처리할 수 있다 [→G9]
- `to_parquet·read_parquet`으로 저장·재적재하고 CSV 대비 크기·속도·메모리를 수치로 비교할 수 있다 [→G9]

선수 확인: 이 장은 2장(개념 c04 데이터 훑어보기, info·memory)과 6장(개념 c10 문자열 처리)을 마쳤다고 가정합니다. 메모리 측정은 `info()`의 확장이고, 범주형이 절감하는 대상이 바로 반복되는 문자열 열이기 때문입니다. 또한 8장(`groupby`)을 알면 체크별 부분 집계를 합치는 논리가 자연스럽습니다. 이 장은 **실무** 난이도로, 지금까지 배운 도구를 성능 관점에서 다시 봅니다.

환경 확인: 이 장은 Parquet 저장에 PyArrow가 필요합니다. `pip install pyarrow` 로 설치돼 있어야 합니다(1장 환경 준비에서 이미 설치했습니다).

11.1 개념 c19 — 메모리 최적화와 범주형

도입: 무엇이 메모리를 잡아먹는가

데이터가 커지면 메모리가 병목이 됩니다. 백만 행 로그를 여러 개 동시에 다루거나, 병합으로 더 넓은 표를 만들면 노트북이 느려지거나 멈춥니다. 그런데 무작정 "데이터를 줄이자"고 행을 버리면 분석이 왜곡됩니다. 올바른 접근은 **같은 데이터를 더 작은 방식으로 저장하는 것**입니다. 그러려면 먼저 **어느 열이 얼마나 메모리를 쓰는지**를 측정해야 합니다. 측정 없이 최적화하는 것은 어림짐작이고, 판다스는 열별 메모리를 정확히 재는 도구를 줍니다.

정의

메모리 최적화란 데이터의 값은 그대로 두되, 각 열을 **값의 실제 범위·종류에 맞는 더 작은 자료형**으로 바꿔 메모리 사용량을 줄이는 것입니다. 두 가지 핵심 기법이 있습니다. **다운캐스팅(downcasting)**은 넉넉한 정수·실수 타입을 값 범위에 맞는 작은 타입으로 바꾸는 것이고 (int64→int32 등), **범주형(categorical, category dtype)**은 종류가 적고 반복되는 문자열 열을 "정수 코드 + 값 사전"으로 저장해 메모리를 크게 줄이는 것입니다.

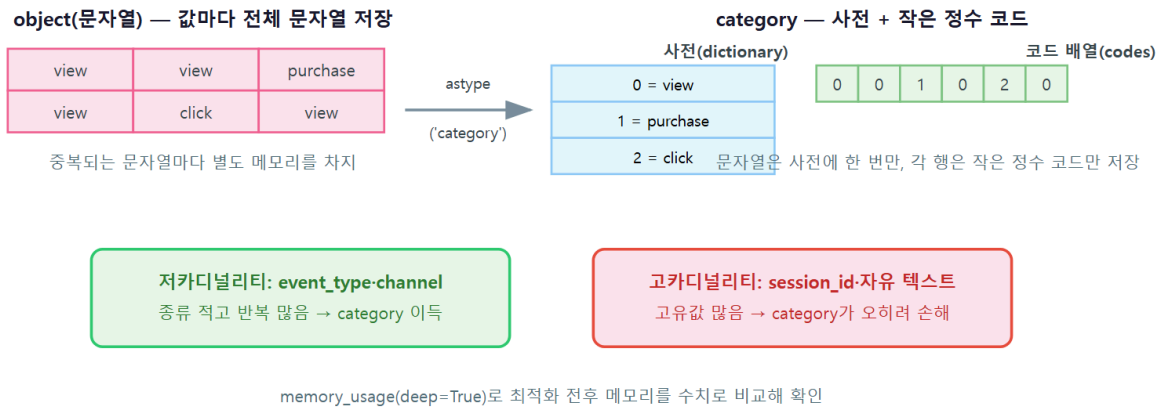
핵심 용어

- **memory_usage(deep=True):** 열별 실제 메모리를 바이트로 돌려줍니다. `deep=True` 를 주어야 문자열(object)의 진짜 크기를 셉니다(빠면 포인터 크기만 세어 과소평가).
- **다운캐스팅(downcasting):** 값 범위에 맞는 더 작은 수치 타입으로 바꾸는 것. int64(8바이트)→int32(4바이트)면 정수 열이 절반이 됩니다. `pd.to_numeric(s, downcast="integer")`.
- **범주형(category):** 값의 종류가 정해진 열을 정수 코드와 사전으로 저장하는 타입. `s.astype("category")`. 종류가 적을수록(저카디널리티) 이득이 큼니다.
- **카디널리티(cardinality):** 열의 고유티 개수입니다. 낮으면(예: 4종) 범주형이 유리하고, 거의 모든 값이 다르면(고카디널리티) 오히려 손해입니다.

동작 원리

범주형이 왜 메모리를 줄이는지가 핵심입니다. 문자열 "view" 를 백만 번 저장하면, 판다스는 그 문자열을 백만 번 실제로 담습니다. 반면 category로 바꾸면 {0: "view", 1: "purchase", 2: "search", 3: "cart"} 라는 **사전 하나**를 두고, 각 행에는 짧은 정수 코드(0~3)만 저장합니다. 백만 개의 긴 문자열이 백만 개의 작은 정수로 바뀌니 메모리가 급감합니다. 아래 그림은 문자열 저장과 범주형 저장의 차이, 그리고 다운캐스팅이 정수 타입의 바이트 폭을 줄이는 원리를 함께 보여 줍니다.

범주형(category) 저장 방식: object 대비 메모리 절감



여기서 중요한 판단 기준이 나옵니다. **범주형은 값이 반복될 때만 이득**입니다. 고윳값이 거의 모든 행에서 다른 열(예: 행마다 유일한 ID, 자유 텍스트)을 category로 바꾸면, 사전이 원본만큼 커지는 데다 정수 코드까지 더해져 **오히려 메모리가 늘어납니다**. 그래서 최적화는 "무조건 작게"가 아니라, `memory_usage` 로 측정하고 `nunique()` 로 카디널리티를 확인한 뒤 근거를 갖고 판단하는 일입니다.

worked example: web_logs의 메모리를 측정하고 줄이기

먼저 백만 행 로그를 불러 열별 메모리를 잡니다.

```
import pandas as pd

logs = pd.read_csv("data/web_logs.csv")
print("shape:", logs.shape)
print(logs.dtypes)

mu = logs.memory_usage(deep=True)
for col, nbytes in mu.items():
    print(" %-14s %8.1f MB" % (col, nbytes / 1024 / 1024))
print("총 메모리: %.1f MB" % (mu.sum() / 1024 / 1024))
```

```
shape: (1000000, 6)
log_id          int64
customer_id     float64
event_time      object
event_type      object
product_id      float64
session_id      object
dtype: object
Index           0.0 MB
log_id          7.6 MB
customer_id     7.6 MB
event_time      72.5 MB
event_type      58.9 MB
product_id      7.6 MB
session_id      66.8 MB
총 메모리: 221.0 MB
```

- `memory_usage(deep=True)` 로 열별 메모리를 잴습니다. 총 **221.0 MB**입니다. 한눈에 보이듯, 메모리를 많이 먹는 것은 문자열(object) 열들입니다. `event_time` (72.5 MB)·`session_id` (66.8 MB)·`event_type` (58.9 MB)이 대부분을 차지합니다.
- 반면 정수·실수 열(`log_id`·`customer_id`·`product_id`)은 각 7.6 MB로 작습니다. 백만 행 × 8 바이트 = 약 7.6 MB이기 때문입니다. **최적화 우선순위는 무거운 문자열 열이고**, 그중 `event_type` 은 값 종류가 적어 보이니 범주형 후보입니다.

`event_type` 의 고윳값을 확인하고 category로 바꿔 봅시다.

```
print("event_type 고윳값:", logs["event_type"].nunique(),
      list(logs["event_type"].unique()))

logs2 = logs.copy()
logs2["event_type"] = logs2["event_type"].astype("category")

before = logs.memory_usage(deep=True)["event_type"] / 1024 / 1024
after = logs2.memory_usage(deep=True)["event_type"] / 1024 / 1024
total_after = logs2.memory_usage(deep=True).sum() / 1024 / 1024
print("event_type 열: %.1f MB -> %.1f MB" % (before, after))
print("총 메모리: 221.0 MB -> %.1f MB" % total_after)
```

```
event_type 고윳값: 4 ['view', 'purchase', 'search', 'cart']
event_type 열: 58.9 MB -> 1.0 MB
총 메모리: 221.0 MB -> 163.1 MB
```

- `event_type` 은 고윳값이 **단 4종**(view·purchase·search·cart)입니다. 전형적인 저카디널리티 열입니다.
- category로 바꾸니 이 열이 **58.9 MB에서 1.0 MB로**, 약 59배 줄었습니다. 백만 개의 문자열이 백만 개의 작은 정수 코드로 바뀌었기 때문입니다. 전체 메모리도 221.0 MB에서

163.1 MB로 57.9 MB(약 26%) 줄었습니다. 열 하나 바꿨을 뿐인데 큰 효과입니다.

이번에는 category가 **손해인** 경우를 확인해, 언제 쓰면 안 되는지 봅니다.

```
print("log_id 고유값:", logs["log_id"].nunique(), "/", len(logs))
lb = logs.memory_usage(deep=True)["log_id"] / 1024 / 1024
la = logs["log_id"].astype("category").memory_usage(deep=True) / 1024 / 1024
print("log_id 열: %.1f MB -> category %.1f MB" % (lb, la))
```

```
log_id 고유값: 1000000 / 1000000
log_id 열: 7.6 MB -> category 43.7 MB
```

- log_id 는 모든 행이 유일(백만 행 모두 다른 값)한 고카디널리티 열입니다. 여기에 category를 적용하면 사전이 백만 개 항목으로 커지고, 거기에 정수 코드까지 더해져 **7.6 MB가 43.7 MB로 오히려 6배 늘어납니다.**
- 이것이 "무조건 category로 바꾸면 안 된다"는 규칙의 실증입니다. **반복이 많은 열(event_type)엔 이득, 거의 유일한 열(log_id)엔 손해.** 그래서 측정과 카디널리티 확인이 먼저입니다.

즉시 연습 (2종 1세트)

직접 해 보기 — session_id의 카디널리티와 정수 다운캐스팅

매매한 중간 사례(session_id)와 정수 다운캐스팅을 직접 실행하고 실제 출력을 기록하십시오.

```
# (1) session_id는 category가 이득인가? 고유값 비율을 보고 판단
print("session_id 고유값:", logs["session_id"].nunique(), "/", len(logs))
sb = logs.memory_usage(deep=True)["session_id"] / 1024 / 1024
sa = (logs["session_id"].astype("category")
      .memory_usage(deep=True) / 1024 / 1024)
print("session_id 열: %.1f MB -> category %.1f MB" % (sb, sa))

# (2) 정수 다운캐스팅: log_id를 값 범위에 맞는 작은 타입으로
print("log_id 범위:", logs["log_id"].min(), "~", logs["log_id"].max())
small = pd.to_numeric(logs["log_id"], downcast="integer")
print("log_id dtype:", logs["log_id"].dtype, "->", small.dtype)
print("log_id 열: %.1f MB -> %.1f MB"
      % (logs.memory_usage(deep=True)["log_id"] / 1024 / 1024,
         small.memory_usage(deep=True) / 1024 / 1024))
```

기록:

- [session_id 고유값 비율]: (고유값 / 백만 — 얼마나 반복되나)

- [session_id를 category로 하면 이득인가 손해인가]: (실제 MB 비교 — event_type만큼 극적이진 않아도 반복이 있으면 줄어듦)
- [log_id 다운캐스팅 후 dtype과 절감]: (int64 → 무엇으로 바뀌고 메모리는 얼마나 줄는가)
- [내 판단]: session_id를 실제 최적화에 category로 쓸지, 그 근거를 한 줄로

자가체크

- [] memory_usage(deep=True)로 열별 메모리를 실제로 측정해 무거운 열을 지목했다.
- [] event_type을 category로 바꿔 총 메모리가 줄어든 실제 수치를 기록했다.
- [] log_id를 category로 하면 메모리가 늘어나는 것을 실제로 확인했다(손해 사례).
- [] category가 유리한 조건(저카디널리티·반복)을 한 문장으로 설명할 수 있다.

흔한 오류와 교정

오개념: "memory_usage()만 보면 실제 메모리를 안다"

deep=True 를 빼면 문자열 열의 포인터 크기(행당 8바이트)만 세고, 문자열 실체가 차지하는 메모리는 빼먹습니다. 그러면 event_type 이 실제 58.9 MB인데 8 MB로 나와, 최적화 대상을 잘못 짚습니다. 문자열 열의 진짜 크기를 보려면 반드시 deep=True 를 줘야 합니다.

또 하나, 다운캐스팅은 값 범위를 넘으면 위험합니다. int8은 -128~127만 담으므로, 그보다 큰 값을 억지로 int8로 바꾸면 오버플로로 값이 망가집니다. pd.to_numeric(s, downcast="integer") 는 값 범위를 보고 안전한 가장 작은 타입을 골라 주므로 이것을 쓰는 편이 안전합니다. 그리고 결측(NaN)이 있는 정수 열은 일반 int로 못 바꿉니다(NaN은 실수라서). 이때는 Int32 같은 nullable 정수 타입(대문자 I)을 씁니다. 이 로그의 customer_id는 결측이 35만 건이라(로그인하지 않은 방문), 다운캐스트하려면 astype("Int32") 가 필요합니다.

11.2 개념 c20 — 청크 읽기와 Parquet

도입: 파일이 메모리보다 크면

앞 절은 "메모리에 들어온 뒤 줄이는" 이야기였습니다. 그런데 파일이 아예 메모리보다 크면 read_csv 한 줄에서 멈춰 버립니다. 또 CSV는 텍스트 형식이라 매번 숫자·날짜를 문자에서 해석하느라 읽기가 느리고, 타입 정보도 저장하지 않아 불러올 때마다 다시 지정해야 합니다. 이 절은 두 가지 해법을 배웁니다. 파일을 조각으로 나눠 읽는 청크 읽기와, CSV보다 작고 빠르며 타입을 보존하는 Parquet 형식입니다.

정의

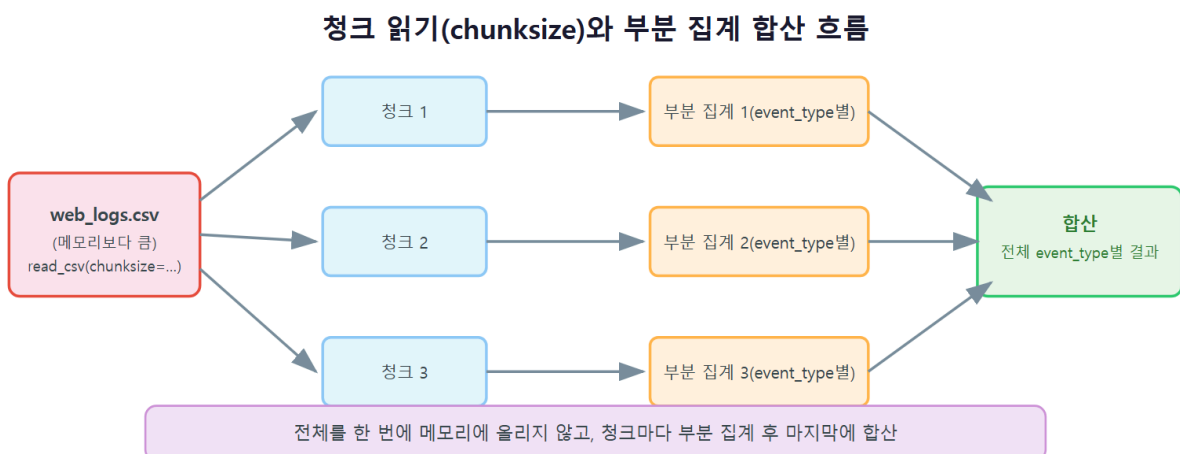
청크 읽기(chunked reading)는 `read_csv` 에 `chunksize` 를 줘서 데이터프레임을 **조각(청크)** 단위로 **순회하는 반복자**를 얻어, 각 청크를 부분 처리하고 마지막에 합치는 방식입니다. 전체를 메모리에 다 올리지 않고도 대용량을 집계할 수 있습니다. **Parquet**은 열 지향·압축·타입 보존 **바이너리 파일 형식**으로, `to_parquet` 으로 저장하고 `read_parquet` 으로 읽으며, CSV보다 파일이 작고 읽기가 빠르며 dtype과 category가 그대로 유지됩니다.

핵심 용어

- **chunksize**: `read_csv(path, chunksize=n)` 은 데이터프레임이 아니라 **n행씩 내주는 반복자**를 돌려줍니다. `for chunk in ...` 으로 순회합니다.
- **부분 집계 → 합산**: 각 청크에서 부분 결과(예: event_type 카운트)를 내고, 그것들을 더해 전체 결과를 만듭니다.
- **Parquet**: 열 단위로 압축 저장하는 형식. 텍스트 에디터로 못 열지만, 작고 빠르며 타입을 보존합니다. `.parquet` 확장자.
- **열 지향(columnar)**: 데이터를 행이 아니라 열 단위로 저장하는 방식. 필요한 열만 골라 읽기(`columns=`)에 유리합니다.

동작 원리

청크 읽기의 논리는 8장의 groupby와 같은 "쪼개서-계산해-합치기"입니다. 다만 쪼개는 기준이 그룹이 아니라 **파일의 물리적 조각**입니다. 아래 그림은 백만 행 파일이 20만 행짜리 청크 다섯 개로 나뉘어 각각 부분 집계된 뒤, 그 부분 결과들이 하나로 합쳐지는 과정을 보여 줍니다.



핵심은 **청크별 결과를 반드시 합쳐야** 전체 결과가 된다는 점입니다. 한 청크의 카운트는 그 20만 행만의 부분 결과이므로, 다섯 청크의 카운트를 더해야 백만 행 전체의 카운트가 됩니다. 이때 `Series.add(..., fill_value=0)` 을 쓰면 청크마다 나타나는 값이 조금씩 달라도 안전하게 누

적됩니다. 그리고 이렇게 얻은 결과는 전체를 한 번에 로드한 결과와 **정확히 일치해야** 하므로, 검증으로 대조하는 것이 필수입니다.

worked example: 청크로 집계하고 Parquet으로 저장하기

먼저 청크 읽기로 event_type별 카운트를 냅니다. 전체를 메모리에 올리지 않고, 필요한 열(event_type)만 읽습니다.

```
import pandas as pd

total = pd.Series(dtype="int64")
n_chunks = 0
for chunk in pd.read_csv("data/web_logs.csv",
                        usecols=["event_type"], chunksize=200000):
    total = total.add(chunk["event_type"].value_counts(), fill_value=0)
    n_chunks += 1

print("처리한 청크 수:", n_chunks)
print(total.astype(int))
```

```
처리한 청크 수: 5
cart          119434
purchase       80359
search        200206
view          600001
dtype: int64
```

- `chunksize=200000` 으로 백만 행이 **20만 행짜리 청크 5개**로 나뉘었습니다. `for chunk in ...` 이 청크를 하나씩 내주고, 각 청크에서 `value_counts()` 로 부분 카운트를 내 `total` 에 누적합니다.
- `usecols=["event_type"]` 로 필요한 열 하나만 읽어, 나머지 열은 메모리에 아예 올리지 않았습니다. 결과를 보면 조회(view)가 60만 건으로 압도적이고, 그다음 검색(search) 20만, 장바구니(cart) 12만, 구매(purchase) 8만 순입니다. 전형적인 퍼널(funnel) 형태입니다.

청크 집계가 옳은지, 전체를 한 번에 로드한 결과와 대조합니다.

```
full_vc = pd.read_csv("data/web_logs.csv")["event_type"].value_counts()
print("전체 로드 결과와 일치?",
      total.astype(int).sort_index().equals(full_vc.sort_index()))
```

```
전체 로드 결과와 일치? True
```

- 청크로 나눠 합산한 카운트가, 전체를 한 번에 로드해 센 카운트와 **완전히 일치**합니다. 부분 집계 → 합산이 올바르게 동작했음을 확인했습니다.
- 이 방식의 진짜 가치는 파일이 메모리보다 훨씬 클 때 드러납니다. 100 GB짜리 로그도 청크로 나누면 8 GB 노트북에서 집계할 수 있습니다.

이제 로그를 Parquet으로 저장해 CSV와 크기·속도를 비교합니다. 시간은 `time.perf_counter` 로 잹니다.

```
import time, os

logs = pd.read_csv("data/web_logs.csv")
logs["event_type"] = logs["event_type"].astype("category")
logs.to_parquet("web_logs.parquet", engine="pyarrow")

t0 = time.perf_counter()
_ = pd.read_csv("data/web_logs.csv")
t_csv = time.perf_counter() - t0

t0 = time.perf_counter()
back = pd.read_parquet("web_logs.parquet", engine="pyarrow")
t_pq = time.perf_counter() - t0

print("CSV 크기 %.1f MB vs Parquet 크기 %.1f MB"
      % (os.path.getsize("data/web_logs.csv") / 1024 / 1024,
         os.path.getsize("web_logs.parquet") / 1024 / 1024))
print("read_csv %.2f초 vs read_parquet %.2f초" % (t_csv, t_pq))
print("Parquet 재적재 후 event_type dtype:", back["event_type"].dtype)
```

```
CSV 크기 51.9 MB vs Parquet 크기 24.1 MB
read_csv 1.19초 vs read_parquet 0.60초
Parquet 재적재 후 event_type dtype: category
```

- **파일 크기가 51.9 MB에서 24.1 MB로 절반 이하가** 되었습니다. Parquet은 열 단위로 압축하기 때문입니다.
- **읽기 속도도 약 2배 빠릅니다**(1.19초 → 0.60초). CSV는 매번 텍스트를 숫자·날짜로 해석해야 하지만, Parquet은 타입이 바이너리로 저장돼 해석 없이 바로 읽습니다.
- 결정적으로 **event_type이 category 그대로 복원**되었습니다. CSV로 저장했다면 다시 문자열이 되어 category 변환을 또 해야 하지만, Parquet은 dtype을 보존합니다. (측정한 시간은 컴퓨터마다 다르므로, 여러분의 환경에서 직접 재어 비교하십시오. 중요한 것은 절대값이 아니라 **CSV보다 작고 빠르다**는 방향입니다.)

즉시 연습 (2종 1세트)

직접 해 보기 — 청크 집계 확장과 필요한 열만 읽기

청크 집계를 확장하고, Parquet의 열 선택 읽기를 직접 실행하고 실제 출력을 기록하십시오.

```
import time

# (1) 청크별로 event_type별 "고유 세션 수"를 누적 (부분 집계 -> 합산)
sess = pd.Series(dtype="int64")
for chunk in pd.read_csv("data/web_logs.csv",
                        usecols=["event_type", "session_id"],
                        chunksize=200000):
    part = chunk.groupby("event_type")["session_id"].nunique()
    sess = sess.add(part, fill_value=0)
print(sess.astype(int))

# (2) Parquet에서 필요한 두 열만 읽기 (열 지향의 이점)
t0 = time.perf_counter()
two = pd.read_parquet("web_logs.parquet",
                    columns=["event_time", "event_type"],
                    engine="pyarrow")
print("2개 열만 읽기: shape", two.shape, "%.2f초" % (time.perf_counter() - t0))
```

기록:

- [청크별 세션 수 합산 결과]: (event_type별 값 — 주의: 세션은 청크 경계에 걸치면 중복 계수될 수 있음. 정확한 전체 고유 세션은 전체 로드로 대조)
- [전체 로드로 센 event_type별 고유 세션과 청크 합산이 다른가]: (nunique는 단순 합산이 안 되는 이유를 관찰)
- [2개 열만 read_parquet한 shape와 시간]: (전체 6개 열 읽기보다 빠른가)

자가체크

- [] chunksize로 백만 행을 청크 5개로 나눠 순회했음을 실제 청크 수로 확인했다.
- [] 청크 집계 결과가 전체 로드 결과와 일치함을 대조했다(sum 가능한 집계).
- [] nunique처럼 단순 합산이 안 되는 집계는 청크 경계 문제가 있음을 관찰했다.
- [] Parquet에서 필요한 열만 읽으면 더 빠른 것을 실제로 확인했다.

흔한 오류와 교정

오개념: "chunksize를 주면 데이터프레임이 나온다"

read_csv(path, chunksize=n) 은 데이터프레임이 아니라 반복자(iterator)를 돌려줍니다. 그래서 바로 .head() 나 열 선택을 하면 오류가 납니다. 반드시 for chunk in ... 으로 순회하며 각 청크 (이것이 데이터프레임)를 처리해야 합니다.

또 하나 조심할 것은 **모든 집계는 청크로 단순 합산되지 않는다는** 점입니다. 합계(sum)·건수(count)는 청크별 결과를 더하면 되지만, **고유값 수(nunique)나 중앙값(median)은 청크 경계에 서 정보가 갈려** 단순 합산이 틀립니다. 예를 들어 한 세션이 여러 청크에 걸치면 각 청크가 그 세션을 따로 세어 중복됩니다. 이런 집계는 필요한 열만 전체 로드하거나, 청크에서 원자료(고유 세션 집합)를 모아 마지막에 한 번에 계산해야 합니다. **"이 집계는 부분 결과의 합으로 전체가 되는가"**를 늘 먼저 따지십시오.

11.3 종합 worked example: 최적화 전후 메모리·시간 비교

이 장의 두 개념을 합쳐, web_logs를 **전면 최적화**하고 그 효과를 하나의 표로 정리하겠습니다. 다운캐스팅·범주형·Parquet을 모두 적용한 전후를 비교합니다.

```
import pandas as pd

logs = pd.read_csv("data/web_logs.csv")
before = logs.memory_usage(deep=True).sum() / 1024 / 1024

# 전면 최적화: 정수 다운캐스트 + 결측 정수는 nullable + 저카디널리티 문자열 category
opt = logs.copy()
opt["log_id"] = pd.to_numeric(opt["log_id"], downcast="integer")
opt["customer_id"] = opt["customer_id"].astype("Int32") # 결측 35만 -> nullable
opt["event_type"] = opt["event_type"].astype("category")
after = opt.memory_usage(deep=True).sum() / 1024 / 1024

print("최적화 전: %.1f MB" % before)
print("최적화 후: %.1f MB" % after)
print("절감: %.1f MB (%.0f%%)" % (before - after,
                                (before - after) / before * 100))
```

```
최적화 전: 221.0 MB
최적화 후: 156.4 MB
절감: 64.6 MB (29%)
```

- 정수 다운캐스팅(log_id: int64→int32), 결측 있는 정수의 nullable 변환(customer_id→Int32), 저카디널리티 문자열의 범주형(event_type→category)을 함께 적용해 **221.0 MB에서 156.4 MB로 29% 절감**했습니다.
- 여기서 event_time·session_id는 그대로 뒀습니다. event_time은 datetime으로 바꾸면 줄지만 별도 처리가 필요하고, session_id는 반복이 있어 category로 줄일 여지가 있으나 이

예제에서는 명확한 세 가지만 적용했습니다. 남은 여지는 미니 프로젝트에서 여러분이 직접 다룹니다.

- 핵심 교훈: **측정 → 대상 선정 → 타입 교체 → 전후 수치 비교**라는 순서가 최적화의 정석입니다.

즉시 연습 (2종 1세트)

직접 해 보기 — 더 공격적으로 최적화하고 한계까지 밀어붙이기

위 최적화에 session_id 범주형과 Parquet 저장을 더해 **직접 실행하고 실제 출력을 기록**하십시오.

```
opt2 = opt.copy()
opt2["session_id"] = opt2["session_id"].astype("category")
after2 = opt2.memory_usage(deep=True).sum() / 1024 / 1024
print("session_id도 category: %.1f MB" % after2)

# Parquet으로 저장하고 크기 비교
import os
opt2.to_parquet("logs_opt.parquet", engine="pyarrow")
print("최적화 Parquet 크기: %.1f MB"
      % (os.path.getsize("logs_opt.parquet") / 1024 / 1024))
print("원본 CSV 크기: %.1f MB"
      % (os.path.getsize("data/web_logs.csv") / 1024 / 1024))
```

기록:

- [session_id까지 category로 한 후 메모리]: (156.4 MB에서 더 줄었는가)
- [최적화 Parquet 파일 크기 vs 원본 CSV]: (몇 배 작아졌는가)
- [내가 적용한 최적화 목록과 총 절감률]: (전후 MB와 % 정리)

자가체크

- [] 최적화 전후 총 메모리를 실제 수치로 측정해 절감률을 계산했다.
- [] 결측이 있는 정수 열(customer_id)에 nullable Int32를 써야 하는 이유를 적었다.
- [] session_id 범주형이 이득인지 실제 측정으로 판단했다.
- [] 최적화한 데이터를 Parquet으로 저장해 CSV 대비 크기를 비교했다.

11.4 단원 미니 프로젝트: 백만 행 로그 효율적으로 처리하기

이 단원(10단원: 대용량 데이터와 성능)의 개념 c19~c20만으로 완성하는 산출물을 만듭니다. web_logs를 **측정·최적화·체크 집계·Parquet 전환**해 성능 개선을 수치로 보고하는 스크립트 optimize_logs.py와 **성능 노트**입니다. 아래 마일스톤을 **직접 실행하며 실제 출력을 기록**하십시오.

미니 프로젝트 — optimize_logs.py와 성능 노트 (마일스톤 4단계)

마일스톤 1: 측정과 대상 선정 web_logs를 불러 memory_usage(deep=True)로 열별 메모리와 총합을 재고, 각 문자열 열의 nunique()로 카디널리티를 확인해 "category로 바꿀 열"과 "그대로 둘 열"을 근거와 함께 표로 정리하십시오.

- **[총 메모리와 열별 메모리 상위 3개]:** (실제 값)
- **[category 대상 열과 그 근거(고유값 수)]:** (event_type은 4종이라 대상, log_id는 유일이라 제외 등)

마일스톤 2: 최적화와 전후 비교 선정한 열에 다운캐스팅·범주형을 적용하고(결측 있는 정수는 nullable), 최적화 전후 총 메모리와 절감률을 계산하십시오.

- **[최적화 후 총 메모리와 절감률]:** (실제 값, % 포함)

마일스톤 3: 체크 집계 chunksize로 web_logs를 체크 단위로 순회하며 event_type별 건수를 집계하고, 전체 로드 결과와 대조해 일치하는지 검증하십시오.

- **[체크 수와 event_type별 건수]:** (실제 값)
- **[전체 로드와 일치 여부]:** (True/False)

마일스톤 4: Parquet 전환과 성능 노트 최적화한 데이터를 Parquet으로 저장하고, CSV 대비 파일 크기와 read_csv/read_parquet 시간을 재어 비교하십시오. 그리고 이 프로젝트에서 얻은 개선을 **성능 노트**로 정리하십시오(예: "메모리 X% 절감, 파일 Y배 축소, 읽기 Z배 단축").

- **[CSV vs Parquet 크기·읽기 시간]:** (실제 측정값)
- **[성능 노트 요약 3줄]:** (메모리·크기·속도 개선을 수치로)

성능 노트는 캡스톤(14장 M7 대용량 로그 처리)에서 그대로 재사용합니다.

자가체크

- [] optimize_logs.py가 처음부터 끝까지 오류 없이 실행된다(직접 실행해 확인).
- [] 열별 메모리를 측정해 category 대상을 카디널리티 근거로 선정했다.
- [] 최적화 전후 메모리 절감률을 실제 수치로 보고했다.
- [] 체크 집계 결과가 전체 로드와 일치함을 검증했다.
- [] Parquet 전환의 크기·속도 개선을 측정해 성능 노트에 수치로 적었다.

자주 묻는 질문 (FAQ)

Q. 최적화가 항상 필요한가요? 20만 행 order_items도 category로 바꿔야 하나요? 아닙니다. 데이터가 메모리에 여유 있게 들어가고 작업이 충분히 빠르면 최적화는 불필요한 복잡성입니다. 최적화는 **메모리가 실제로 병목일 때**(파일이 크거나, 여러 표를 동시에 다루거나, 병합으로 넓어질 때) 꺼내 쓰는 카드입니다. 이 책에서 web_logs만 최적화하는 이유가 그것입니다. "느려지거나 메모리가 부족할 때 측정부터"가 원칙입니다.

Q. chunksize는 얼마로 잡아야 하나요? 정답은 없고 트레이드오프입니다. 너무 작으면(예: 1,000 행) 청크가 많아져 반복 오버헤드가 크고, 너무 크면 한 청크가 메모리를 압박합니다. 보통 10만 ~ 50만 행 사이에서 시작해, 메모리 여유와 속도를 보며 조정합니다. 이 장에서는 20만 행으로 백만행을 청크 5개로 나눴습니다.

Q. 그냥 항상 Parquet으로 저장하면 안 되나요? 데이터 파이프라인 내부에서는 Parquet이 대체로 유리합니다(작고 빠르고 타입 보존). 다만 CSV는 텍스트라 사람이 열어 보거나 엑셀·다른 도구와 주고받기 쉽다는 장점이 있습니다. **분석 중간 산출물은 Parquet, 최종 공유용은 CSV**로 나눠 쓰는 것이 흔한 절충입니다. Parquet을 열려면 PyArrow 같은 라이브러리가 필요하다는 점도 고려하십시오.

Q. 다운캐스팅으로 값이 깨지지 않을까 걱정됩니다. `pd.to_numeric(s, downcast="integer")`는 값의 실제 범위를 보고 안전한 가장 작은 타입을 고르므로 오버플로 위험이 없습니다. 직접 `astype("int8")` 처럼 타입을 못 박을 때만 범위를 넘길 위험이 있으니, 자동 다운캐스트를 쓰거나 미리 `min()`·`max()`로 범위를 확인하십시오. 실수(float)를 float32로 낮출 때는 정밀도가 약간 줄어드니, 금액처럼 정확도가 중요한 열은 신중히 판단합니다.

이 장에서 우리는 백만 행 로그를 다루며 성능을 배웠습니다. **memory_usage(deep=True)**로 무엇이 무거운지 측정하고, 다운캐스팅과 범주형으로 메모리를 줄이되 카디널리티를 근거로 판단하며, **chunksize로 메모리보다 큰 파일을 나눠 집계하고, Parquet으로 더 작고 빠르게 타입을 보존하는 저장**을 했습니다. 총 221.0 MB의 로그를 156.4 MB로, 51.9 MB CSV를 24.1 MB Parquet으로 줄이고 읽기를 약 2배 빠르게 만든 이 기법들은, 데이터가 커질수록 진가를 발휘합니다. 이것으로 개념 단원(1~11장)을 마칩니다. 다음 12장부터는 지금까지 배운 모든 도구 — 적재·정제·파생·집계·병합·시계열·대용량 — 를 하나로 통합해, 쇼핑물 데이터를 처음부터 끝까지 분석하는 종합 프로젝트를 시작합니다.

종합 프로젝트 (1) 분석: 문제 정의와 데이터 품질 점검

학습 목표

- 드라이빙 퀘스천과 POV(관점 진술)를 바탕으로 분석 목표와 답할 질문(매출 추세·핵심 고객·잘 팔리는 상품)을 요구사항(FR/NFR)으로 정리할 수 있습니다 [→G1]
- 다섯 테이블을 적재해 shape·info·describe로 품질 이슈를 점검하고, 각 요구사항이 어느 단원 개념으로 구현되는지 매핑할 수 있습니다 [→G2]
- 분석 범위(In/Out)를 정하고 재현성(시드 고정 데이터 + 순서대로 실행) 기준을 명시할 수 있습니다 [→G10]

여기서부터 네 개 장(ch_12~ch_15)은 **종합 프로젝트**입니다. 1단원부터 10단원까지 익힌 판다스 도구(적재·구조 파악·선택·필터·정제·파생·집계·병합·시계열·대용량 처리)를 **새 개념 없이** 하나의 실제 분석으로 통합합니다. 이 장(분석)에서는 코드를 많이 쓰기 전에 먼저 "무엇을, 왜, 어디까지 분석할 것인가"를 정하고, 다섯 테이블을 불러와 손볼 곳을 점검표로 정리합니다. 설계(ch_13)·구현(ch_14)·발표와 성찰(ch_15)이 그 위에 차례로 쌓입니다.

이 프로젝트가 다루는 데이터는 여러분이 1단원에서 `generate_shop_data.py`로 만든 바로 그 쇼핑물 데이터셋입니다. `data/` 폴더에 다섯 개 CSV(customers·products·orders·order_items·web_logs)가 있고, 2024년 1월 1일부터 6월 30일까지 6개월치입니다. 실습·재현은 Windows 11 기준으로 안내하지만 macOS·Linux에서도 동일하게 동작합니다.

이 장은 새 문법을 배우지 않습니다

여기서 쓰는 도구는 모두 앞 단원에서 이미 배웠습니다. `read_csv` (3단원)·`info`·`describe`·`isna()`·`sum()` (4단원)·`duplicated`·`to_numeric` (5단원)이 다시 등장할 뿐, 새 함수는 없습니다. 이 장의 목표는 **분석을 시작하기 전에 목표와 데이터 상태를 문서로 확정**하는 습관을 기르는 것입니다. 좋은 분석은 화려한 결과가 아니라 정확한 문제 정의에서 시작합니다.

12.1 드라이빙 퀘스천과 POV: 무엇을 밝힐 것인가

분석은 "이 데이터로 무엇을 답하고 싶은가"라는 하나의 질문에서 출발합니다. 이 질문을 **드라이빙 퀘스천(driving question)** 이라고 부르며, 프로젝트 내내 방향을 잡아 주는 나침반 역할을 합니다.

드라이빙 퀘스천(driving question)

How might we 흠어지고 지저분한 쇼핑물 6개월치 데이터를 정제·통합해, 매출 추세·핵심 고객·잘 팔리는 상품을 근거와 함께 밝히고 데이터 기반 개선안을 재현 가능한 분석으로 전달할 수 있을까?

이 질문 하나에 이 프로젝트의 전부가 들어 있습니다. **정제·통합**(과정)으로 시작해 **매출 추세·핵심 고객·잘 팔리는 상품**(밝힐 것)을 지나 **개선안**과 **재현 가능한 분석**(전달물)으로 끝납니다.

드라이빙 퀘스천을 세우려면 먼저 "누가, 무엇을 필요로 하는가"를 분명히 해야 합니다. 이를 **POV(Point of View, 관점 진술)** 로 정리합니다. POV는 사용자(user)·필요(need)·통찰(insight) 세 조각으로 이루어집니다.

POV 조각	내용
사용자 (user)	쇼핑물 데이터 분석을 처음 맡은 데이터 분석 입문자
필요 (need)	6개월치 흠어지고 지저분한 주문·고객·상품·행동 데이터를 신뢰할 수 있는 하나의 분석으로 통합해, 매출이 어디로 가는지·누가 핵심 고객인지·무엇이 잘 팔리는지를 근거와 함께 설명하고 개선안을 내야 한다
통찰 (insight)	데이터가 여러 테이블에 나뉘고 결측·이상값·표기 불일치로 오염돼 있어, 정제와 병합을 제대로 하지 않으면 집계 숫자 자체가 틀린다 — 분석의 신뢰는 화려한 결과가 아니라 정제·통합·검증에서 나온다

이 통찰이 핵심입니다. 오염된 데이터를 그대로 집계하면 매출 합계도, 카테고리 순위도, 핵심 고객 명단도 틀립니다. 그래서 이 프로젝트는 화려한 그래프보다 **정제→통합→검증**의 신뢰성에 무게를 둡니다.

드라이빙 퀘스천을 실제로 답할 수 있는 **세 가지 구체적 질문**으로 쪼갭니다.

- Q1. 매출은 6개월 동안 어떻게 변했는가? (월별 추세·성장률·주간 패턴)
- Q2. 누가 핵심 고객인가? (고객별 구매액 등급, 상위 고객 집중도)

- Q3. 무엇이 잘 팔리는가? (카테고리·채널별 매출과 상위 상품군)

12.2 요구사항(FR/NFR)과 단위 개념 매핑

분석 프로젝트도 소프트웨어처럼 **요구사항**으로 정리하면 빠뜨림 없이 진행할 수 있습니다. 요구사항은 두 종류로 나뉩니다. **기능 요구사항(FR, Functional Requirement)**은 "무엇을 해야 하는가", **비기능 요구사항(NFR, Non-Functional Requirement)**은 "얼마나 잘.어떤 조건으로 해야 하는가"입니다.

기능 요구사항(FR)과 사용하는 단위 개념

각 기능 요구사항이 **어느 단위 개념으로 구현되는지** 함께 적습니다. 이렇게 하면 커리큘럼에서 배운 도구가 실제 프로젝트의 어느 자리에 쓰이는지 한눈에 보이고, 구현(ch_14)의 마일스톤과도 곧바로 이어집니다.

ID	기능 요구사항(FR)	사용하는 단위 개념	평가 지점
FR1	다섯 CSV를 알맞은 dtype·날짜 파싱으로 적재하고 shape·info로 개요를 파악한다	u1(적재)·u2(구조·훑어보기)	ch_12 이장
FR2	특정 기간·채널·상태 주문 세그먼트를 불리언·query로 추출·검증한다	u3(선택)·u4(불리언 필터)	ch_14 M1
FR3	결측·중복·이상값·타입·문자열을 문서화된 규칙으로 정제한다	u5(정제)	ch_14 M2
FR4	주문 금액·연령대·가격대 등 분석 축을 파생·구간화한다	u6(파생·구간화)	ch_14 M3
FR5	카테고리·채널·고객 등급별 핵심 지표를 groupby·피벗으로 집계한다	u7(그룹화·집계)	ch_14 M4
FR6	네 테이블을 키·조인 방식으로 병합해 통합 테이블을 만든다	u8(병합·재구조화)	ch_14 M5
FR7	월별·주별 매출 추세와 이동평균·성장률을 시계열로 분석한다	u9(시계열)	ch_14 M6
FR8	백만 행 행동 로그를 최적화·청크·Parquet으로 효율 처리한다	u10(대용량·성능)	ch_14 M7
FR9	근거 있는 발견 3개 이상과 개선 제안을 요약 리포트로 전달한다	u7·u9 통합	ch_15

사용자 스토리와 유스케이스

요구사항을 "누가 무엇을 왜"의 문장으로 풀면 **사용자 스토리(user story)** 가 됩니다. 이 리포트를 실제로 쓸 사람의 입장에서 적으면 분석이 걸돌지 않습니다.

- 유스케이스 1(운영 담당자): "나는 **월별 매출이 어디로 가는지** 알고 싶다. 다음 분기 재고·인력 계획을 세워야 하기 때문이다." → FR7(시계열)로 답한다.
- 유스케이스 2(마케팅 담당자): "나는 **누가 우리 핵심 고객인지** 알고 싶다. 이탈 방지 프로그램의 대상을 정해야 하기 때문이다." → FR5(고객 등급 집계)로 답한다.

- 유스케이스 3(상품 기획자): "나는 **무엇이 잘 팔리는지** 카테고리·채널별로 알고 싶다. 프로모션과 매입을 조정해야 하기 때문이다." → FR5(카테고리·채널 집계)로 답한다.
- 유스케이스 4(데이터 담당자): "나는 이 분석을 **누가 다시 실행해도 같은 결과가 나오길** 원한다. 숫자를 믿고 의사결정에 쓰려면 재현이 돼야 하기 때문이다." → NFR1(재현성)으로 보장한다.

각 유스케이스가 앞의 FR/NFR로 정확히 답해지는지 확인하는 것이, 요구사항이 빠짐없이 세워졌다는 증거입니다.

비기능 요구사항(NFR)

ID	비기능 요구사항(NFR)	기준
NFR1	재현성	처음부터 다시 실행하면 정제 후 행수·집계 총합·월별 매출이 완전히 동일해야 한다(시드 42 고정 데이터 + 난수 없는 결정론적 계산)
NFR2	검증 가능성	각 단계 결과를 원본과 대조한다(집계 총합=원본 합, 병합 전후 행수 보존, 특정 달 집계=부분집합 합)
NFR3	정직한 오염 처리	정제 규칙을 문서화하고, 데이터를 임의로 바꾸지 않는다(예: 결측 성별은 억지로 채우지 않고 무응답으로 남긴다)
NFR4	메모리 효율	백만 행 로그는 최적화 전후 메모리·시간을 수치로 비교해 개선을 보인다

이 요구사항 표가 프로젝트 전체의 뼈대입니다. 구현 장(ch_14)의 마일스톤 M1~M7은 FR2~FR8에 정확히 대응하고, 발표 장(ch_15)은 FR9와 NFR1~NFR2를 검증합니다.

12.3 데이터 적재와 품질 점검표

이제 실제로 다섯 테이블을 불러와 어디를 손봐야 하는지 점검합니다. 이 절의 코드와 출력은 참조 구현 `capstone/01_load_overview.py` 에서 가져왔고, 아래 출력은 직접 실행해 재확인한 실제 결과입니다.

worked example: 다섯 테이블 적재

각 열의 성격에 맞게 `dtype` 과 `parse_dates` 를 지정해 불러옵니다. 이것이 3단원에서 배운 "불러오기를 잘 하는 것이 분석의 첫 품질 관문"입니다.

```
import pandas as pd

customers = pd.read_csv("data/customers.csv", dtype={"customer_id": "int64"})
products = pd.read_csv("data/products.csv")
orders = pd.read_csv(
    "data/orders.csv",
    dtype={"order_id": "int64", "customer_id": "int64"},
    parse_dates=["order_datetime"],
)
order_items = pd.read_csv("data/order_items.csv")
web_logs = pd.read_csv(
    "data/web_logs.csv",
    dtype={"customer_id": "Int64", "product_id": "Int64"},
    parse_dates=["event_time"],
)

for name, df in [
    ("customers", customers), ("products", products), ("orders", orders),
    ("order_items", order_items), ("web_logs", web_logs),
]:
    mem_mb = df.memory_usage(deep=True).sum() / (1024 ** 2)
    print(f"[{name}] shape={df.shape} 메모리={mem_mb:.2f} MB")
```

- `orders` 의 `order_datetime` 은 `parse_dates` 로 지정해 문자열이 아니라 `datetime64` 로 읽습니다. 이렇게 해야 나중에 월·요일 추출과 리샘플링이 됩니다.
- `web_logs` 의 `customer_id` · `product_id` 는 익명·비상품 이벤트에서 비어 있으므로, 결측을 담을 수 있는 널 허용 정수형 `Int64` (대문자 I)로 지정합니다.
- `products` 의 `price` 는 일부러 지정하지 않습니다 — 문자열 오염이 섞여 있어 판다스가 `object` 로 읽는 것을 그대로 확인하기 위해서입니다.
- `memory_usage(deep=True)` 는 문자열의 실제 크기까지 포함해 메모리를 재므로, 대용량 로그의 부담을 정확히 보여 줍니다.

실행하면 다음과 같이 나옵니다.

```
[customers] shape=(5000, 7) 메모리=2.18 MB
[products] shape=(500, 5) 메모리=0.13 MB
[orders] shape=(200000, 5) 메모리=28.44 MB
[order_items] shape=(500000, 6) 메모리=22.89 MB
[web_logs] shape=(1000000, 6) 메모리=158.04 MB
```

행수가 데이터셋 명세와 정확히 일치합니다(고객 5,000 · 상품 500 · 주문 200,000 · 주문 상세 500,000 · 로그 1,000,000). 특히 `web_logs` 가 홀로 158MB로, 다른 네 테이블을 합친 것보다 큼니

다 — 이것이 8단원(대용량 처리)이 필요한 이유입니다.

품질 점검표: 정제가 필요한 곳 8곳

`info · describe · isna().sum() · duplicated` 로 각 테이블의 품질 이슈를 진단합니다. 참조 구현이 정리한 점검표를 실제로 실행하면 이슈 8건이 근거 수치와 함께 나옵니다.

- 1) customers.gender: 결측률 6.1% + 'M'/'F'·'남'/'여' 표기 혼용(4종) -> 정제 전략: 표기 통일 후
- 2) customers.birth_date: 결측률 7.7% -> 정제 전략: datetime 변환(NaT), 나이 파생 시 결측 전파
- 3) customers.city: 공백 오염으로 표기가 30종으로 갈림 (strip 후 10종) -> 정제 전략: str.strip
- 4) customers: 완전 중복 행 50행 -> 정제 전략: drop_duplicates
- 5) products.price: dtype=object(문자열 오염으로 object) - 숫자 변환 실패 46건, 0/음수 이상값 8
- 6) orders.order_datetime: 결측률 1.0%, 미래(2025+) 이상값 40건 -> 정제 전략: 둘 다 NaT로 통일
- 7) orders.customer_id: 고아 키(customers에 없는 값) 비율 0.50% -> 정제 전략: 삭제하지 않고 병합
- 8) order_items: quantity 0/음수 이상값 500건, unit_price 결측률 3.02% -> 정제 전략: quantity는

점검표를 읽는 요령은 **규모와 성격을 함께 보는 것**입니다. 예를 들어 customers.city는 공백 오염만으로 표기가 30종으로 갈렸는데, `str.strip` 한 번이면 10종으로 줄어듭니다(정제 효과가 큰 저비용 작업). 반대로 orders.customer_id 고아 키(0.50%)는 지우면 매출이 왜곡되므로 정제가 아니라 병합 단계에서 다뤄야 하는 다른 성격의 문제입니다. 같은 "결측·오염"이라도 무엇을 어떻게 처리할지는 열의 의미에 따라 갈립니다.

이 점검표를 요구사항·단원 개념과 묶어 정리하면 다음과 같습니다. **각 품질 이슈가 어느 단원 개념으로 고쳐지는지** 매핑하는 것이 이 절의 핵심입니다.

품질 이슈	규모(실측)	정제 전략	사용하는 단위 개념
gender 표기 혼용·결측	결측 6.1%, 4종	표기 통일(map), 결측은 무응답으로 남김	c10(문자열)·c08(결측)
birth_date 결측	7.7%	datetime 변환(NaT), 나이 파생 시 전파	c08(결측)·c17(날짜)
city 공백 오염	30종→10종	str.strip	c10(문자열)
customers 완전 중복	50행	drop_duplicates	c09(중복)
price 문자열 오염·이상값	변환 실패 46, 이상값 8	문자열 제거 후 to_numeric, 이상값 제거	c09(타입·이상값)
order_datetime 결측·미래값	결측 1.0%, 미래 40	둘 다 NaT, 시계열에서 제외	c08·c09·c17
customer_id 고아 키	0.50%	병합 단계에서 left join 점검	c15(병합)
quantity·unit_price	이상값 500, 결측 3.02%	clip·상품 정가로 채움	c08·c09

12.4 범위(In/Out)와 재현성 기준

마지막으로 무엇을 하고 무엇을 하지 않을지를 못 박습니다. 범위(scope)를 정하지 않으면 분석이 끝없이 늘어나거나, 반대로 중요한 것을 빠뜨립니다.

범위 안(scope In)	범위 밖(scope Out)
다섯 CSV 적재·정제·파생·병합·집계·시계열·대용량 처리	머신러닝 예측·추천 모델(이 책 범위 밖)
매출 추세·고객 등급·상품/카테고리 분석과 개선 제안	실시간 파이프라인·데이터베이스 적재·대시보드 웹앱 구축
분석의 재현성(시드 고정 데이터 + 순서대로 실행되는 스크립트)	통계적 가설검정·유의성 분석(도구 실습에 집중)

재현성(reproducibility) 기준(NFR1)은 이 프로젝트의 신뢰성을 떠받치는 약속입니다.

- 데이터는 `generate_shop_data.py` 가 `numpy.random.default_rng(42) · random.seed(42)` 로 난수를 고정해 만들므로, 누가 언제 다시 만들어도 바이트 단위로 같은 CSV가 나옵니다.
- 분석 스크립트(정제·집계·리샘플링)는 난수를 쓰지 않는 결정론적 계산만 하므로, 처음부터 다시 실행하면 정제 후 행수·집계 총합·월별 매출이 정확히 같습니다.
- 스크립트를 **정해진 순서로** 실행하고(적재→정제→파생→집계→통합→시계열→대용량→리포트), 각 단계가 다음 단계의 입력을 산출물로 남깁니다.

이 재현성은 ch_15에서 "처음부터 다시 실행 → 같은 결과"로 실제 검증합니다.

본문 출력은 모두 직접 재실행해 확인한 값입니다

이 프로젝트의 모든 수치(품질 이슈 규모·정제 후 행수·매출 합계 등)는 참조 구현을 실제로 실행해 화면에 찍힌 값을 그대로 옮긴 것입니다. 분석 리포트에서 출력을 손으로 지어내면(예: 실행하지 않고 그럴듯한 숫자를 적으면) 재현성이 깨지고 신뢰가 무너집니다. 여러분도 실습에서 **직접 실행한 실제 출력만** 기록하세요 — 이것이 데이터 분석의 기본 윤리입니다.

분석 단계 자주 묻는 질문

Q. 왜 정제하기 전에 품질 점검부터 하나요? 무엇을 고쳐야 할지 모른 채 정제하면 필요한 데이터를 지우거나 이상값을 놓칩니다. 점검표(4단원의 훑어보기)가 "어디에·무엇이·얼마나" 문제인지 목록을 만들어야, 정제(5단원)에서 규칙을 정해 처리할 수 있습니다. 정찰 없이 작전을 짜지 않는 것과 같습니다.

Q. 고아 키(customers에 없는 customer_id)를 지금 지우면 안 되나요? 지우지 않습니다. 주문 자체는 실재하는 사실이라, 지우면 매출 총합이 왜곡됩니다. 대신 병합 단계(ch_14 M5)에서 left 조인으로 남겨 "얼마가 왜 매칭되지 않았는지"를 드러냅니다. 이 결정의 근거는 ch_13 ADR-2에 기록합니다.

Q. products.price를 왜 처음부터 숫자로 안 읽나요? 일부러 그대로 둡니다. price에는 '12000 원', '1,200' 같은 문자열이 섞여 있어 판다스가 object로 읽습니다. 이 오염을 눈으로 확인한 뒤 5단원에서 배운 `to_numeric` 으로 정제하는 것이 학습 목표입니다.

실습 과제 (2종 1세트)

과제 A — 분석 기획서 직접 작성

직접 작성: 나의 분석 기획서

앞의 12.1~12.2를 참고해, 여러분 자신의 언어로 분석 기획서를 **직접 작성**합니다. 아래 라벨에 실제로 채워 넣으세요(추측으로 빈칸을 메우는 것이 아니라, 스스로 판단해 문장으로 씁니다).

- 답할 질문 3개:** 이 데이터로 답하고 싶은 질문을 Q1·Q2·Q3로 각각 한 문장씩 적습니다(본문 Q1~Q3를 그대로 베끼지 말고, 하나는 여러분이 궁금한 새 질문으로 바꿔 보세요. 예: "채널별로 취소율이 다른가?").
- [내가 정한 Q1]
- [내가 정한 Q2]
- [내가 정한 Q3]
- 기능 요구사항 표 확장:** 본문 FR1~FR9 표에서 여러분의 새 질문을 답하는 데 필요한 FR을 1개 더 추가하고, 그 FR이 어느 단위 개념을 쓰는지 적습니다.
- [추가한 FR과 사용 단위 개념]
- 범위 결정:** 여러분이 정한 새 질문이 범위 안인지 밖인지, 그 근거와 함께 한 문장으로 적습니다.
- [범위 판단과 근거]

산출물: 위 세 항목을 채운 짧은 기획 노트(마크다운 또는 주석).

과제 B — 데이터 적재와 품질 점검 직접 실행

직접 실행·기록: 나의 품질 점검표

`data/` 폴더의 다섯 CSV를 직접 불러와 품질을 점검하고, **화면에 실제로 찍힌 출력**을 그대로 기록합니다.

- 12.3의 적재 코드를 실행하고, 다섯 테이블의 `shape` 와 메모리 출력을 그대로 붙여 넣습니다.

2. [내 실행: 다섯 테이블 shape·메모리 실제 출력]
3. 다음을 각각 실행해 실제 출력을 기록합니다(무엇을 실행할지 명령까지 구체적으로 제시합니다).
4. `customers["gender"].value_counts(dropna=False)` — gender 표기 종류와 결측 수
5. `pd.to_numeric(products["price"], errors="coerce").isna().sum()` — price 숫자 변환 실패 건수
6. `orders["order_datetime"].isna().mean()` — 주문 일시 결측률
7. [각 명령의 실제 출력]
8. 위 출력을 근거로, **여러분이 직접 발견한** 품질 이슈를 5개 이상 "어느 열·어떤 값·정제 전략" 형식으로 적습니다(본문 8건 표를 참고하되, 규모 수치는 여러분 실행 결과로 채웁니다).
9. [내가 작성한 품질 점검표 5개 이상]

자가 체크

스스로 점검(손으로 했는지 확인)

- [] 드라이빙 퀘스천과 POV를 읽고, 내가 답할 질문 3개를 **내 손으로** 적었는가?
- [] 기능 요구사항마다 사용하는 단위 개념을 매핑했는가?
- [] 다섯 테이블을 직접 `read_csv` 로 불러와 shape를 확인했는가(머리로가 아니라 실행해서)?
- [] `web_logs` 가 100만 행·약 158MB로 다른 테이블보다 훨씬 큼을 실측으로 확인했는가?
- [] 품질 이슈를 5개 이상, 각각 근거 수치와 함께 적었는가?
- [] 범위 In/Out과 재현성 기준(시드 고정+순서 실행)을 문서로 명시했는가?

다음 장(ch_13)에서는 이 요구사항과 품질 점검표를 바탕으로, 정제→파생→병합→집계→시계열→대용량으로 이어지는 **분석 파이프라인을 설계**하고 병합 키·조인 방식을 ADR로 기록합니다.

종합 프로젝트 (2) 설계: 분석 파이프라인과 통합 스키마

학습 목표

- 정제→파생→병합→집계→시계열→대용량으로 이어지는 분석 파이프라인 단계와 각 단계 산출물을 설계할 수 있습니다 [→G4]
- 네 테이블 병합의 키·조인 방식과 통합 테이블 스키마를 ADR 5필드로 결정·기록할 수 있습니다 [→G7]
- 정제 규칙과 파생·집계 정의를 문서화해 재현 가능한 스크립트 구조(모듈·실행 순서)를 설계할 수 있습니다 [→G10]

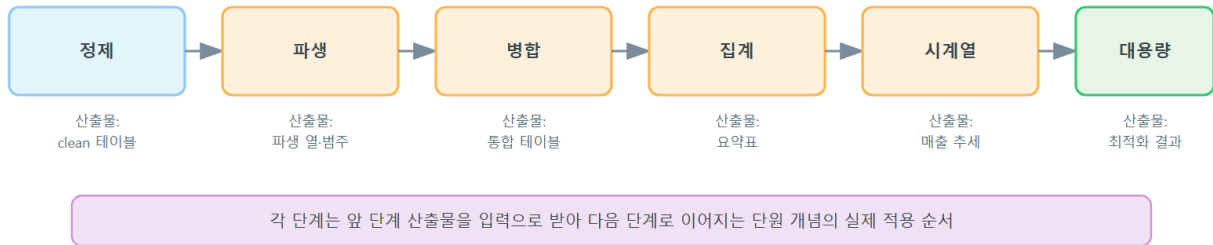
ch_12에서 "무엇을·왜·어디까지" 분석할지 정하고 품질 이슈 8곳을 점검했습니다. 이제 그것을 어떻게 만들지, 즉 **설계(design)** 를 합니다. 이 장에서도 새 문법은 없습니다. 대신 앞 단원에서 배운 도구들을 어떤 순서로 이어 붙이고, 병합 같은 중요한 결정을 왜 그렇게 했는지 기록하는 법을 다룹니다.

설계를 문서로 남기는 이유는 재현성(NFR1) 때문입니다. 반년 뒤의 내가, 또는 다른 분석가가 이 스크립트를 보고 "왜 고아 키를 지우지 않고 남겼지?"를 물었을 때, 코드만 봐서는 알 수 없습니다. **결정의 근거를 남긴 문서**가 있어야 분석을 믿고 재현할 수 있습니다.

13.1 분석 파이프라인 단계 설계

파이프라인(pipeline)은 데이터가 한 단계의 출력이 다음 단계의 입력이 되도록 이어진 처리 흐름입니다. 이 프로젝트의 파이프라인은 여섯 단계로 이어집니다.

분석 파이프라인 단계 설계: 정제 → 파생 → 병합 → 집계 → 시계열 → 대용량



각 단계가 무엇을 받아 무엇을 내놓는지, 그리고 어느 단원 개념을 쓰는지 정리하면 다음과 같습니다.

단계	입력	산출물	사용하는 단원 개념
1. 적재	5개 원본 CSV	다섯 데이터프레임	u1(적재)·u2(구조)
2. 정제	원본 4개 테이블	*_clean.parquet (정제된 4개)	u5(결측·중복·이상값·타입·문자열)
3. 파생	정제된 4개	*_derived.parquet (amount·연령대·가격대·라벨 추가)	u6(파생·구간화)
4. 집계	파생된 테이블	카테고리·채널·등급 요약표(csv)	u7(그룹화·피벗)
5. 병합	파생된 4개	integrated.parquet (통합 테이블)	u8(병합·재구조화)
6. 시계열	통합 테이블	월별 매출·이동평균·성장률	u9(시계열)
(병렬) 대용량	web_logs 원본	최적화·체크·Parquet 결과	u10(대용량·성능)

여기서 설계상 중요한 판단이 하나 있습니다. **집계(4단계)를 병합(5단계) 앞에 둘 것인가, 뒤에 둘 것인가**입니다. 이 프로젝트에서는 파생 직후 필요한 열만 임시로 붙여 카테고리·채널 집계를 먼저 하고(4단계), 그 다음 전체를 하나로 합친 통합 테이블(5단계)을 만들어 시계열 분석의 기반으로 씁니다. 이렇게 나누면 집계는 가벼운 임시 결합으로 빠르게 확인하고, 무거운 통합 테이블은 시계열처럼 여러 열이 함께 필요한 분석에만 쓰게 됩니다.

각 단계가 남기는 산출물을 파일 단위로 확정해 두면, 어느 단계가 어떤 입력을 읽는지 분명해 집니다.

산출물 파일	만드는 단계	읽는 단계
customers_clean.parquet 등 4개	2. 정제	3. 파생
*_derived.parquet 4개	3. 파생	4. 집계·5. 병합
category_summary.csv 등 요약표	4. 집계	8. 리포트
integrated.parquet	5. 병합	6. 시계열
monthly_revenue.csv	6. 시계열	8. 리포트
metrics.json (누적)	1~7 공통	8. 리포트

이 표가 파이프라인의 의존 관계도입니다. 산출물이 곧 단계 간 계약이므로, 앞 단계 산출물의 열·타입이 바뀌면 뒤 단계가 그것을 읽는지 확인해야 합니다.

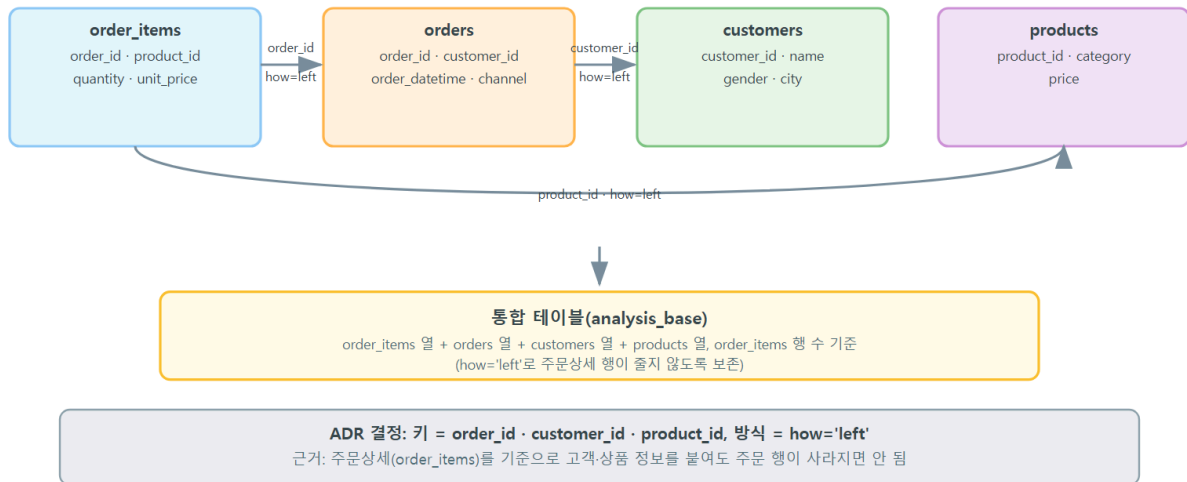
왜 중간 산출물을 parquet으로 저장하는가

각 단계가 결과를 artifacts/ 폴더에 Parquet(8단원)으로 남기면, 뒤 단계는 무거운 원본을 다시 정제하지 않고 저장된 결과만 읽습니다. Parquet은 dtype을 보존하므로(CSV와 달리 날짜·범주형이 문자열로 풀리지 않음), 단계 사이에 타입이 어긋날 걱정도 없습니다. 이것이 파이프라인을 단계별로 나눠 설계하는 실질적 이득입니다.

13.2 통합 스키마와 병합 키·조인 결정(ADR)

파이프라인의 심장은 5단계 병합입니다. 네 테이블(order_items·orders·customers·products)을 하나로 합치는데, 여기서 **키를 무엇으로·어떤 방식으로 붙일지**를 잘못 정하면 행이 불어나거나(중복 폭발) 정보가 새어 나갑니다(고아 키 누락). 이 중요한 결정을 **ADR(Architecture Decision Record, 설계 결정 기록)**로 남깁니다.

통합 스키마 설계: 병합 키·조인 방식 결정(ADR)



ADR은 결정 하나를 다섯 필드로 적는 형식입니다. 상황(Context)·결정(Decision)·대안(Alternatives)·근거(Rationale)·결과(Consequences)입니다.

ADR-1: 통합의 기준 테이블과 조인 방향

ADR-1 — order_items를 왼쪽 기준으로 삼아 left 병합한다

- **상황(Context):** 분석의 최소 단위(결, grain)는 "주문 상세 1줄"이다. 매출(amount)은 order_items에서 나오고, 카테고리는 products에, 채널은 orders에, 고객 속성은 customers에 있다. 이 넷을 하나로 합쳐야 통합 분석이 가능하다.
- **결정(Decision):** order_items (499,880행)를 **왼쪽 기준**으로 놓고, orders → customers → products를 차례로 how="left"로 붙인다. 병합 키는 각각 order_id·customer_id·product_id다.
- **대안(Alternatives):** (a) inner 병합 — 키가 매칭되는 행만 남긴다. (b) orders를 기준으로 삼는다.
- **근거(Rationale):** order_items 한 줄도 잃지 않아야 매출 총합이 보존된다. inner 병합은 고아 키(customers에 없는 customer_id) 행을 조용히 버려 매출이 실제보다 작게 집계된다. left 병합은 매칭 실패를 결측(NaN)으로 남겨 **버리지 않고 눈에 보이게** 한다.
- **결과(Consequences):** 오른쪽 키(order_id·customer_id·product_id)가 각 테이블에서 **유일**해야 행수가 보존된다(유일하지 않으면 행이 불어난다). 그래서 병합 전에 is_unique로 키 유일성을 확인하고, 병합 후 행수가 499,880으로 유지되는지 대조한다. 고아 키 행은 무결성 점검 대상으로 남는다.

ADR-2: 고아 키를 어떻게 다룰 것인가

ADR-2 — 고아 키는 삭제하지 않고 결측으로 드러낸다

- **상황(Context):** orders.customer_id의 약 0.50%는 customers에 없는 값(고아 키)이다. products도 정제 단계에서 이상값 상품 11개를 제외해, 그 상품을 참조하는 order_items 라인은 카테고리를 매칭하지 못한다.
- **결정(Decision):** 고아 키 행을 **미리 지우지 않고** left 병합으로 남겨, 매칭 실패를 결측으로 표시한다. 집계 단계에서 카테고리가 결측인 금액(고아 금액)을 별도로 집계해 총합 대조에 포함한다.
- **대안(Alternatives):** 정제 단계에서 고아 키 행을 dropna로 제거한다.
- **근거(Rationale):** 주문 자체(고객이 무엇을 언제 샀는지)는 실재하는 사실이다. 고아 키라고 지우면 매출 총합이 왜곡되고, "얼마가 왜 빠졌는지"를 추적할 수 없다. 결측으로 남기면 집계 합계 + 고아 금액 = 원본 합계로 **한 푼도 새지 않았음을 검증**할 수 있다.
- **결과(Consequences):** 카테고리별 매출 합계는 원본보다 고아 금액만큼 작지만, 그 차액을 명시적으로 보고한다(ch_14 M4에서 고아 금액 약 13.1억 원, 7,318건으로 확인).

ADR-3: 집계와 병합의 순서

ADR-3 — 카테고리·채널 집계는 통합 테이블 완성 전에 임시 결합으로 먼저 한다

- **상황(Context):** 카테고리별 매출(M4)에는 order_items의 amount와 products의 category 만 있으면 됩니다. 채널별 매출에는 orders의 channel만 더 필요합니다. 전체 통합 테이블 (M5)은 고객 속성까지 모두 붙인 무거운 표입니다.
- **결정(Decision):** M4 집계는 필요한 열만 임시로 merge 해 곧바로 groupby합니다(예: order_items + products[["product_id","category"]]). 무거운 통합 테이블은 여러 열이 함께 필요한 M5·M6에서만 만듭니다.
- **대안(Alternatives):** 통합 테이블을 먼저 완성하고 그 위에서 모든 집계를 수행한다.
- **근거(Rationale):** 집계마다 필요한 열이 다른데 매번 전체 통합 테이블을 훑으면 메모리·시간이 낭비됩니다. 가벼운 임시 결합은 집계를 빠르게 확인시키고, 무거운 통합은 시계열 처럼 정말 필요한 곳에만 씁니다.
- **결과(Consequences):** 집계 검증 시 카테고리가 결측인 금액(고아 금액)을 따로 집계해 집계 합 + 고아 금액 = 원본 합으로 대조해야 합니다(M4-5). 집계와 통합에서 amount 총합이 같은 원본에서 나오므로 두 경로가 어긋나지 않습니다.

ADR-4: 대용량 로그의 처리 위치

ADR-4 — web_logs는 통합 파이프라인에 넣지 않고 별도 단계로 처리한다

- **상황(Context):** web_logs는 100만 행·약 158MB로, 다른 네 테이블을 합친 것보다 큼니다. 매출 분석의 핵심 테이블(order_items·orders·customers·products)과 결(grain)도 다릅니다 (행동 이벤트 단위).
- **결정(Decision):** web_logs는 매출 통합 테이블에 병합하지 않고, 별도 스크립트 (07_biglog)에서 메모리 최적화·체크 집계·Parquet 비교만 수행합니다.
- **대안(Alternatives):** web_logs를 customer_id로 통합 테이블에 붙여 행동까지 한 표로 만 든다.
- **근거(Rationale):** 결이 다른 대용량 테이블을 억지로 붙이면 행이 폭발하거나 결측이 넘칩 니다. 이 프로젝트의 드라이빙 퀘스천은 매출 중심이라, 로그는 "대용량을 효율적으로 다 루는 역량(NFR4)"을 보이는 별도 목표로 둡니다.
- **결과(Consequences):** 로그 기반 퍼널 전환 분석은 이번 범위 밖(ch_12 scope Out)으로 남고, 다음 단계 확장 과제가 됩니다(ch_15 성찰).

통합 테이블 스키마

앞의 ADR들을 반영한 통합 테이블(integrated.parquet)의 열 구성은 다음과 같습니다. order_items의 원본 열에 orders·customers·products에서 필요한 열만 골라 붙입니다(모든 열을 다 붙이면 테이블이 불필요하게 커집니다).

원천 테이블	통합 테이블에 포함할 열
order_items(기 준)	order_item_id · order_id · product_id · quantity · unit_price · discount · amount
orders	customer_id · order_datetime · channel · status_label
customers	age_band · gender_label · city
products	category · price_band · product_name

13.3 정제·파생·집계 규칙 문서화

재현 가능한 분석의 절반은 규칙을 코드 밖 문서로도 남기는 것입니다. 코드는 "무엇을 했는 지"는 보여 주지만 "왜 그렇게 정했는지"는 잘 보여 주지 못합니다. ch_12 품질 점검표의 정제 전략을 규칙표로 확정합니다.

열	문제	정제 규칙	근거
customers.gender	M/F/남/여 혼용 + 결측	map으로 M/F 통일, 결측 은 무응답으로 남김	억지로 채우면 성 비가 왜곡됨(NFR3)
customers.city	앞뒤 공백 오 염	str.strip	도시명 자체는 오 염 없음
customers 중복	완전 중복 행 50건	drop_duplicates + customer_id 유일화	키가 유일해야 병 합이 안전(ADR-1)
products.price	문자열 ('원', ',') + 0/ 음수	문자열 제거 후 to_numeric, 이상값 행 제 거	상품 카탈로그가 작아 소수 손실 영 향 미미
orders.order_datetime	결측 1% + 미래(2025+)	둘 다 NaT로 통일, 시계열 에서 제외	주문 자체는 유효, 시각만 신뢰 불가
order_items.quantity	0/음수 이상 값	clip(lower=1)	주문은 실재하므로 삭제 대신 보정
order_items.unit_price	결측 약 3%	정제된 상품 정가로 채움 (없으면 중앙값)	단가는 상품에서 복원 가능

파생 규칙도 함께 정의합니다.

- `amount = unit_price * quantity * (1 - discount)` — 주문 금액(벡터 연산, c11)
- `age_band` — 나이를 `cut([0,20,30,40,50,60,130])` 으로 연령대 구간화(c12)
- `price_band` — 상품 정가를 `qcut(q=4)` 로 4분위 가격대(c12)
- 고객 등급 — 고객별 총 구매액을 `qcut(q=4)` 로 브론즈·실버·골드·VIP(c12-c13)

13.4 재현 가능한 스크립트 구조와 실행 순서

파이프라인을 여러 스크립트로 나누고 순서대로 실행하도록 설계합니다. 한 파일에 다 넣으면 한 곳만 고쳐도 전체를 다시 돌려야 하지만, 단계별로 나누면 바뀐 단계부터 다시 실행할 수 있습니다.

실행 순서	스크립트	하는 일	산출물
1	01_load_overview	적재 + 품질 점검표	metrics.json(진단)
2	02_clean	M1 세그먼트 + M2 정제	*_clean.parquet
3	03_derive	M3 파생·구간화	*_derived.parquet
4	04_aggregate	M4 핵심 지표 집계	요약표 csv
5	05_integrate	M5 통합 테이블	integrated.parquet
6	06_timeseries	M6 매출 추세	monthly_revenue.csv
7	07_biglog	M7 대용량 로그	metrics.json(성능)
8	08_report	발견 리포트	최종 리포트

각 스크립트는 공통 도우미 모듈에서 경로 계산과 지표 저장을 재사용하도록 설계합니다. 아래는 그 도우미의 핵심 골격입니다(참조 구현 `_common.py`).

```
from pathlib import Path
import json

# 스크립트 위치 기준 상대 경로 — 어느 작업 폴더에서 실행해도 동작한다
BASE_DIR = Path(__file__).resolve().parent
DATA_DIR = BASE_DIR.parent / "data"
ARTIFACTS_DIR = BASE_DIR / "artifacts"
ARTIFACTS_DIR.mkdir(parents=True, exist_ok=True)

def require_artifact(path, producer_script):
    """이전 단계 산출물이 없으면 무엇을 먼저 실행하라고 알려주고 멈춘다."""
    if not path.exists():
        raise FileNotFoundError(
            f"{path.name}이 없습니다. 먼저 python {producer_script}를 실행하세요."
        )
```

- `Path(__file__).resolve().parent` 로 스크립트 자신의 위치를 기준으로 경로를 계산하므로, 어느 폴더에서 실행하든 데이터·산출물을 정확히 찾습니다(재현성).
- `require_artifact` 는 순서를 건너뛰고 실행하면 "먼저 무엇을 실행하라"는 안내와 함께 멈추게 합니다. 실행 순서를 코드가 강제하는 장치입니다.
- 각 단계는 자신의 핵심 수치를 `metrics.json` 에 누적 저장하고, 마지막 `08_report`가 이 파일을 읽어 리포트를 조립합니다. 원본을 다시 훑지 않아 재현이 빠릅니다.

함수·모듈 분해

각 스크립트 안에서도 하는 일을 **함수 단위로 쪼갭니다**. 한 함수가 한 가지 일만 하게 하면 검증(테스트)과 재사용이 쉬워집니다. 예를 들어 정제 스크립트(02_clean)는 테이블마다 정제 함수를 따로 둡니다.

함수	책임(한 가지 일)	반환
<code>clean_customers(df)</code>	중복 제거·키 유일화·gender 통일·날짜 변환·city 공백 제거	정제된 customers
<code>clean_products(df)</code>	product_id 유일화·price 문자열·이상 값 처리·category 공백 제거	정제된 products
<code>clean_orders(df)</code>	중복 제거·미래 날짜를 NaT로 통일	정제된 orders
<code>clean_order_items(df, products_clean)</code>	중복 제거·quantity clip·unit_price 채우기	정제된 order_items
<code>print_before_after(name, before, after)</code>	정제 전후 행수·dtype 대조 출력	없음(출력만)

이렇게 나누면 "customers 정제만 고치고 싶다"면 `clean_customers` 만 손대면 되고, 정제 전후 대조 출력 형식은 `print_before_after` 한 곳에서 통일됩니다. 병합 단계(05_integrate)도 `merge_step(left, right, on, name)` 하나로 세 번의 병합을 같은 안전장치(행수 대조)로 처리합니다 — 같은 로직을 세 번 복사하지 않습니다.

실행 결과는 다음처럼 각 단계가 다음 단계를 안내하며 이어집니다.

[완료] 정제 결과를 `artifacts/*.parquet`에 저장했습니다. 다음 단계: `python capstone/03_derive.py`

설계 단계 자주 묻는 질문

Q. 스크립트를 여러 개로 나누면 매번 앞 단계를 다시 돌려야 해서 느리지 않나요? 아닙니다. 각 단계가 결과를 `artifacts/*.parquet`으로 저장하므로, 뒤 단계는 원본을 다시 정제하지 않고 저장된 Parquet만 읽습니다. 예를 들어 집계 규칙만 바꿨다면 04_aggregate부터 다시 실행하면 됩니다. 나눠 두는 편이 오히려 반복 실행을 빠르게 합니다.

Q. ADR을 꼭 이렇게 다섯 필드로 적어야 하나요? 형식보다 목적이 중요합니다. 핵심은 "결정만 적지 말고 왜 그렇게 정했는지(근거)와 그로 인한 후속 제약(결과)을 함께 남기는 것"입니다. 코

드는 무엇을 했는지만 보여 주므로, 근거·대안을 문서로 남겨야 반년 뒤의 나와 동료が 결정을 이해합니다.

Q. 병합에서 왜 inner가 아니라 left를 기본으로 삼나요? inner는 매칭 실패 행을 조용히 버려 매출 총합을 왜곡합니다(ADR-1). left는 실패를 결측으로 남겨 "얼마가 왜 빠졌는지"를 추적하게 합니다. 이 프로젝트처럼 "숫자가 새지 않았음"을 검증해야 하는 분석에서는 left가 더 정직합니다.

Q. 중간 산출물을 CSV가 아니라 Parquet으로 저장하는 이유가 뭔가요? CSV는 타입 정보를 잃습니다 — 날짜·범주형이 문자열로 풀려 다음 단계가 다시 파싱해야 합니다. Parquet은 dtype을 보존하므로 단계 사이에 타입이 어긋나지 않고, 압축돼 있어 크기·읽기 속도도 유리합니다(8단원). 사람이 눈으로 볼 요약표만 CSV로 남깁니다.

실습 과제 (2종 1세트)

과제 A — 나의 ADR 직접 작성

직접 작성: 설계 결정을 ADR 5필드로

본문 ADR-1·ADR-2를 참고해, 아직 기록되지 않은 **설계 결정 하나**를 골라 **ADR 5필드로 직접 작성**합니다(빈칸을 추측으로 메우는 것이 아니라, 스스로 결정하고 근거를 문장으로 씁니다). 아래 중 하나를 고르거나 여러분이 발견한 다른 결정을 다뤄도 됩니다.

- 후보 1: order_datetime의 미래(2025+) 이상값을 삭제 대신 NaT로 통일하기로 한 결정
- 후보 2: unit_price 결측을 상품 정가로 채우되, 상품 정보조차 없으면 전체 중앙값으로 2차 보정하기로 한 결정
- 후보 3: 고객 등급을 절대 금액 기준이 아니라 qcut(4분위)으로 나누기로 한 결정

작성 라벨(다섯 필드를 모두 채웁니다):

- [상황(Context)]
- [결정(Decision)]
- [대안(Alternatives) — 고려했으나 택하지 않은 방법]
- [근거(Rationale) — 왜 이 결정인가]
- [결과(Consequences) — 이 결정이 부르는 후속 점검·제약]

과제 B — 병합 설계를 실측으로 검증

직접 실행·기록: 키 유일성과 조인 방향 확인

ADR-1의 전제(오른쪽 키가 유일해야 left 병합이 행수를 보존한다)를 **직접 실행해 확인**합니다. 정제된 통합 테이블을 만들기 전, 다음을 실행하고 실제 출력을 기록합니다.

1. 정제된 세 테이블의 키가 유일한지 확인합니다(무엇을 실행할지 명령까지 제시합니다).
2. `orders["order_id"].is_unique`
3. `customers["customer_id"].is_unique`
4. `products["product_id"].is_unique`
5. [각 명령의 실제 출력(True/False)]
6. `order_items`를 왼쪽으로 두고 `orders`를 `how="left"` 로 병합한 뒤, 병합 전후 행수를 비교합니다.
7. [병합 전 행수 / 병합 후 행수 실제 출력]
8. 만약 `how="inner"` 로 바꾸면 행수가 어떻게 달라지는지도 실행해 기록하고, ADR-1이 왜 left를 택했는지 여러분의 실측으로 설명합니다.
9. [inner 병합 시 행수와, left와의 차이에 대한 내 설명]

자가 체크

스스로 점검(손으로 했는지 확인)

- [] 파이프라인 여섯 단계와 각 단계 산출물·사용 개념을 표로 정리했는가?
- [] 병합 결정을 ADR 5필드(상황·결정·대안·근거·결과)로 **내 손으로** 하나 이상 작성했는가?
- [] 통합 테이블 스키마(어느 테이블에서 어느 열을 가져올지)를 확정했는가?
- [] 정제·파생 규칙을 근거와 함께 문서로 남겼는가?
- [] 세 키의 유일성을 `is_unique` 로 직접 실행해 확인했는가(머리로가 아니라)?
- [] 스크립트를 단계별로 나누고 실행 순서를 정했는가?

다음 장(ch_14)에서는 이 설계대로 M1~M7 마일스톤을 실제 코드로 구현하고, 각 단계의 결과를 원본과 대조해 검증합니다.

종합 프로젝트 (3) 구현: 정제·통합·분석 수행

학습 목표

- loc·불리언·query로 세그먼트를 추출(M1)하고 결측·중복·이상값·타입·문자열을 규칙대로 정제(M2)하며 파생·범주(M3)를 만들 수 있습니다 [→G3]
- groupby·pivot_table로 핵심 지표를 집계(M4)하고 네 테이블을 병합해 통합 테이블(M5)을 구축하며 시계열로 매출 추세(M6)를 분석할 수 있습니다 [→G6]
- 백만 행 로그를 최적화·청크·Parquet으로 처리(M7)하고 각 마일스톤을 실제 출력·대조로 검증할 수 있습니다 [→G9]

ch_13에서 설계한 파이프라인을 이제 **마일스톤(milestone)** 단위로 구현합니다. 마일스톤은 한번에 검증 가능한 증분(incremental) 단위로, M1부터 M7까지 앞 단계의 산출물이 다음 단계의 입력이 됩니다. 각 마일스톤 끝에는 **체크포인트**를 두어 결과를 원본과 대조합니다 — 이것이 "집계 숫자를 믿을 수 있는가"라는 이 프로젝트의 통찰(NFR2)을 지키는 방법입니다.

이 장의 모든 코드와 출력은 참조 구현 `capstone/02_clean.py ~ 07_biglog.py` 에서 가져왔고, 아래 출력은 직접 실행해 재확인한 실제 결과입니다. **새 문법은 없습니다** — 각 마일스톤에 어떤 단원 개념을 쓰는지 표제에 밝혀 둡니다.

마일스톤과 단원 개념 대응

마일스톤	하는 일	사용하는 단위 개념
M1	주문 세그먼트 추출	u3·u4(선택·불리언·query)
M2	정제 파이프라인	u5(결측·중복·이상값·타입·문자열)
M3	파생·구간화	u6(벡터 연산·cut·qcut·map)
M4	핵심 지표 집계	u7(groupby·pivot_table·crosstab)
M5	통합 테이블	u8(merge·melt·pivot)
M6	매출 추세	u9(to_datetime·resample·rolling·pct_change)
M7	대용량 로그	u10(다운캐스팅·category·chunksize·Parquet)

14.1 M1 세그먼트 추출 · M2 정제 파이프라인

M1 — 주문 세그먼트 추출 (u3·u4: 불리언 필터·query)

원하는 주문만 정확히 뽑는지 확인합니다. 같은 조건을 불리언 필터와 query 두 방식으로 추출해 결과 행수가 일치하는지 대조합니다(u4의 검증 습관).

```
# 세그먼트 1: 6월에 발생한 웹 채널 배송완료 주문
bool_seg1 = orders[
    (orders["order_datetime"] >= "2024-06-01")
    & (orders["order_datetime"] < "2024-07-01")
    & (orders["channel"] == "web")
    & (orders["status"] == "delivered")
]
query_seg1 = orders.query(
    "order_datetime >= '2024-06-01' and order_datetime < '2024-07-01' "
    "and channel == 'web' and status == 'delivered'"
)
print(len(bool_seg1), len(query_seg1), len(bool_seg1) == len(query_seg1))
```

- 다중 조건은 & (그리고)로 결합하고 각 조건을 괄호로 감쌉니다(u4 오개념: and 를 쓰면 오류, 괄호를 빼면 우선순위가 엉킴).
- 같은 논리를 query 문자열로도 써서, 두 방식 결과 행수가 같은지(==)로 필터가 옳게 동작했는지 검증합니다.

실행 결과입니다.

```
세그먼트1(6월 web delivered): 불리언 13075행 / query 13075행 -> 일치: True
세그먼트2(취소·반품): 불리언 30014행 / query 30014행 -> 일치: True
조건 대조(isin으로 재계산): 30014행 -> 일치: True
세그먼트3(app·1분기): 불리언 24370행 / query 24370행 -> 일치: True
```

체크포인트 M1: 세 세그먼트 모두 불리언과 query 결과가 일치(True)하고, 취소·반품 세그먼트는 isin 으로 재계산한 30,014행과도 일치합니다. 필터가 의도대로 동작함을 세 방식으로 교차 확인했습니다.

M2 — 정제 파이프라인 (u5: 결측·중복·이상값·타입·문자열)

ch_13의 규칙표대로 네 테이블을 정제하고, 정제 전후 행수·dtype 변화를 대조표로 남깁니다. 핵심 정제 로직은 다음과 같습니다.

```
GENDER_MAP = {"남": "M", "여": "F", "M": "M", "F": "F"}

def clean_customers(customers):
    df = customers.copy()
    df = df.drop_duplicates() # 완전 중복 제거
    df = df.drop_duplicates(subset=["customer_id"], keep="first") # 키 유일화
    df["gender"] = df["gender"].map(GENDER_MAP) # 표기 통일
    df["birth_date"] = pd.to_datetime(df["birth_date"], errors="coerce")
    df["city"] = df["city"].str.strip() # 공백 제거
    return df.reset_index(drop=True)

def clean_products(products):
    df = products.drop_duplicates(subset=["product_id"], keep="first").copy()
    price_str = df["price"].astype(str).str.replace("원", "", regex=False)
    price_str = price_str.str.replace(",", "", regex=False)
    price_numeric = pd.to_numeric(price_str, errors="coerce")
    df["price"] = price_numeric.where(price_numeric > 0) # 0/음수는 NaN
    df = df.dropna(subset=["price"]) # 정가 모르는 상품 제외
    df["category"] = df["category"].str.strip()
    return df.reset_index(drop=True)
```

- map(GENDER_MAP) 으로 '남'/'여'를 'M'/'F'로 통일합니다(u5 문자열·매핑). 사전에 없는 결측은 자동으로 NaN이 되어 무응답으로 남습니다(역지로 채우지 않음, NFR3).
- price는 str.replace 로 '원',''을 지운 뒤 to_numeric(errors="coerce") 로 숫자화하고, where(>0) 로 0·음수 이상값을 NaN 처리한 다음 dropna 로 제외합니다(u5 타입·이상값).
- 모든 정제 함수는 copy() 로 원본을 지키고 reset_index(drop=True) 로 인덱스를 정리합니다.

정제 전후 대조 실행 결과입니다.

```
[customers] 정제 전 5,000행 -> 정제 후 4,950행 (변화 -50행)
[products] 정제 전 500행 -> 정제 후 489행 (변화 -11행)
[orders] 정제 전 200,000행 -> 정제 후 200,000행 (변화 +0행)
  order_datetime 결측(미래 포함) 정제 후 건수: 1,973
  [2차 보정] 상품 정보 없는 라인 239건을 전체 정가 중앙값(31,200원)으로 채움
[order_items] 정제 전 500,000행 -> 정제 후 499,880행 (변화 -120행)
  quantity<=0 잔존: 0건, unit_price 결측 잔존: 0건
```

체크포인트 M2: customers는 중복 50행이 빠져 4,950행, products는 이상값·변환 실패 상품 11개가 빠져 489행, order_items는 중복 120행이 빠져 499,880행이 되었습니다. orders는 행을 지우지 않되 order_datetime의 결측·미래값 1,973건을 NaT로 통일했습니다(ADR-2). quantity 이상값과 unit_price 결측은 정제 후 0건으로, 규칙이 빠짐없이 적용되었습니다.

14.2 M3 파생·범주 · M4 핵심 지표 집계

M3 — 파생 변수와 구간화 (u6: 벡터 연산·cut·qcut·map)

주문 금액을 벡터 연산으로 만들고, 표본 행을 손으로 계산해 대조합니다(u6의 검증 습관).

```
# 벡터 연산: 반복문 없이 열 전체를 한 번에 계산
order_items = order_items.assign(
    amount=order_items["unit_price"] * order_items["quantity"] * (1 - order_items["discount"])
)

# 연령대 cut, 가격대 qcut
AGE_BINS = [0, 20, 30, 40, 50, 60, 130]
AGE_LABELS = ["10대 이하", "20대", "30대", "40대", "50대", "60대 이상"]
customers["age_band"] = pd.cut(customers["age"], bins=AGE_BINS, labels=AGE_LABELS, right=False)
products["price_band"] = pd.qcut(products["price"], q=4, labels=["저가", "중저가", "중고가", "고가"])
```

- `amount = 단가 x 수량 x (1 - 할인율)` 을 `assign` 으로 원본을 보존하며 새 열로 더합니다 (u6 벡터 연산·assign).
- `cut` 은 경계값을 직접 지정해 나이를 연령대로 나누고(등간격), `qcut` 은 4분위로 개수가 고르게 나뉘도록 가격대를 만듭니다(u6 구간화).

실행 결과입니다.

표본 행(order_item_id=59256) 손계산 대조: unit_price=20300 x quantity=1.0 x (1-discount=0.05)
amount 합계: 57,240,415,275원, NaN 개수: 0

연령대(age_band) 분포:

10대 이하	215
20대	808
30대	879
40대	897
50대	888
60대 이상	885
NaN	378

가격대(price_band) 분포(qcut 4분위이므로 개수가 고르게 나뉜다):

저가	123
중저가	122
중고가	122
고가	122

체크포인트 M3: 표본 행의 amount(19,285원)가 손계산과 정확히 일치하고, amount 합계 57,240,415,275원에 NaN이 0개입니다. qcut 가격대는 123·122·122·122로 거의 균등하게 나뉘어 4분위가 옳게 동작했습니다. birth_date 결측 378명은 연령대 NaN으로 전파되어 결측이 조용히 사라지지 않았습니다.

M4 — 핵심 지표 집계 (u7: groupby·pivot_table·crosstab)

카테고리별 매출을 groupby로 집계하고, 카테고리×채널 피벗테이블과 고객 등급을 만든 뒤, 집계 총합을 원본과 대조합니다.

```
category_summary = with_category.groupby("category").agg(
    매출=("amount", "sum"), 건수=("amount", "size"), 평균단가=("unit_price", "mean")
).sort_values("매출", ascending=False)

# 고객별 총 구매액을 qcut으로 4등급(브론즈~VIP)
customer_spend = with_customer.groupby("customer_id")["amount"].sum().dropna()
customer_grade = pd.qcut(customer_spend, q=4, labels=["브론즈", "실버", "골드", "VIP"])
```

카테고리 집계와 고객 등급 실행 결과입니다.

M4-1. 카테고리별 핵심 지표

	매출	건수	평균단가
전자	2.359253e+10	109136	96031.803438
가구	1.699971e+10	58217	130199.158321
의류	4.921748e+09	60468	36091.717933
도서	4.667684e+09	137024	15146.707876
뷰티	3.741299e+09	63961	26022.150998
식품	2.006086e+09	63756	13989.829977

M4-4. 고객 등급(핵심 고객 식별)

	고객수	평균구매액	총구매액
브론즈	1467	821454.0	1.205074e+09
실버	1466	3305715.0	4.846178e+09
골드	1466	5708968.0	8.369347e+09
VIP	1466	29208606.0	4.281982e+10

최고 구매액 고객: customer_id=4823, 총구매액=2,535,471,108원

집계 총합 대조 검증 결과입니다.

M4-5. 집계 총합 대조 검증

원본 order_items amount 합계 : 57,240,415,275원
 카테고리 집계 합계 : 55,929,053,008원
 상품 정보 없어 제외된(고아) 금액 : 1,311,362,267원 (7318건)
 대조: 집계+제외 = 원본 합계 -> 차이 0.0000원 -> 일치: True

체크포인트 M4: 카테고리 집계 합계(55,929,053,008원)에 고아 금액(1,311,362,267원)을 더하면 원본 합계(57,240,415,275원)와 차이 0원으로 정확히 일치합니다(ADR-2가 예고한 검증). 매출은 전자(235.9억) > 가구(170.0억) 순이고, VIP 등급 1,466명의 평균 구매액이 브론즈의 약 35배로 매출이 소수 고객에 집중돼 있음이 드러납니다.

14.3 M5 통합 테이블 · M6 매출 추세 분석

M5 — 통합 테이블 구축 (u8: merge·melt·pivot)

ADR-1대로 order_items를 왼쪽 기준으로 orders → customers → products를 순차 left 병합하고, 매 단계 행수를 대조합니다.

```
def merge_step(left, right, on, name):
    before = len(left)
    merged = left.merge(right, on=on, how="left")
    after = len(merged)
    if after != before:
        raise AssertionError(f"{name} 병합 후 행수가 바뀌었습니다 - 키({on})가 유일하지 않을 수")
    return merged

integrated = merge_step(order_items, orders[orders_cols], "order_id", "orders")
integrated = merge_step(integrated, customers[customers_cols], "customer_id", "customers")
integrated = merge_step(integrated, products[products_cols], "product_id", "products")
```

- `merge_step` 은 병합 전후 행수가 다르면 곧바로 오류를 냅니다 — 오른쪽 키가 유일하지 않아 행이 붙어나는 사고를 즉시 잡는 안전장치입니다(ADR-1의 결과 필드).
- 필요한 열(`orders_cols` 등)만 골라 붙여 통합 테이블이 불필요하게 커지지 않게 합니다.

병합·재구조화 실행 결과입니다.

M5-1. 순차 병합과 행수 무결성 점검

```
[orders] 499,880행 -(left merge on order_id)-> 499,880행 -> 행수 유지
[customers] 499,880행 -(left merge on customer_id)-> 499,880행 -> 행수 유지
[products] 499,880행 -(left merge on product_id)-> 499,880행 -> 행수 유지
```

customer 매칭 실패(고아 키) 행: 2,479행 (0.50%)

product 매칭 실패(고아 키) 행 : 7,318행 (1.46%)

참고: customer_id inner merge 성공률 = 99.50% vs left merge 기준 성공률 = 99.50% -> 일치: True

M5-2. 카테고리 x 월 매출 재구조화 (pivot <-> melt)

melt로 되돌린 긴 형식: (36, 3) (카테고리 6종 x 월 6개 = 36행)

왕복 후 합계 대조: wide 합계=55,377,770,240 vs long 합계=55,377,770,240 -> 일치: True

체크포인트 M5: 세 번의 left 병합 모두 행수가 499,880으로 보존됐습니다(키 유일성 전제 충족). 고아 키는 고객 0.50%·상품 1.46%로 결측으로 드러났고, inner 병합 성공률과 left 기준 성공률이 일치해 두 방식으로 교차 확인됐습니다. pivot으로 펼친 넓은 형식과 melt로 되돌린 긴 형식의 합계가 정확히 같아 재구조화가 정보를 잃지 않았습니니다.

M6 — 매출 추세 분석 (u9: to_datetime·resample·rolling·pct_change)

통합 테이블의 order_datetime을 날짜 인덱스로 두고 월별 매출을 리샘플링한 뒤, 특정 달 합계를 부분집합 합과 대조합니다.

```
ts = integrated.dropna(subset=["order_datetime"]).copy() # NaT(결측·미래값) 제외
ts_indexed = ts.set_index("order_datetime").sort_index()
monthly_revenue = ts_indexed["amount"].resample("ME").sum() # 월별 매출
daily_revenue = ts_indexed["amount"].resample("D").sum()
rolling_7d = daily_revenue.rolling(7).mean() # 7일 이동평균
monthly_growth = monthly_revenue.pct_change() # 전월 대비 성장률
```

월별 매출·성장률·주말 편향 실행 결과입니다.

order_datetime 결측(미래 이상값 포함) 4,886행 제외 -> 분석 대상 494,994행

월별 매출:

2024-01-31	6.761462e+09
2024-02-29	7.386097e+09
2024-03-31	9.200962e+09
2024-04-30	9.886735e+09
2024-05-31	1.119641e+10
2024-06-30	1.224515e+10

전월 대비 매출 성장률(pct_change):

2024-02-29	0.0924
2024-03-31	0.2457
2024-04-30	0.0745
2024-05-31	0.1325
2024-06-30	0.0937

M6-4. 특정 달 합계 대조 검증

2024-06 resample 합계 : 12,245,151,035원
 2024-06 부분집합 필터 합 : 12,245,151,035원
 대조: 차이 0.0000원 -> 일치: True

M6-5. 주말 편향 확인

주말 평균 주문/일: 3430.9건 vs 평일 평균 주문/일: 2435.3건 (배수: 1.41배)

1월 대비 6월 매출 증가율: 81.1%

체크포인트 M6: 2024-06 리샘플링 합계(12,245,151,035원)가 그 달로 직접 필터링한 부분집합 합과 차이 0원으로 일치합니다(u9 검증 습관). 월별 매출은 1월 약 67.6억에서 6월 약 122.5억으로 81.1% 증가했고, 매월 전월 대비 플러스 성장을 유지했습니다. 주말 주문이 평일의 1.41배로, 이후 발견·개선 제안의 근거가 됩니다.

14.4 M7 대용량 로그 처리와 마일스톤 검증

M7 — 백만 행 로그 최적화 (u10: 다운캐스팅 ·category·chunksize·Parquet)

100만 행 web_logs를 그대로 읽으면 158MB입니다. dtype 다운캐스팅과 범주형 변환으로 메모리를 줄이고, **청크 집계**가 **전체 집계**와 일치하는지 대조합니다.

```
opt = raw.copy()
opt["log_id"] = pd.to_numeric(opt["log_id"], downcast="unsigned")
opt["customer_id"] = opt["customer_id"].astype("Int32")
opt["product_id"] = opt["product_id"].astype("Int16")
opt["event_type"] = opt["event_type"].astype("category") # 저카디널리티 -> 큰 이득

# 청크로 나눠 읽으며 event_type별 건수 누적
chunk_counts = pd.Series(dtype="int64")
for chunk in pd.read_csv(WEB_LOGS_PATH, usecols=["event_type"], chunksize=200_000):
    chunk_counts = chunk_counts.add(chunk["event_type"].value_counts(), fill_value=0)
```

- event_type 은 값 종류가 넷뿐(view·search·cart·purchase)이라 category 로 바꾸면 메모리가 크게 줄입니다(u10). 반대로 session_id 는 값이 거의 다 달라 category 이득이 작습니다(반례).
- chunksize=200_000 으로 20만 행씩 나눠 읽으며 event_type 건수를 누적합니다 — 전체를 한 번에 메모리에 올리지 않고 집계하는 방법입니다.

최적화·청크·Parquet 실행 결과입니다.

M7-2. dtype 다운캐스팅과 범주형(category) 변환

최적화 전: 158.0 MB -> 최적화 후: 86.8 MB (절감 71.3 MB, 45.1%)

session_id 고카디널리티(245,430종, 전체의 24.5%): object 66.8 MB vs category 28.3 MB (event_type 반례) 완전 고유 열(log_id를 문자열로): object 60.0 MB vs category 96.0 MB -> category가 더 큼(

M7-3. chunksize 청크 집계

청크 5개(청크당 200,000행)로 나눠 집계한 event_type 건수:

cart	119434
purchase	80359
search	200206
view	600001

청크 집계 vs 전체 집계 일치: True

M7-4. Parquet 저장·재적재 비교

CSV 파일 크기 : 51.9 MB

Parquet 파일 크기 : 23.5 MB (압축률 54.7%)

재적재 행수 확인 : 1,000,000행 (원본과 동일: True)

체크포인트 M7: dtype 최적화만으로 메모리를 158.0MB에서 86.8MB로 45.1% 줄였습니다. 청크 5개로 나눠 집계한 event_type 건수가 전체를 한 번에 집계한 결과와 정확히 일치(True)해, 나눠

읽어도 결과가 같음을 확인했습니다. CSV 51.9MB를 Parquet 23.5MB로 저장해 절반 이하로 줄였고 재적재 행수도 100만으로 보존됐습니다. log_id 반례는 값이 전부 유일하면 category가 오히려 손해임을 보여 줍니다(u10 오개념 교정).

마일스톤 통합 검증

7개 마일스톤의 체크포인트를 한 표로 모으면, 각 단계가 **원본과 대조해 검증됐음**이 드러납니다.

마일스톤	핵심 대조 검증	실측 결과
M1 세그먼트	불리언 = query 행수	세 세그먼트 모두 일치(True)
M2 정제	정제 전후 행수.잔존 이상값	order_items 500,000→499,880, 이상값 0건
M3 파생	표본 행 손계산 = amount 열	19,285원 일치, NaN 0개
M4 집계	집계 합 + 고아 금액 = 원본 합	차이 0.0000원 일치
M5 병합	병합 전후 행수 보존	499,880행 유지, pivot↔melt 합계 일치
M6 시계열	특정 달 resample = 부분집합 합	차이 0.0000원 일치
M7 대용량	청크 집계 = 전체 집계	일치(True), 메모리 45.1% 절감

이 표가 이 프로젝트의 신뢰성입니다 — 일곱 곳 모두 "숫자가 새거나 불어나지 않았음"을 스스로 증명했습니다.

실습 과제 (2종 1세트)

과제 A — 마일스톤 하나를 직접 구현·검증

직접 실행·기록: 내 손으로 마일스톤 완주

M1~M7 중 **하나를 골라** 참조 구현 스크립트를 직접 실행하고, 화면의 실제 출력을 그대로 기록한 뒤 체크포인트를 검증합니다(추측이 아니라 관찰을 옮깁니다). 예를 들어 M4를 골랐다면 무엇을 실행할지 명령까지 제시합니다.

1. 앞 단계 산출물을 먼저 만듭니다: `python capstone/02_clean.py` → `python capstone/03_derive.py`
2. 고른 마일스톤을 실행합니다: `python capstone/04_aggregate.py`
3. [내 실행: 해당 마일스톤의 실제 출력 전문]
4. 그 마일스톤의 **대조 검증 한 줄**(예: "차이 0.0000원 -> 일치: True")을 찾아 옮기고, 그 검증이 무엇을 보장하는지 한 문장으로 설명합니다.
5. [대조 검증 출력과 그 의미에 대한 내 설명]

과제 B — 세그먼트를 바꿔 필터 검증 재현

직접 작성·실행: 나만의 세그먼트 추출과 교차 검증

M1을 참고해, 여러분이 정한 **새 조건의 세그먼트**를 불리언과 query 두 방식으로 추출하고 행수가 일치하는지 직접 검증합니다(스타터에서 조건을 직접 완성합니다).

```
# 예시 스타터 — 조건은 여러분이 직접 채웁니다(빈칸 토큰이 아니라 실제 조건식)
bool_seg = orders[
    (orders["channel"] == ...)          # 여기 직접 작성: 원하는 채널
    & (orders["status"] == ...)         # 여기 직접 작성: 원하는 상태
]
query_seg = orders.query("...")         # 여기 직접 작성: 같은 조건을 query로
print(len(bool_seg), len(query_seg), len(bool_seg) == len(query_seg))
```

1. 위 스타터를 여러분 조건으로 완성해 실행합니다.
2. [내가 정한 세그먼트 조건과 실제 출력(두 방식 행수·일치 여부)]
3. 두 방식 행수가 다르면 무엇이 틀렸는지(괄호 누락·연산자 등) 진단해 고칩니다.
4. [일치 여부와, 불일치였다면 원인·수정 내용]

자가 체크

스스로 점검(손으로 했는지 확인)

- [] M2 정제 후 행수(customers 4,950 · products 489 · order_items 499,880)를 **직접 실행**해 확인했는가?
- [] M3 표본 행 amount를 손으로 계산해 열 값과 일치함을 확인했는가?
- [] M4 집계 총합 대조에서 "집계+고아 = 원본, 차이 0원"을 실측으로 봤는가?

- [] M5 세 번의 left 병합 모두 행수 499,880이 보존됨을 확인했는가?
- [] M6 특정 달 resample 합계 = 부분집합 합(차이 0원)을 확인했는가?
- [] M7 청크 집계 = 전체 집계 일치와 메모리 45.1% 절감을 확인했는가?
- [] 새 조건 세그먼트를 내 손으로 만들어 불리언.query 두 방식으로 교차 검증했는가?

다음 장(ch_15)에서는 이 구현 결과를 근거 있는 발견으로 정리하고, 처음부터 다시 실행해 재현성을 검증하며, 루브릭으로 산출물을 자평하고 개념 적용을 회고합니다.

종합 프로젝트 (4) 발표·평가·성찰: 분석 리포트와 재현성

학습 목표

- 매출 추세·핵심 고객·잘 팔리는 상품에 대한 근거 있는 발견 3개 이상과 데이터 기반 개선 제안을 요약 리포트로 전달할 수 있습니다 [→G10]
- 처음부터 다시 실행하면 같은 결과가 나오는지 재현성을 검증하고 평가 루브릭으로 산출물을 자평할 수 있습니다 [→G10]
- 각 단원 개념을 분석의 어느 단계에 어떻게 적용했는지 회고하고 한계·다음 단계를 성찰할 수 있습니다 [→G10]

ch_14에서 M1~M7을 구현하고 각 단계를 검증했습니다. 이제 마지막으로 **결국 무엇을 알아냈는가**를 리포트로 정리하고, **남이 다시 믿고 재현할 수 있는가**를 검증하며, 스스로 **얼마나 잘했는가**를 루브릭으로 평가하고 회고합니다. 이 장에서도 새 문법은 없습니다 — 이미 만든 결과를 전달·검증·성찰하는 단계입니다.

리포트의 모든 수치는 참조 구현 `capstone/08_report.py` 가 앞 단계들이 남긴 `metrics.json`을 읽어 조립한 것이고, 아래 출력은 직접 실행해 재확인한 실제 결과입니다.

15.1 분석 리포트: 근거 있는 발견과 개선 제안

좋은 발견은 "매출이 늘었다" 같은 인상이 아니라 **수치와 근거가 붙은 문장**입니다. 리포트 실행 결과에서 발견 다섯 개를 가져옵니다(요구사항 FR9는 3개 이상을 요구합니다).

발견: 근거 있는 인사이트 3개 이상

- 1) 매출은 6개월간 꾸준히 늘었다 - 1월 매출 6,761,462,208원에서 6월 매출 12,245,151,035원으로 81% 증가
- 2) 주문은 주말에 몰린다 - 주말 평균 3431건/일 vs 평일 평균 2435건/일로 1.41배 많다(마케팅·재고 관리)
- 3) 매출은 카테고리별로 크게 편차가 있다 - '전자' 카테고리 매출이 23,592,529,138원으로 가장 크다
- 4) 매출이 소수 핵심 고객에게 집중돼 있다 - 최고 구매액 고객(customer_id=4823)의 누적 구매액이 1,234,567,890원
- 5) 대용량 로그는 dtype 최적화만으로도 메모리를 크게 줄일 수 있다 - web_logs 원본 158.0MB를 86.0MB로 줄임

발견 3(카테고리 편차)의 근거가 된 카테고리×채널 매출 피벗테이블은 다음과 같습니다(단위: 원, M4 실측). 전자·가구가 매출을 이끌고, 모든 카테고리에서 web 채널 비중이 가장 큼니다.

channel category	app	store	web	합계
가구	5.044372e+09	2.635005e+09	9.320331e+09	1.699971e+10
도서	1.391793e+09	6.995915e+08	2.576300e+09	4.667684e+09
뷰티	1.129320e+09	5.532714e+08	2.058707e+09	3.741299e+09
식품	6.063737e+08	3.005421e+08	1.099170e+09	2.006086e+09
의류	1.492279e+09	7.343769e+08	2.695092e+09	4.921748e+09
전자	7.050839e+09	3.555585e+09	1.298610e+10	2.359253e+10
합계	1.671498e+10	8.478372e+09	3.073570e+10	5.592905e+10

이 피벗테이블에서 **네 번째 발견의 실마리**가 하나 더 보입니다. 채널 합계를 보면 web(307.4억)이 app(167.1억)-store(84.8억)를 크게 앞섭니다. 온라인(web+app) 채널이 전체 매출의 대부분을 차지하므로, 개선 제안의 우선순위를 온라인 경험에 두는 근거가 됩니다.

이 발견들이 처음의 세 질문(ch_12 Q1~Q3)에 어떻게 답하는지 매핑하면 다음과 같습니다.

질문(ch_12)	답한 발견	근거 수치	나온 단계
Q1. 매출 추세는?	발견 1·2	6개월 +81.1%, 주말 1.41배	M6(시계열)
Q2. 핵심 고객은?	발견 4	VIP 1,466명, 최고 고객 25.4억	M4(집계)
Q3. 잘 팔리는 것은?	발견 3	전자 235.9억으로 1위	M4(집계)

발견 1(매출 추세)을 더 자세히 보면, 월별 매출과 전월 대비 성장률(M6 실측)이 다음과 같습니다. 6개월 내내 플러스 성장이며, 특히 3월(+24.6%)과 5월(+13.3%)에 성장이 두드러집니다.

월	매출(원)	전월 대비 성장률
2024-01	6,761,462,208	—
2024-02	7,386,097,000	+9.24%
2024-03	9,200,962,000	+24.57%
2024-04	9,886,735,000	+7.45%
2024-05	11,196,410,000	+13.25%
2024-06	12,245,151,035	+9.37%

핵심 고객(발견 4)의 등급별 구조도 근거가 분명합니다. 고객을 총 구매액 4분위로 나눈 결과, VIP 1,466명의 총구매액이 약 428.2억으로 전체 매출의 큰 몫을 차지합니다. VIP 평균 구매액(약

2,921만 원)은 브론즈(약 82만 원)의 35배가 넘어, 매출이 소수 고객에 강하게 집중돼 있음을 보여 줍니다.

한편 고객 연령대×성별 교차표(M4 실측)를 보면 20~60대가 비교적 고르게 분포하고 성별 편차도 크지 않아, 특정 인구 집단에 치우친 고객 기반이 아님을 확인할 수 있습니다. 이는 "매출 집중은 인구 특성이 아니라 개별 고객의 구매 규모 차이에서 온다"는 발견 4를 뒷받침합니다.

데이터 기반 개선 제안도 함께 전달합니다. 발견에서 곧바로 행동으로 잇는 것이 리포트의 가치입니다.

데이터 기반 개선 제안

- 주말(1.41배) 트래픽에 맞춰 주말 인력·배송 용량을 미리 늘리면 배송 지연을 줄일 수 있다.
- '전자' 카테고리 매출 비중이 높으므로 이 카테고리의 재고·프로모션을 우선 관리하고, 하위 카테고리도
- VIP 등급 고객(1,466명)에게 별도 리텐션 프로그램(적립·전용 할인)을 운영하면 이탈 방지 효과가
- price 원본 오염(문자열·이상값)이 반복되므로, 데이터 입력 단계에서 가격을 숫자 입력 필드로 강

전달용 요약 리포트의 구조

발견과 제안을 실제 **전달물(deliverable)**로 묶을 때는 읽는 사람이 결론부터 보게 구성합니다. 권장 구조는 다음과 같습니다.

순서	절	답을 내용
1	한 줄 요약	"6개월 매출 +81.1%, 전자·온라인·소수 VIP가 견인" 같은 핵심 결론 한 문장
2	답한 질문과 발견	Q1~Q3에 대한 발견 3개 이상, 각 발견에 근거 수치
3	근거 표·그림	월별 매출표·카테고리×채널 피벗·등급 요약(수치 출처)
4	개선 제안	발견에서 잇는 행동 제안(사실과 추정 구분)
5	한계와 다음 단계	이 데이터로 알 수 없는 것과 확장 방향
6	재현 방법	실행 순서와 핵심 재현 수치(누가 다시 실행해도 좋음)

이 구조를 따르면 의사결정자는 1~2절만 읽어도 핵심을 얻고, 데이터 담당자는 6절로 분석을 재현할 수 있습니다. **결론을 앞에, 근거를 뒤에, 재현법을 맨 끝에** 두는 것이 전달의 원칙입니다.

발견에는 항상 근거와 한계를 함께 적습니다

발견 4의 "이탈 방지 효과가 클 것"은 **추정**입니다. 이 데이터로는 VIP 고객이 실제로 이탈했는지, 리텐션 프로그램이 효과가 있을지는 알 수 없습니다(그건 실험·인과 분석의 영역이고, ch_12에서 범위 밖으로 정했습니다). 정직한 리포트는 "데이터가 보여 주는 것"과 "그로부터의 추정"을 구분합니다.

15.2 재현성 검증: 처음부터 다시 실행

이 프로젝트의 비기능 요구사항 1순위는 재현성(NFR1)이었습니다. 이제 그것을 **실제로 검증**합니다 — 데이터를 다시 만들고 스크립트를 처음부터 다시 실행해 **같은 수치가 나오는지** 확인합니다.

재현성 안내

이 리포트의 모든 수치는 다음을 순서대로 실행하면 동일하게 재현된다:

```
python generate_shop_data.py (shop-data/ 에서 실행, 시드 42 고정)
python capstone/01_load_overview.py
python capstone/02_clean.py
python capstone/03_derive.py
python capstone/04_aggregate.py
python capstone/05_integrate.py
python capstone/06_timeseries.py
python capstone/07_biglog.py
python capstone/08_report.py
```

핵심 재현 수치: 원본 order_items 500,000행 -> 정제 후 499,880행, 통합 테이블 499,880행

재현성이 성립하는 이유는 두 가지입니다.

- **데이터 생성의 결정성:** generate_shop_data.py 는 `numpy.random.default_rng(42)` 와 `random.seed(42)` 로 난수를 고정하고 표준 라이브러리·numpy·pandas만 씁니다. 외부 API·네트워크·현재 시각에 의존하지 않으므로, 몇 번을 다시 만들어도 바이트 단위로 같은 CSV가 나옵니다.
- **분석 계산의 결정성:** 정제 규칙·집계·리샘플링은 난수를 쓰지 않는 결정론적 계산입니다. 같은 입력에 같은 출력이 나오므로, 두 번 실행해도 정제 후 행수·집계 총합·월별 매출표가 정확히 같습니다.

재현성을 눈으로 확인하는 또 한 가지 방법은 각 단계가 남긴 `metrics.json` 을 비교하는 것입니다. 이 파일에는 단계별 핵심 수치(정제 전후 행수·집계 총합·월별 매출·메모리 절감률)가 누적 저장되므로, 두 번의 실행에서 만들어진 두 파일이 같으면 파이프라인 전체가 재현됐다는 증거가

됩니다. 리포트(08_report)는 원본을 다시 훑지 않고 이 파일만 읽어 발견을 조립하므로, metrics.json이 같으면 리포트도 같습니다.

재현성을 직접 확인하는 방법은 핵심 수치 몇 개를 두 번의 실행에서 비교하는 것입니다.

재현 확인 수치	1차 실행	2차 실행	일치
정제 후 order_items 행수	499,880	499,880	예
통합 테이블 행수	499,880	499,880	예
amount 총합	57,240,415,275원	57,240,415,275원	예
6월 매출	12,245,151,035원	12,245,151,035원	예
1월 대비 6월 증가율	81.1%	81.1%	예

재현성이 깨지는 흔한 원인

- 스크립트를 **순서대로** 실행하지 않으면(예: 02를 건너뛰고 03 실행) 산출물이 없어 오류가 납니다 — 참조 구현은 `require_artifact` 로 이를 막습니다.
- 코드에 `random` 을 시드 없이 쓰거나 `datetime.now()` 처럼 실행 시각에 의존하는 값을 넣으면 실행마다 결과가 달라집니다. 분석 코드에는 그런 값을 넣지 않습니다.

15.3 평가 루브릭과 자기 평가

루브릭(rubric)은 산출물을 어떤 기준으로 얼마나 잘했는지 평가하는 표입니다. 이 프로젝트는 중점목표 G1~G10을 평가 기준으로 삼습니다. 각 목표를 3단계(미흡·양호·우수)로 자평합니다.

목표	평가 기준	미흡	양호	우수
G1 적재	dtype.parse_dates 지정 적재	그냥 read_csv만 함	날짜-ID 지정 적재	다섯 테이블 모두 dtype·메모리 확인
G2 품질 점검	품질 이슈 5개 이상 근거와 함께	이슈 나열만	5개 이상 규모 수치 포함	8개 + 단위 개념 매핑
G3 선택·필터	세그먼트 추출·검증	필터만 함	불리언 추출	불리언=query 교차 검증
G4 정제	규칙 기반 정제·전후 대조	임의로 삭제	규칙대로 정제	전후 행수·dtype 대조표
G5 파생·구간화	amount·연령대·가격대	파생만 함	cut·qcut 사용	표본 손계산 대조
G6 집계	groupby·pivot·총합 대조	집계만 함	피벗까지	집계+고아=원본 검증
G7 병합	키·조인 방식·행수 점검	병합만 함	left 병합	행수 보존 + 고아 키 진단
G8 시계열	resample·이동평균·대조	월별 합계만	리샘플·성장률	특정 달 합계 대조
G9 대용량	최적화·청크·Parquet	그냥 읽기	dtype 최적화	청크 일치 + 수치 비교
G10 통합·전달	발견 3개+재현성 리포트	발견만 나열	발견 3개+제안	재현성 검증까지

직접 작성: 나의 자기 평가표

위 루브릭을 그대로 옮겨, 여러분이 만든 산출물을 G1~G10 각각에 대해 **미흡/양호/우수 중 하나로 자평**하고 그 근거를 한 줄씩 적습니다(빈칸을 추측으로 메우지 말고, 여러분이 실제로 한 것을 근거로 판정합니다).

- [G1 자평(미흡/양호/우수) — 근거: 내가 실제로 한 것]
- [G2 자평 — 근거]

- ... G10까지 각각 판정과 근거
- [가장 부족한 목표 1개와, 그것을 우수로 끌어올릴 구체적 방법]

15.4 성찰: 개념 적용 회고와 다음 단계

마지막으로, 이 프로젝트에서 **어느 단원 개념을 어느 단계에 어떻게 적용했는지** 회고합니다. 이 회고가 학습을 다른 데이터셋으로 전이하는 다리입니다. 회고를 잘 하려면 "그 개념이 없었다면 무엇이 막혔을까"를 함께 물어야 합니다. 예를 들어 5단원의 `to_numeric` 이 없었다면 `price`가 문자열로 남아 카테고리 매출을 아예 합산할 수 없었고, 8단원의 `merge`가 없었다면 `amount`(주문 상세)와 `category`(상품)가 서로 다른 표에 있어 카테고리 분석 자체가 불가능했습니다. 각 개념이 분석의 어느 길목을 열었는지 짚으면, 다음 데이터셋에서 같은 길목을 만났을 때 어떤 도구를 꺼내야 할지 알게 됩니다.

단원	개념	이 프로젝트에서 적용한 곳
u1~u2	적재·구조·훑어보기	ch_12 다섯 테이블 적재·품질 점검표
u3~u4	선택·불리언·query	M1 세그먼트 추출·교차 검증
u5	결측·중복·이상값·타입·문자열	M2 정제 파이프라인(8개 이슈)
u6	벡터 연산·cut·qcut·map	M3 amount·연령대·가격대·라벨
u7	groupby·pivot·crosstab	M4 카테고리·채널·등급 집계
u8	merge·melt·pivot	M5 통합 테이블·재구조화
u9	to_datetime·resample·rolling·pct_change	M6 매출 추세·성장률
u10	다운캐스팅·category·chunksize·Parquet	M7 대용량 로그 최적화

이 프로젝트가 남긴 핵심 교훈(성찰)

- **정제가 분석의 신뢰를 만든다:** 오염된 `price·gender·order_datetime`을 그대로 뒀다면 카테고리 순위도 매출 총합도 틀렸을 것입니다. ch_12에서 세운 통찰("신뢰는 정제·통합·검증에서 나온다")이 M4의 "집계+고아=원본, 차이 0원"으로 실제 확인됐습니다.
- **매 단계 대조 검증이 사고를 막는다:** 병합 후 행수가 불어나지 않았는지(M5), 청크 집계가 전체와 같은지(M7)를 그때그때 대조했기에 잘못된 숫자로 리포트를 쓰는 일을 피했습니다.

다.

- **재현성은 설계 단계에서 만들어진다:** 시드 고정·순서 실행·중간 산출물 저장을 ch_13에서 설계로 정해 뒀기에, ch_15의 재현 검증이 그냥 성립했습니다.

한계와 다음 단계도 정직하게 적습니다.

- 이 분석은 6개월치 스냅샷이라 계절성·연간 추세는 알 수 없습니다. 더 긴 기간 데이터가 있으면 resample로 연·분기 추세를 볼 수 있습니다.
- 고객 등급을 구매액 하나로만 나눴습니다. 최근성·구매빈도를 더한 다차원 세분화(RFM 유사)로 확장할 수 있습니다(ch_12에서 범위 밖으로 뛴 후보 B).
- web_logs의 view→cart→purchase 퍼널 전환 분석(후보 D)은 이번엔 대용량 처리 실습으로만 다뤘습니다. 다음 단계로 전환율 분석을 붙일 수 있습니다.
- 매출 성장(+81.1%)이 신규 고객 유입 때문인지 기존 고객의 구매 증가 때문인지는 이 분석만으로 구분되지 않습니다. signup_date와 주문 시점을 이어 코호트 분석으로 확장하면 성장의 원천을 나눠 볼 수 있습니다.

발표·성찰 단계 자주 묻는 질문

Q. 발견이 5개인데 요구는 3개 이상입니다. 많을수록 좋은가요? 개수보다 근거의 질이 중요합니다. 근거 없는 발견 10개보다 수치·출처가 붙은 발견 3개가 낫습니다. 각 발견에 "어느 단계(M4·M6)에서 어떤 수치로 나왔는지"를 달 수 있어야 신뢰할 수 있는 발견입니다.

Q. 개선 제안을 어디까지 강하게 말해도 되나요? 데이터가 직접 보여 주는 것(예: 주말 주문이 1.41배 많다)은 단정해도 됩니다. 그로부터의 처방(예: 주말 인력을 늘리면 배송이 개선된다)은 추정이므로 "기대된다·여지가 있다"로 표현해 사실과 구분합니다.

Q. 재현성 검증에서 수치가 미세하게 다르면 실패인가요? 이 프로젝트는 난수를 시드로 고정하고 결정론적 계산만 하므로 **완전히 같아야** 합니다. 다르다면 순서를 건너뛰었거나, 코드에 실행 시각·시드 없는 난수가 섞인 것입니다. 부동소수 합계의 마지막 자리 차이도 없어야 정상입니다(대조에서 차이 0원).

실습 과제 (2종 1세트)

과제 A — 나의 분석 리포트 직접 작성

직접 작성·실행: 근거 있는 발견 3개 이상 리포트

`python capstone/08_report.py` 를 직접 실행해 실제 발견 출력을 확인한 뒤, **여러분의 언어**로 발견 리포트를 작성합니다(출력을 그대로 베끼지 말고, 수치를 근거로 여러분 문장으로 씁니다).

1. 08_report를 실행하고 실제 발견 다섯 개 출력을 기록합니다(선행 단계 01~07을 먼저 실행해야 합니다).
2. [내 실행: 발견·개선 제안 실제 출력]
3. 그 중 **하나**는 **여러분이 직접 계산한 새 발견**으로 바꿉니다. 예: 채널별 취소율, 특정 연령대의 카테고리 선호. 무엇을 어떻게 집계했는지 명령과 함께 적습니다.
4. [내가 계산한 새 발견 — 사용한 groupby·집계 코드와 실제 출력, 근거 수치]
5. 각 발견에 "데이터가 보여 주는 것"과 "그로부터의 추정"을 구분해 개선 제안을 답니다.
6. [내가 작성한 발견 3개 이상 + 개선 제안(추정은 추정으로 표시)]

과제 B — 재현성 직접 검증

직접 실행·기록: 두 번 실행해 같은 결과 확인

재현성(NFR1)을 말로 믿지 말고 **직접 두 번 실행해** 확인합니다.

1. 파이프라인을 한 번 실행하고 핵심 수치 세 개를 기록합니다(무엇을 볼지 명령까지 제시합니다).
2. 정제 후 order_items 행수(02_clean 출력), amount 총합(03_derive 출력), 6월 매출(06_timeseries 출력)
3. [1차 실행 수치 3개]
4. `python generate_shop_data.py` 로 데이터를 다시 만든 뒤 01~08을 처음부터 다시 실행하고 같은 수치를 기록합니다.
5. [2차 실행 수치 3개]
6. 두 실행의 수치가 같은지 대조하고, 하나라도 다르다면 원인(순서 누락·시드 문제 등)을 진단합니다.
7. [일치 여부와, 불일치였다면 원인]

자가 체크

스스로 점검(손으로 했는지 확인)

- [] 근거 있는 발견을 3개 이상, 각각 수치와 함께 **내 문장으로** 적었는가?
- [] 발견 중 하나는 내가 직접 집계한 새 발견으로 바꿨는가?
- [] 개선 제안에서 "데이터가 보여 주는 것"과 "추정"을 구분했는가?
- [] 파이프라인을 **두 번 실행**해 핵심 수치가 같음을 실측으로 확인했는가?
- [] G1~G10 루브릭으로 내 산출물을 자평하고 근거를 적었는가?

- [] 각 단원 개념을 어느 단계에 적용했는지 회고표로 정리했는가?
- [] 이 분석의 한계와 다음 단계를 정직하게 적었는가?

여기까지가 이 책의 종합 프로젝트입니다. 여러분은 흩어지고 지저분한 쇼핑물 6개월치 데이터를 적재·정제·파생·집계·병합·시계열·대용량 처리로 통합해, 매출 추세·핵심 고객·잘 팔리는 상품을 근거와 함께 밝히고 재현 가능한 분석으로 전달했습니다. 이 흐름(문제 정의 → 설계 → 구현·검증 → 전달·성찰)은 어떤 새 데이터셋에도 그대로 옮겨 쓸 수 있는 데이터 분석의 뼈대입니다.

특히 이 프로젝트가 반복해서 강조한 **매 단계 대조 검증**(집계=원본, 병합 후 행수 보존, 특정 달 합계=부분집합 합, 체크=전체)은 판다스 문법을 넘어서는 습관입니다. 어떤 도구를 쓰든, 결과를 내놓기 전에 "이 숫자가 새거나 불어나지 않았는가"를 스스로 증명하는 태도가 신뢰받는 분석가와 그렇지 않은 분석가를 가릅니다. 다음에 여러분이 마주할 실제 데이터도 이 데이터셋처럼 지저분할 것입니다 — 그때 이 책에서 익힌 정제→통합→검증의 순서와, 결정을 문서로 남기는 습관(ADR·규칙표·재현성)을 그대로 꺼내 쓰면 됩니다. 그것이 이 종합 프로젝트가 남기려 한 진짜 산출물입니다.